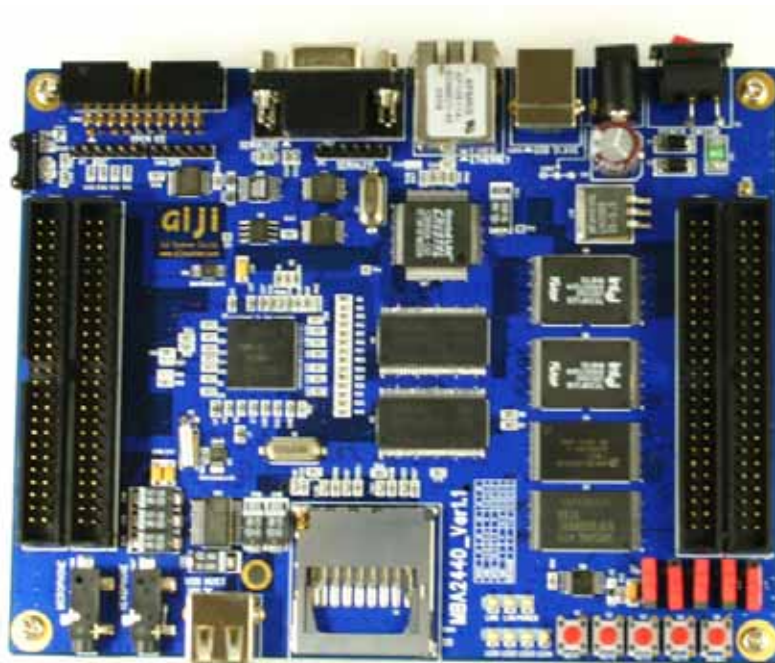


MBA2440 Reference Board Manual



Version 2.0



The present document has been developed within Embedded Division of AIJI System and may be further elaborated for the purposes of AIJI System.
This document is provided for future development work within AIJI System and membership companies of AIJI System only.

Document Summary

Created by: Embedded System Solution Business Division of AIJI System Co. Ltd.
Published by: AIJI System Co. Ltd.
Phone: 031-223-6611
Fax: 031-223-6613
openice@aijisystem.com
www.aijisystem.com, www.mculand.com
2F Samho Tower #1122-10 Ingye-dong, Paldal-gu, Suwon-city, Gyonggi-do, Korea

Keywords: MBA2440 Reference Board Manual

- Version : V2.0 (21. Feb. 2006)

Editor Information

- Name: Choong Heon Kim

- E-mail: honeyme@aijisystem.com

- Tel.: +82-31-231-9868

- Address: Samho-Park-Tower Bldg. 2F, 1122-10, Ingye-Dong, Paldal-Gu, Suwon-City, Gyeonggi-Do, Korea

Version Information

Copyright Notification



AIJI System Co. Ltd.

Copyright ©2005 AIJI System Co., Ltd.

Company names and product names mentioned hereafter are all their own brands or registered brands. No part in this document can be changed without notification in advance. This document is an intellectual asset of AIJI System Co. Ltd. It is prohibited to copy or publish this document without permission of AIJI System Co. Ltd..

No part may be reproduced except as authorized by written permission.

The copyright and the foregoing restriction extend to reproduction in all media.

© 2005, Embedded Div. AIJI System

All rights reserved.

Revision History

Rev.	Date	Remark
1.0	1. Jul. 05	First Edition.
1.3	17. Oct. 05	6.4" LCD upgrade. Firmware, u-boot, Linux WinCE upgrade
2.0	21. Feb. 06	3.5" landscape LCD upgrade, Firmware, Linux WinCE upgrade

Contents

1	Introduction	1
1.1	System Overview	2
1.2	MBA2440 Specification	4
1.3	MBA2440 H/W configuration.....	8
1.3.1	System Memory Configuration	8
1.3.2	Boot flash Configuration	8
1.3.3	NAND controller Configuration	14
2	MBA2440 Quick Start	16
2.1	Environment Setup	16
2.1.1	Connect Cables	17
2.1.2	DNW Setup	18
2.1.3	Hyper-terminal Setup	18
2.1.4	Set up TFTP Server Program for Windows	20
2.2	MBA2440 Test Program	22
2.3	Bootting WinCE.....	24
2.3.1	Bootting WinCE with U-boot	24
2.3.2	Bootting WinCE with TFTP	27
2.3.3	Bootting WinCE with USB.....	28
2.4	Bootting Linux	30
2.4.1	Bootting Linux with TFTP.....	30
2.4.2	Bootting Linux in NAND flash	31
2.4.3	Bootting Linux with USB	36
3	Set up Development Environment.....	38
3.1	Examples of establishing development environment.....	39
3.2	Firmware Development Environment.....	42
3.3	Embedded Linux Development Environment.....	43
3.3.1	Install Cross Compiler.....	43
3.3.2	Install TFTP Server on Linux	47
3.3.3	Set up NFS	49
3.3.4	Minicom Setup	53
3.4	WinCE Development Environment	58
3.5	OPENice Debugger Equipments.....	59
3.5.1	OPENice-A900, OPENice-A1000.....	59
3.5.2	Write image on Boot flash.....	61
3.5.3	Load image in Memory	65
3.5.4	Read Data in Memory.....	67

3.5.5	Matters to be Attended when using OPENice	68
4	MBA2440 Test Program	69
4.1	Compile Programs	69
4.1.1	Set up Test Program	70
4.1.2	Compiling	72
4.2	Matters to be Attended for MBA2440 Test Program	74
4.2.1	In case of setting up 0x0 address as RO_BASE	74
4.2.2	In case of setting up 0x30000000 address as RO_BASE	75
4.3	Test Command	76
4.3.1	UART	76
4.3.2	Power/Clock	76
4.3.3	Timer	77
4.3.4	RTC	77
4.3.5	ADC test	78
4.3.6	Touch Screen test	78
4.3.7	NOR flash test	79
4.3.8	NAND flash test	79
4.3.9	DMA	79
4.3.10	Interrupt	80
4.3.11	SPI	80
4.3.12	IIC	81
4.3.13	SDIO	81
4.3.14	IIS	82
4.3.15	IrDA test	82
4.3.16	TFT-LCD	83
4.3.17	CS8900_Ethernet	83
4.3.18	USB download	84
5	U-Boot	90
5.1	What is Bootloader?	91
5.2	U-boot for MBA2440	92
5.2.1	U-Boot memory map	92
5.2.2	U-Boot Commands	93
5.3	Bootloader Compile	107
5.3.1	Extract Source	107
5.3.2	U-boot configuration	107
5.3.3	Compile U-boot	109
6	Linux Kernel	111
6.1	Kernel compile	112
6.1.1	Extract Linux Kernel	112

6.1.2	Set up Linux Kernel.....	112
6.1.3	Perform Kernel Compile Command.....	118
6.2	File System.....	119
6.2.1	Root File System.....	119
6.2.2	RAMDISK File System.....	120
6.2.3	CRAMFS.....	124
6.2.4	JFFS2 File System.....	130
6.3	Device Driver.....	134
6.4	Application.....	136
6.5	MBA2440 Linux Tips.....	138
6.5.1	Mount USB memory.....	138
7	WinCE.....	141
7.1	WinCE 5.0 BSP for MBA2440.....	141
7.1.1	BSP Build.....	141
7.1.2	Support 6.4" TFT-LCD.....	152
7.1.3	Set up SDRAM Memory.....	153
7.1.4	Support Korean Language.....	153
7.2	Eboot boot loader.....	156
7.3	Nboot boot loader.....	159
7.3.1	Booting WinCE with Nboot.....	160
8	UbiFOS.....	163
8.1	What is UbiFOS.....	163
8.1.1	Kernel.....	164
8.2	Booting UbiFOS.....	169
9	Appendix. Errata.....	오류! 책갈피가 정의되어 있지 않습니다.

1 Introduction

This document is a manual for MBA2440 Board of AIJI System Co. Ltd.

MBA2440 is focused on providing reference to develop mobile system and other embedded systems controlling various types hardware devices based on S3C2440 MCU of Samsung.

As reference board, MBA2440 has been embodied to achieve the only objective checking if MBA2440 hardware works properly and OS widely used such as Linux, WinCE and UbiFOS which is domestic RTOS basically work fine.

Please be aware that developers should test parts which are not explained in this document themselves, because those parts are matters which haven't been tested yet.

Besides, the realization of various applications and the establishment and modification of hardware for them may be possible according to the capability of developers, and please be fully aware that developers are also responsible for developing process for that.

This document has been created for the following objectives targeting beginners.

- ☞ Explaining basically how to activate the board easily when users access the board for the first time
- ☞ Explaining how to establish development environments to build up firmware and OS for the board
- ☞ Explaining how to compile programs using development environments

It is strongly recommended to read documents provided by CD and the board manual to solve problems or satisfy curiosity.

We will continuously update and provide source or information related to MBA2440 through the website, www.mculand.com or www.aijisystem.com. You can use it anytime.

Other inquiries and technical support for the embodiment of applications and the setup and modification of them we don't provide are matters of solution development depending on situations.

1.1 System Overview

The heart of MBA2440 is MCU, S3C2440. Please refer to S3C2440 datasheet for detailed explanation about MCU.

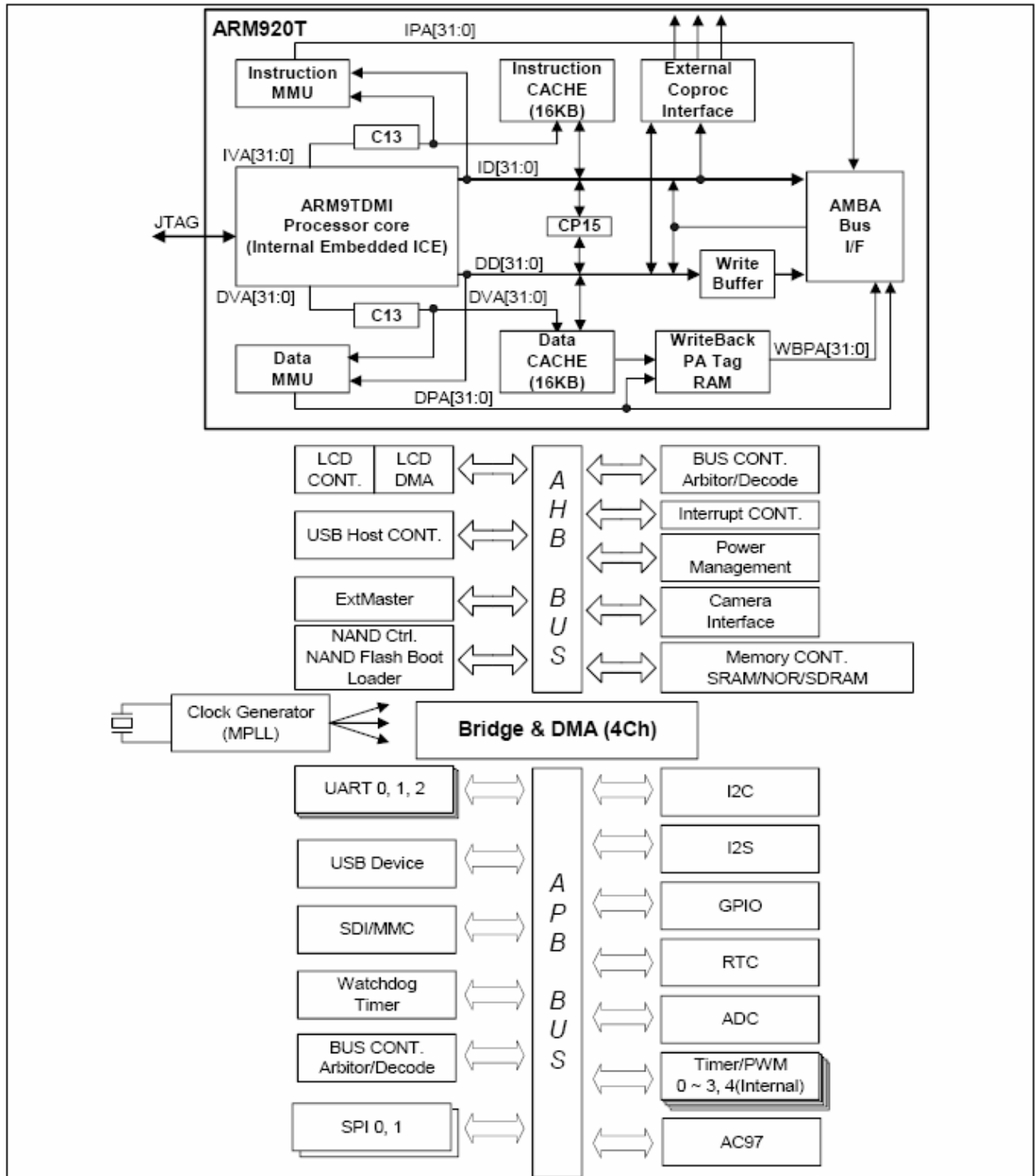
In this chapter, we would like to mention about the structure and characteristics of architecture including basic explanation about S3C2440.

S3C2440 suits applications and hand-held devices based on ARM920T core requiring high performance and consumption of low power as 16/32bit RISC micro-controller developed by Samsung. Also, many kinds of various IP are included in S3C2440 chip to reduce costs for developing the system, and AMBA(Advanced Micro controller Bus Architecture) bus is embodied for the connection between those IP.

The summary of characteristics of S3C2440 is as the following.

- CPU clock 300MHz(1.20V for core), 400MHz(1.30V for core)
- Memory power 1.8V/2.5V/3.3V, external I/O of 3.3V
- 16KB² I/D cache and MMU
- External memory controller (SDRAM control and chip select logic)
- controller having the exclusive DMA channel for one LCD (4K color STN, 256K color TFT)
- Four external DMA channels
- Three UART channels and two SPI channels
- IIC/IIS(AC97 I/F) support
- SD host version 1.0/MMC protocol version 2.11
- Two USB host channels / one USB device channel (version 1.1) (it doesn't support USB isochronous)
- Four PWM timer channels / one internal timer
- WDT(Watch Dog Timer)
- 130 of multi-functional GPIO/ 24 of external interrupt channels
- 8 of 10bit ADC/ Touch screen I/F
- RTC(Real Time Clock)

[Picture 1.1] is a block diagram of S3C2440.

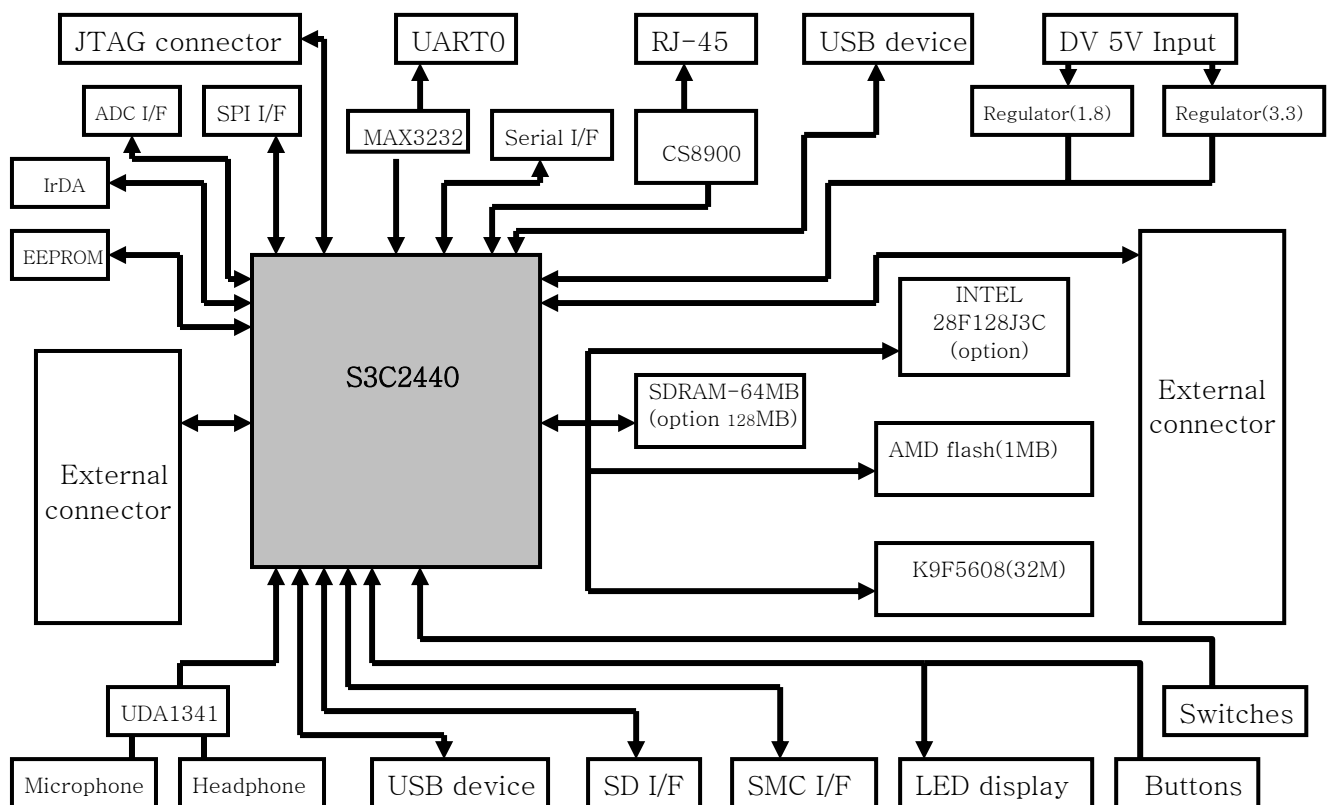


[Picture 1.1] S3C2440 Block Diagram

1.2 MBA2440 Specification

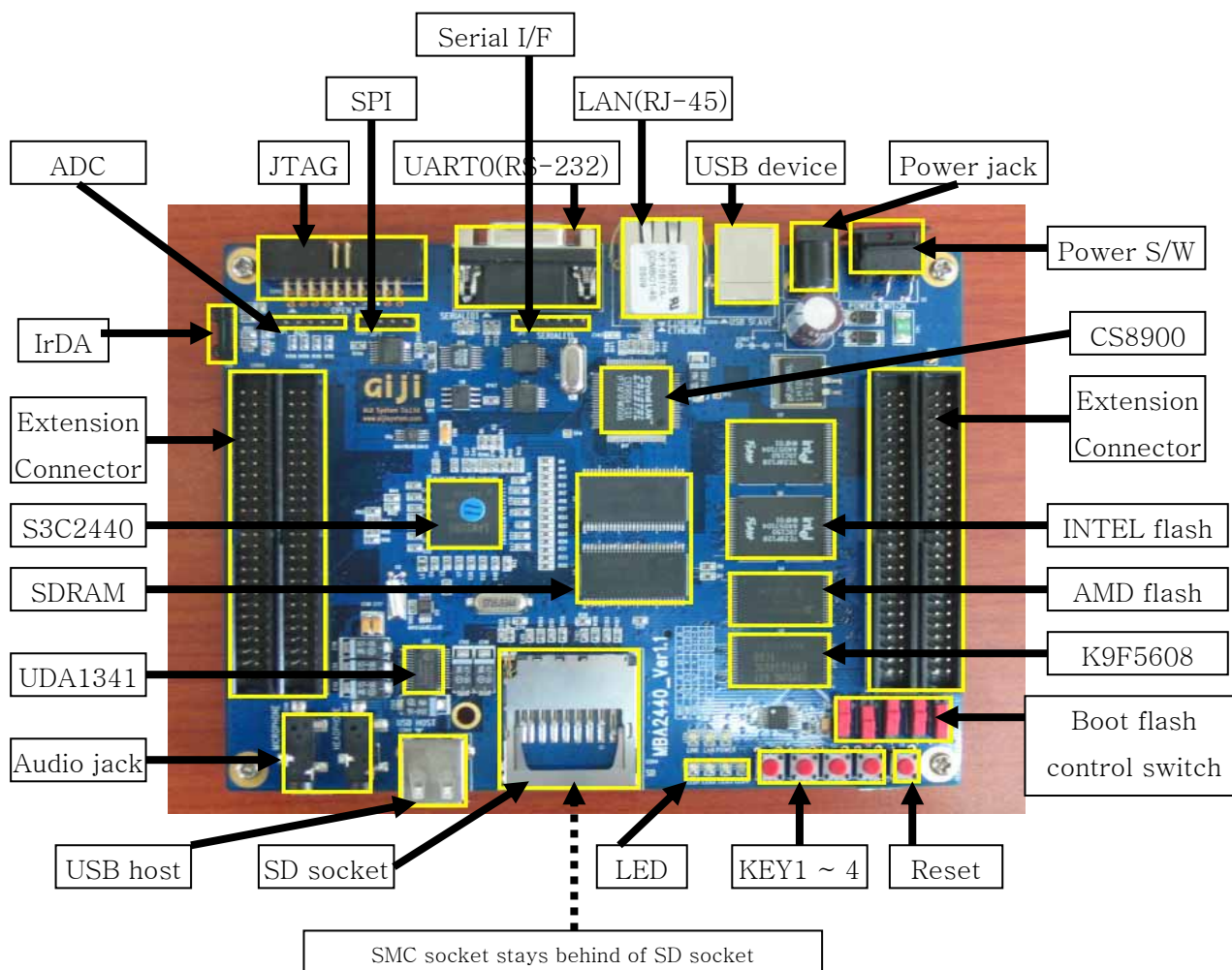
MBA2440 board based on S3C2440 connects various kinds of devices with hardware as a reference board to show many different solutions can be developed by it.

[Picture 1.2] shows a block diagram of MBA2440.



[Picture 1.2] MBA2440 Block Diagram

[Picture 1.3] shows what MBA2440 looks like and names of each part.



[Picture 1.3] What MBA2440 looks like

Let's simply learn about each device.

☞ SDRAM

- used as system memory, basically supporting 64MB, possible to extend to 128MB
- Connecting to nGCS6(0x30000000)

☞ AMD flash(AM29LV800BB)

- NOR Flash memory to be used as boot flash, 1MB size
- Connecting to nGCS0(0x0) or nGCS4(0x20000000) according to jumper setting

☞ INTEL flash(28F128J3C)

- NOR flash memory used as boot flash or nonvolatile save device
- Optional when purchase, 32MB size
- Connecting to nGCS0(0x0) or nGCS1(0x08000000) according to jumper setting

☞ NAND flash(K9F5608)

- Used as boot flash or nonvolatile save device, 32MB size, connecting to S3C2440 NAND controller

☞ UART0

- Used as serial console through MAX3232, RS-232 connector

☞ Serial I/F

- 6 pin connector directly connected to S3C2440 UART1,
not connecting to Serial chip(MAX3232)

☞ EEPROM

- KS24C080 serial EEPROM of IIC interface. 1KB size

☞ IrDA

- Infrared rays communication port, connecting to S3C2440 UART2

☞ ADC I/F

- 6 pin connector directly connected to S3C2440 ADC controller

☞ SPI I/F

- 5 pin connector directly connected to S3C2440 SPI controller

☞ JTAG

- connector for 20 pins JTAG. Used for linking various emulator equipments

☞ CS8900

- 10BASE-T Ethernet controller. Linking to RJ-45 connector. Linking with 20MHz clock source

☞ USB host , device

- S3C2440 USB host, linking to device controller

☞ Power

- Using 5V adaptor over 2A. 1.3V(core power) through regulator, providing 3.3v(VDD, device power)

☞ UDA1341

- Connecting to audio codec chip. S3C2440 IIS

☞ SD I/F

- Connecting to S3C2440 SDIO controller. SD socket.

☞ SMC I/F

- Connecting to S3C2440 NAND controller like K9F5608. chosen by chip select jumper (J3)
SMC socket on the lower part of SD socket

☞ Extension Connector

- Total 200 pins of connector. Connecting address, data bus and most of control signals of S3C2440

☞ LED display

- LED displaying the status of power, Ethernet and 4 of LED which can control by themselves

☞ Buttons

- Reset button and 4 buttons for external signal input (KEY1 ~ 4)

☞ Boot flash control switch

- 5 switches for setting up NAND flash and MBA2440 Boot Rom

☛ S3C2440

- MCU which is the heart of MBA2440. 400MHz clock (1.3V core power)
- Using 16.9344MHz System clock
- Using 32.768KHz RTC clock

1.3 MBA2440 H/W configuration

Let's learn what should be set up to use MBA2440 efficiently in hardware ways.

Setting up hardware is divided into the following 3 matters.

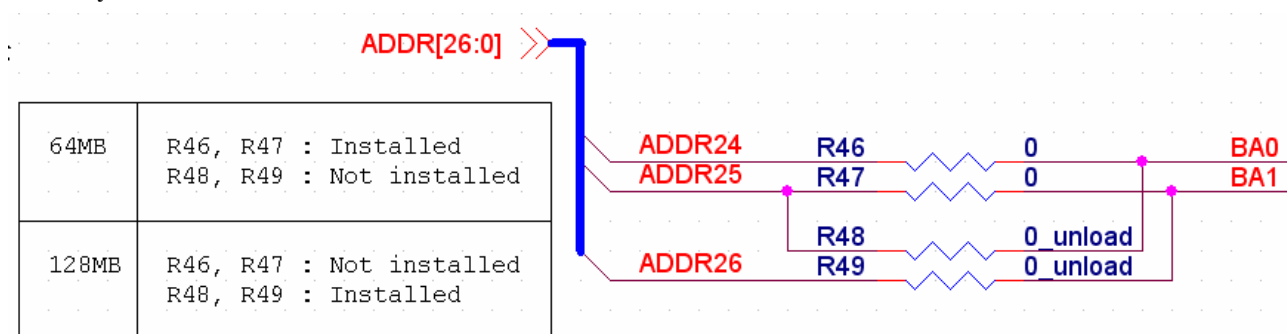
- System Memory Configuration
- Boot flash Configuration
- NAND controller Configuration

1.3.1 System Memory Configuration

Basically, MBA2440 is released attaching two of 32MB Samsung memory chip (K4S561632E-TC75) on the board to use 64MB system memory at the time releasing products, however, sometimes the board is released being upgraded to 128MB according to developer's request.

It can be upgraded to 128MB if develops can get a chip themselves. In this case, 2 of 64MB Samsung chip (K4S511632D-TC75 : 8M x 6bits x 4 banks) are used.

If it is upgraded from 64MB to 128MB, there should be changes on address bus, because it can't approach the whole 128MB memory area with address bus which can approach 64MB memory.



[Picture 1.4]

As shown on [Picture 1.4], there is 0Ω resistance for R46 and R47 (resistance) to short on the back of board in case of 64MB. When upgrading to 128MB, open R46, R47 and short R49.

If you put together with two SDRAM of 64MB after changing the status of resistance as above, it is upgraded to 128MB.

1.3.2 Boot flash Configuration

MBA2440 has many flash memories.

What is flash memory?

Flash memory is a nonvolatile memory consisting of one chip unlike a system memory, SDRAM. It is the most-widely used as data save media in the embedded system, because it has both

merits of ROM and RAM: Rom still saves data even while the power of board is off, and Ram freely inputs and outputs data.

The flash memory is divided into two types, NOR type and NAND type according to the internal process.

NOR flash has cells connected parallel. It is the memory mostly used for cell phones, because it has fast reading capability due to short data access time compared to NAND. But the manufacture unit cost is higher compared to NAND.

NAND flash has cells connected serially. It performs far better functions as high capacity memory compared to NOR because it is easy to embody high capacity accumulation, and it is usually used for voice or moving picture multi media because reading speed is faster than NOR.

The following can be used as NOR flash memory in MBA2440,

- **AM29LV800BB** (AMD flash) : 1MB [default]
- **INTEL 28F128J3C** (INTEL flash) : 32MB(16MB x 2) [optional]

The following can be used as NAND flash memory in MBA2440,

- **K9F5608** (SOP) : 32MB [default]
- **K9S1208** (SMC) : 64MB [optional]

MBA2440 can use one of these 4 flash memories as boot flash memory.

Boot flash is recognized as 0x0 in S3C2440 when the power is on, saves the code which is supposed to perform first, and helps the code perform at the same time as power on.

It can be set up to determine what flash memory will be used as boot flash through 5 of boot flash control switch on the right lower part of the board.

How to set up regarding this is simply remarked on the board with silk.

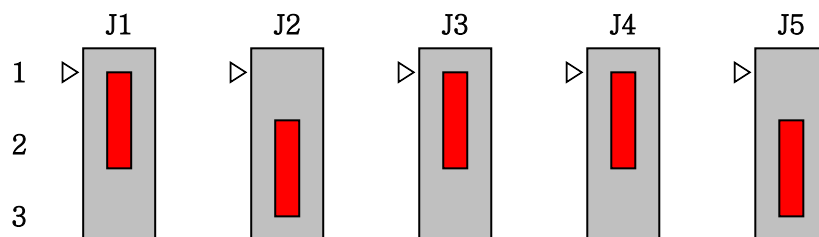
If you look through boot flash set up table and boot flash control switch, they are as the following. On the table, "x" means "don't care".

[Table 1.1] boot flash set up table

BOOT	J1	J2	J3	J4	J5
AMD	1-2	2-3	X	1-2	2-3
INTEL	2-3	1-2	X	2-3	1-2
SMC	x	x	2-3	2-3	2-3
K9F5608	x	x	1-2	2-3	2-3

Boot flash control switch is placed on the edge of right lower part of board.

It is basically set up like [Picture 1.5] when the board is released.



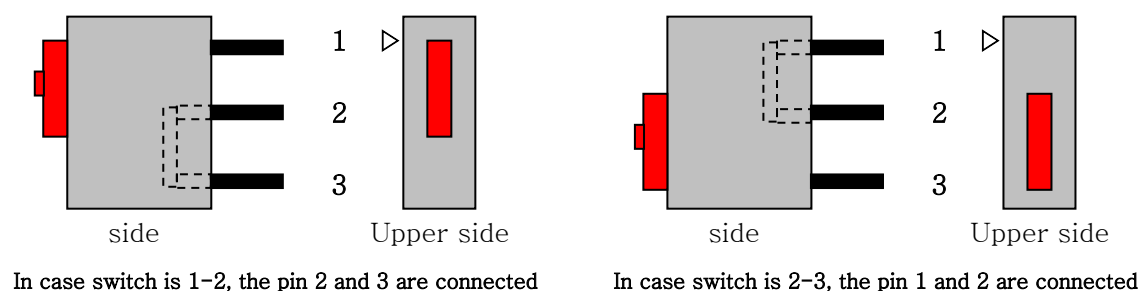
[Picture 1.5]

The set up of [Picture 1.5] fixes AM29LV800 NOR flash as boot flash, and NAND flash which can be controlled is SOP (K9F5608) NAND flash.

Here's a suggestion.

There are 3 pins on a boot flash control switch connected to the board.

If the switch is pulled up as shown on [Picture 1.6], it means that pin 2 and pin 3 are actually connected to each other, and if the switch is pulled down, it means that pin 1 and pin 2 are actually connected to each other.



[Picture 1.6]

On this document, the explanation is not based on the status of pin's connection, but based on the status of switch.

Let's learn about each switch. (please refer to the circuit diagram for this part)

The switch needed to learn about first is J4 and J5.

J4 and J5 control OM0, OM1 signals of S3C2440. OM0, OM1 signals are for setting up data bus of boot flash which will be connected to nGCS0. Please refer to the chapter 5 memory controller of S3C2440 manual for details.

OM0 signal controls through J4, and OM1 signal does through J5.

If both OM0 and OM1 signals are connected to the ground (both J4 and J5 are set up as 2-3), it comes to fix NAND flash as boot flash.

If OM0 is connected to VDD (J4 is set up as 1-2) and OM1 is connected to the ground (J5 is set up as 2-3), data bus is fixed as 16bit. This comes under a case using AMD flash as boot flash. You

can see data bus of AMD flash connected as 16bit through the circuit diagram.

If OM0 is connected to the ground (J4 is set up as 2-3) and OM1 is connected to VDD (J5 is set up as 1-2), data bus is fixed as 32 bit. This comes under a case using INTEL flash as boot flash. You can see data bus of INTEL flash connected as 32 bit through the circuit diagram.

Let's learn about J1 and J2.

J1 and J2 determines which to use as boot flash among two NOR flash memories.

If J1 is connected to 1-2 and J2 is connected to 2-3, nCE16 signal is connected to nGCS0 of S3C2440. nCE16 is the signal connected to nCE of AM29LV800BB flash memory. **That's why AM29LV800BB flash is set up as boot flash.**

If J1 is connected to 2-3 and J2 is connected to 1-2, nCE32 signal is connected to nGCS0 of S3C2440 on the circuit diagram. nCE32 is the signal connected to nCE0 of INTEL 28F128J3C flash memory. **That's why INTEL 28F128J3C flash is set up as boot flash.**

If both J1 and J2 are set up 1-2, it is natural it doesn't work because S3C2440 gets confused to select boot flash among NOR flash. And if both are set up 2-3, it becomes same as the board without boot flash because there is no flash memory connected to nGCS0. Be aware of this point!!!

In case NAND flash is set up as boot flash through J4 and J5, J3 determines which NAND flash memory among two shall be used as boot flash.

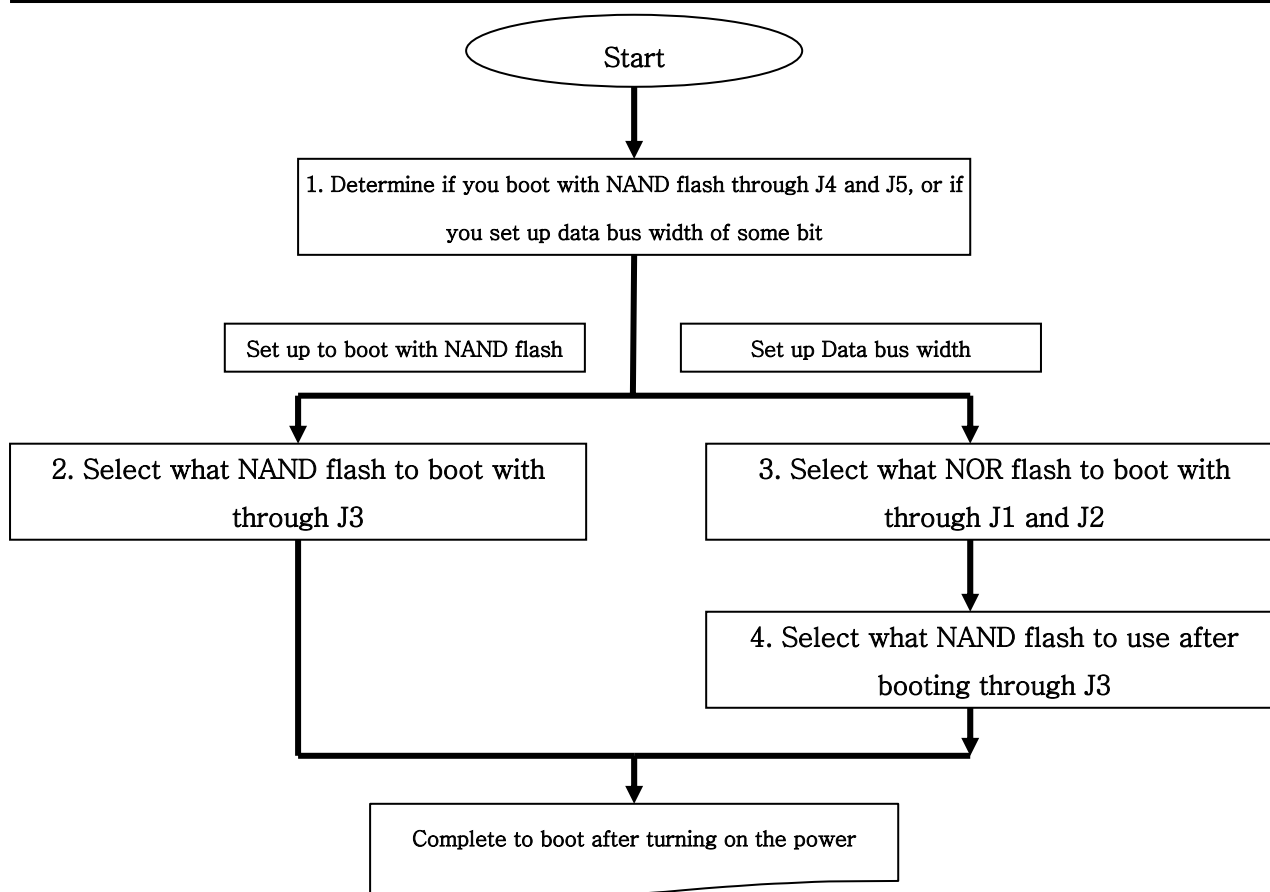
In case NOR flash is set up as boot flash through J4 and J5, J3 determines what NAND flash shall be controlled.

When NAND flash is set up as boot flash through J4 and J5, and J3 is connected to 1-2, it can control K9F5608 NAND flash through S3C2440 NAND controller.

When NAND flash is set up as boot flash through J4 and J5 and J3 is connected to 2-3, it can control K9S1208(SMC) NAND flash through S3C2440 NAND controller.

NAND flash memory can't control both NAND flash at the same time, because it only controls through NAND controller of S3C2440.

Please set up boot flash control switch as the following orders.



[Picture 1.7]

We are providing a simple order chart as [Picture 1.7] for your understanding about setting up jumper.

1.3.2.1 Using NOR flash as Boot Flash

Based on the above order chart, we would like to explain how to set up NOR flash as boot flash.

In case of using AM29LV800BB as boot flash, [Picture 1.8]

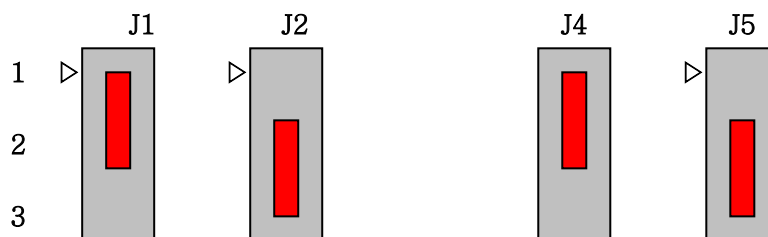
1. If you want to set up AM29LV800BB, adjust J2 to 1-2, J5 to 2-3 and data bus to 16bit.

2. Adjust J1 to 1-2, J2 to 2-3 and connect nGCS0 to nCE of AM29LV800BB

3. After booting, set up J3 and select NAND flash you are willing to control in test program, Linux and WinCE. J3 doesn't affect NOR flash to be set up as boot flash.

In case J3 is 1-2, it is possible to control K9F5608 NAND flash

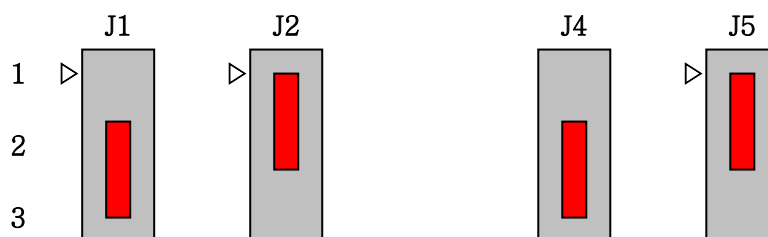
In case J3 is 2-3, it is possible to control K9S1208 NAND flash



[Picture 1.8] Set up AMD flash as boot flash

In case of using INTEL 28F128J3C as boot flash [Picture 1.9]

1. If you want to set up E28F128J3C, adjust J4 to 2-3, J5 to 1-2 and data bus to 32bit
2. Adjust J1 to 2-3, J2 to 1-2 and connect nGCS0 to nCE0 of E28F128J3C
3. After booting, set up J3 and select NAND flash you are willing to control in test program, Linux and WinCE. J3 doesn't affect NOR flash to be set up as boot flash.
In case J3 is 1-2, it is possible to control K9F5608 NAND flash
In case J3 is 2-3, it is possible to control K9S1208 NAND flash



[Picture 1.9] Set up INTEL flash as boot flash

1.3.2.2 Using NAND flash as Boot Flash

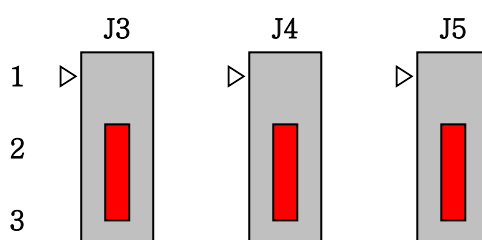
Based on the above order chart, we would like to explain how to set up NAND flash as boot flash.(Picture 1.10). [Picture 1.11]

1. Adjust J4 and J5 to 2-3
2. Set up K9F5608 or K9S1208(SMC) as boot flash through J3
In case J3 is 1-2, K9F5608 NAND flash is set up as boot flash
In case J3 is 2-3, K9S1208 NAND flash is set up as boot flash

In case of booting by NAND flash, it is impossible to approach NOR flash connected with nGCS0 of S3C2440, however, it is possible to control NOR flash connected with nGCS1 or nGCS4. Set up J1 and J2 not to connect NOR flash with nGCS0.

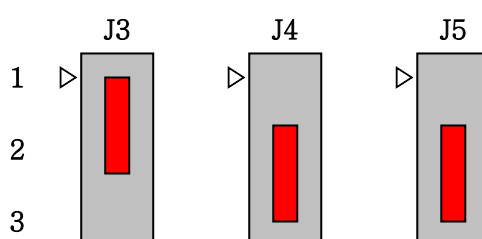
J1 and J2 doesn't affect NAND flash to be set up as boot flash.

In case of using SMC(K9S1208) as boot flash,



[Picture 1.10] Set up SMC(K9S1208) as boot flash

In case of using SOP(K9F5608) as boot flash



[Picture 1.11] Set up SOP(K9F5608) as boot flash

1.3.3 NAND controller Configuration

S3C2440 has NAND controller. NAND controller is useful to control NAND flash memory easily through NAND flash control signal.

There are various types of NAND flash memory which can be connected with S3C2440. But S3C2440 NAND controller is in the most suitable status for favorable gearing with NAND flash memory provided by Samsung.

NAND controller set up **normal mode** and **advanced mode** according to the type of NAND flash memory connected, and setting up memory page size, memory address cycle, I/O bus width is subject to change by it. **It is supposed to be set up in hardware, not in software.**

Signals to set it up is NCON, GPG13(EINT21), GPG14(EINT22), GPG15(EINT23) signals in S3C2440.

NAND FLASH MEMORY CONFIGURATION TABLE

NCON0	GPG13	GPG14	GPG15
0: Normal NAND	0: 256Words	0: 3-Addr	0: 8-bit bus width
	1: 512Bytes	1: 4-Addr	
1: Advance NAND	0: 1Kwords	0: 4-Addr	1: 16-bit bus width
	1: 2Kbytes	1: 5-Addr	

Note

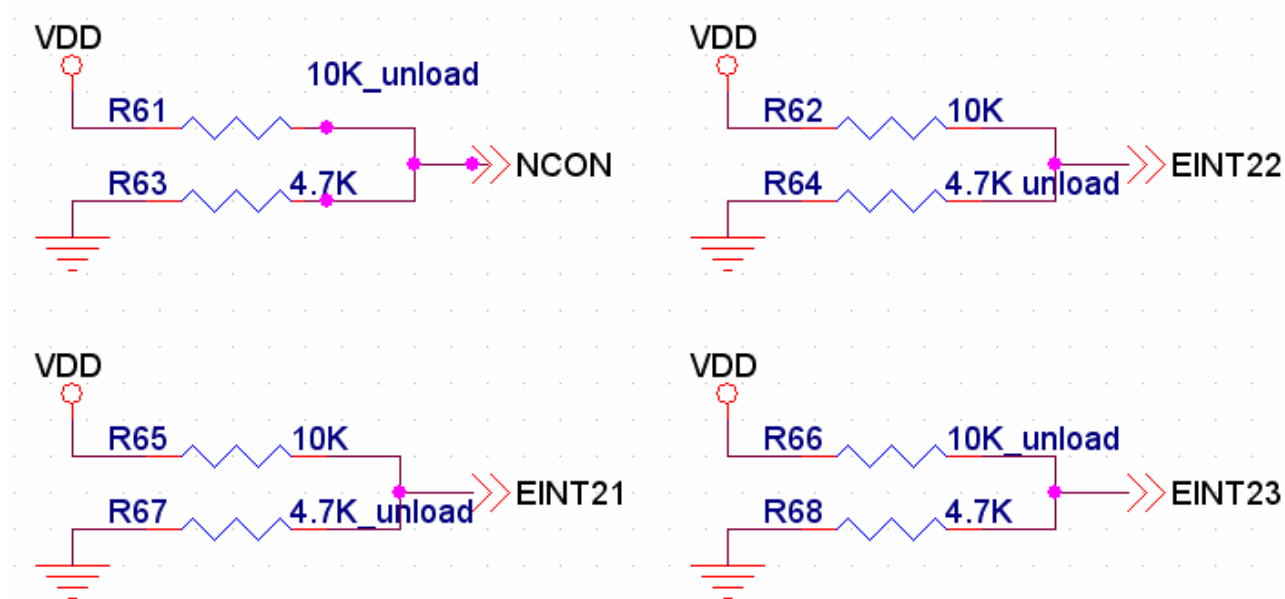
With above 4-bit, Possible total combinations are 16, but not all the value can be used.

Example) Nand flash configuration setting.

Parts	Page size/Total size	NCON0	GPG13	GPG14]	GPG15
K9S1208V0M-xxxx	512Byte / 512Mbit	0	1	1	0
K9K2G16U0M-xxxx	1KW / 2Gbit	1	0	1	1

Setting up hardware to activate normal mode or advanced mode is as above.

MBA2440 composes the circuit diagram as [Picture 1.12] to set up hardware of NAND controller.



[Picture 1.12]

MBA2440 is set up to activate normal mode.

To connect and use NAND flash memory like K9K2G08U0M, you should set up advanced mode. You can connect signals as NCON = 1, GPG13 = 1, GPG14 = 1, GPG15 = 0 for advanced mode, upload R63 in the circuit diagram, and connect the resistance with R61.

Please refer to datasheet of S3C2440 for details.

2 MBA2440 Quick Start

In this chapter, we would like to introduce how to activate firmware or OS already set up when purchased in proper procedures and how to connect cables to MBA2440 and PC for developers who are beginners for MBA2440.

Let's learn how to drive firmware for the first step and how to drive Linux and WinCE for the next step.

2.1 Environment Setup

You need to connect with desktop PC to drive MBA2440 first, and the first thing you need to do is to install and set up programs which will be performed in PC.

You need to do basic environment set up which will be explained on this chapter in Windows XP, and the following applications shall be installed.

◆ **DNW, Hyper-terminal serial emulator**

: Set up programs for serial console such as minicom in Linux

◆ **TFTP programs for Windows**

: TFTP server program for windows performing a role like TFTP program in Linux

We would like to let you know in advance that there might be some problem for environment set up if you try in Windows 98, however, no problem in Windows 2000 and higher versions.

Firstly, let's learn how to set up Hyper-terminal and DNW, serial communication emulator in Windows OS.

Please refer to [3.3.4 Minicom setup](#) in this document for how to set up serial communication emulator in Linux OS.

Next, you will learn how to set up TFTP server program to transmit images to the board through LAN. Please refer to [3.3.2 Install TFTP server on Linux](#) on this document for how to set up TFTP server program in Linux OS.

Setting up IP address of Windows PC, Linux PC, MBA2440 is also needed for using TFTP.

Construct and use network privately established for setting up address.

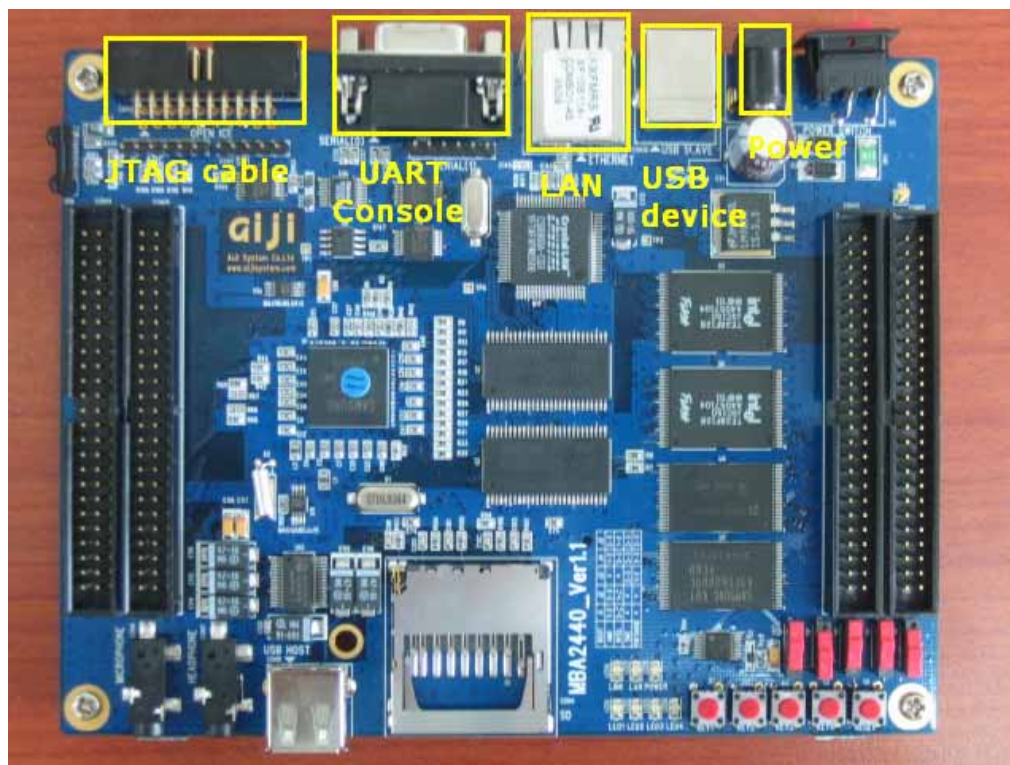
We also recommend you to construct environments referring to [3.1 Examples of establishing development environment](#) for this matter.

Let's learn how to connect cables first.

2.1.1 Connect Cables

There are many cables connected to PC and MBA2440. Let's learn about cables which are supposed to connect MBA2440 with PC.

Ports which should be connected are shown on [Picture 2.1]



[Picture 2.1] Ports which should be connected

2.1.1.1 Power Cable

The first cable to be connected is power cable, because it provides electricity for MBA2440. Connect 5V/2A adaptor which should be connected basically.

2.1.1.2 UART Console Cable

Connect UART console cable to UART0 port.

You can use serial cable provided when purchasing the board. This cable is a direct cable.

The one side of serial cable shall be connected to the port for UART0 consol of MBA2440 and the other side shall be connected to COM1 or COM2 of PC. Mostly, it's connected to COM1.

you can monitor MBA2440 with DNW, Hyper-terminal and minicom only when this cable connects MBA2440 with PC. Please be aware this cable is basically connected.

2.1.1.3 Connect LAN Cable

There are two types of LAN cables.

Cross cables is used for direct connection of MBA2440 with PC. You can use direct cable to connect MBA2440 with PC by hub. You can also use NFS file system after booting Linux and "tftp" command of u-boot only when LAN cable is connected.

2.1.1.4 Connect JTAG Cable

There is a JTAG port to connect debugger equipments such as OpenICE-A900, OpenICE-A1000 and etc. in MBA2440. it is possible to connect with debugger equipments by the port. Connect it when you use debugger equipments.

2.1.1.5 USB device

Connect USB device cable provided when purchasing the board when you perform “USB download”, the last menu of MBA2440 test program and “usbdown” command in u-boot.

Connect one side of cable to USB port on PC and the other side of cable to MBA2440 USB device port.

When you download using USB cable, you should download through DNW program in Windows OS PC.

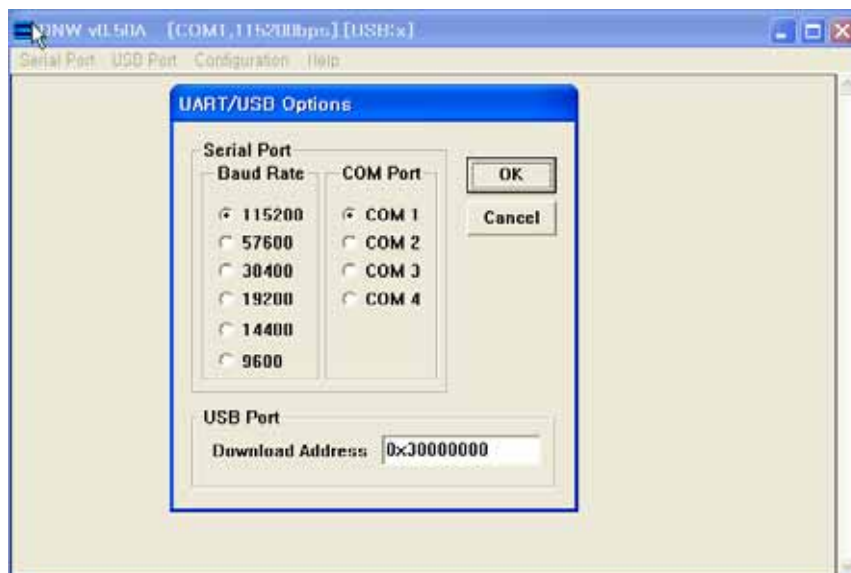
USB cable also can be used as the power cable of board.

Please refer to [4.3.18 USB download](#) for details.

2.1.2 DNW Setup

DNW program is **dnw.exe** file in the utility/DNW directory in CD provided. You can activate this file.

Set up as [Picture 2.2]. Do not forget to set up the correct COM port connected with serial cable. In this document, connect the cable to COM1 port on PC.



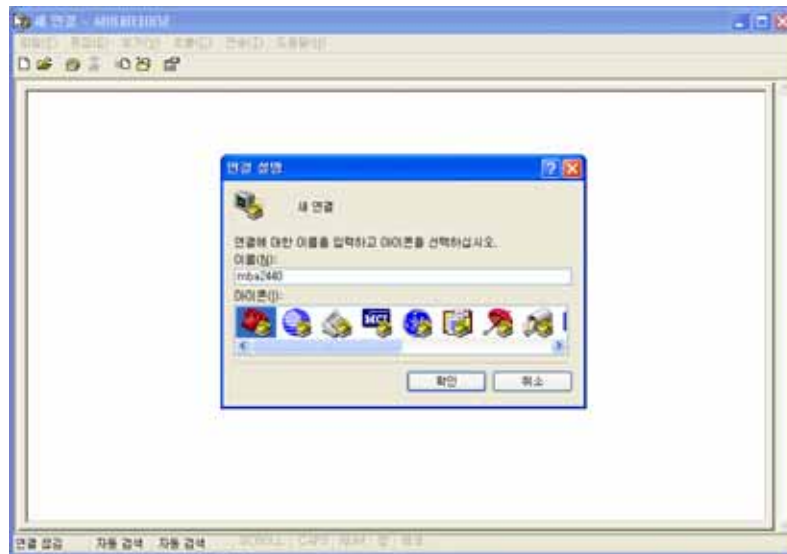
[Picture 2.2]

After completing setup, push OK button. If you push Serial Port->Connect next, it will be connected.

2.1.3 Hyper-terminal Setup

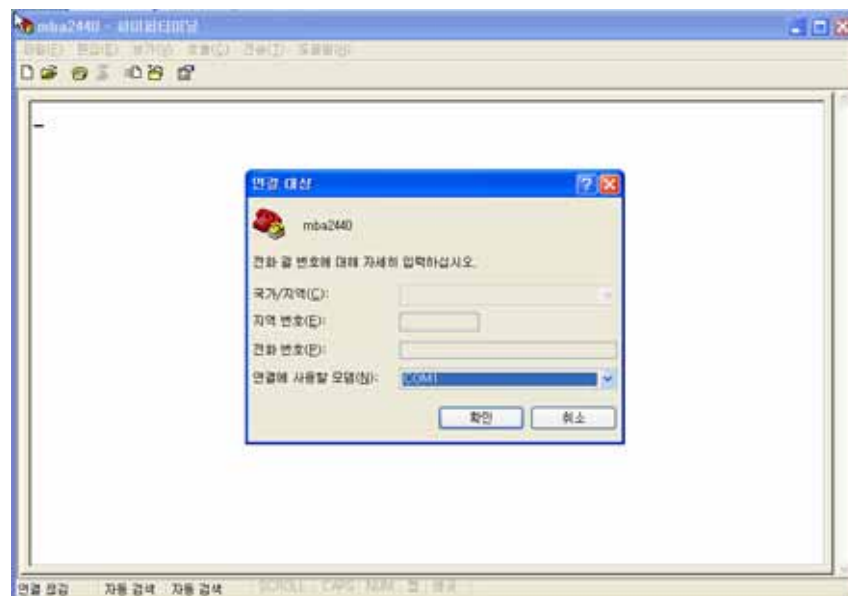
1. Start -> Program -> Support Program -> Communication -> Hyper-terminal

2. Push cancel button when a window setting up location information is shown
3. Write mba2440 when a window explaining about the connection after hyper-terminal is shown.
You can write down a free sentence. [Picture 2.3]



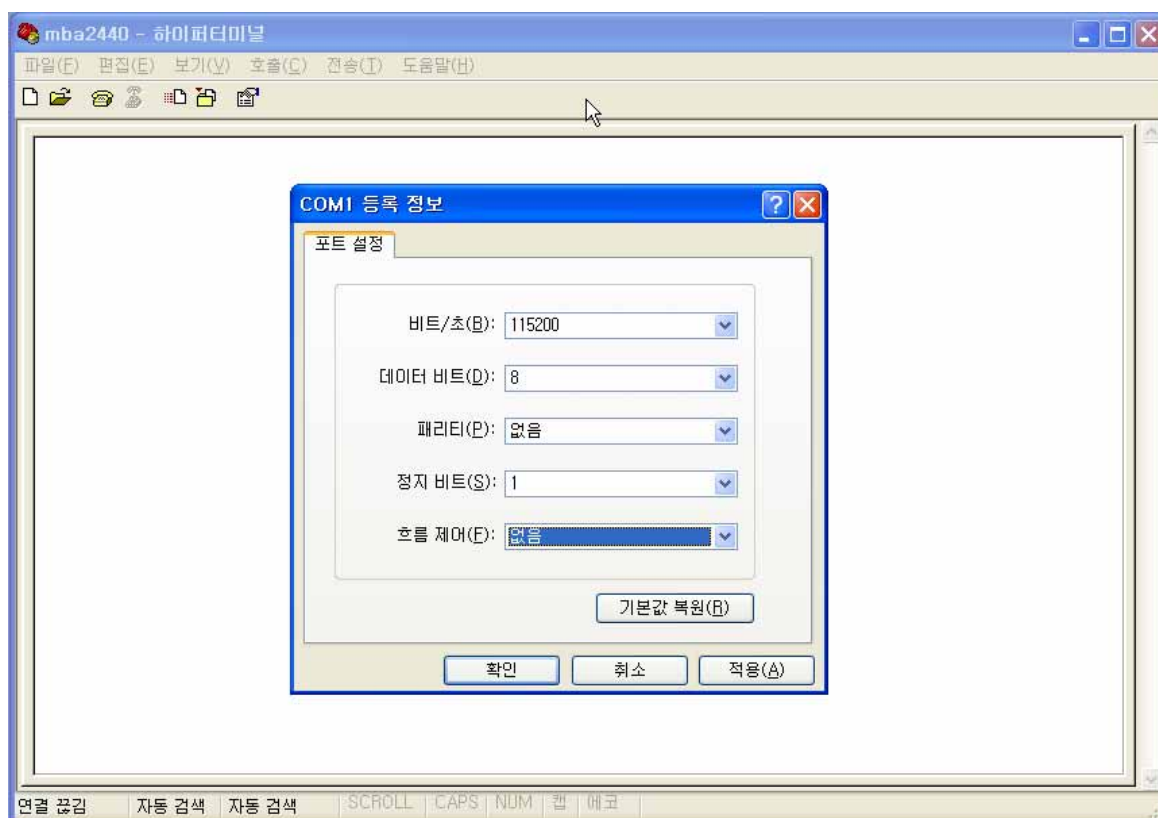
[Picture 2.3]

4. If you push confirm button, you will see a window setting up the same location information as before. Push cancel button.
5. Set up the modem which will be used as COM port when you see a window to select what to connect. Select COM1 and push confirm button. [Picture 2.4]



[Picture 2.4]

6. When you see a window setting up registration information for COM1, set up information such as baudrate, data bit and etc. Set up like [Picture 2.5].
when the setup is completed, push confirm button.



[Picture 2.5]

2.1.4 Set up TFTP Server Program for Windows

U-boot, a firmware of MBA2440 uses LAN to receive big-sized image files from outside. TFTP server and client program are used for this.

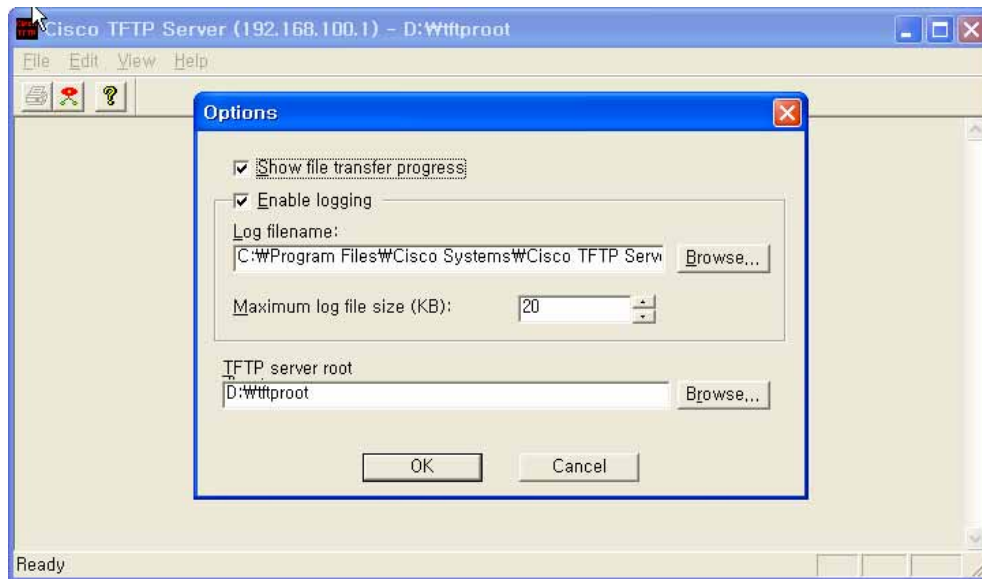
OS images performed in MBA2440 are usually big sized, and they exist in desktop PC. TFTP server program should exist in desktop PC to bring images from desktop PC to MBA2440. There is TFTP server program for Windows in CD provided. In this chapter, you will learn how to set it up.

There is **TFTPServer.exe** file in utility/TFTP4win directory in CD. Drive the file and install TFTP server program.

After completing to install it, **set up root directory of TFTP server program. Be aware that you should put files or images you want to send in root directory for sending files to TFTP.**

In this document, install D:/tftproot as root directory.

Create tftproot directory under D:, click View -> Option and set up root.



[Picture 2.6]

2.2 MBA2440 Test Program

There are 4 of firmware provided by MBA2440.

- **MBA2440 Test Program**
: you can learn how to control by testing each device in the board by this program
- **U-boot Boot Loader Program**
: Boot Loader needed for driving WinCE or Linux
- **nboot Boot Loader Program**
: Boot loader needed for the case of setting up NAND flash as boot flash and driving WinCE
- **eboot Boot Loader Program**
: Boot loader used for developing WinCE.

In this chapter, you will only learn how to drive MBA2440 test program for the first time and detailed matters regarding this in the chapter [4. MBA2440 Test Program](#).

You will learn the detailed matters regarding U-boot boot loader program which is an important boot loader for MBA2440. (refer to [5. U-Boot](#))

You will learn Eboot boot loader program and Nboot boot loader which are boot loaders for WinCE. (refer to [7.WinCE](#))

There is a compressed file, **MBA2440_test_v2.0.zip** in **firmware/test_program** folder in CD

As explained earlier, this program can test many devices in MBA2440, and can be useful for developers to develop programs using this program source as an example program.

MBA2440 is released loading MBA2440 test program in AMD flash, therefore, you can confirm menus of MBA2440 test program through consoles such as minicom in Linux, DNW or Hyper-terminal if you turn on the power after setting up AMD flash as boot flash.

MBA2440 test program is loaded in the address 0x0 of AMD flash when releasing the board.

MBA2440 test program is programmed to work properly in case AMD flash is set up as boot flash.

Please be aware that specific menus don't work properly or don't work at all in case of setting up INTEL flash as boot flash and booting after loading test program in INTEL flash, or in case of setting up NAND flash as boot flash and booting after loading test program in NAND flash.

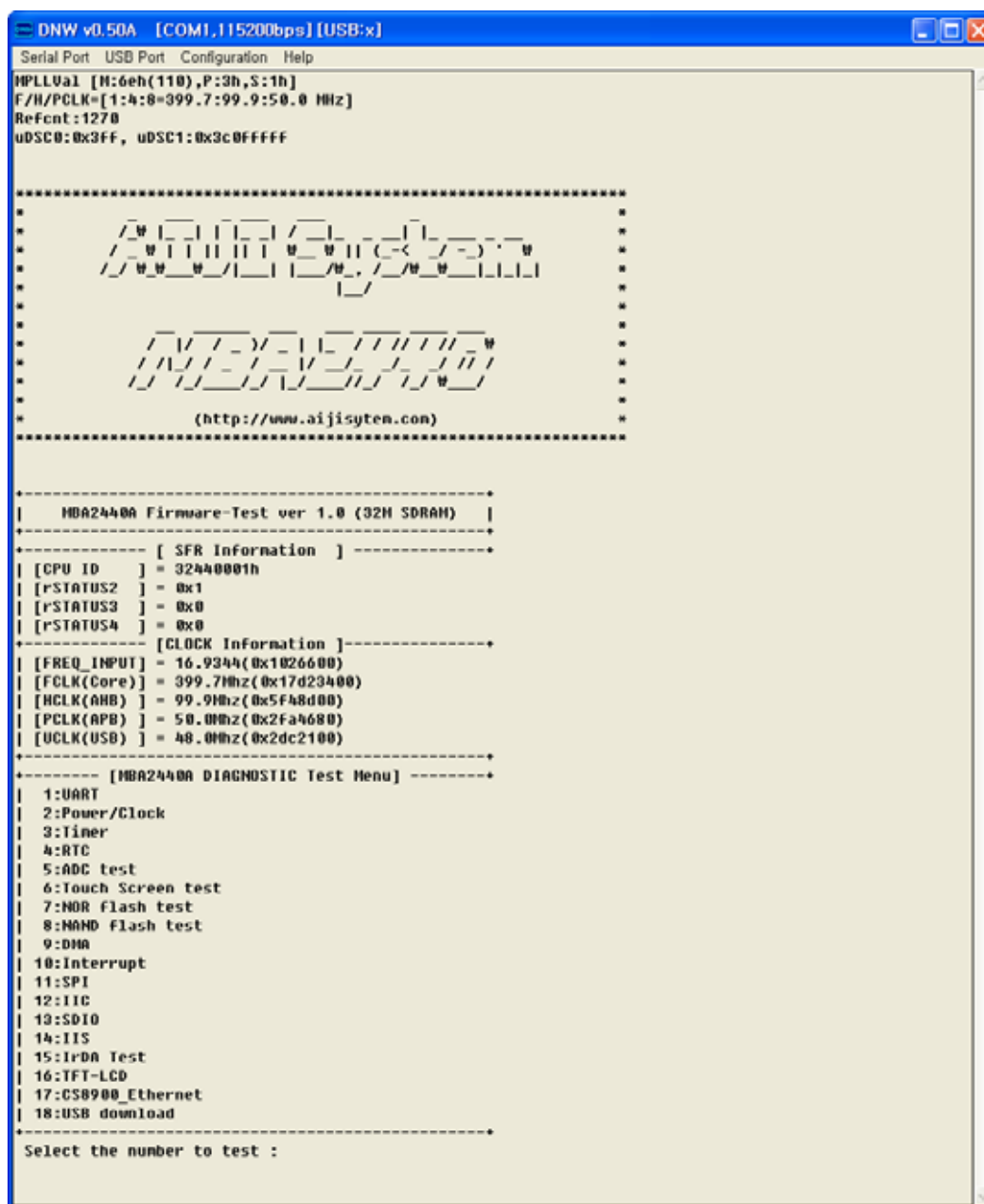
If the cable of board is connected properly, set up AMD flash as boot flash.

(refer to [1.3.2.1 Using NOR flash as Boot flash](#))

After performing serial console emulators such as DNW or minicom, set it up for working properly.

(refer to [2.1.1 DNW Setup](#), [2.1.2 Hyper-terminal Setup](#) and [3.3.4 Minicom Setup](#).)

Turn on the power of board.



[Picture 2.7]

[Picture 2.7] shows when MBA2440 test program prints out menu message through DNW.

2.3 Booting WinCE

In this chapter, let's learn how to perform WinCE using u-boot and WinCE image loaded in K9F5608 NAND flash and how to download WinCE images using TFTP when releasing MBA2440.

You will learn how to operate E-boot in [7.2 Eboot boot loader](#) and how to operate nboot in [7.3 Nboot boot loader](#).

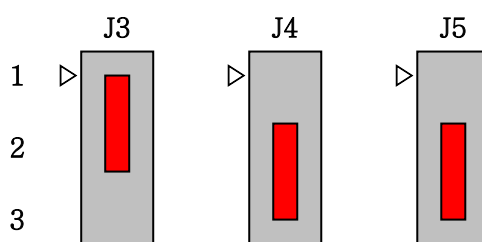
You can find u-boot and WinCE images in CD.

2.3.1 Booting WinCE with U-boot

You can boot u-boot if you set up K9F5608 NAND flash as boot flash and turn on the power. Boot WinCE if there is no input 2 second after booting U-boot, however, if you push any key before 2 second, u-boot will wait for command input in the standby status.

Connect cables properly between the board and PC. (refer to [2.1.2 Connect Cables](#))

Set up switches like the following picture.



[Picture 2.8]

Here's an attention!

There is a matter to be referred for J1 and J2. J1 and J2 don't affect NAND flash to be set up as boot flash.

If you set up NAND flash as boot flash,

it is impossible to approach and control NOR flash connected to nGCS0 after u-boot, because S3C2440 will recognize the address 0x0 as stepping stone buffer (refer to S3C2440 memory map), not as nGCS0.

However, it is possible to approach and control NOR flash connected to other chip select pins, not to nGCS0. Please be aware that developers should set up J1 and J2 to connect NOR flash they want to control to other chip select pins, not to nGCS0 in case of operating u-boot in NAND flash.

Now, let's operate WinCE

Turn on the power of board.

[Picture 2.9] shows the prompt printed out by inputting keys after the power turns on. U-boot boots as NAND flash as shown on [Picture 2.9], and you can confirm what NAND flash through NAND flash information. And you can realize that it recognizes AMD flash as NOR flash memory.

It will take about 15 seconds.

[Picture 2.10] shows WinCE images copied into the memory from K9F5608 NAND flash after the power turns on. If copying is completed, WinCE is booted, and you can see if WinCE works through LCD.

You can perform WinCE with NAND flash read command (nandr). Load kernel images in the memory first.

```
MBA2440 # nandr t 30200000 200000 1400000
Read start point : block[128] page[0] size = 0x1400000
Reading data ...-
Read complete(0x1400000): from 0x200000(128) NAND flash to 0x30200000 SDRAM
```

For nandr command,

30200000 is the address of memory loading WinCE images

200000 is the address of NAND flash where WinCE images are located

1400000 is the size of data you want to copy.

Each number is a unit of 16 hexadecimal.

Lastly, operates WinCE images. The address of kernel images copied is 0x30200000

```
MBA2440 # go wince      (또는 go 30200000)
## Starting WinCE kernel at 0x08lx ... ?Windows CE Kernel for ARM (Thumb Enabled) Built on Jun 7
ProcessorType=0920  Revision=0
sp_abt=ffff5000 sp_irq=ffff2800 sp_undef=ffffc800 OEMAddressTable = 8c201480

Windows CE Firmware Init
INFO: Initializing system interrupts...
.....
```

2.3.2 Booting WinCE with TFTP

Let's operate WinCE after loading WinCE images using "tftp" commands in U-boot.

The connection of network should be established first to operate WinCE with TFTP. (refer to [3.1 Examples of establishing development environment](#))

Copy WinCE images into root directory of TFTP server. (refer to [2.1.4 Set up TFTP server Program for Windows](#) and [3.3.2 Install TFTP server on Linux](#))

Let's just say the name of WinCE image is NK.nb0.

Set up TFTP server address of u-boot in accordance of its environment as mentioned in 2.3.2. (refer to [3.1 Examples of establishing development environment](#) for the establishment of IP address)

In case TFTP server works in Windows, set up the IP address of u-boot environment factor of MBA2440, serverip as 192.168.100.1, and in case TFTP server works in Linux, set up the IP address 192.168.100.2.

IP address of MBA2440 is 192.168.100.3.

u-boot is basically established for the case TFTP server works in Linux.

If TFTP server works in Windows, modify TFTP server address among u-boot environment factors.

After modifying to **setenv** command, confirm if it is modified to **printenv** command.

```
MBA2440 # setenv serverip 192.168.100.1
MBA2440 # printenv
bootdelay=3
baudrate=115200
ethaddr=08:00:3e:26:0a:5b
stdin=serial
stdout=serial
```

```
stderr=serial
filesize=1c9dab
fileaddr=30800000
netmask=255.255.255.0
ipaddr=192.168.100.3
serverip=192.168.100.1
```

Environment size: 195/131068 bytes

ipaddr is MBA2440's own IP address, and **serverip** is the IP address of TFTP server.

Download WinCE images through tftp command.

```
MBA2440 # tftp 30200000 NK.nb0
TFTP from server 192.168.100.2; our IP address is 192.168.100.3
Filename 'NK.nb0'.
Load address: 0x30200000
Loading:/
done
Bytes transferred = 20971520 (1400000 hex)
```

Boot WinCE images.

```
MBA2440 # go 30200000      (또는 go wince)

## Starting application at 0x30200000 ..._FMD::FMD_Init
FMD:: BOOT_LOADER not defied
FMD:: NOSYSCALL not defined
FMD:: NOBINFS not defined
FMD::FMD_Init Done
FMD::FMD_OEMIoControl = 0x71c24
PWR: Process Attach
>PWR_Init(602F1C0)
.....
```

2.3.3 Booting WinCE with USB

Let's operate WinCE after loading WinCE images using “**usbdown**” commands in U-boot.

USB driver should be installed on Windows PC first to operate WinCE with USB. (refer to [4.3.18 USB download](#))

Activate DNW.exe after connecting USB cable between Windows PC and MBA2440

Turn on the Power of board and activate **usbdown** command.

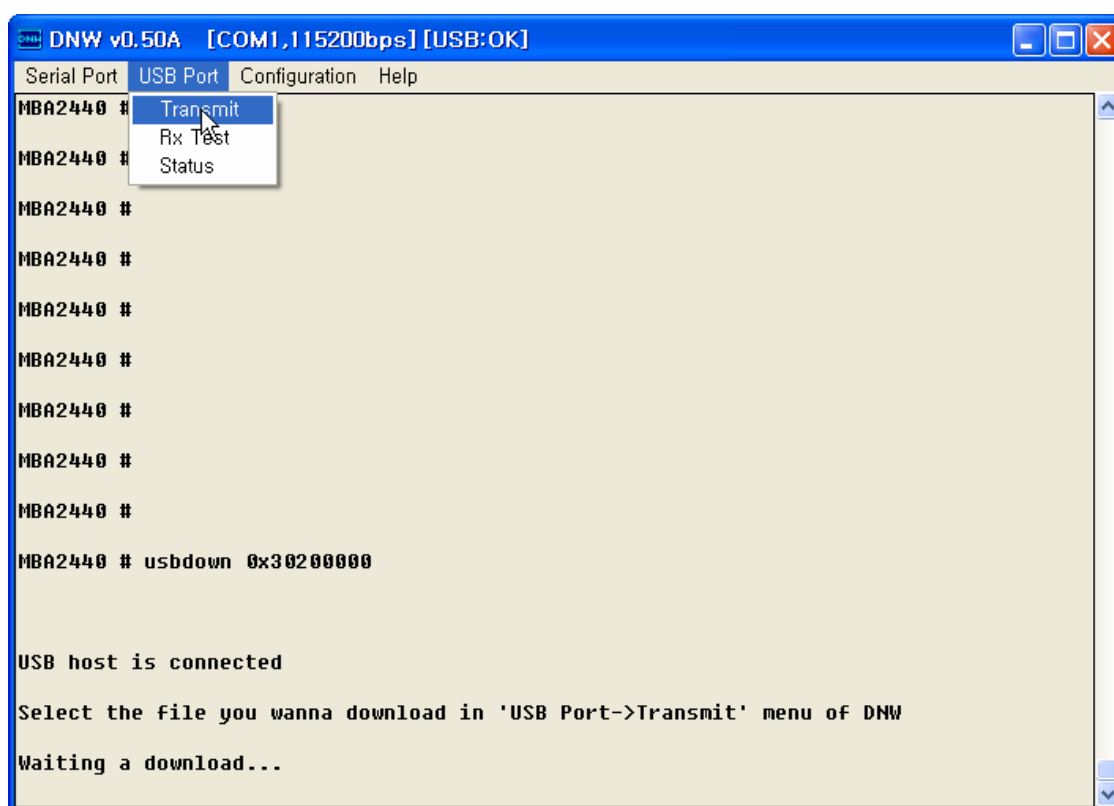
```
MBA2440 # usbdown 0x30200000
```

USB host is connected

Select the file you wanna download in 'USB Port->Transmit' menu of DNW

Waiting a download...

Select USB Port->Transmit menu of DNW window like [Picture 2. 11], and then download WinCE image(NK.nb0).



[Picture 2.11]

Execute WinCE image after completing download.

```
MBA2440 # go wince (또는 go 0x30200000)
```

```
## Starting WinCE kernel at 0x30200000 ... 1/3 Windows CE Kernel for ARM (Thumb Enabled)
```

```
Built on Jun 24 2004 at 18:25:00
```

```
ProcessorType=0920 Revision=0
```

```
sp_abt=ffff5000 sp_irq=ffff2800 sp_undef=ffffc800 OEMAddressTable = 8c201480
```

```
INFO: Initializing driver globals area...->
```

2.4 Booting Linux

You should use u-boot boot loader modified suitably for MBA2440 to operate Linux. U-boot takes the same part as bios of desktop PC.

In this chapter, you will learn how to operate Linux downloading kernel images and RAMDISK using TFTP, how to write RAMDISK image, Linux kernel in NAND flash and how to operate Linux with u-boot.

You can find images of u-boot, Linux kernel, file system in CD.

2.4.1 Booting Linux with TFTP

If you set up K9F5608 NAND flash as boot flash and boot it, you can operate Linux after loading kernel images and file system images into the memory by “tftp” commands of u-boot.

The connection of network should be established first to operate Linux with TFTP. Please refer to [3.1 Examples of establishing development environment](#) for this matter.

Copy Linux kernel, RAMDISK images into root directory of TFTP server. (refer to [2.1.4 Set up TFTP server for Windows](#) and [3.3.2 Set up TFTP in Linux](#))

Let's just say kernel image is zImage.bin and RAMDISK image is ram2440-ram.gz.

In case TFTP server works in Windows, set up IP address of u-boot environment factor, serverip as 192.168.100.1, and in case TFTP server works in Linux, set up IP address as 192.168.100.2.

U-boot is basically established for the case TFTP server works in Linux.

If TFTP server works in Windows, modify TFTP server address of u-boot.

After modifying to setenv command, confirm if it is modified to printenv command.

```
MBA2440 # setenv serverip 192.168.100.1
MBA2440 # printenv
bootdelay=3
baudrate=115200
ethaddr=08:00:3e:26:0a:5b
stdin=serial
stdout=serial
stderr=serial
filesize=1c9dab
fileaddr=30800000
netmask=255.255.255.0
ipaddr=192.168.100.3
serverip=192.168.100.1
```

Environment size: 195/131068 bytes

ipaddr is MBA2440's own IP address, and **serverip** is the Ip address of TFTP server.

Download kernel images by tftp command.

```
MBA2440 # tftp 30c00000 zImage.bin
TFTP from server 192.168.100.2; our IP address is 192.168.100.3
Filename 'zImage.bin'.
Load address: 0x30c00000
Loading:/
done
Bytes transferred = 746208 (b62e0 hex)
```

Download RAMDISK images by tftp command.

```
MBA2440 # tftp 30800000 ram2440-ram.gz
TFTP from server 192.168.100.2; our IP address is 192.168.100.3
Filename 'ram2440.gz'.
Load address: 0x30800000
Loading:-
done
Bytes transferred = 2782866 (2a7692 hex)
```

Operate Linux Kernel.

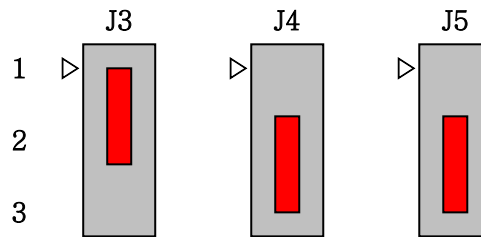
```
MBA2440 # go linux      (또는 go 30c00000)
Uncompressing Linux..... done, booting the kern.
Linux version 2.4.20_elfin-d1.5 (root@localhost.localdomain) (gcc version 2.95.3 20010315 (releaT
CPU: ARM/CIRRUS Arm920Tid(wb) revision 0
.....
```

2.4.2 Booting Linux in NAND flash

If you set up K9F5608 NAND flash as boot flash, u-boot works when booting. Write and load u-boot, Linux kernel, RAMDISK in K9F5608 NAND flash, and operate Linux using images loaded.

In this chapter, we will explain K9F5608 NAND flash as an example, however, you can also use K9S1208 SMC..

Set up the switch and turn on the power of board like [Picture 2.11]



[Picture 2.12]

```

*****
*                                     *
*  Aiji System                      *
*                                     *
*  MBA2440                          *
*                                     *
*  (http://www.aijisystem.com)      *
*                                     *
*****

U-Boot 1.1.1 (Jun 23 2005 - 23:52:32)

U-Boot code: 0x31F80000 -> 0x31F9D228 BSS: -> 0x31FA1CF0
RAM Configuration:
nGCS6 : 0x30000000 32 MB

U-Boot is booted from NAND flash

#### NOR flash information ####
Bank # 0 => [missing or unknown FLASH type]
Bank # 1 => [<nGCS4:0x20000000> = AMD-AM29LV800BB , 1024 KB]

#### NAND flash information ####
Manufacture ID [0xec], Device ID [0x75]
[Samsung K9F5608U0B mem size(32MB), Block count(2048)]

*** Warning - bad CRC, using default environment

In: serial
Out: serial
Err: serial
MBA2440 # 
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.00.0 | VT102 | Offline
[영어] [완성] [두벌식]

```

[Picture 2.13]

As shown on [Picture2.13] U-boot boots as NAND flash, and you can confirm what NAND flash through NAND flash information.

Download u-boot, kernel image, RAMDISK image using TFTP. You can find these three types of images in CD, and please do not forget to copy into root directory of TFTP server.

Let's just say u-boot image is u-boot.bin, kernel image is zImage.bin and RAMDISK image is ram2440-ram.gz.

Download u-boot image through TFTP.

```
MBA2440 # tftp 30000000 u-boot.bin
```

```
TFTP from server 192.168.100.2; our IP address is 192.168.100.3
Filename 'mba2440/u-boot.bin'.
Load address: 0x30000000
Loading:-
done
Bytes transferred = 120492 (1d6ac hex)
```

Download kernel image

```
MBA2440 # tftp 30080000 zImage.bin
TFTP from server 192.168.100.2; our IP address is 192.168.100.3
Filename 'mba2440/zImage.bin'.
Load address: 0x30080000
Loading:/
done
Bytes transferred = 746208 (b62e0 hex)
```

Download RAMDISK image.

```
MBA2440 # tftp 30200000 ram2440-ram.gz
TFTP from server 192.168.100.2; our IP address is 192.168.100.5
Filename 'mba2440/ram2440.1.2.gz'.
Load address: 0x30200000
Loading:-
done
Bytes transferred = 2782866 (2a7692 hex)
```

After erasing all NAND flash, write images downloaded at once.

```
MBA2440 # nande all
erase all blocks [0 ~ 2047]
Erasing block .../
Complete erasing block

MBA2440 # nandw t 30000000 0 600000
Write start point : block[0] page[0] size = 0x600000
Writing data .../
Write complete(0x600000): from 0x30000000 SDRAM to 0x0(0) NAND flash
```

You can operate Linux with NAND flash read command (nandr).

Load kernel images in the memory first.

```
MBA2440 # nandr t 30c00000 80000 180000
Read start point : block[32] page[0] size = 0x180000
Reading data ...-
Read complete(0x180000): from 0x80000(32) NAND flash to 0x30c00000 SDRAM
```

For nandr command,

30c00000 is the address of memory loading kernel images

80000 is the address of NAND flash where kernel images are located

180000 is the size you want to copy

Each number is a unit of 16 hexadecimal.

Next, load RAM images into the memory.

```
MBA2440 # nandr t 30800000 200000 400000
Read start point : block[128] page[0] size = 0x400000
Reading data .../
Read complete(0x400000): from 0x200000(128) NAND flash to 0x30800000 SDRAM
```

Lastly, operates kernel images. The address of kernel images copied is 0x30c00000.

```
MBA2440 # go linux      (또는 go 30c00000)
Uncompressing Linux..... done, booting the kern.
Linux version 2.4.20_elfin-d1.5 (root@localhost.localdomain) (gcc version 2.95.3 20010315 (releaT
CPU: ARM/CIRRUS Arm920Tid(wb) revision 0
.....
```



```
iic elfin init: initialized iic-bus at 0xf4000000.
ELFTIN UDA1341 audio driver initialized
NAND device: Manufacture ID: 0xec, Chip ID: 0x75 (Samsung NAND 32MB 3.3V)
Creating 3 MTD partitions on "NAND 32MB 3.3V":
0x00000000-0x00030000 : "NAND partition 0 : Bootloader"
0x00030000-0x00200000 : "NAND partition 1 : Kernel"
0x00200000-0x02000000 : "NAND partition 2"
usb.c: registered new driver hub
S3C2440 USB Controller Core Initialized
USB Function Ethernet Driver Interface
NET4: Linux TCP/IP 1.0 for NET4.0
IP Protocols: ICMP, UDP, TCP, IGMP
IP: routing cache hash table of 512 buckets, 4Kbytes
TCP: Hash tables configured (established 2048 bind 2048)
IP-Config: No network devices available.
NET4: Unix domain sockets 1.0/SMP for Linux NET4.0.
NetWinder Floating Point Emulator V0.95 (c) 1998-1999 Rebel.com
RAMDISK: Compressed image found at block 0
Freeing initrd memory: 4096K
EXT2-fs warning: mounting unchecked fs, running e2fsck is recommended
VFS: Mounted root (ext2 filesystem).
Freeing init memory: 76K
INIT: version 2.74 booting
SIOCSIFADDR: No such device
eth0: unknown interface: No such device
SIOCSIFNETMASK: No such device
eth0: unknown interface: No such device
SIOCADDRT: Netlink: 4155) Starting system logger: syslogd
Starting portmapper: portmap
Starting network Starting network SIOCSIFADDR: No such device
eth0: unknown interface: No such device
SIOCSIFNETMASK: No such device
SIOCADDRT: No such device

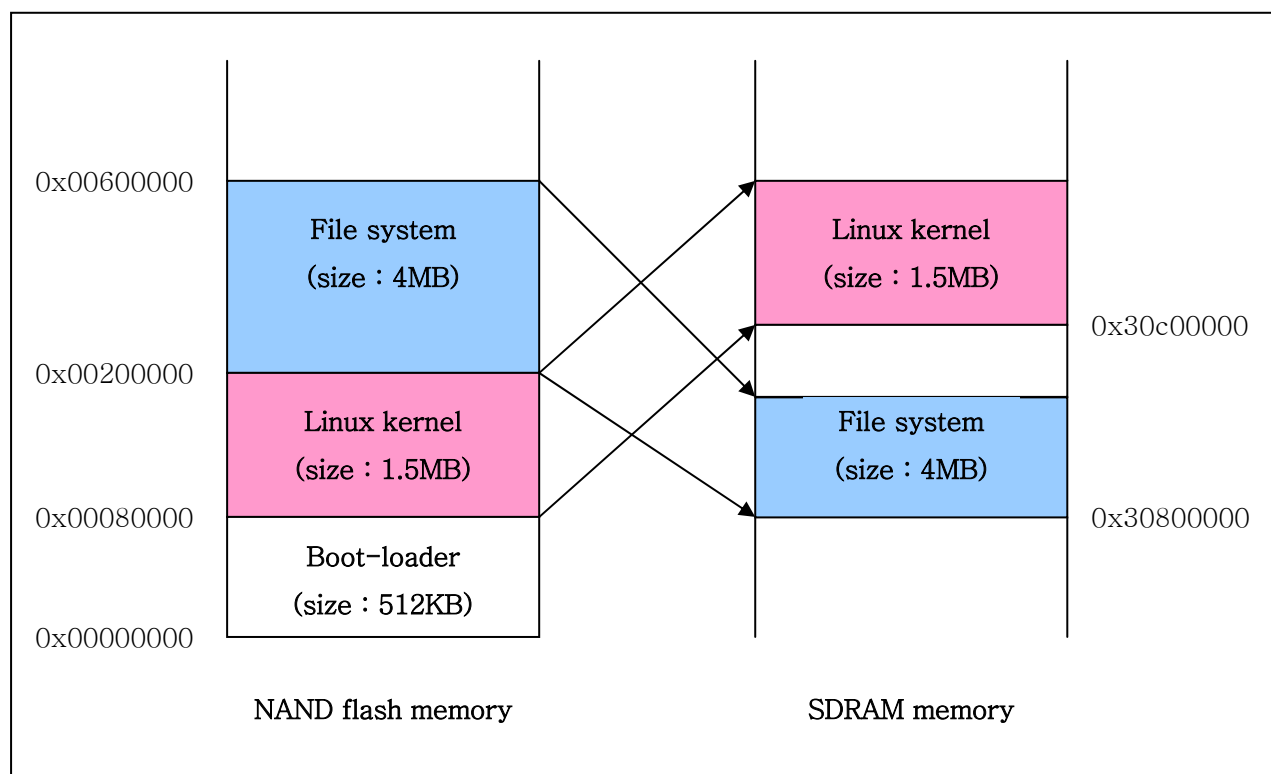
+-----+
| Welcome to MBA2440 !!! |
+-----+

Linux login: root
[root@Linux /root]$
CTRL-Z for help |115200 8N1 | NOR | Minicom 2.00.0 | VT102 | Offline
[영어][완성][두벌식]
```

[Picture 2.14]

You can see Linux prompt shown if you input root on Linux login: after booting.

[Picture 2.15] is the memory map between NAND flash and SDRAM memory which help easy understanding.



[Picture 2.15]

2.4.3 Booting Linux with USB

Let's operate Linux after loading Linux kernel and ramdisk image using “**usbdown**” commands in U-boot.

USB driver should be installed on Windows PC first to operate Linux with USB. (refer to [4.3.18 USB download](#))

Activate DNW.exe after connecting USB cable between Windows PC and MBA2440

Turn on the Power of board and activate **usbdown** command.

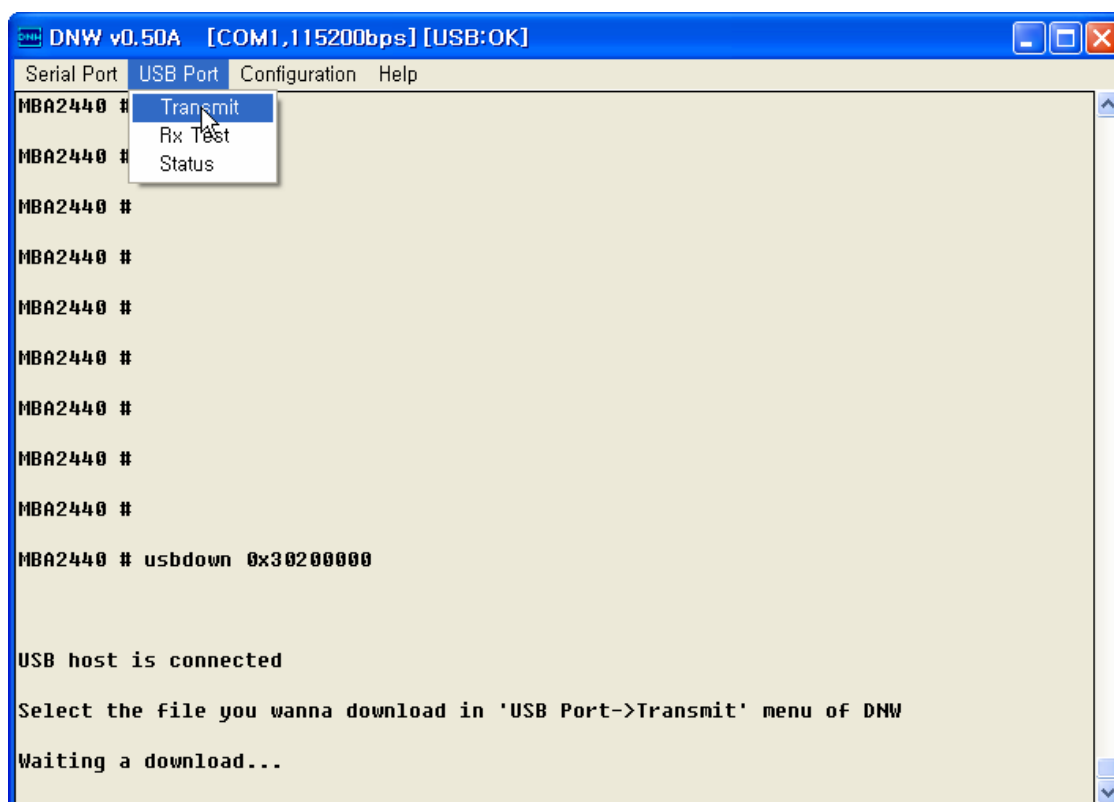
```
MBA2440 # usbdown 0x30800000
```

USB host is connected

Select the file you wanna download in 'USB Port->Transmit' menu of DNW

Waiting a download...

Select USB Port->Transmit menu of DNW window like [Picture 2. 16], and then download image. You first download ramdisk image(mba2440-ram.gz).



[Picture 2.16]

Download linux kernel(zImage-ramdisk.bin) after completing download ramdisk.

```
MBA2440 #  usbdown  0x30c00000

USB host is connected
Select the file you wanna download in 'USB Port->Transmit' menu of DNW
Waiting a download...
```

Finally, execute linux kernel.

```
MBA2440 #  go  linux  (또는 go  0x30c00000)

Uncompressing Linux..... done, booting the kernel.

Linux version 2.4.20_elfin-d1.5 (root@localhost.localdomain) (gcc version 2.95.3 20010315
(release)) #111 2005. 07. 06. (수) 10:03:59 KST
```

3 Set up Development Environment

There are necessary development environments needed for developing embedded system using MBA2440. The objective of this chapter is to teach what are those development environments and how to construct them.

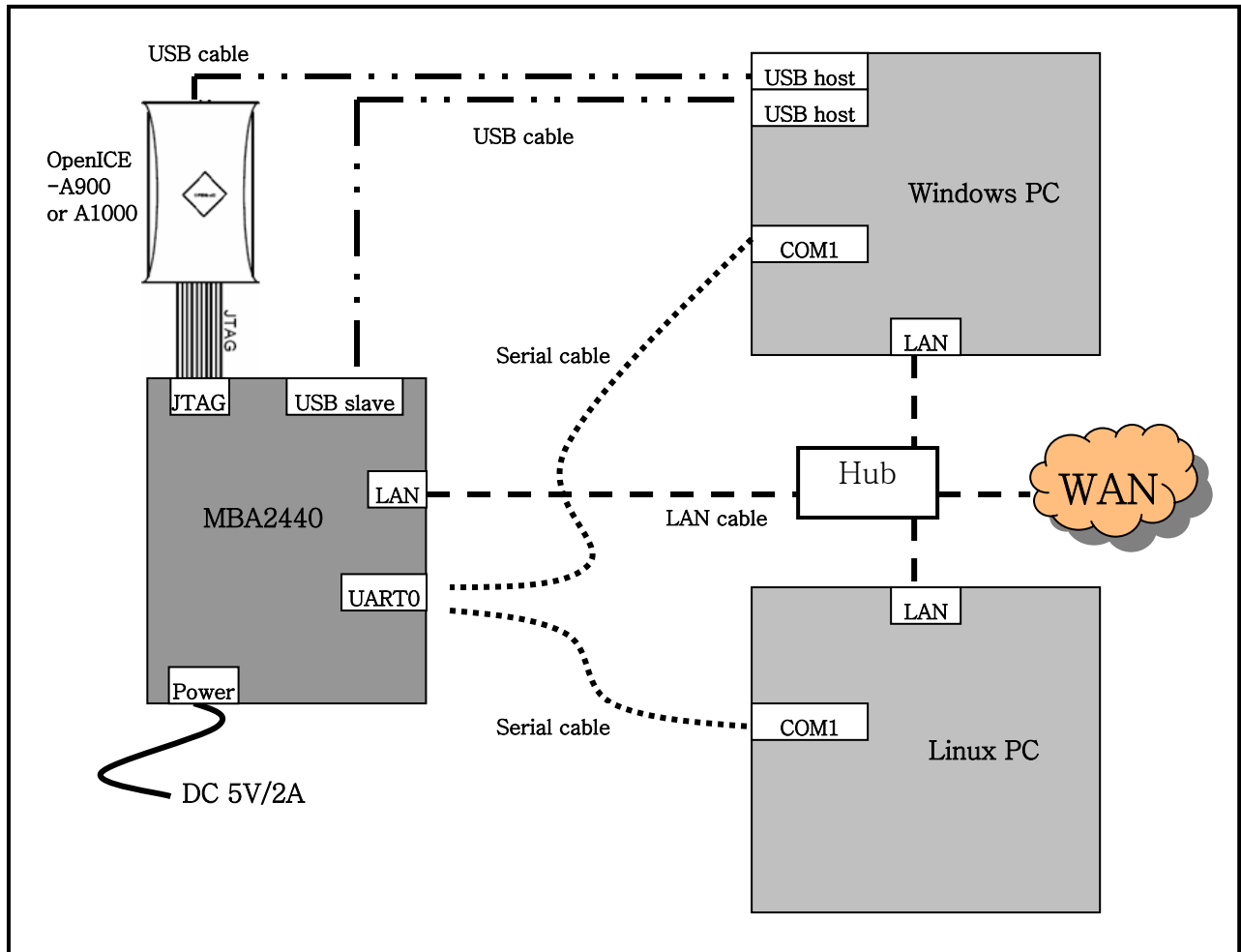
This chapter consists of **firmware development environments to control MBA2440**, **embedded Linux development environment** and **WinCE development environment**.

There are platforms, Windows OS and Linux OS to construct development environments.

It constructs the environment to develop U-boot bootloader and embedded Linux in Linux OS and the environment to develop MBA2440 test program development environment and WinCE in Windows OS..

3.1 Examples of establishing development environment

Development environments are consisted as [Picture 3.1] in this document.



[Picture 3.1] Examples of establishing development environments

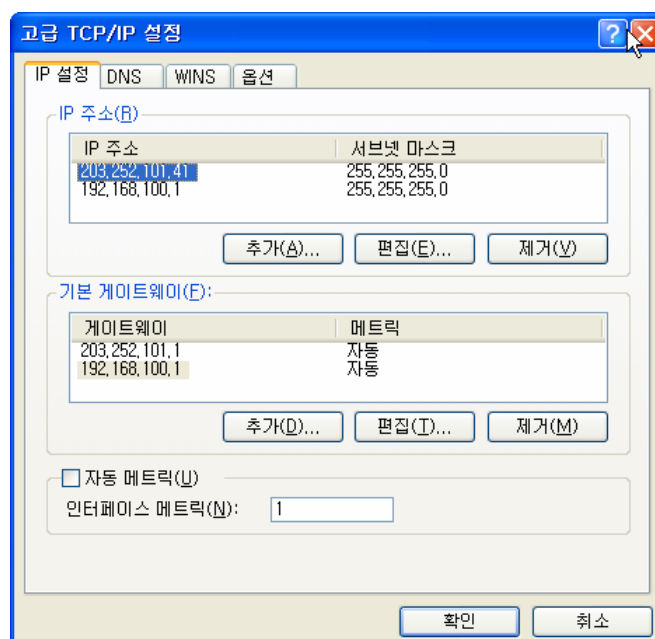
The establishment of network is an important part for the convenience of development to construct development environments.

In this chapter, we will show you how to divide network address and all works in accordance with it. Developers can modify network address for developer's convenience. We would like to establish it in this document only for convenience.

Desktop PC having Windows installed	192.168.100.1
Desktop PC having Linux installed	192.168.100.2
MBA2440 board	192.168.100.3

Windows or Linux can give another IP besides IP address generally used on one network card. Of course, the IP address given on one network card should be in different area from each other.

☞ If you push ‘advanced’ button on registration information of “local area connection” which comforts to its network card in Windows, the window of “set up advanced TCP/IP” is shown. Add a basic gateway address and IP address privately established on the window of “set up advanced TCP/IP” (push the button for adding). In this document, 192.168.100.1 is added in Windows PC.



[Picture 3.2]

☞ Create a file named as ifcfg-eth0:1 under /etc/sysconfig/network-scripts/ in Linux PC first. After creating it, write down as the following, save it and get out of there.

```
DEVICE=eth0:1
ONBOOT=yes
BOOTPROTO=none
IPADDR=192.168.100.2
NETMASK=255.255.255.0
GATEWAY=192.168.100.1
TYPE=Ethernet
USERCTL=no
PEERDNS=no
NETWORK=192.168.100.0
BROADCAST=192.168.100.255
```

Perform the following command and reboot network.

```
[root@localhost]# /etc/rc.d/init.d/network restart
```

You can confirm that two IP which are in different areas from each other are divided on one network card through **ifconfig** command.

```
[root@localhost]# ifconfig
```

```
eth0      Link encap:Ethernet  HWaddr 00:20:ED:35:DD:CE
          inet addr:203.252.101.42  Bcast: 203.252.101.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1679305 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1757078 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:100
          RX bytes:464794008 (443.2 Mb)  TX bytes:947711009 (903.8 Mb)
          Interrupt:11 Base address:0xa000 Memory:e2000000-e2000038

eth0:1    Link encap:Ethernet  HWaddr 00:20:ED:35:DD:CE
          inet addr:192.168.100.2  Bcast:192.168.100.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1679305 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1757078 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:100
          RX bytes:464794008 (443.2 Mb)  TX bytes:947711009 (903.8 Mb)
          Interrupt:11 Base address:0xa000 Memory:e2000000-e2000038

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:378001 errors:0 dropped:0 overruns:0 frame:0
          TX packets:378001 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:110928505 (105.7 Mb)  TX bytes:110928505 (105.7 Mb)
```

You can set up IP of MBA2440 when compiling u-boot, eboot, embedded linux and WinCE or after driving them.

3.2 Firmware Development Environment

MBA240 test program, nboot except u-boot and eboot among firmware for MBA2440 are firmware developed in the basis of ADS1.2. In case of modifying these firmware, install ADS1.2 and recompile after modifying through this.

ADS1.2 is development suite version 1,2 which is exclusive for ARM core.

This development tool can be used on the platform of Windows, SunSPARC workstation, HP workstation, Redhat linux.

Please refer to ARM homepage for details.

Directly purchase and install ADS 1.2. We don't deal with how to install it. Please install it yourself referring to the manual after purchase.

This document will explain everything under the status installing ADS1.2 on Windows OS when we explain matters regarding nboot and MBA2440 test program.

3.3 Embedded Linux Development Environment

This chapter will show you how to construct environments to develop embedded Linux. All process to construct development environments explained on 3.3 are performed on desktop PC where RedHat 9.0 Linux OS is installed in full option. We recommend developers to refer to this document in this kind of environment.

We can develop u-boot bootloader and embedded Linux system for MBA2440 by constructing embedded Linux development environment explained in this chapter.

Development environments can be divided as the following.

- ◆ **Install cross compiler**
: Install compiler which can compile Linux source for MBA2440
- ◆ **Install tftp server**
: Install the server to receive files on MBA2440 bootloader as trivial FTP
- ◆ **Install NFS server**
: Install the server to use Network File System
- ◆ **Set up serial console (minicom)**
: Set up serial communication emulator of Linux which can monitor MBA2440

3.3.1 Install Cross Compiler

3.3.1.1 What is Cross Compiler?

Cross compiler means compilers to compile Linux source modified for MBA2440 besides compilers provided on Linux OS installed in desktop.

As an example of Linux OS installed in general desktop, an image which can be operated on desktop is created if it compiles gcc compiler source provided on RedHat 9.0. This operating image doesn't work in MBA2440. S3C2440, CPU of MBA2440 doesn't understand the operating image, because it is compiled for CPU in desktop to understand.

Therefore, it can be operated in MBA2440 only when compiling it for MBA2440 to understand. The compiler for this is called cross compiler.

We can call other compilers for many embedded systems, platform builder and etc. besides MBA2440 board as cross compiler.

3.3.1.2 Set up Cross Compiler gcc-2.95.3

There are two kinds of cross compilers in linux/toolchain directory in CD.

One of them is **arm-gcc-2.95.3.tar.bz2**.

This cross compiler is used when compiling lower versions of source than Linux 2.4 version.

Bring compressed cross compiler from CD first.

```
[root@localhost]# pwd
/root
[root@localhost]# cp /mnt/cdrom/toolchain/arm-gcc-2.95.3.tar.bz2 ./
[root@localhost]# ls
arm-gcc-2.95.3.tar.bz2
```

The directory 2.95.3 is created if you extract the cross compiler compressed.

```
[root@localhost]# tar xjf arm-gcc-2.95.3.tar.bz2
[root@localhost]# ls
2.95.3 arm-gcc-2.95.3.tar.bz2
```

After you create a directory named as arm under /usr/local/ directory, move the 2.95.3 directory created.

```
[root@localhost]# mkdir /usr/local/arm
[root@localhost]# ls /usr/local
arm bin etc games include lib libexec man sbin share src
[root@localhost]# mv 2.95.3 /usr/local/arm
[root@localhost]# ls /usr/local/arm
2.95.3
```

Now, put the full path of this cross compiler into PATH environment factors.

et up PATH environment factors in /root/.bash_profile file.

```
[root@localhost]# vi /root/.bash_profile
```

If you open the file, you can see as the following. Add sentences bolded to the original.

```
# .bash_profile
# Get the aliases and functions
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
fi

# User specific environment and startup programs

PATH=$PATH:$HOME/bin
PATH=$PATH:/usr/local/arm/2.95.3/bin/

BASH_ENV=$HOME/.bashrc
USERNAME="root"
```

```
export USERNAME BASH_ENV PATH
```

After adding them, save them and get out of there, and then, apply the environment factor modified.

```
[root@localhost]# source /root/.bash_profile
```

input arm-linux- and push tab key to confirm if it is applied. If you can see a message as the following, it means the environment factor is applied properly.

```
[root@localhost]# arm-linux-
arm-linux-addr2line    arm-linux-g77          arm-linux-objcopy     arm-linux-strings
arm-linux-ar           arm-linux-gasp         arm-linux-objdump     arm-linux-strip
arm-linux-as           arm-linux-gcc          arm-linux-protolize   arm-linux-unprotolize
arm-linux-c++          arm-linux-gcj          arm-linux-ranlib
arm-linux-c++filt      arm-linux-ld           arm-linux-readelf
arm-linux-g++          arm-linux-nm           arm-linux-size
```

3.3.1.3 Install Cross Compile gcc-3.3.4

Another cross compiler for Linux provided by CD is **arm-gcc-3.3.4.tar.bz2**. This cross compiler is to compile Linux 2.6 version.

Bring the cross compiler compressed from CD first.

```
[root@localhost]# pwd
/root
[root@localhost]# cp /mnt/cdrom/toolchain/ arm-gcc-3.3.4.tar.bz2 ./
[root@localhost]# ls
arm-gcc-3.3.4.tar.bz2
```

If you extract the cross compiler compressed, a directory named as **arm-gcc-3.3.4** is created.

```
[root@localhost]# tar xjf arm-gcc-3.3.4.tar.bz2
[root@localhost]# ls
arm-gcc-3.3.4  arm-gcc-3.3.4.tar.bz2
```

Create a directory named as arm under the directory. If the directory is already created, you don't need to do it again. Move arm-gcc-3.3.4 directory to arm directory.

```
[root@localhost]# mkdir /usr/local/arm
[root@localhost]# ls /usr/local
arm bin etc games include lib libexec man sbin share src
[root@localhost]# mv arm-gcc-3.3.4 /usr/local/arm
```

```
[root@localhost]# ls /usr/local/arm
arm-gcc-3.3.4
```

Now, put the full path of cross compiler into PATH environment factors.

Set up PATH environment factors in /root/.bash_profile file.

```
[root@localhost]# vi /root/.bash_profile
```

If you open the file, you can see as the following. Add sentences bolded to the original.

```
# .bash_profile

# Get the aliases and functions
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
fi

# User specific environment and startup programs

PATH=$PATH:$HOME/bin
PATH=$PATH:/usr/local/arm/2.95.3/bin/
PATH=$PATH:/usr/local/arm/ arm-gcc-3.3.4/bin/

BASH_ENV=$HOME/.bashrc
USERNAME="root"
export USERNAME BASH_ENV PATH
```

Add and save it, and get out of there, and then, apply the environment factor modified.

```
[root@localhost]# source /root/.bash_profile
```

Input arm-linux-gnu- and push tab key to confirm if it is applied. If you can see a message as the following, it means the environment factor is applied properly.

```
[root@localhost]# arm-linux-gnu-
arm-linux-gnu-addr2line  arm-linux-gnu-gcc          arm-linux-gnu-objcopy
arm-linux-gnu-ar         arm-linux-gnu-gcc-3.4.4      arm-linux-gnu-objdump
arm-linux-gnu-as         arm-linux-gnu-gccbug        arm-linux-gnu-ranlib
arm-linux-gnu-c++        arm-linux-gnu-gcov          arm-linux-gnu-readelf
arm-linux-gnu-c++filt    arm-linux-gnu-gprof         arm-linux-gnu-size
arm-linux-gnu-cpp        arm-linux-gnu-ld            arm-linux-gnu-strings
arm-linux-gnu-g++        arm-linux-gnu-nm            arm-linux-gnu-strip
```

3.3.2 Install TFTP Server on Linux

3.3.2.1 What is TFTP?

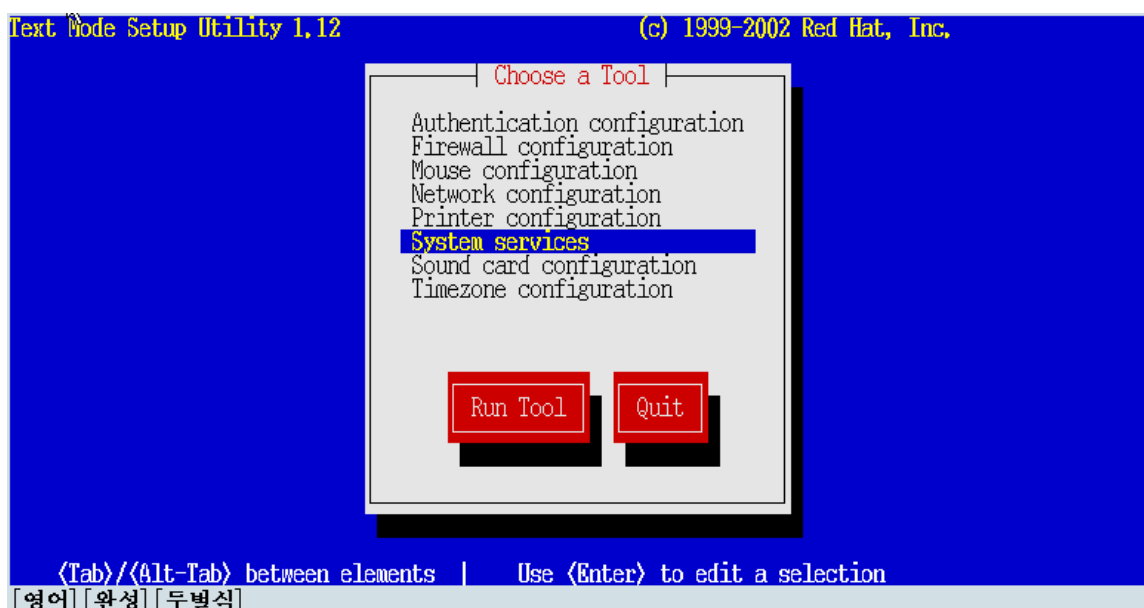
TFTP is a protocol based on UDP to send and receive files as a FTP protocols.

It is very useful for the cases frequently occurred to send and receive big sized data such as Linux kernel, file system and WinCE images in MBA2440 u-boot in the process of developing embedded system.

Install TFTP server operated as a server based on this protocol.

3.3.2.2 Install TFTP Server

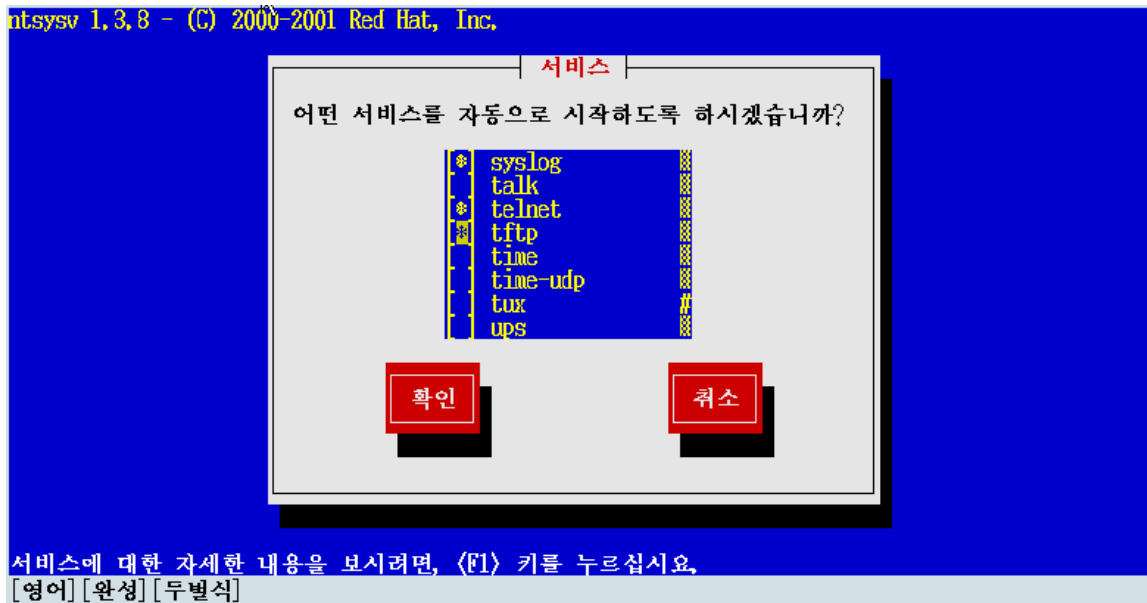
In case of installing RedHat 9.0 in full option, you can use right away because TFTP is installed already. If you call a console window on Linux PC and input setup command, utility setup window in the text mode and many menus will be shown as [Picture 3.3]. If you select system services and push enter key, you will find windows which enable you to choose services.



[Picture 3.3]

Find and select TFTP among service menus, check "*" pushing the space bar like [Picture 3.4] if "*" mark is not checked. And move to 'confirm' by pushing the tab key and apply setup by pushing the enter key.

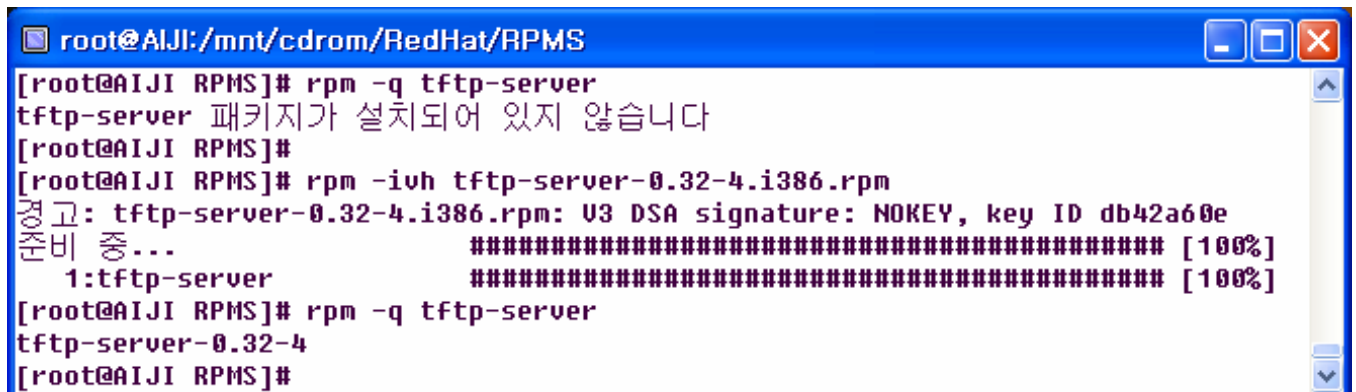
You can see the screen go back to the last menu, [Picture 3.3]. Move to 'Quit' by pushing the tab key and get out of there by pushing the enter key.



[Picture 3.4]

In the case it is not installed, you can find and install rpm from RedHat CD.

RedHat CD consists of three chapters. Search for **tftp-server-0.32-4.i386.rpm** file in CD, copy it into /root directory, and install with rpm command.

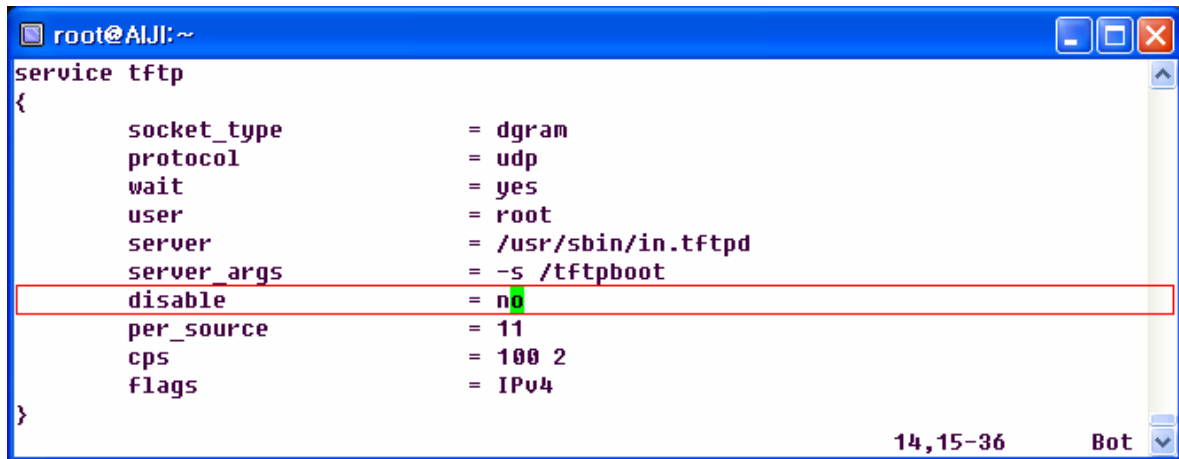


[Picture 3.5]

3.3.2.3 Set up TFTP

There are matters which will be modified in /etc/xinetd.d/tftp file.

```
[root@localhost]# vi /etc/xinetd.d/tftp
```



```

root@AIJI:~
service tftpd
{
    socket_type      = dgram
    protocol         = udp
    wait             = yes
    user             = root
    server           = /usr/sbin/in.tftpd
    server_args      = -s /tftpboot
    disable          = no
    per_source       = 11
    cps              = 100 2
    flags            = IPv4
}
14,15-36 Bot

```

[Picture 3.6]

Set up disable, no like [Picture 3.6].

server_args is a part to set up root directory of TFTP server. Please remember you can transmit TFTP files only if you put files and images you want to send in the directory.

You can input the absolute path of directory located according to developer's needs instead of /tftpboot. (ex: -s /aiji)

3.3.2.4 Restart xinetd

Restart xinetd daemon to make TFTP service available.



```

root@AIJI:~
[root@AIJI root]# /etc/rc.d/init.d/xinetd restart
xinetd 를 정지함: [ 확인 ]
xinetd (을)를 시작합니다: [ 확인 ]
[root@AIJI root]#

```

[Picture 3.7]

3.3.3 Set up NFS

3.3.3.1 What is NFS?

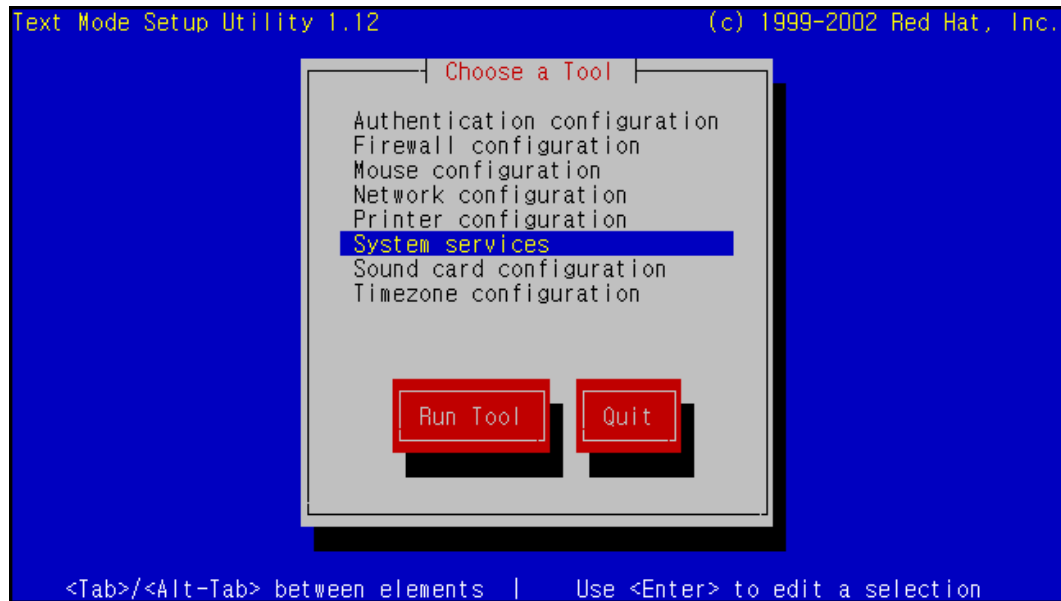
NFS enables users to use the directory of remote host like their own local directory using RPC as one of file systems.

his file system has to support NFS on remote host and the system used by users as well.

3.3.3.2 NFS Service

The NFS also can use service through setup command in case RedHat 9.0 is installed in full option.

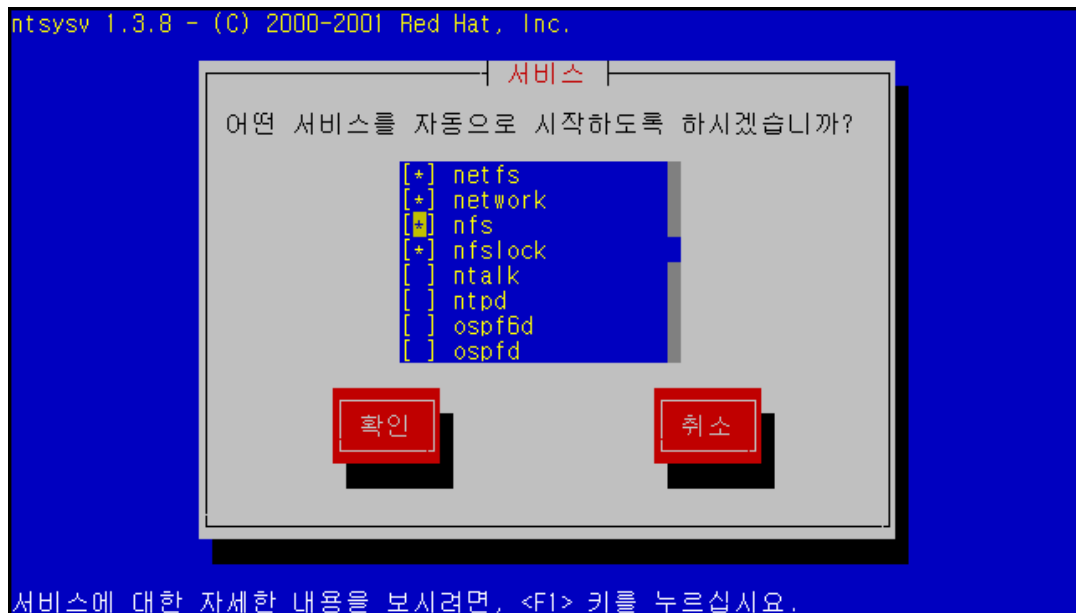
You can do the same as the case of TFTP like [Picture 3.8].



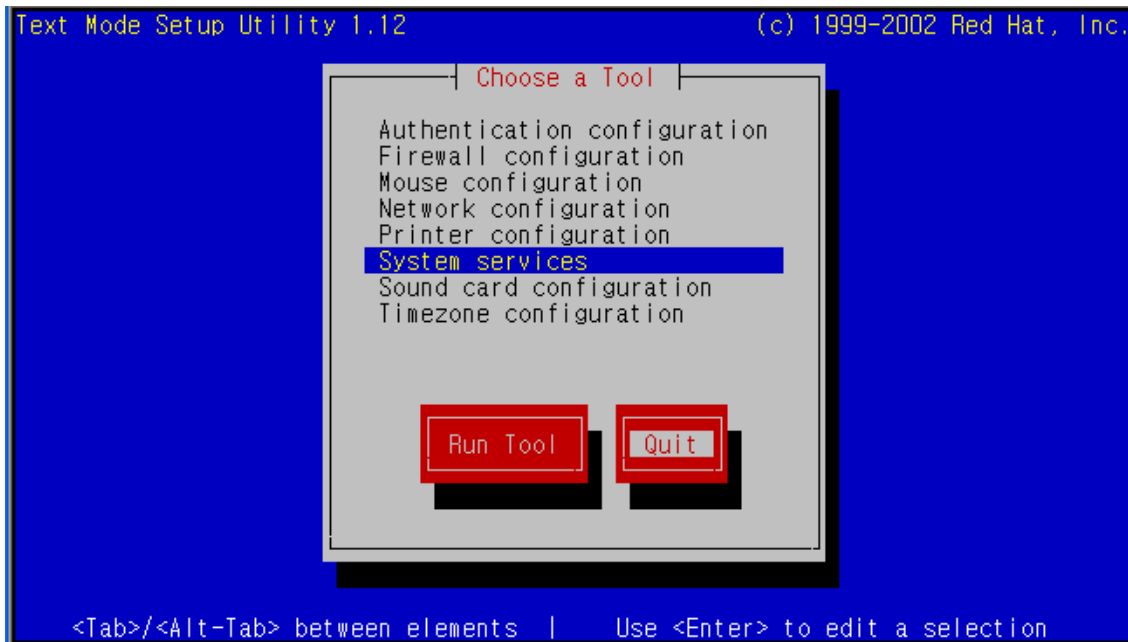
[Picture 3.8]

If you operate setup after calling a terminal window on Linux, a window like above will be shown. You can support for NFS service now. Move to system services and push the enter key, and you will move to the window for supporting many system services.

Select nfs and nfslock among many services shown after moving to the window. If you push the space bar, * mark repeats to appear and disappear. If * mark appears, it means selected. After confirming it, get out of there.



[Picture 3.9]



[Picture 3.10]

3.3.3.3 NFS Setup

Create a directory you want to share as NFS. In this chapter, we will explain it creating a directory named as nfsroot in the highest directory ("/).

```
[root@localhost]# mkdir /nfsroot
[root@localhost]# cd /
[root@localhost]# chmod 777 /nfsroot
[root@localhost]# chown nobody /nfsroot
[root@localhost]# chgrp nobody /nfsroot
```

Modify the /etc/exports file next.

```
[root@localhost]# vi /etc/exports
```

```
/nfsroot          localhost(rw,no_root_squash)
/nfsroot          192.168.100.0/24(rw, no_root_squash)
```

/nfsroot is the name of directory to share mentioned before.

192.168.100.0/24 means all computers having a substitute IP address of 192.168.100 are allowed to approach.

The explanation about () is as the following.

Option name	Description
ro	Read only
rw	Read/Write possible
no_root_squash	permit root to mount file system

root_squash	permit root to access as anonymous UID/GID
insecure	No certification

Confirm setup and reboot nfs daemon after environment setup.

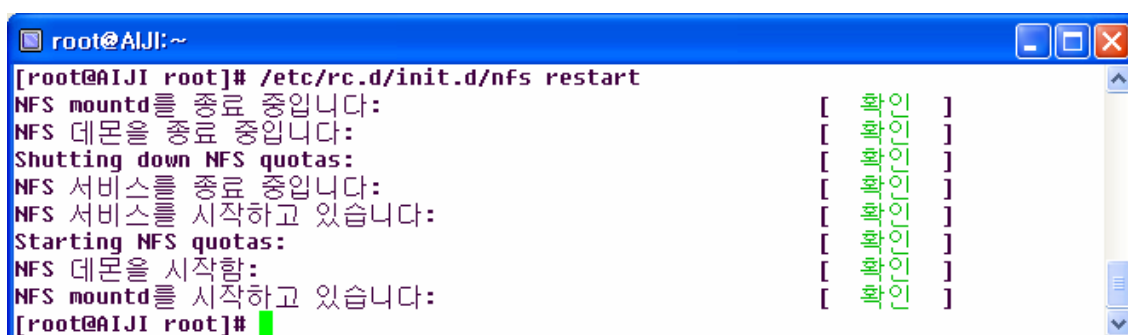


```

root@AIJI:~
[root@AIJI root]# exportfs
/tftpboot <world>
[root@AIJI root]#

```

[Picture 3.11]



```

root@AIJI:~
[root@AIJI root]# /etc/rc.d/init.d/nfs restart
NFS mountd를 종료 중입니다: [ OK ]
NFS 데몬을 종료 중입니다: [ OK ]
Shutting down NFS quotas: [ OK ]
NFS 서비스를 종료 중입니다: [ OK ]
NFS 서비스를 시작하고 있습니다: [ OK ]
Starting NFS quotas: [ OK ]
NFS 데몬을 시작함: [ OK ]
NFS mountd를 시작하고 있습니다: [ OK ]
[root@AIJI root]#

```

[Picture 3.12]

3.3.3.4 Confirm if NFS works

Let's confirm if NFS works properly.

Boot MBA2440 on Linux. Refer to [2.4 Booting Linux](#) for how to boot it.

If booting is completed, you can see Linux prompt through minicom or DNW console window.

If Linux PC is installed properly to support NFS, perform as the following on the console window.

```
[root@localhost root]# mount -t nfs 192.168.100.2:/nfsroot /mnt/nfs/
```

192.168.100.2 is the IP address of Linux PC.

:/nfsroot is the absolute path address of NFS root directory in Linux PC.

/mnt/nfs/ is a directory in Linux file system operated on MBA2440, and /nfsroot directory of Linux PC is mounted in the directory through network.

You will realize that considering /nfsroot directory as Is command on Linux PC and considering /mnt/nfs directory as Is command on console window (minicom or DNW) are same.

```
[root@localhost root]# cd /nfsroot
[root@localhost nfsroot]# ls
ram2440.gz root-mba2440.cramfs zImage-cramfs.bin zImage.bin
[root@localhost nfsroot]# █
```

[영어] [완성] [두벌식]

[Picture 3.13] Showing /nfsroot directory on Linux PC

```
[root@Linux /root]$ mount -t nfs 192.168.100.2:/nfsroot /mnt/nfs/
nfs warning: mount version older than kernel
[root@Linux /root]$ cd /mnt/nfs/
[root@Linux nfs]$ ls
zImage.bin      ram2440.gz      zImage-cramfs.bin
root-mba2440.cramfs
[root@Linux nfs]$ █
```

[영어] [완성] [두벌식]

[Picture 3.14] Showing /mnt/nfs directory on minicom

If you copy files in /mnt/nfs on console window of MBA2440 board into /root directory, it is copied as it is through network, and the reverse is also possible.

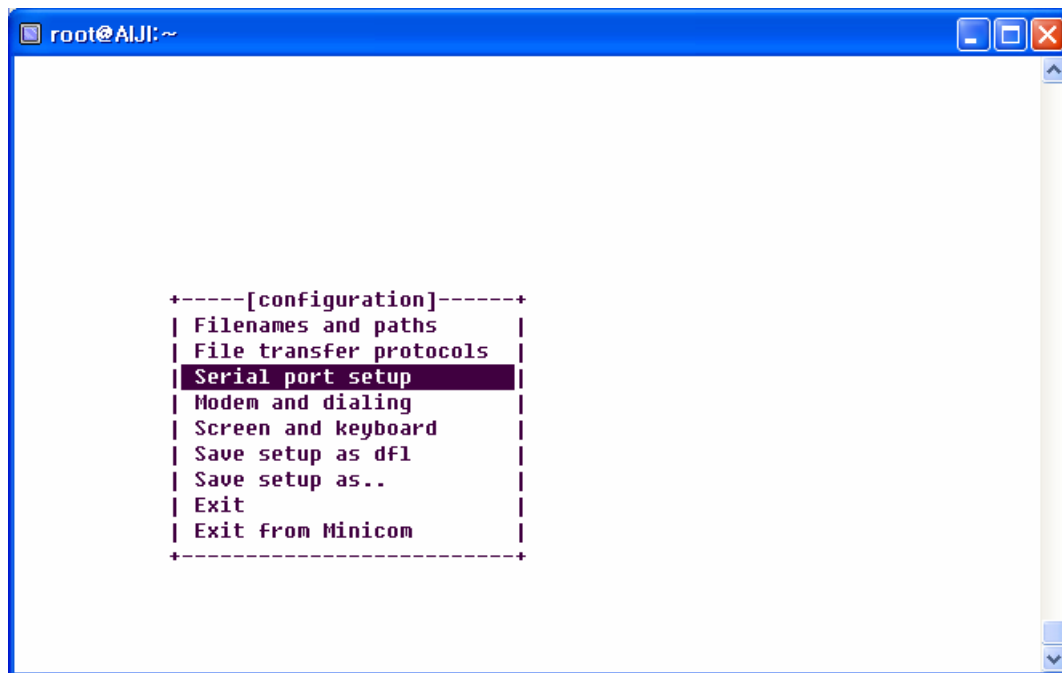
Please test it yourself.

3.3.4 Minicom Setup

Minicom on Linux is a utility taking a role of monitor for MBA2440 as a serial communication emulator like Hyper-terminal of Windows.

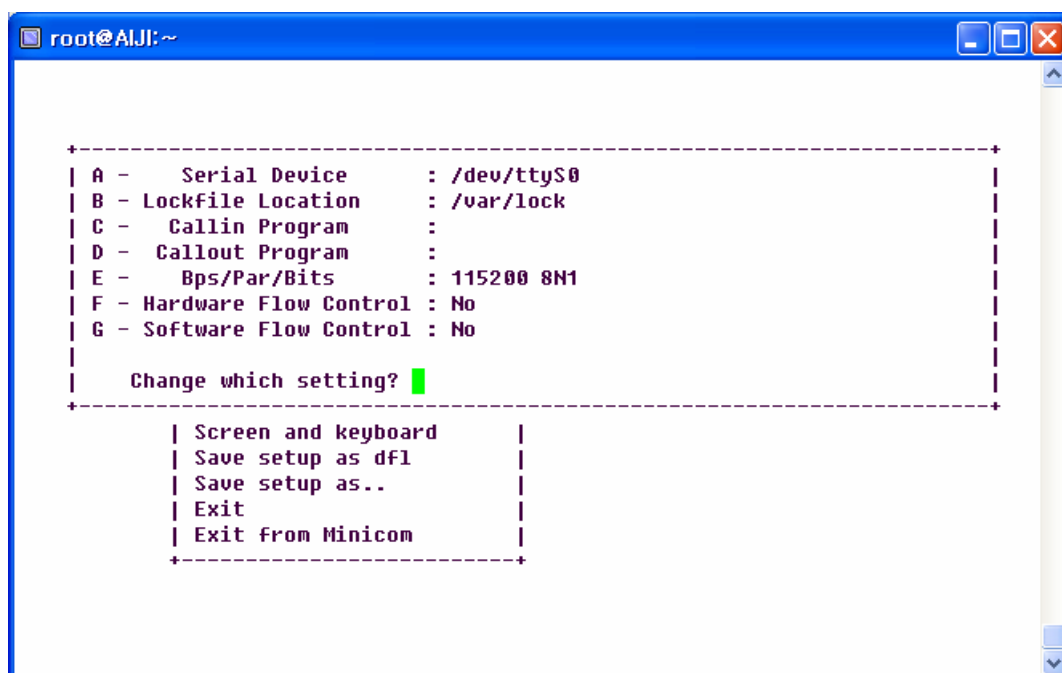
Perform the following command for Minicom Setup.

```
[root@localhost]# minicom -s
```



[Picture 3.15]

[Select serial port setup as shown on [Picture 3.15]. In case of COM1 on PC, it is /dev/ttyS0, and in case of COM2, it is /dev/ttyS1.



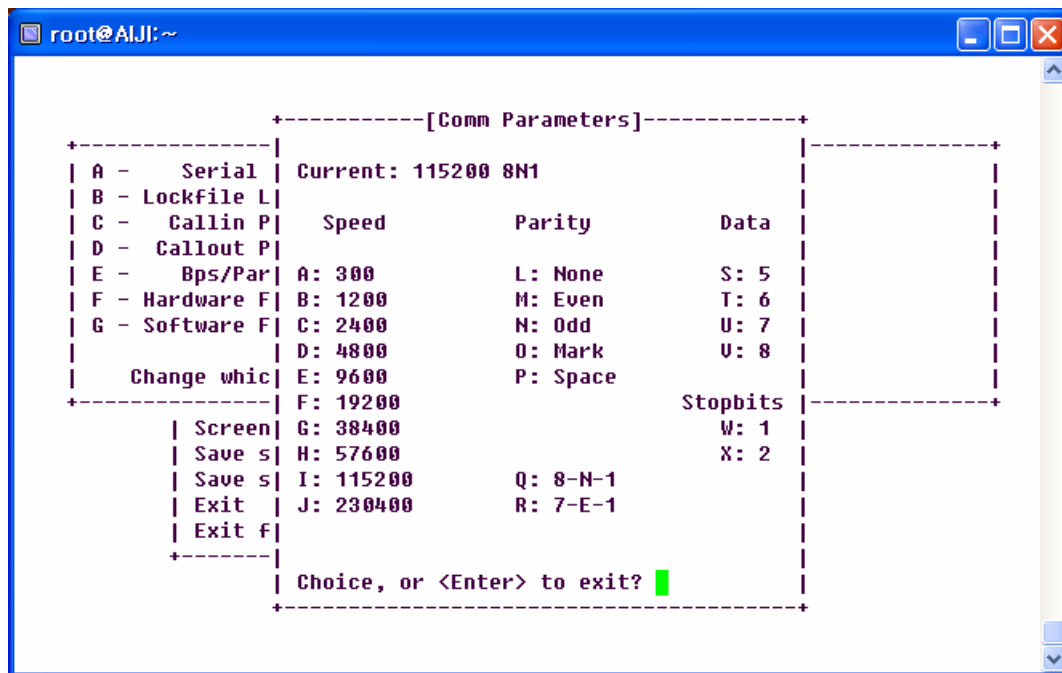
[그림 3.16]

Set up Parameters.

The value which should be set up :

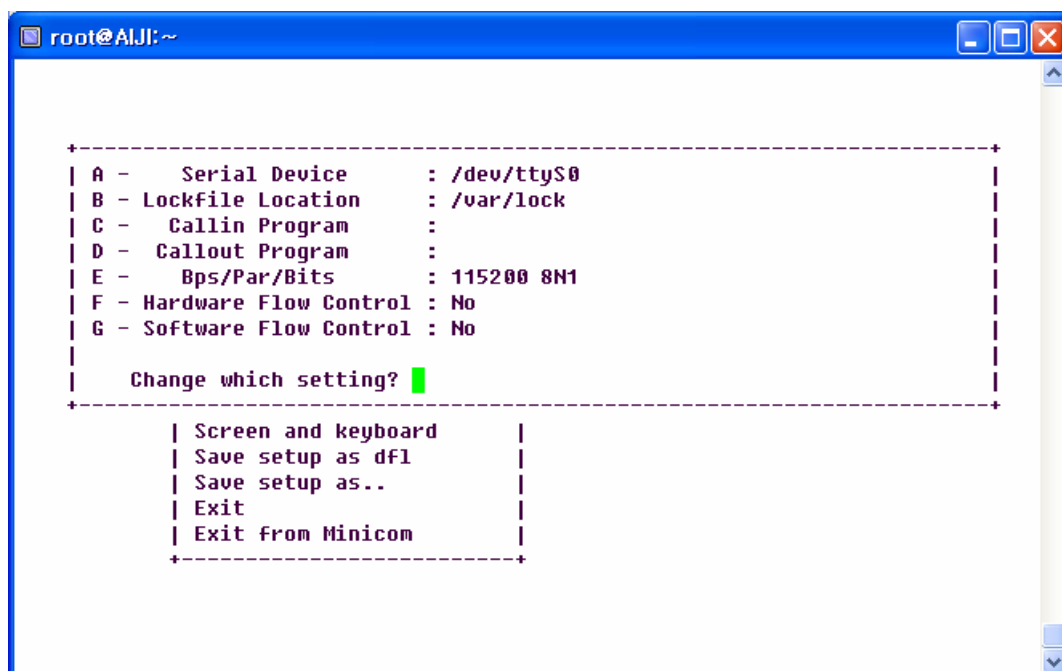
[Baudrate(speed) : 115200 Parity : none Data bits : 8 Stop bits : 1]

If you push E, you will see as [Picture 3.17]



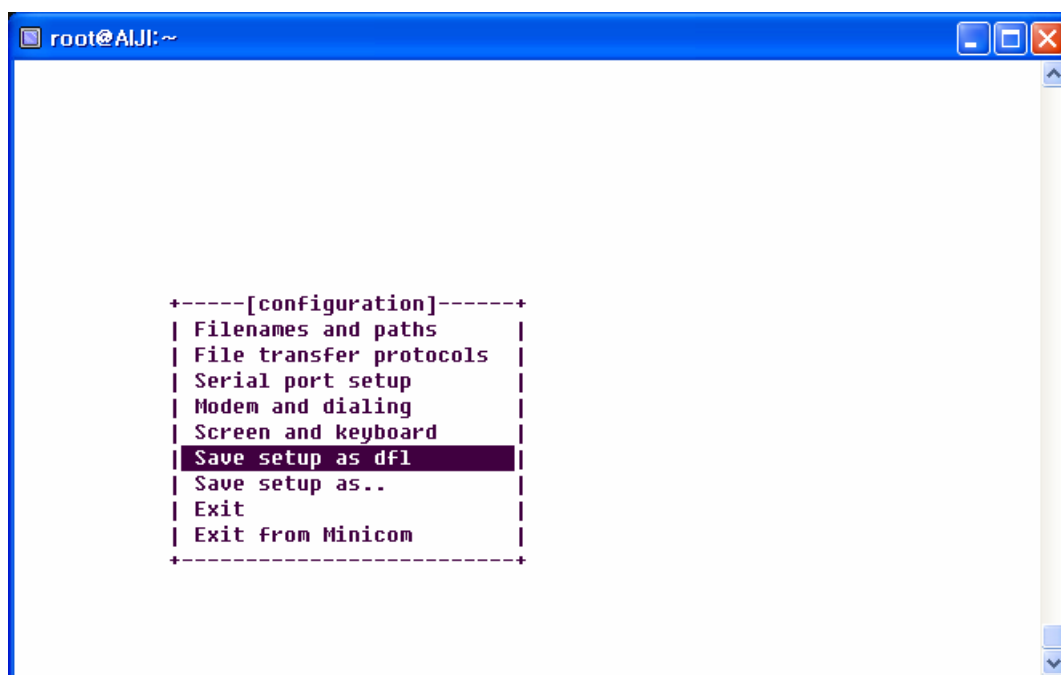
[Picture 3.17]

Push I (select 115200) and Q (select 8-N-1) and push the enter key.



[Picture 3.18]

Setup Hardware Flow Control and Software low Control as No by pushing F and G next.

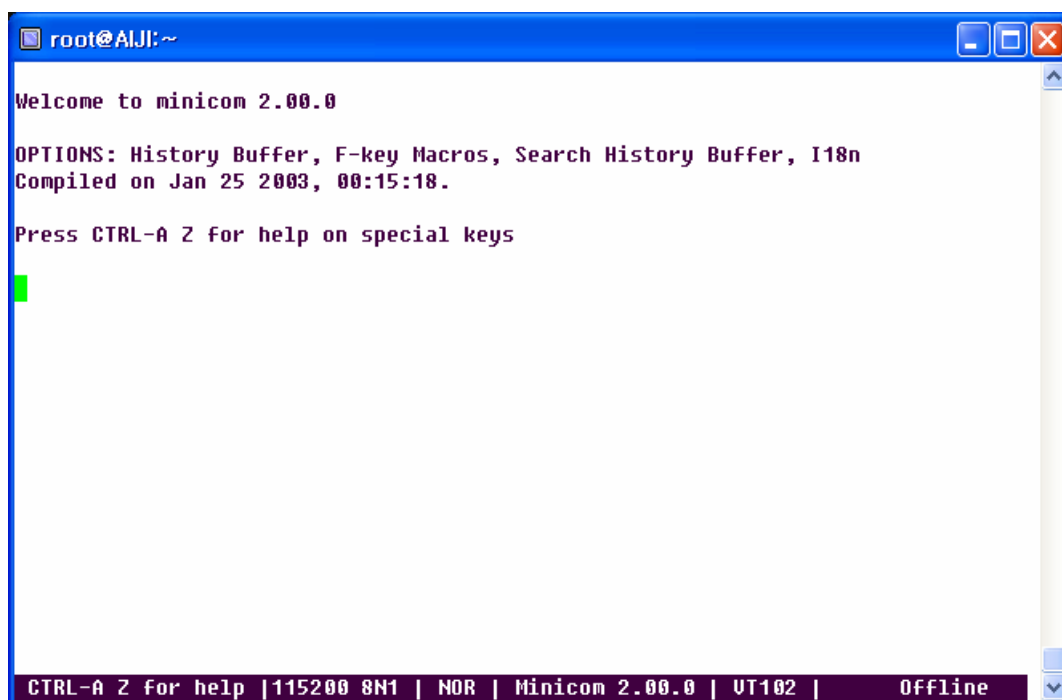


[Picture 3.19]

Save the setup. Select 'Save setup as df1' and push the enter key. If the save is completed, select 'Exit from minicom', push the enter key, and get out of there.

From now on, minicom will be operated according to the value set up only if you input the command, minicom.

```
[root@localhost]# minicom
```

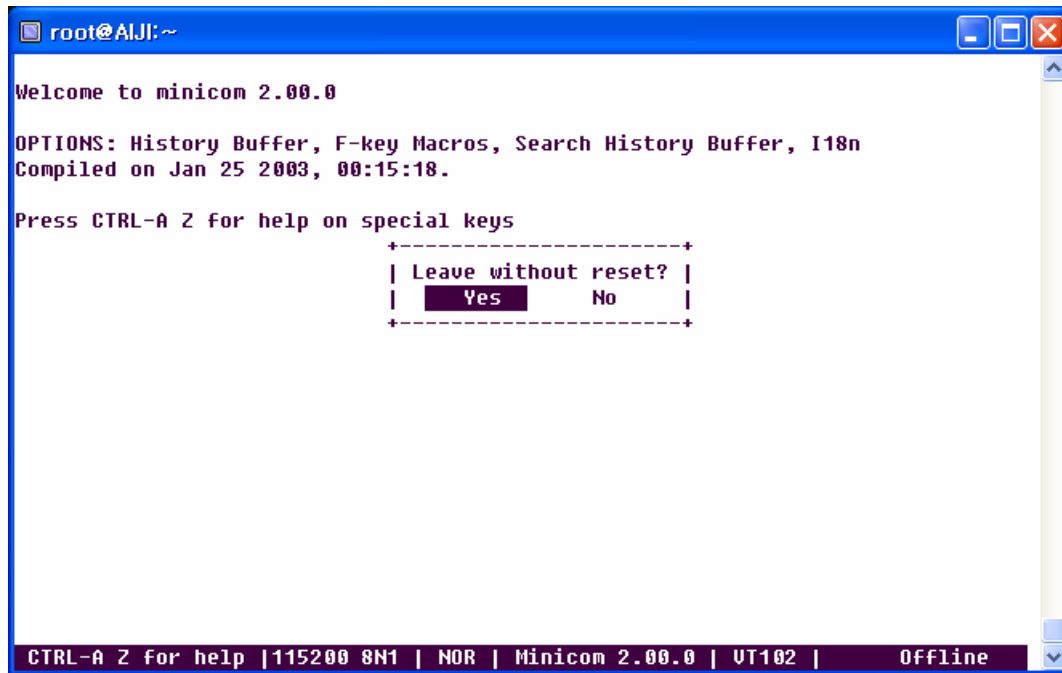


[Picture 3.20]

If you push Z after Ctrl + A, minicom menu as the following will be shown.

We don't explain about each menu. Please do it yourself.

If you push X after Ctrl + A while minicom is operated, minicom will be terminated.



[Picture 3.21]

3.4 WinCE Development Environment

There are images compiled and BSP for WinCE 5.0 in WinCE 5.0 directory in CD to develop WinCE for MBA2440.

WinCE 5.0 platform builder is needed to develop WinCE. Developers shall purchase it themselves, and we don't explain how to install it in this document.

Please refer to the chapter [7. WinCE](#) for how to compile WinCE 5.0 BSP for MBA2440 and how to boot WinCE on MBA2440 using eboot or nboot.

3.5 OPENice Debugger Equipments

In this chapter, we will introduce MDS equipments developed by AIJI System and teach how to upload and activate operating files in flash memory or the memory of MBA2440.

3.5.1 OPENice-A900, OPENice-A1000

OPENice is the only MDS equipment in the nation developed by AIJI system in national technology. It has great capabilities enough to compare with other MDS equipments.

Characteristics of OPENice are as the following.

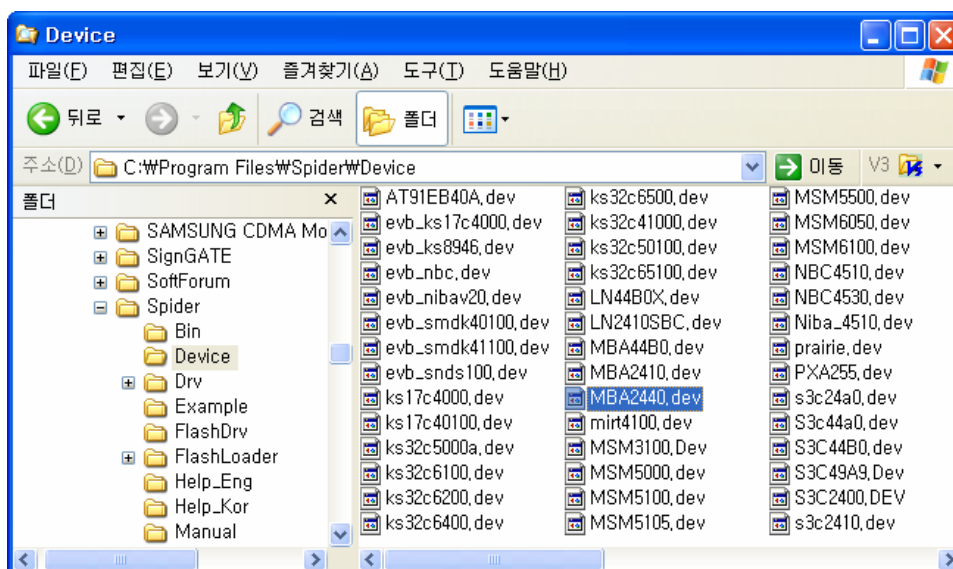
-
- ◆ Support various ARM core Processor
 - OPENice-A1000: ARM7, ARM9, ARM10, ARM11, XScale
 - OPENice32-A900¹: ARM7, ARM9, ARM10, XScale
 - ◆ Support CDMA/GSM/GPRS/UMTS Device
 - Support processor for cell phones based on ARM core
(Supporting in OPENice-A1000 and OPENice32-A900)
 - ¹ OPENice32-A900 doesn't support some devices of Xscale and new devices (in particular, multi core (ARM + non ARM based core))
 - ◆ Support multi core/processor
 - ◆ Provide convenient GUI SMU/SFR setup window
 - ◆ JTAG Interface
 - ◆ High-speed flash downloading and debugging
 - Supporting in OPENice-A1000 and OPENice32-A900
 - ◆ Stand Alone Flash programming (128MB)
 - Support only OPENice-A1000
 - ◆ High-speed image downloading through USB
 - USB 2.0: OPENice-A1000
 - USB 1.1: OPENice32-A900
 - ◆ Simple firmware upgrade
 - ◆ Provide an exclusive debugger (Spider) and compatible with other debuggers
 - Exclusive debugger : Spider – for OPENice-A1000 and OPENice32-A900
 - Compatible debugger: ARM SDT/ADS, IAR EWARM, Linux GDB,
Windows CE Platform Builder

¹ OPENice32-A900 doesn't support some devices of Xscale and new devices (in particular, multi core (ARM + non ARM based core)).

3.5.1.1 Install Device file

After you install Spider, a debugger for OPENice, you have to install device file of MBA2400 for Spider. The device file includes information to set up S3C2440 registers for reset when controlling MBA2440 through OPENice.

Copy **MBA2440.dev** in utiliy/devicefile directory in CD into C:/Program files/Spider/Device directory in.



[Picture 3.22]

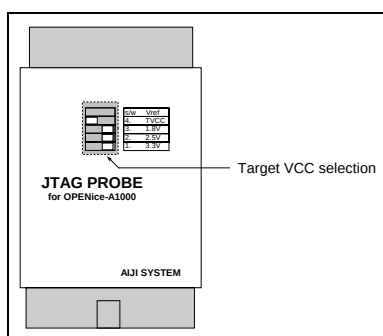
3.5.1.2 Connect with PC

OPENice communicates with PC through USB. After the power of OPENice turns on, install drivers on USB driver install screen and use it. Please refer to "Install products" in OPENice manual for details

3.5.1.3 Connect with MBA2440

Basically, OPENice can be used connecting with JTAG connector consisted of 20 pins or 14 pins of target board.

In the case of OPENice-A1000, there exists JTAG prob. Perform the jumper setting of probe as the following.



[Picture 3.23]

Connect JTAG cable of OPENice with JTAG port of MBA2440.

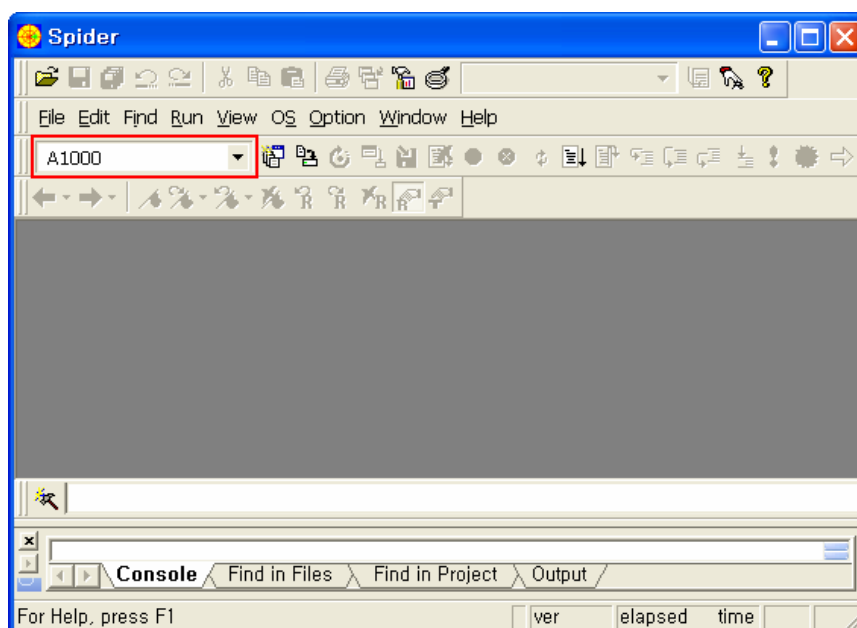
3.5.2 Write image on Boot flash

Let's just say the environment setup for OPENice is completed, and learn about how to write image on boot flash (AMD flash or INTEL flash). It doesn't provide NAND flash write and please use U-boot.

Please refer to the OPENice manual for the environment setup of OPENice.

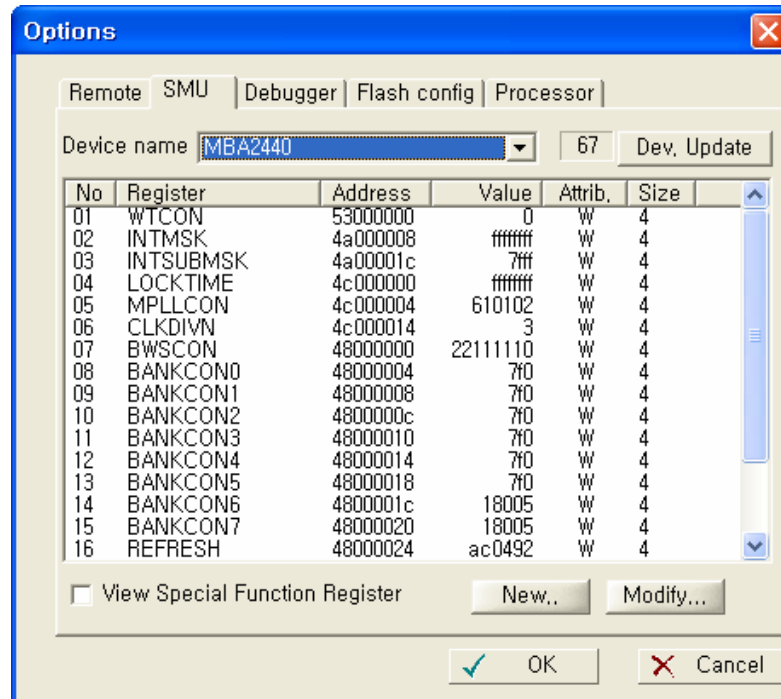
3.5.2.1 Write Image on AMD flash

1. Connect cables
2. set up MBA2440 boot flash as AMD flash (please refer to [1.3.2.1 Use NOR flash as boot flash](#))
3. Turn on the power of MBA2440
4. Select OPENice related on the upper left part after operating Spider
A1000 is selected in this document.



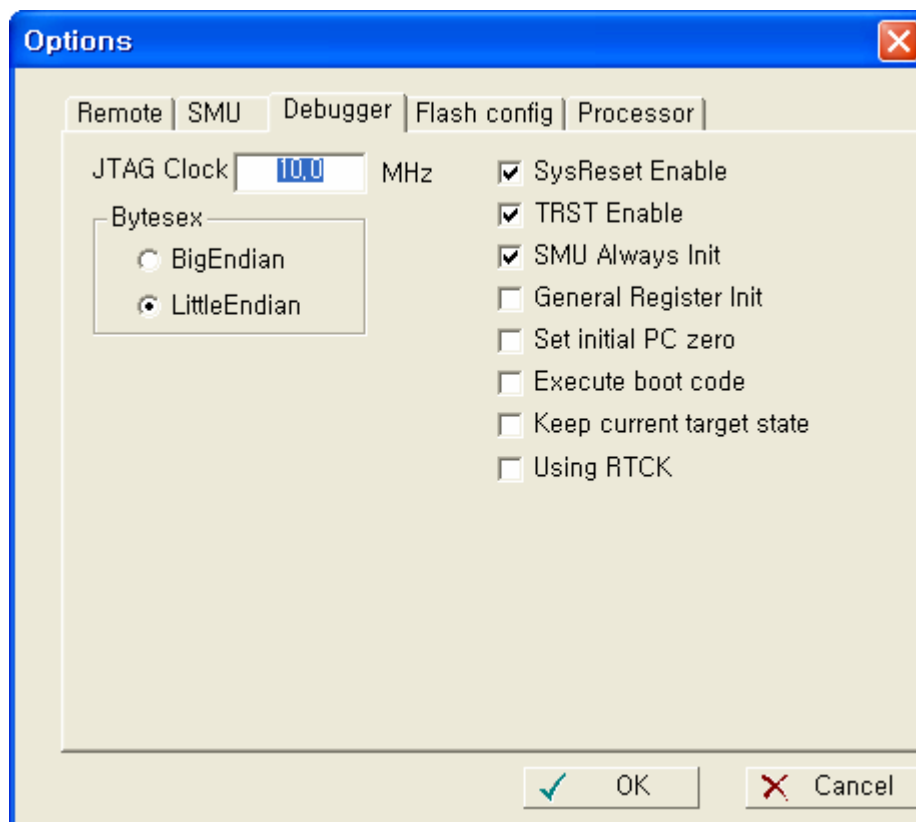
[Picture 3.24]

5. If you select Option->Configure Interface (Cntl+ E) on Spider, options window will be shown. Now choose MBA2440 on SMU tab.



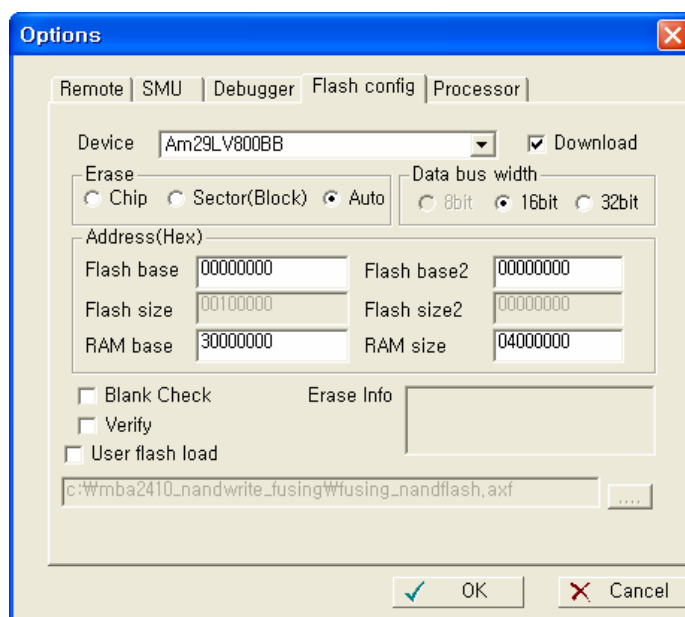
[Picture 3.25]

6. Set up debugger tab on options window as [Picture 3.26]



[Picture 3.26]

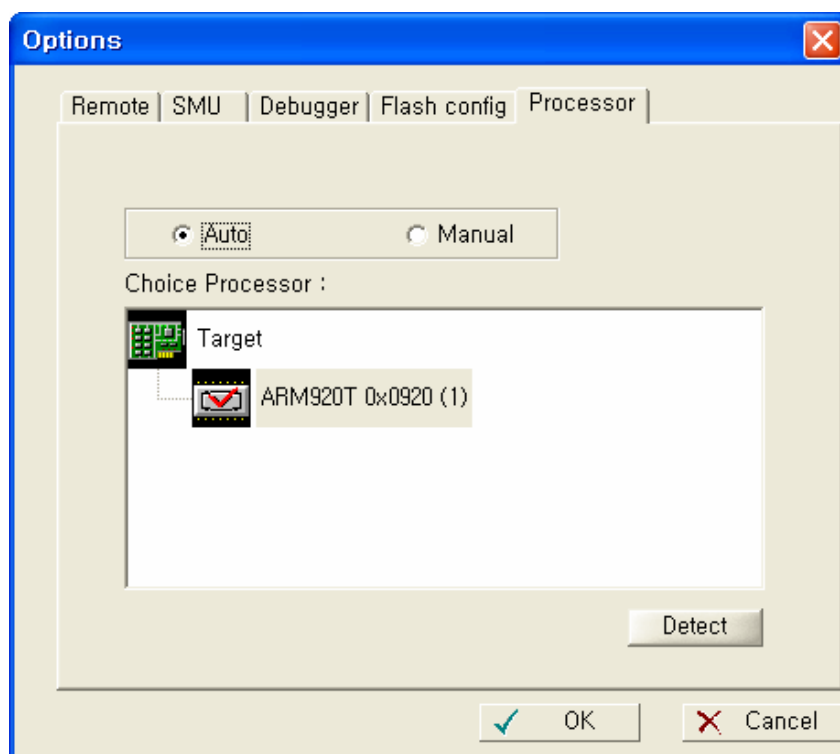
7. Set up Flash config tab on options window as [Picture 3.27]



[Picture 3.27]

You can confirm that AMD flash is selected as device on [Picture 3.27], download is checked and erase is set up as Auto, Data bus width as 16bit, Flash base as 0x0, RAM base as 0x30000000 and RAM size as 0x40000000(64MB).

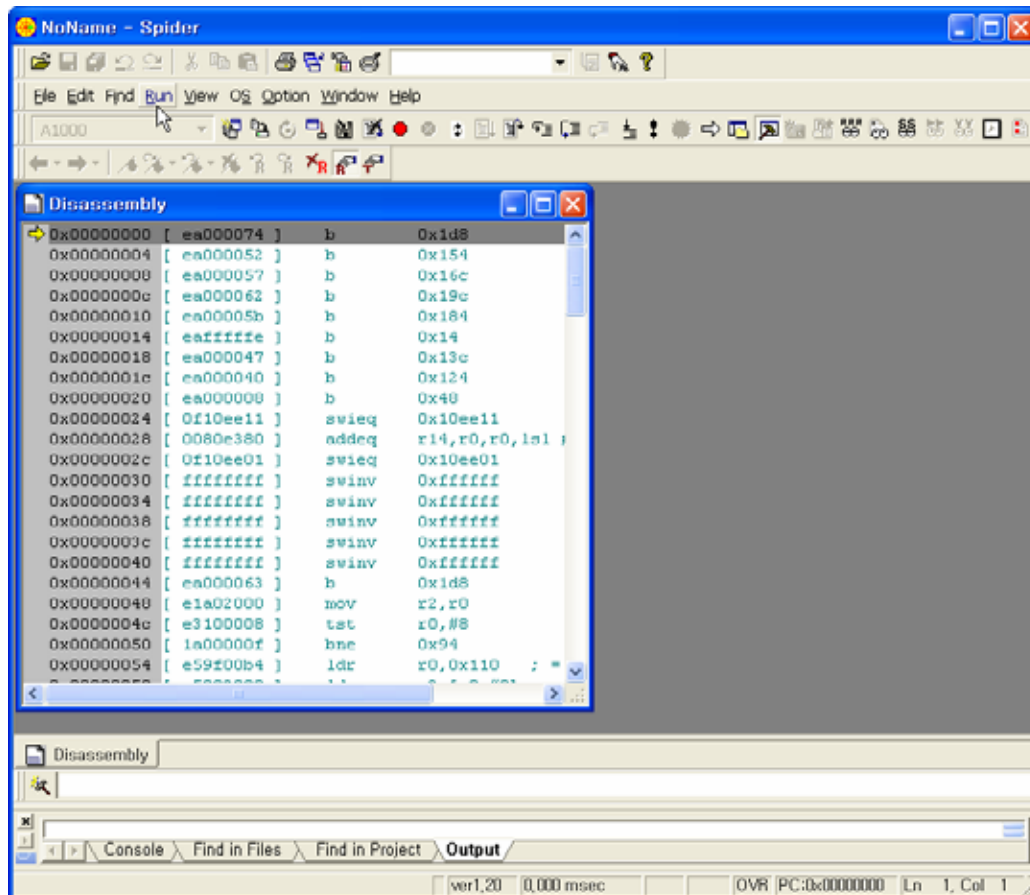
8. Detect ARM920T core on Processor tab of Options window.



[Picture 3.28]

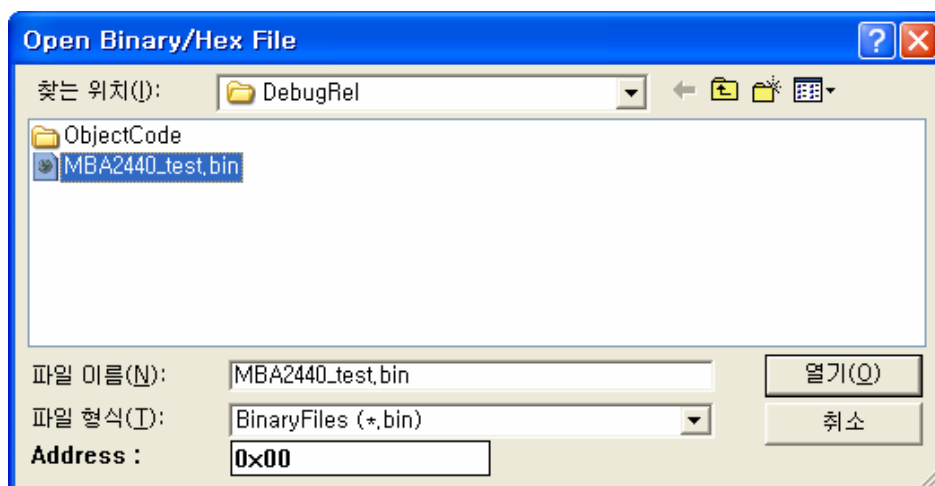
Push the detect button on [Picture 3.28]. If PC and OPENice / OPENice and MBA2440 are connected properly, you can see as [Picture 3.28].

9. If you push Ok button on Options window, you can see it reset as shown on [Picture 3.29]



[Picture 3.29]

10. Select File->Download Bin/Hex File on Spider



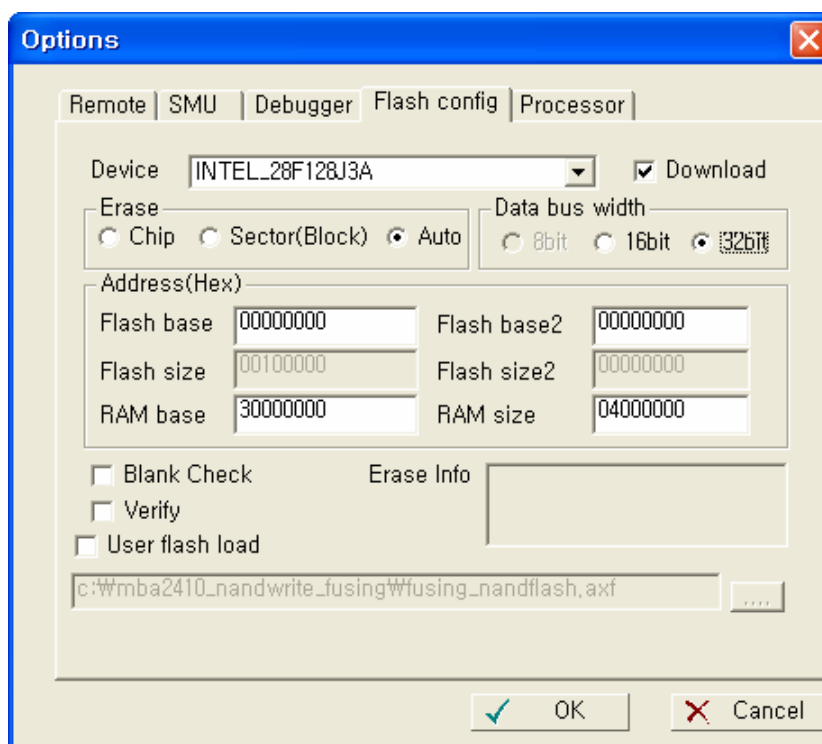
[Picture 3.30]

Select the file you want as shown on [Picture 3.3]. Write the address of location you want in AMD flash on address. If you want to write 0x0 address, write 0x0 address as the picture.

If you push open button, you can write after erasing the area which will be written in AMD flash on Spider.

3.5.2.2 Write Image on INTEL flash

If INTEL flash is an optional matter of MBA2440. If INTEL flash is attached in MBA2440, you can follow [3.5.2.1 Write image on AMD flash](#). Only for number 7, please set up as [Picture 3.31]



[Picture 3.31]

Select device as INTEL_28F128J3A, and please remember Data bus width should be 32bit.

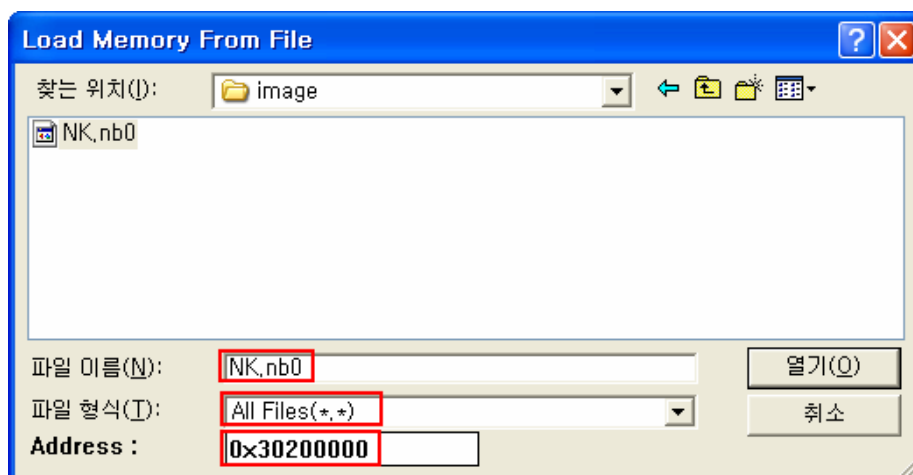
Besides, you can also write on NOR flash using FlashUp utility provided when Spider installed. Please refer to Spider manual for details.

3.5.3 Load image in Memory

In case of WinCE image, load it in 0x30200000 address of SDRAM memory, set PC value as 0x30200000 adress, and select Run->Go of Spider. And then, WinCE will boot.

Let's learn how to upload image directly in SDRAM memory.

1. Choose Option->Configure Interface in Spider and follow number 1 and 9 of [3.5.2.1 write image on AMD flash](#)
2. Choose File->Load Memory From File on the initial page of Spider
3. Choose WinCE image lik [Picture 3.32] on Load Memory From File window

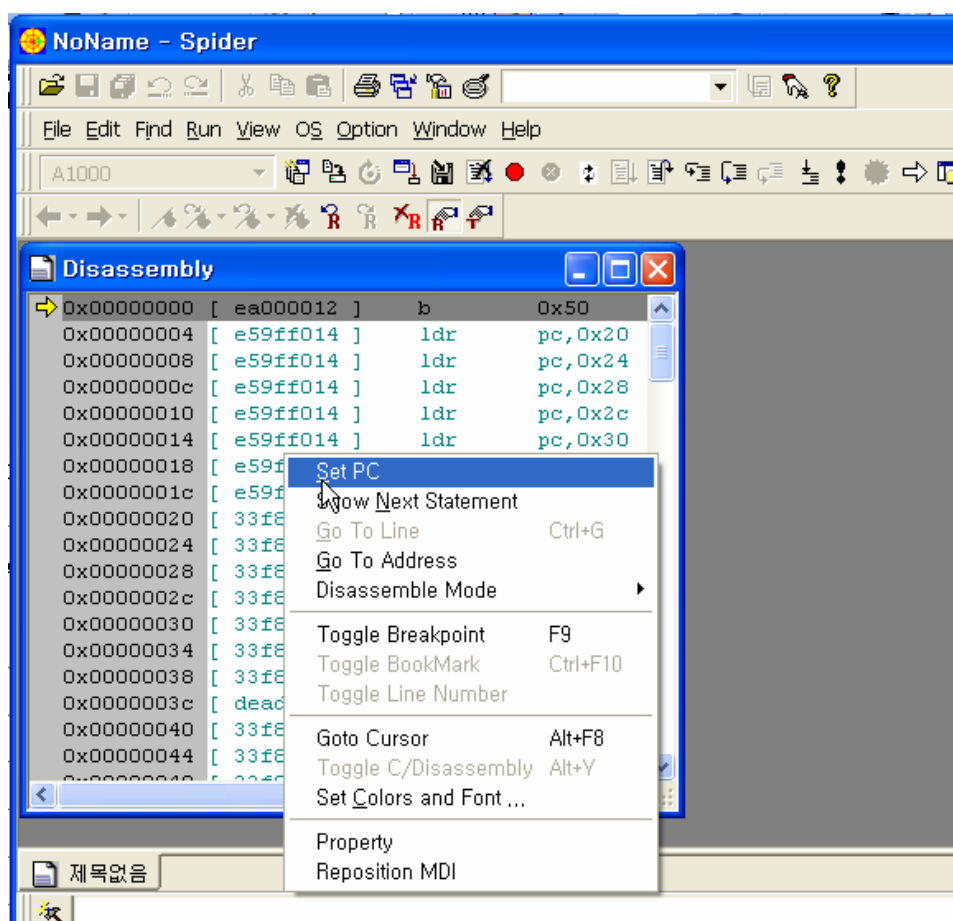


[Picture 3.32]

Set up an address as 0x30200000, the address which will be loaded.

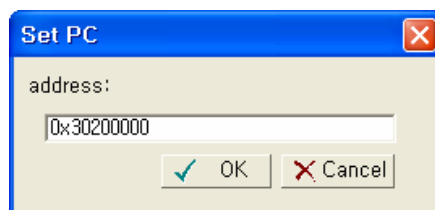
If you push the open button, the WinCE image, NK.nb0 will be loaded to 0x30200000 address. It takes some time to download because the size of image is big.

4. Set up PC value as 0x30200000 after download is completed.



[Picture 3.33]

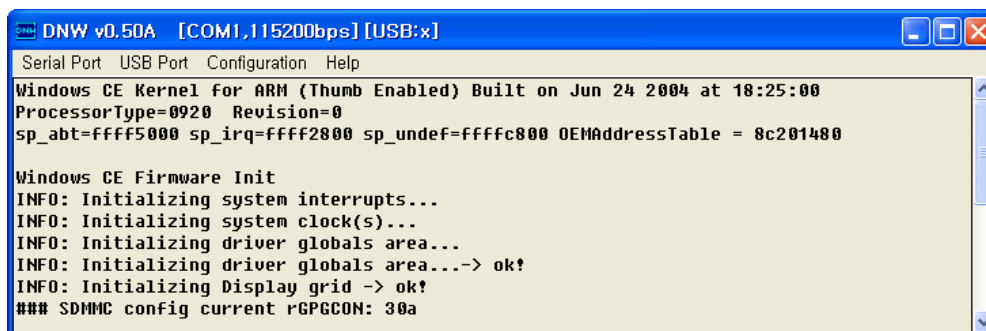
Put the mouse pointer on Disassembly window, click the right button, and click 'Set PC' as shown on [Picture 3.33].



[Picture 3.34]

After setting 0x30200000 address on Set PC window like [Picture 3.34], push the OK button.

5. If you choose Run->Go of Spider or push F8, you can confirm WinCE image boots.

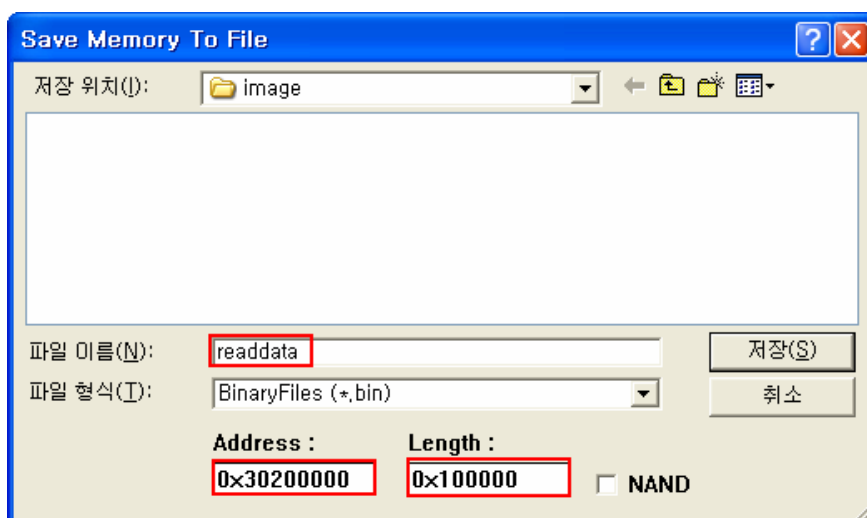


[Picture 3.35]

3.5.4 Read Data in Memory

You can bring data in SDRAM memory of MBA2440 to PC.

1. Choose Option->Configure Interface on Spider, and follow number 1 and 9 of [3.5.2.1 Write image on AMD flash](#).
2. Choose File->Save Memory To File on the initial page of Spider.
3. Set up the booting address of the area you want to bring and read from Save Memory To File window, size and file's name which will be saved.



[Picture 3.36]

Input the booting address of the area you want to bring and read as the address, input the size

on Length, and input the name of file which will be saved on file's name.

If input is completed, push the save button. Please wait long enough until the completion, because it could take much time (if long, 30 ~ 40 minutes).

3.5.5 Matters to be Attended when using OPENice

Let's learn about matters to be attended when connecting OPENice with MBA2440.

1. In case OPENice and MBA2440 are connected through JTAG

- Reset button doesn't work

If you push reset button of OPENice, MBA 2440 will reset together.

- if you turn on the power after turning it off, boot code on boot flash doesn't work

If you want to operate boot code, set up the address as 0 x 0 on Set PC window, and push F8 key.

2. If you choose Option->Configure Interface on Spider and push Ok button on Option window, MBA2440 will reset

But if it doesn't reset and you can see error message printed out, you need to check.

- 518 error message

This error means the case that SysReset Enable is not checked on Debugger tab on Option window or JTAG reset signal is not linked properly.

- 683 error message

You need to check the device file on SMU tab of Option window if it is right for target board

- 778 error message

You need to check if the power of board is on and CPU core is detected properly on Processor tab of Option window.

4 MBA2440 Test Program

In this chapter, you will learn about MBA2440 test firmware programs in detail.

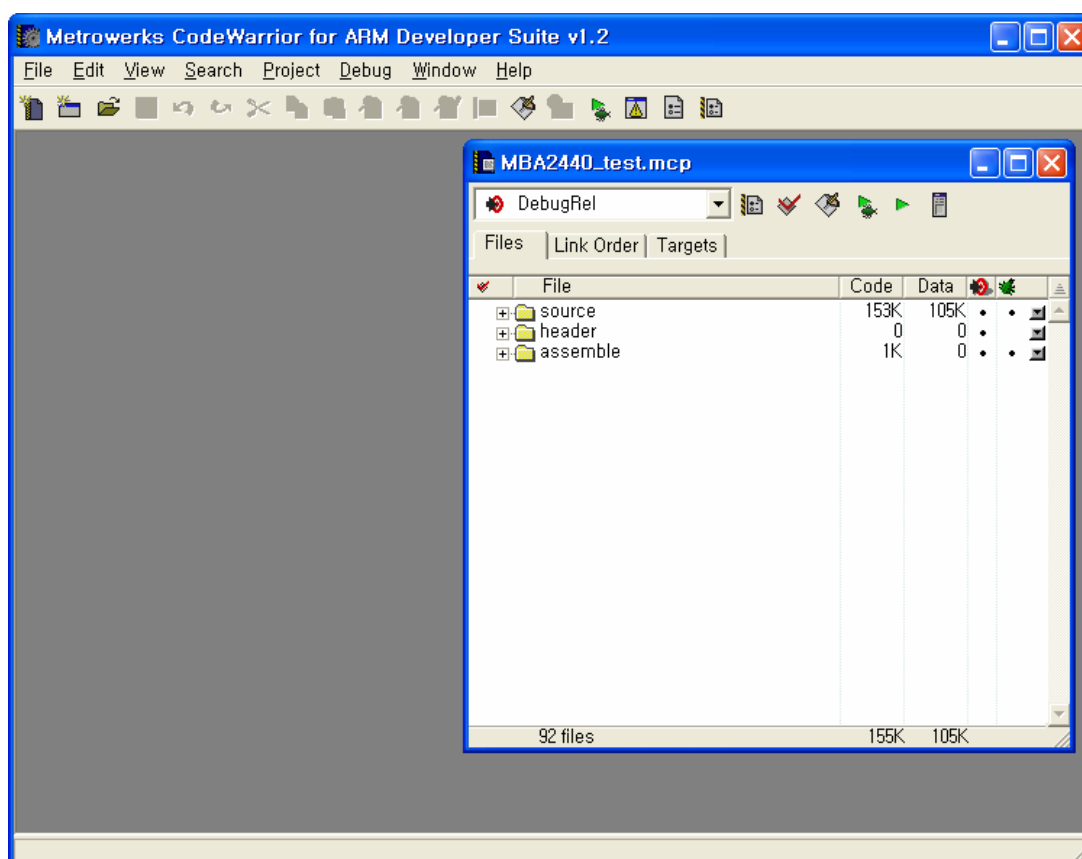
4.1 Compile Programs

MBA2440 test programs have been developed as ADS 1.2. **In this chapter, we will explain about it assuming ADS1.2 is installed in PC and you know how to use ADS1.2.**

There is a compressed file, **MBA2440_test_v2.0.zip** in firmwares/test_program directory in CD. The compressed file is the source of MBA2440 test program.

If you extract it, MBA2440_test_v2.0 directory will be created.

There is **MBA2440_test.mcp** file in 2440_test_v2.0/MBA2440_test directory. If you double-click the file, ADS1.2 will be operated, and you can see project window created already. [Picture 4.1]



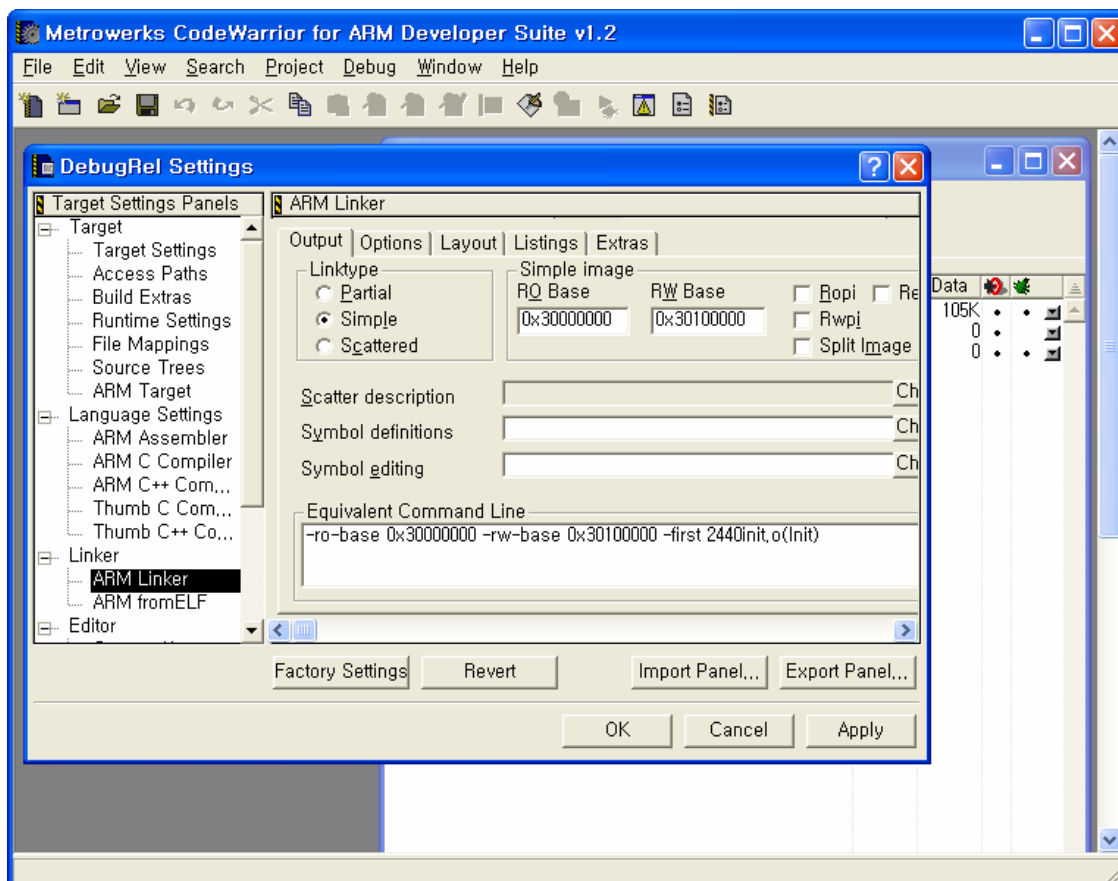
[Picture 4.1]

The source is compiled setting up RO_BASE as 0x0 and RW_BASE as 0x30000000 in compile options. Please be aware that it works only if you write images compiled on 0x0 address.

If you want to operate it in SDRAM, not in boot flash, set up RO_BASE as 0x30000000 (this

address is the opening address of SDRAM address) and RW_BASE as 0x30100000.

The reason setting up RW_BASE as 0x30100000 is that test program images are usually loaded in 0x30000000 ~ 0x3003ffff area. If other data are written in this area, the test program doesn't work. Set up RW_BASE as 0x30100000 to empty about 1M of area for enough memory space besides 0x40000 sized test program image area. Please analyze the reset part (2440init.s) of source for details.



[Picture 4.2]

4.1.1 Set up Test Program

Source provided CD is adjusted to SDRAM 64MB.

You need to modify parts related to memory among source if SDRAM is MBA2440 board extended to 128MB.

You need to modify Option.inc file and Option.h file among source provided.

Modify Option.inc file first.

Contents in the file are matters which need to be referred for assembly code to reset the board.

```
=====
```

```
; NAME: OPTION.A
; DESC: Configuration options for .S files
; HISTORY:
; 02.28.2002: ver 0.0
; 03.11.2003: ver 0.0    attached for 2440.
; jan E, 2004: ver0.03   modified for 2440A01.
; 04.15.2005: modified for MBA2440
;=====
                GBLA   MBA2440_MEM_SIZE
;MBA2440_MEM_SIZE  SETA   0x4000000
MBA2440_MEM_SIZE  SETA   0x8000000
```

The part defining MBA2440_MEM_SIZE macro modifies the value defined as 0x4000000 to 0x8000000.

“;” mark means treating as a note in assembly code.

Modify Option.h next.

```
/******
NAME: option.h
DESC: To measure the USB download speed, the WDT is used.
      To measure up to large time, The WDT interrupt is used.
HISTORY:
Feb.20.2002:Shin, On Pil: Programming start
Mar.25.2002:purnnamu: S3C2400X profile.c is ported for S3C2440X.
Jan.E.2004:DonGo: Modified for S3C2440a.
04.15.2005: modified for MBA2440
*****/

#ifndef __OPTION_H__
#define __OPTION_H__

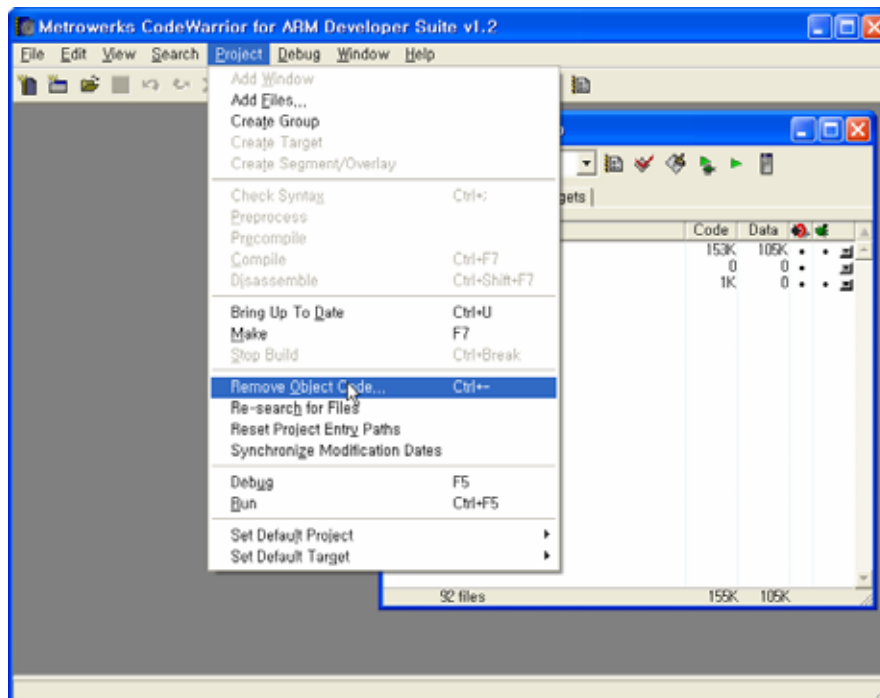
// #define MBA2440_MEM_SIZE          0x4000000
#define MBA2440_MEM_SIZE          0x8000000
```

The part defining MBA2440_MEM_SIZE macro modifies the value defined as 0x4000000 to 0x8000000.

4.1.2 Compiling

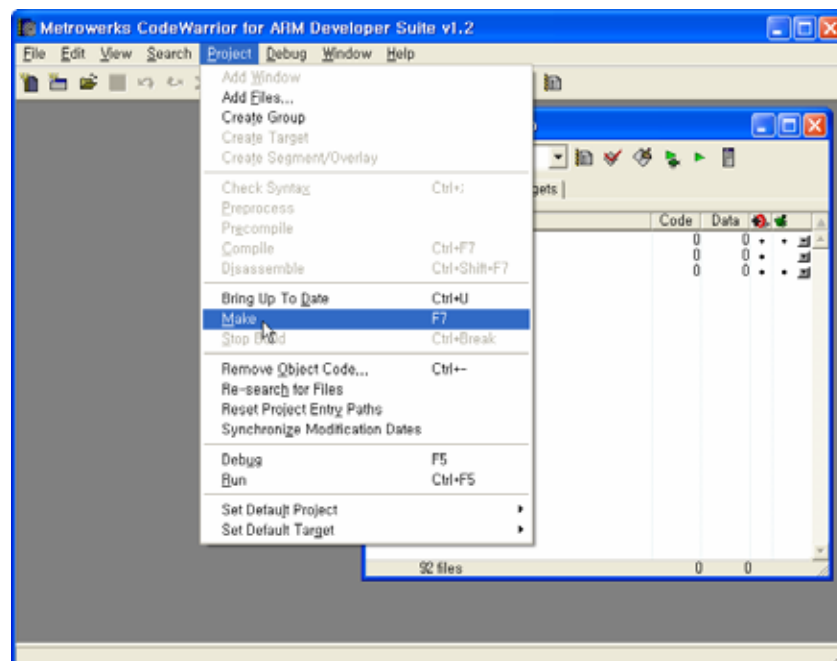
The system memory of source provided is compiled as 64MB. But let's compile it again.

Click 'Project->Remove Object Code' and click All Targets button on Remove Object Code window. [Picture 4.3]



[Picture 4.3]

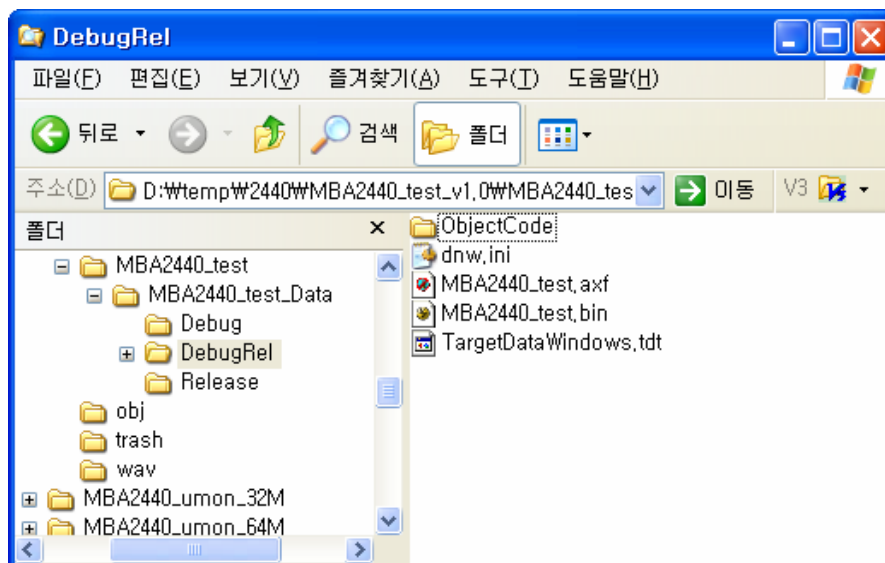
Compile it by choosing Project->Make or pushing F7 button. [Picture 4.4]



[Picture 4.4]

If compiling is completed,

You can confirm MBA2440_test.bin file and MBA2440_test.axf file has been created in MBA2440_test_v2.0/MBA2440_test/MBA2440_test_Data/DebugRel.



[Picture 4.5]

If the image file, MBA2440_test.bin is compiled setting up RO_BASE as 0x0, load it in 0x30000000 address of memory and operate it.

If RO_BASE is set up as 0X30000000 and compiled, load it into 0X30000000 address and operate it.

You can refer to how to use OPENice ([3.5.3 Load image in memory](#)) or how to use u-boot ([5.2.2.4 NOR flash command](#)) for how to operate it after loading it in memory.

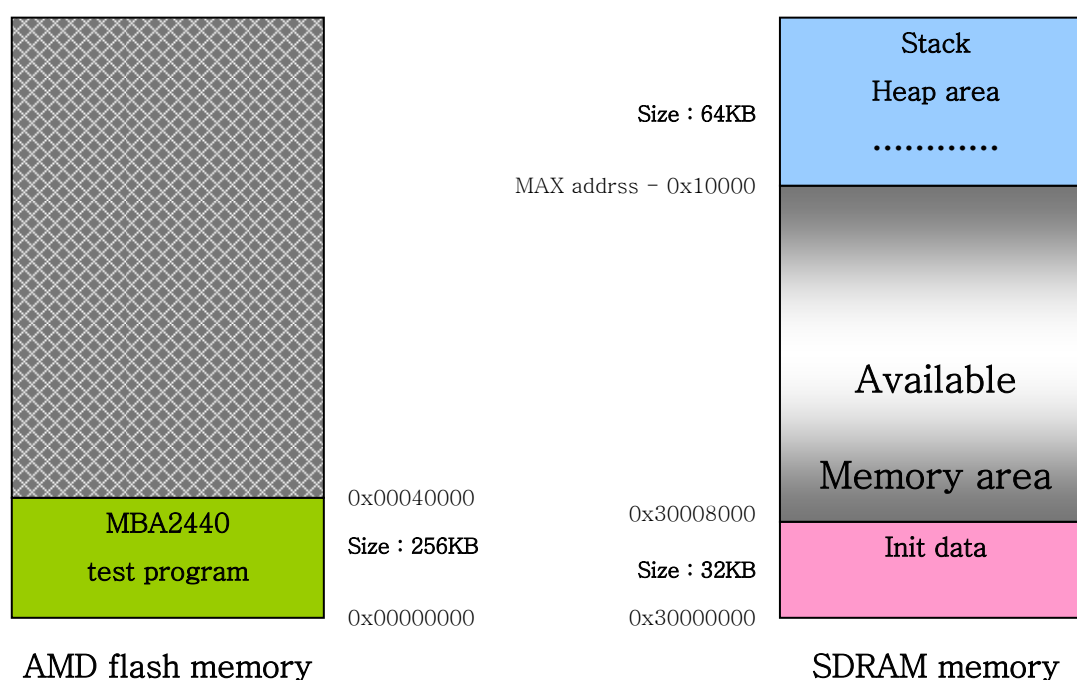
4.2 Matters to be Attended for MBA2440 Test Program

MBA2440 test program is programmed to operate properly under the environment setting up AMD flash as boot flash.

The memory map of test program becomes different according the following two cases, and it determines if AMD flash can be tested in NOR flash test menu.

4.2.1 In case of setting up 0x0 address as RO_BASE

If you set up RO_BASE as 0x0 when compiling source, it will construct the memory map as shown on [Picture 4.6]. (In this case, RW_BASE sets up 0x30000000, the base address of memory.)



[Picture 4.6]

In case of setting up RO_BASE as 0x0 and compiling as shown on [Picture 4.6], MBA2440 test program code will operate on 0x0 ~ 0x3ffff address of AMD flash.

The size of MBA2440 test program image is approximately 256KB (0x40000), however, it will be different in case developers modify or add program.

But its reset data or heap, stack and etc. exist on SDRAM.

In this case, please remember AM29LV800 test menu doesn't work properly on NOR test menu,

because MBA2440 test program code works on AMD flash.

Also in case of SDRAM, the area where reset data part, heap, stack and etc. are constructed can't be used. If this area is modified, MBA2440 test program could stop working..

When you download files or images on USB download menu, please download them in available space of SDRAM.

4.2.2 In case of setting up 0x30000000 address as RO_BASE

In this case, parts except 0x30000000 ~ 0x300fffff address on the memory map of SDRAM and 0x10000, the highest of SDRAM can be used.

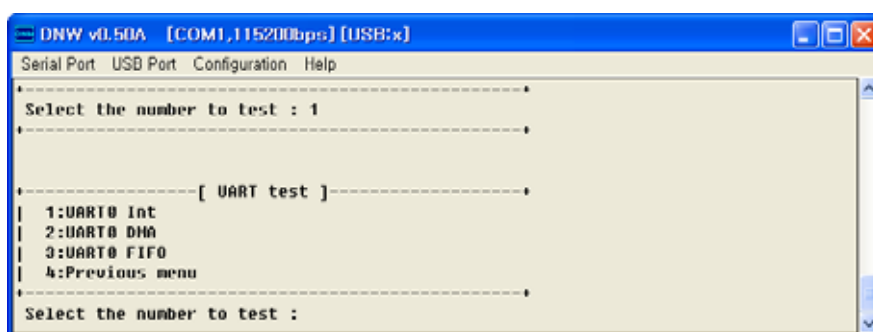
AMD flash test menu will work properly, because MBA2440 test program code doesn't work on AMD flash.

When you download files or images on USB download menu, please download them in available space of SDRAM.

4.3 Test Command

In this chapter, we will explain about commands of MBA2440 test program. We don't explain it in detail. Please analyze test program source yourself for detailed operation of those commands.

4.3.1 UART



[Picture 4.7]

[UART test] menu is to test UART.

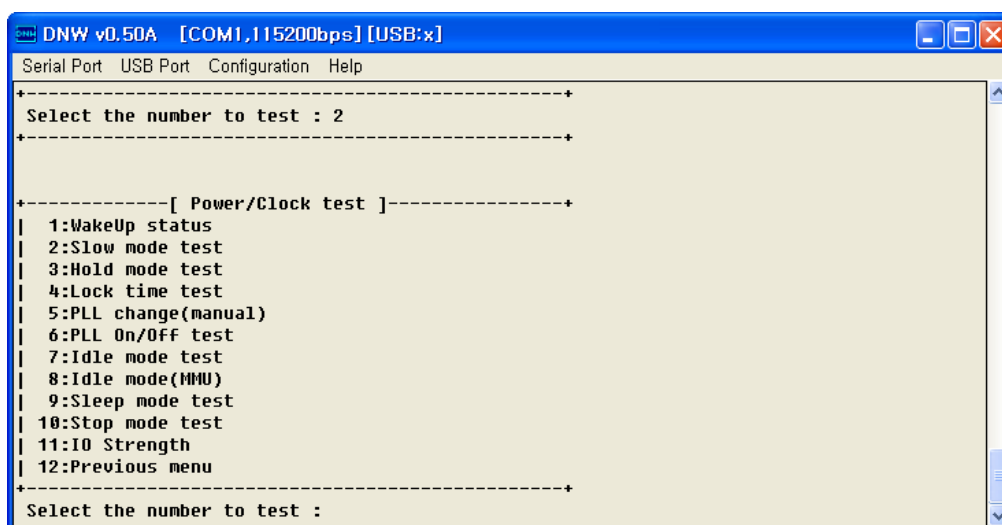
Test it operating UART0 as interrupt mode, DMA mode and FIFO mode for number 1~3 as shown on the menu.

After selecting each menu, test it according to the message printed out. Serial chip is connected only to UART0.

When you test UART1, you can test it connecting the signal of SERIAL[1] 6 pins connectors to serial chip.

UART2 is connected to IrDA.

4.3.2 Power/Clock

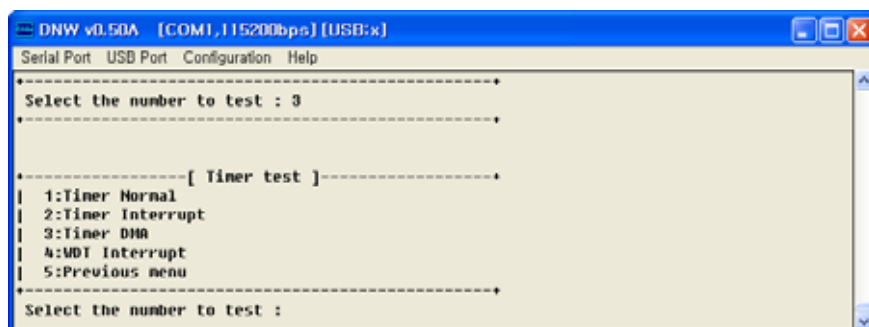


[Picture 4.8]

You can test CPU operation mode or CPU clock. After selecting each menu, test them

according to the message printed out.

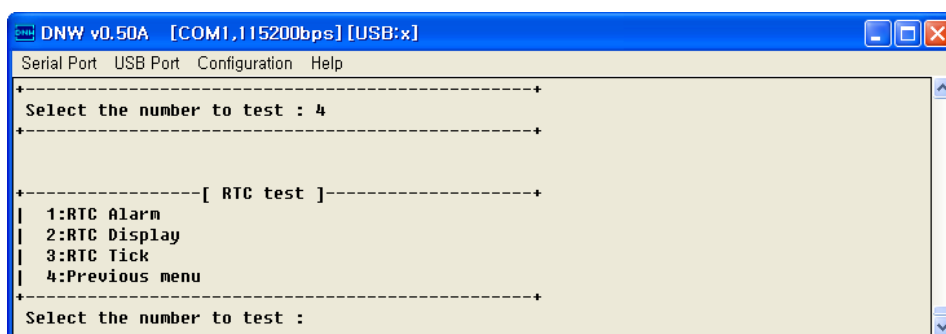
4.3.3 Timer



[Picture 4.9]

It's regarding timer test. After selecting each menu, please test it according to the message printed out.

4.3.4 RTC



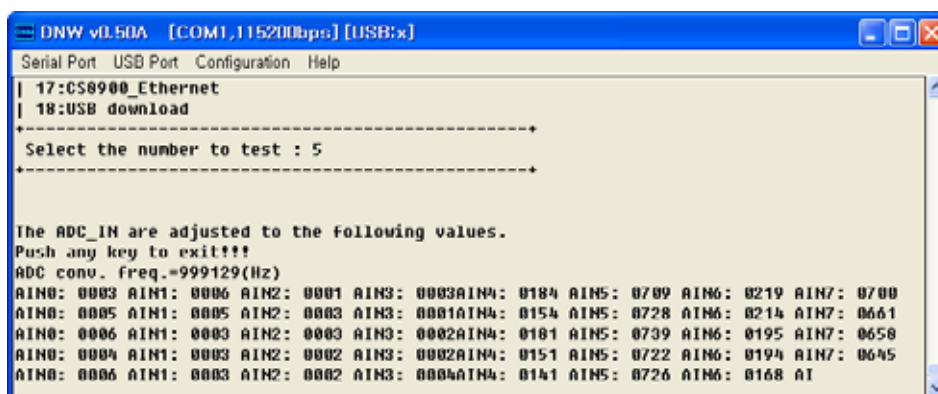
[Picture 4.10]

This is about RTC test. You can do alarm test using RTC.

After you set up time through the menu 2, you can see the time printed out every 1 sec. on the standard of the time set up.

After selecting each menu, please test it according to the message printed out.

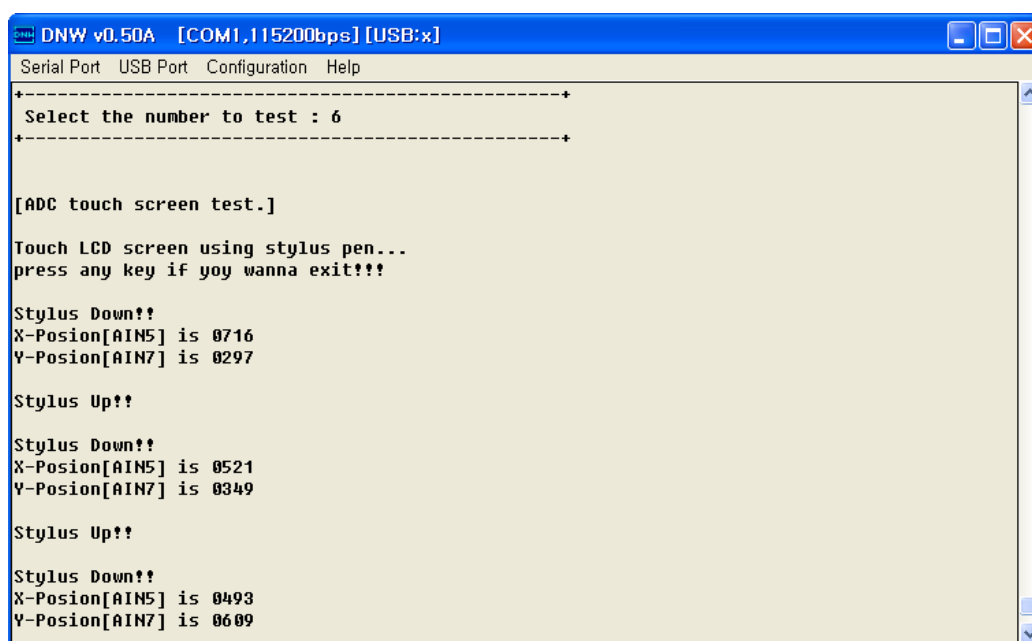
4.3.5 ADC test



[Picture 4.11]

This is about ADC test. Each Ain0 ~ 7 is adjusted with the value printed out at the same time as selecting a menu. If you push the enter key, it stops working.

4.3.6 Touch Screen test



[Picture 4.12]

This is about touch screen test. If you contact LCD with a stylus pen after selecting a menu, coordinates value is printed out.

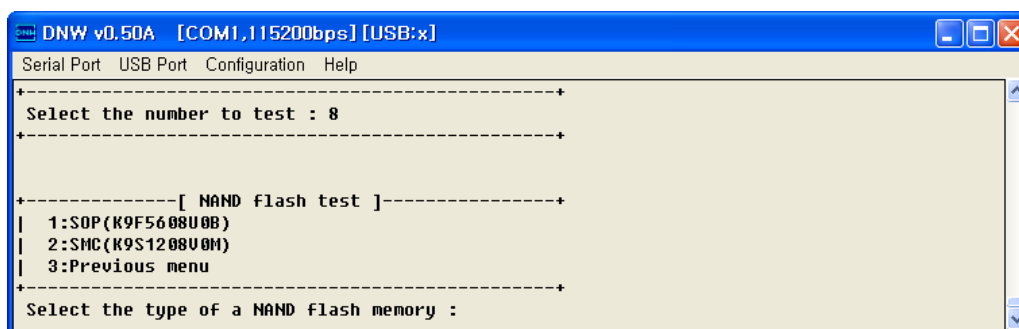
4.3.7 NOR flash test



[Picture 4.13]

This is about NOR flash test. If you try to write on NOR flash set up as boot flash or read ID, it doesn't work properly. Please remember this point.

4.3.8 NAND flash test



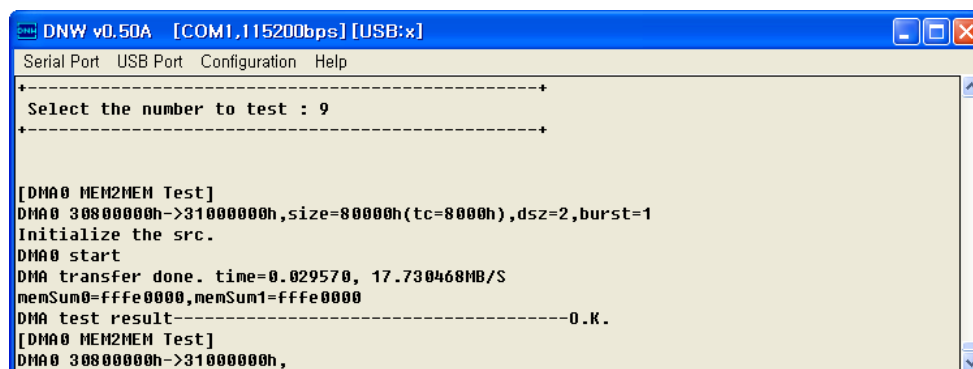
[Picture 4.14]

If you set up J3 switch as 1-2, you can control SOP(K9F5608U0B) NAND flash.

If you set up J3 switch as 2-3, you can control SMC(K9S1208V0M) NAND flash.

You should set up J3 to control NAND flash you want before operating MBA2440 tst program. Please remember the conversion of J3 switch during operating it isn't applied to the program.

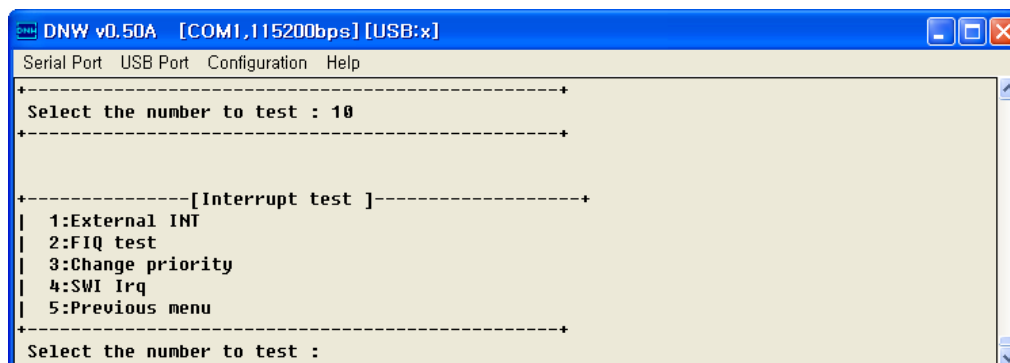
4.3.9 DMA



[Picture 4.15]

You can test DMA at the same time as selecting a menu.

4.3.10 Interrupt

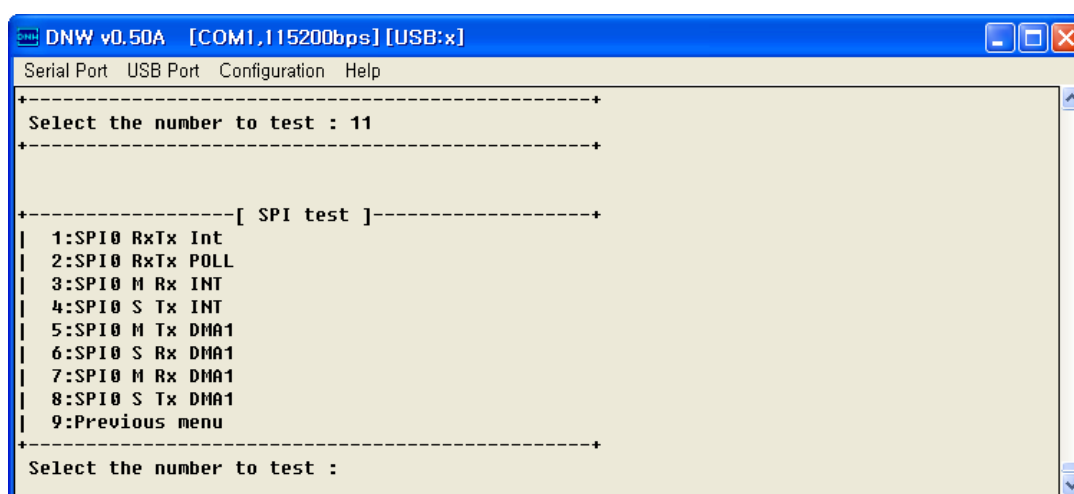


[Picture 4.16]

The menu 1 prints out a message if you set up the switch (Key 1~4) as external interrupt and push the button.

After selecting each menu, please test it according to the message printed out.

4.3.11 SPI



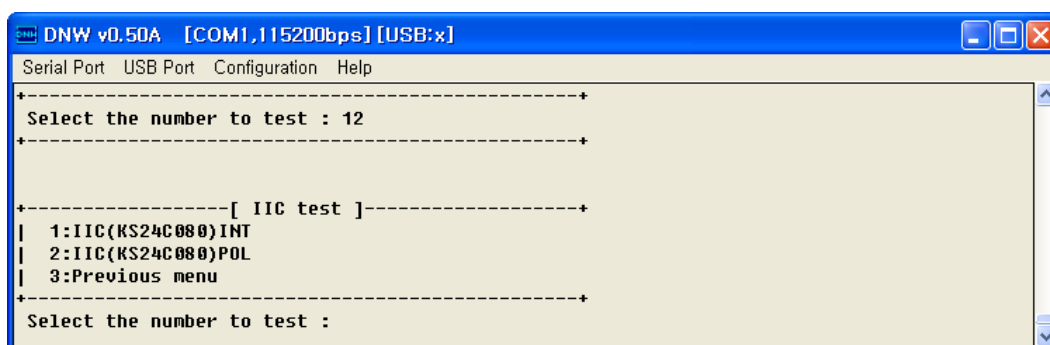
[Picture 4.17]

This is about SPI test. This menu only tests SPI0.

You can test the number 1 and 2 after connecting the pin 2 with the pin 3 on SPI 5 pins connector of MBA2440 board.

You can use 3~8 connecting one board among SPI pins of two MBA2440 with SPIMOSIO pin and another board with SPIMISO0 pin. M means operating in master mode and S means operating in slave mode.

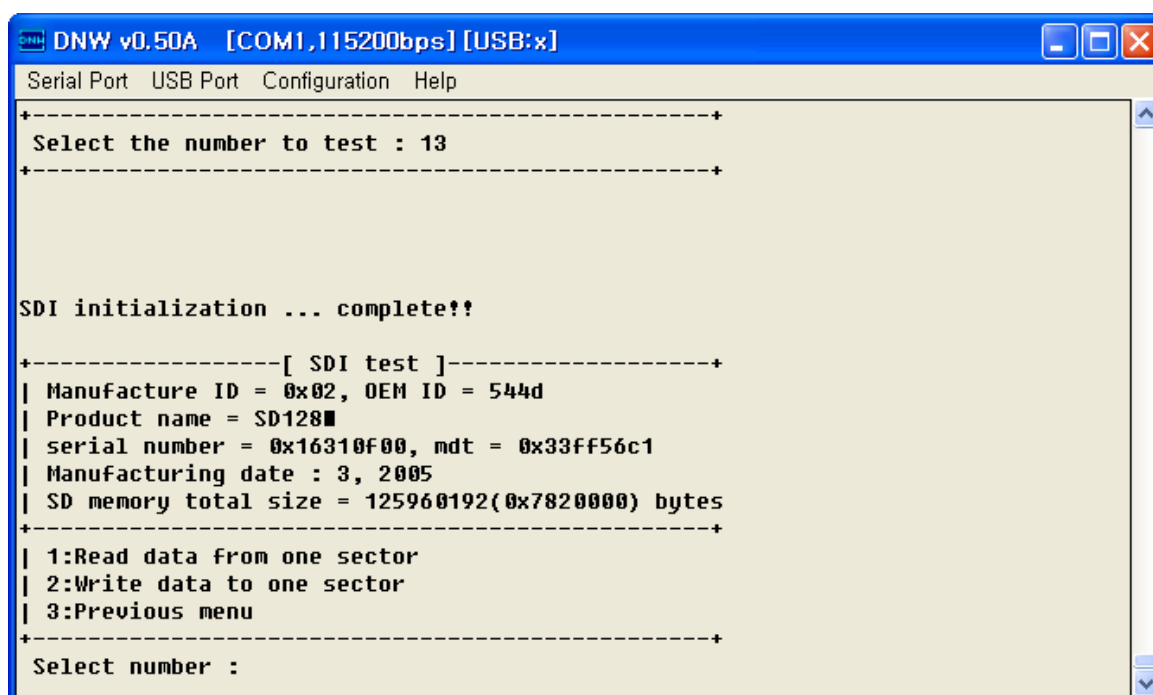
4.3.12 IIC



[Picture 4.18]

You can test writing data through IIC of KS24C080C EEPROM on MBA2440 board.

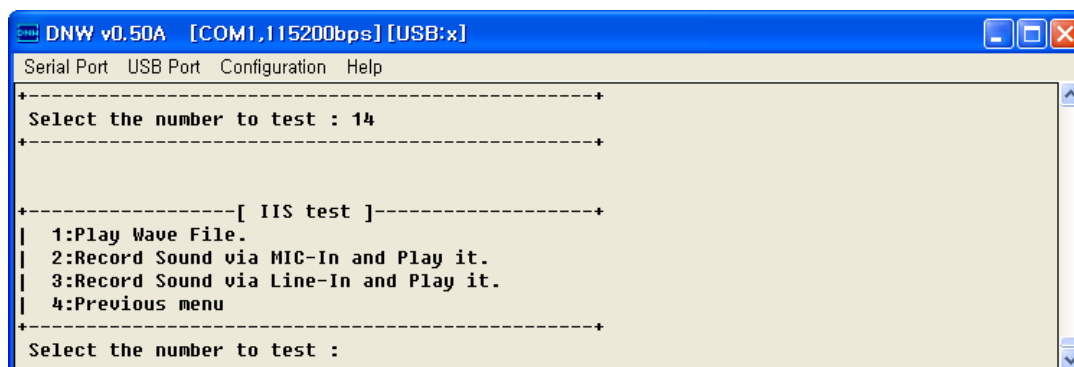
4.3.13 SDIO



[Picture 4.19]

If you choose the menu 13 after inserting SD memory card first, you can test block write test and block read test. After selecting each menu, you can test it according to the message printed out. Read/Write test read or write only one sector(512 Bytes).

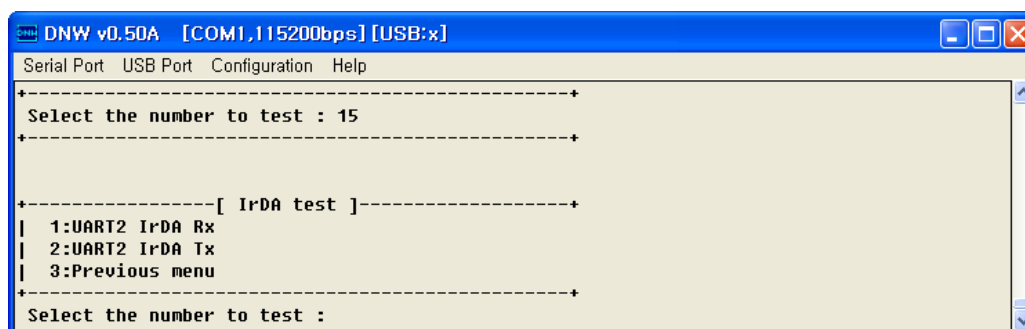
4.3.14 IIS



[Picture 4.20]

It is a program testing UDA1341. after selecting each menu, you can test it according to the message printed out.

4.3.15 IrDA test

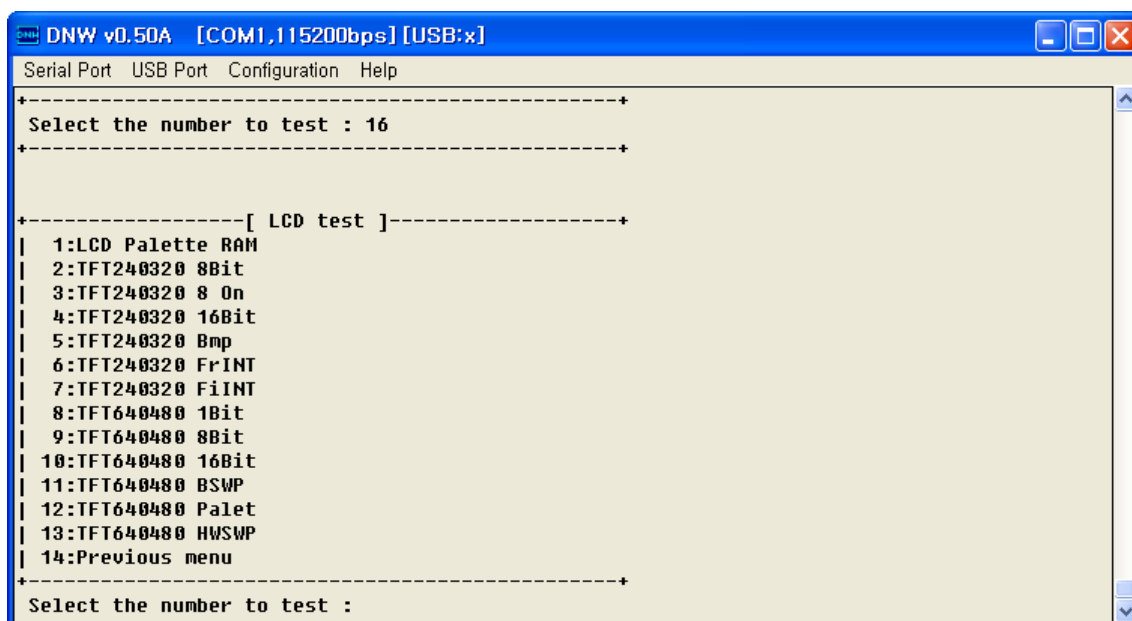


[Picture 4.21]

This is about IrDA test. It needs two MBA2440 boards. One shall be RX and the other shall be TX. Always operate RX first and then, operate TX.

When you operate two boards, you have to make IrDa ports face each other exactly. If they are placed wrong or the distance between them is too far, their transmit-receive could not work properly.

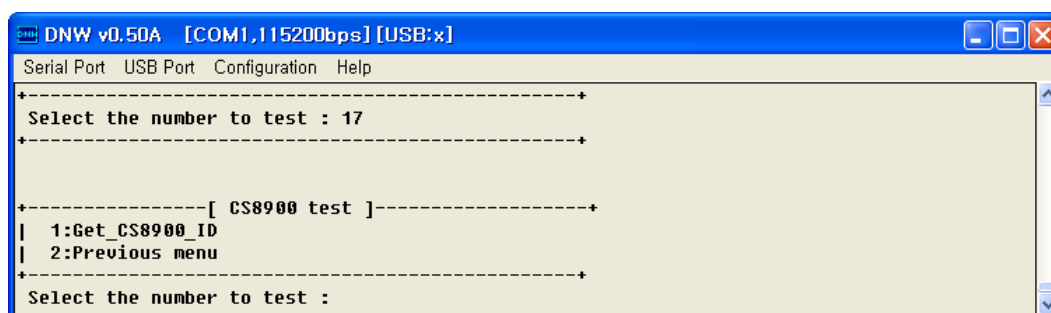
4.3.16 TFT-LCD



[Picture 4.22]

You can test 240x320 TFT-LCD and 640x480 TFT-LCD. After selecting each menu, you can test them according to the message printed out.

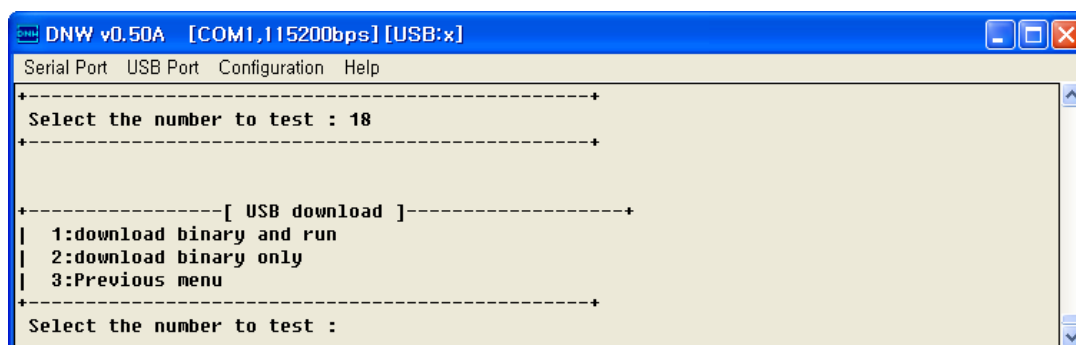
4.3.17 CS8900_Ethernet



[Picture 4.23]

This is about CS8900 Ethernet test. In this menu, you can check if the chip works only through reading CS8900 ID. You can confirm the actual transmit-receive of CS8900 through u-boot ofr MBA2440.

4.3.18 USB download



[Picture 4.24]

This is a menu to download data in SDRAM through USB.

The number 1 downloads an operating binary file and operates it right away.

The number 2 only downloads files. You can write files downloaded by the number 2 on NOR flash or NAND flash.

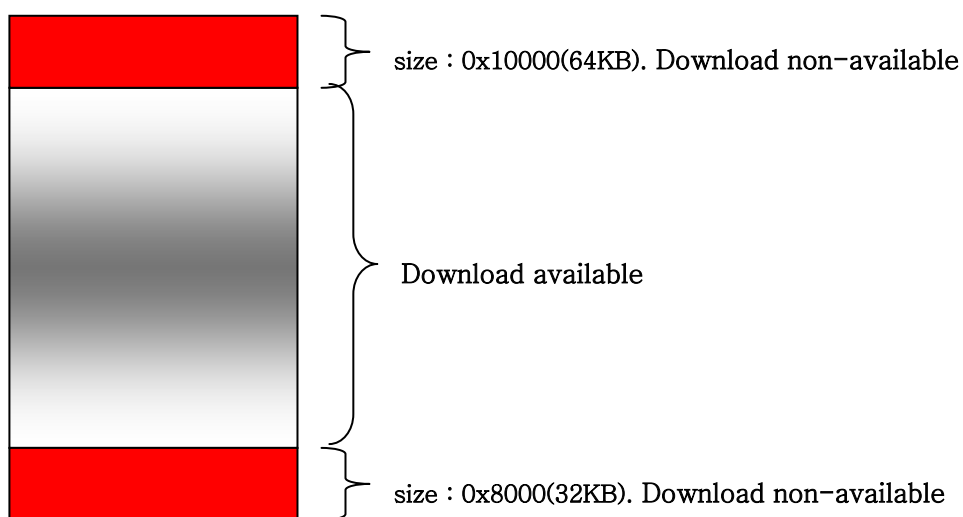
In case of loading test program in 0x0 address of boot flash and booting, you don't use the highest 32KB area and the highest 32KB area when downloading in memory. If you use them, MBA2440 test program doesn't work.

If the size of SDRAM is 64MB, don't download into 0x30000000 ~ 0x30007fff area and 0x33ff8000 ~ 0x33ffffff area.

In case of loading test program in 0x30000000 address of SDRAM and booting, you don't use the lowest 1MB area and the highest 32KB area when downloading in memory. If you use them, MBA2440 test program doesn't work.

If the size of SDRAM is 64MB, don't download into 0x30000000 ~ 0x300fffff area and 0x33ff8000 ~ 0x33ffffff area.

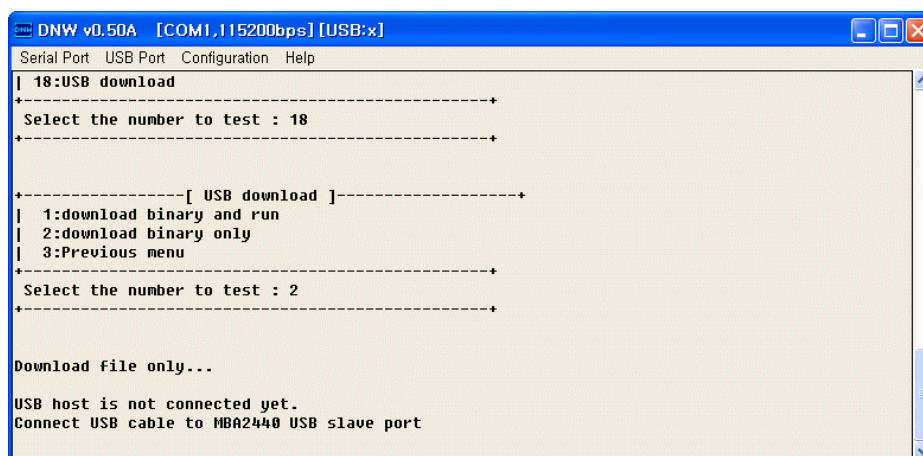
In case of loading test program in 0x0 address of boot flash and booting, the area for downloading is as [Picture 4.25]. [Picture 4.25] is the case of setting up RO_BASE as 0x0 and compiling when compiling MBA2440 test program source. It means when MBA2440 test program code operates on AMD flash.



[Picture 4.25]

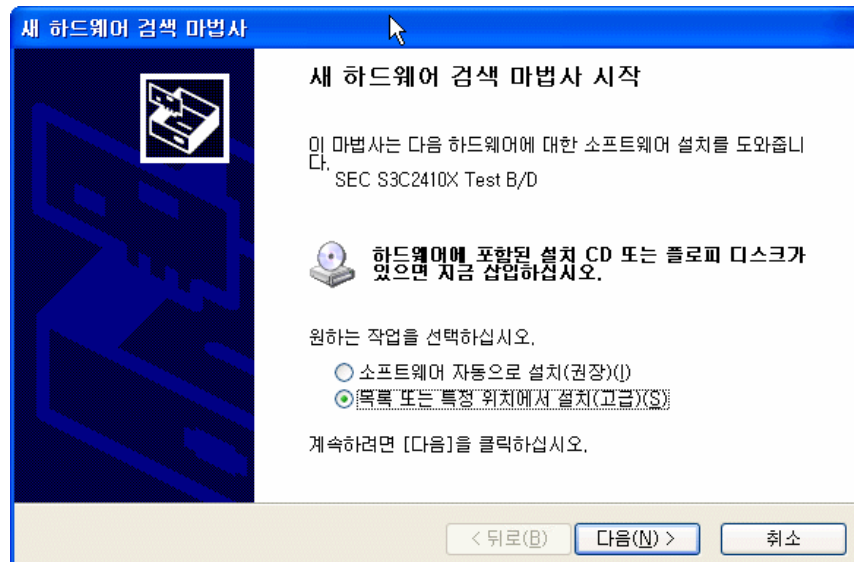
If you test this menu after purchasing the board for the first time, you should install USB driver on Windows OS.

Choose the menu 2



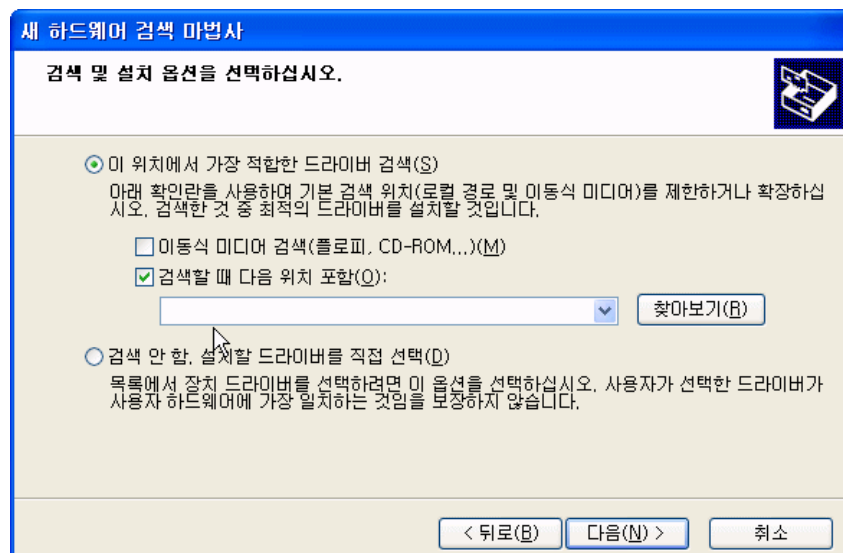
[Picture 4.26]

If USB cable connects PC with MBA2440, you can find a message of 'new hardware found', and new hardware search wizard window will be shown.



[Picture 4.27]

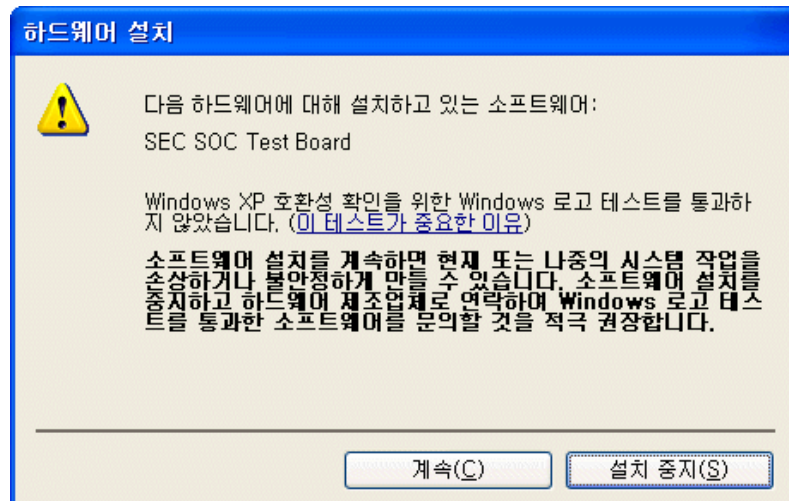
Push the next button after choosing as [Picture 4.27].



[Picture 4.28]

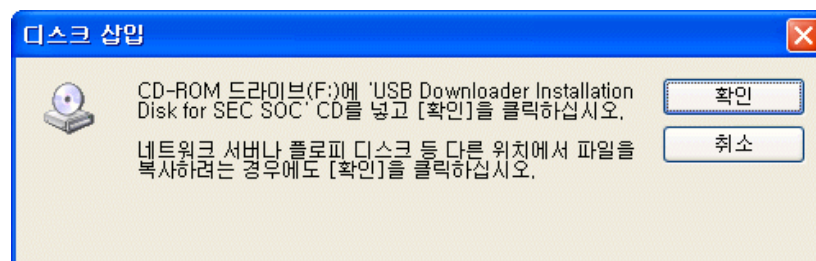
Push 'find' button on [Picture 4.28], select utility/USB_driver directory in CD, and then push the next button.

You will search for hardware. During researching for hardware, windows will keep rising like [Picture 4.29]. Just push the continue button.



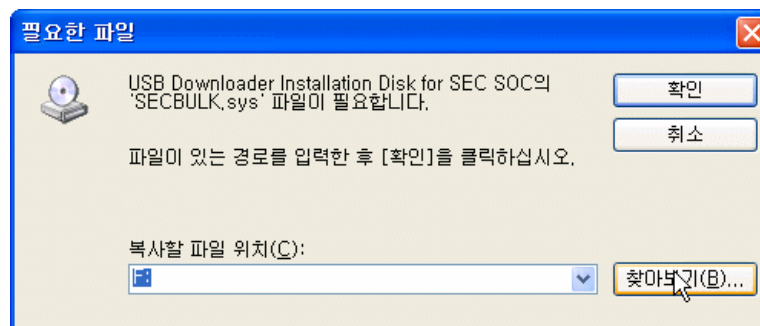
[Picture 4.29]

If a window like on [Picture 4.3] is shown, just push the confirm button.



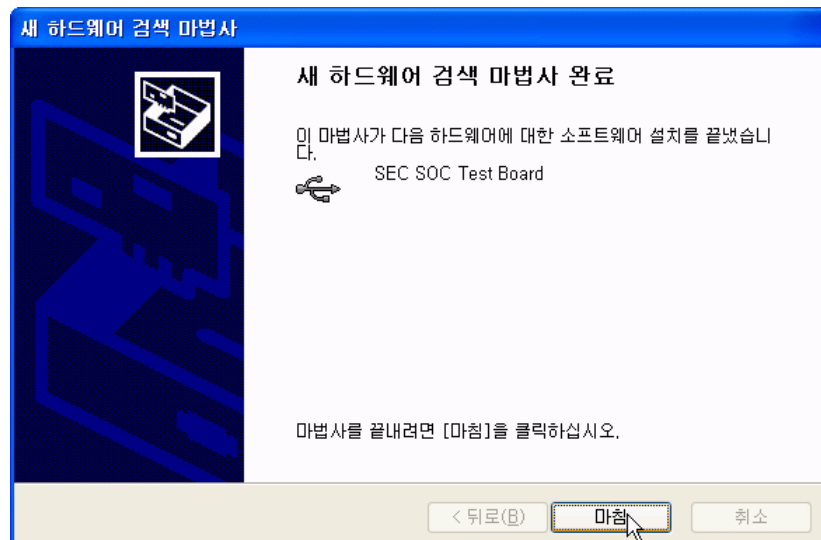
[Picture 4.30]

If a window like on [Picture 4.31] is shown, find and select secbulk.sys file in utility/USB_driver directory in CD.



[Picture 4.31]

If the window, of 'wizard completion' is shown like on [Picture 4.32], push the complete button and finish it.



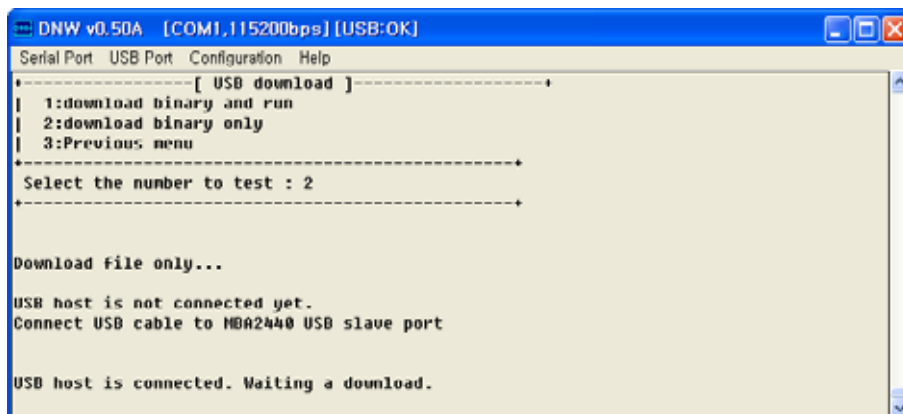
[Picture 4.32]

Now, let's try to download files.

Select the menu you want.

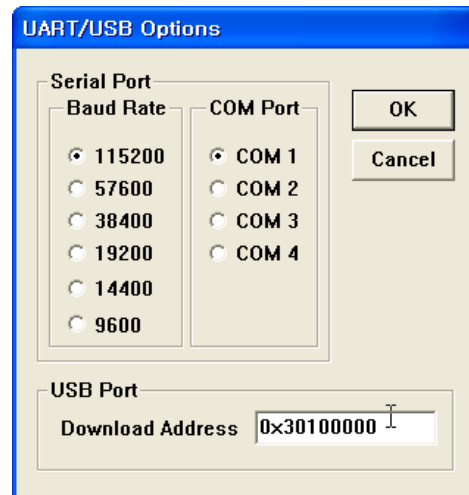
If USB cable is not connected to USB SLAVE port, you will see a message saying to connect the cable.

If the cable is connected, please wait to download.



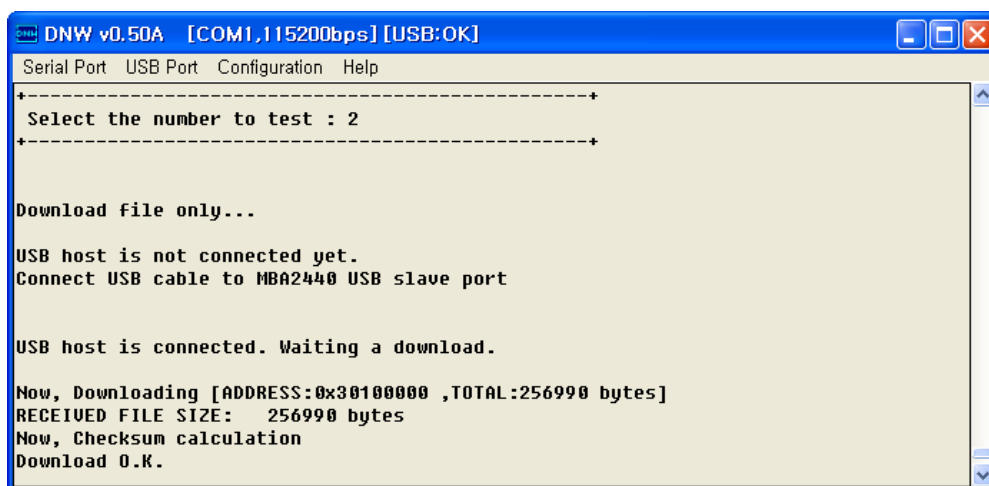
[Picture 4.33]

Set up the address you want to download in Configuration->Option of DNW. This address means the address of memory area on MBA2440, and you don't set up the address for the lowest 32KB area and the highest 32KB area as mentioned earlier. Set up the address on Download Address and push Ok button.



[Picture 4.34]

If you select the file you want to send from USB Port->Transmit of DNW, the file will be transmitted.



[Picture 4.35]

5 U-Boot

There are u-boot, eboot and nboot as boot loader provided for MBA2440.

- **u-boot boot loader**

- : Boot loader for Linux. It is the most important boot loader in MBA2440 among boot loaders modified to adapt to MBA2440 as open source

- It can be used to boot embedded Linux as well as WinCE image.

- **eboot boot loader:** refer to [7.2 Eboot boot loader](#)

- : It is a boot loader included WinCE BSP for MBA2440 to develop WinCE

- **nboot boot loader:** refer to [7.3 Nboot boot loader](#)

- : It is simply firmware to boot and upload WinCE image into memory rather than boot loader in case NAND flash is set up as boot flash

In this chapter, you will learn about u-boot which is the most important boot loader in MBA2440.

We will teach you how boot loaders are first and u-boot, the most important boot loader in MBA2440.

5.1 What is Bootloader?

Boot loader is firmware which takes a part as BIOS if compared to general desktop PC.

Boot loader is the program which activates the very first when the power turns on, and takes a part of constructing basic environments to reset MCU of MBA2440 and other many types of devices (memory and etc.) and activate OS.

Developers can verify hardware themselves through embodying commands which can test if hardware works before controlling it through OS.

In MBA2440, bootloader modified for MBA2440 is provided based on u-boot 1.1.1 version.

All types of development for u-boot are performed in Linux PC (Redhat 9.0), and we will explain about it based on Linux PC development environment.

5.2 U-boot for MBA2440

You can see `mba2440-uboot1.1.1-v1.3.tar.bz2` file, a modified source for MBA2440 in linux/u-boot directory in CD.

It has the following functions in mba2440-uboot1.1.1.

- read, write and erase memory

There are commands to control SDRAM NAND flash, NOR flash which are memory.

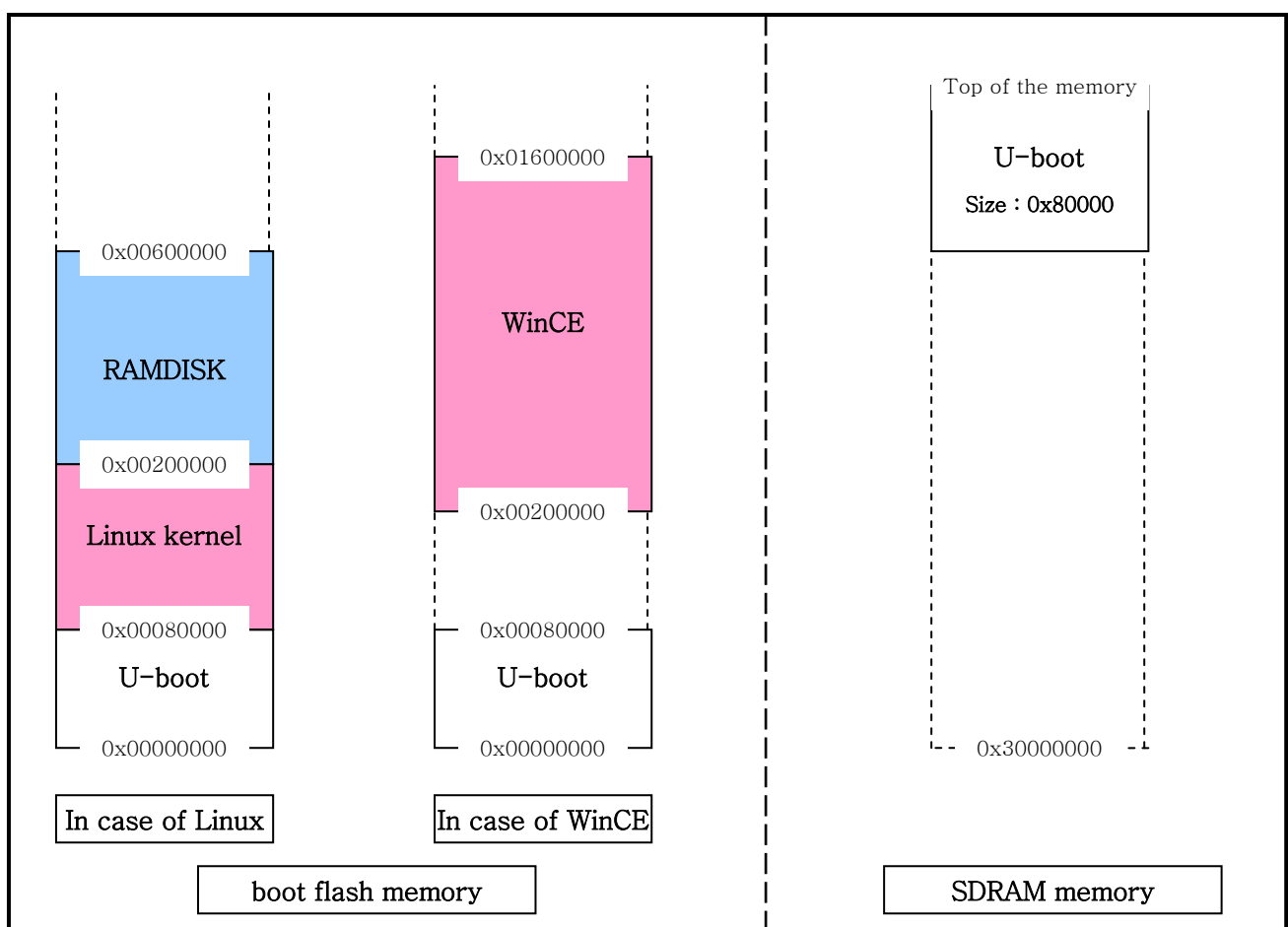
- Network Communication Function

We can check if it is connected to host PC through ping command and receive files with tftp in the available range of system memory in MBA2440.

Besides them, many other commands are provided.

5.2.1 U-Boot memory map

It constructs a memory map like on [Picture 5.1] during its operation after u-boot boots.



[Picture 5.1]

[Picture 5.1] shows a memory map where u-boot recognizes boot flash (in case of NAND flash memory) and a memory map where u-boot recognizes SDRAM.

(Be aware that OS images or file system images can't be loaded, because the size of flash memory is only 1MB in case AMD flash is the boot flash)

In case of Linux, “bootrom kernel” command can work properly only if Linux kernel is within the area of 0x80000 ~ 0x17ffff (1.5MB size).

In the same way, “bootrom kernel” command can work properly only if fliesystem (RAMDISK image) is within the area of 0x200000 ~ 0x5fffff (4MB size).

In case of WinCE, “bootrom wince” command can work properly only if WinCE image is within the area of 0x200000 ~ 0x15ffff 영역 (20MB size).

As soon as u-boot is activated, it copies its own operating image in the space of 0x80000 (512KB) in the highest address of SDRAM memory (system memory), and it performs the rest of duties such as resetting hardware and etc. where it copied.

5.2.2 U-Boot Commands

In this chapter, you will learn about u-boot commands. They have been developed modifying source in u-boot 1.1.1 version, and some of them are modified for MBA2440.

We don't introduce all u-boot commands and explain in detail about those commands introduced in this chapter. Please analyze u-boot source regarding commands.

5.2.2.1 U-boot Information command

- 1) **bdinfo** : It shows board info structure. Bd structure has information such hardware, memory organization and etc. about target board in u-boot.

```
MBA2440 #
MBA2440 # bdinfo
arch_number = 0x000000C1
env_t       = 0x00000000
boot_params = 0x30000100
DRAM bank   = 0x00000000
-> start     = 0x30000000
-> size      = 0x04000000
ethaddr     = 08:00:3E:26:0A:5B
ip_addr     = 192.168.100.5
baudrate    = 115200 bps
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.2]

- 2) **coninfo** : It shows information of console.

```
MBA2440 # coninfo
List of available devices:
serial 80000003 SIO stdin stdout stderr
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.3]

- 3) **help** : It shows the whole command.

```

MBA2440 # help
? - alias for 'help'
autolinux - get default 'linux' image to 0x00020000
autoscr - run script from memory
base - print or set address offset
binfo - print Board Info structure
boot - boot default, i.e., run 'bootcmd'
bootd - boot default, i.e., run 'bootcmd'
bootelf - Boot from an ELF image in memory
bootm - boot application image from memory
bootp - boot image via network using BootP/TFTP protocol
bootrom - just typing bootrom and enter
bootvx - Boot vxWorks from an ELF image
cmp - memory compare
coninfo - print console devices and informations
cp - memory copy
crc32 - checksum calculation
date - get/set/reset date & time
dcache - enable or disable data cache
echo - echo args to console
flerase - erase NOR FLASH memory
flinfo - print NOR FLASH memory information
flread - read NOR FLASH memory
flwrite - write NOR FLASH memory
go - start application at address 'addr'
help - print online help
icache - enable or disable instruction cache
iminfo - print header information for application image
imls - list all images found in flash
itest - return true/false on integer compare
loadb - load binary file over serial line (kermit mode)
loadm - load a image from flash rom to memory
loads - load S-Record file over serial line
loop - infinite loop on address range
md - memory display
me - memory erase
mm - memory modify (auto-incrementing)
mtest - simple RAM test
mw - memory write (fill)
nandb - MBA2410 NAND Flash Bad block check program
nande - MBA2410 NAND Flash Erase program : erase blocks
nandr - MBA2410 NAND Flash Read program : read and copy to memory
nandw - MBA2410 NAND Flash Write program : write from memory to NAND flash
nfs - boot image via network using NFS protocol
nm - memory modify (constant address)
ping - send ICMP ECHO_REQUEST to network host
printenv - print environment variables
protect - enable or disable OR FLASH write protection
rarpboot - boot image via network using RARP/TFTP protocol
reset - Perform RESET of the CPU
run - run commands in an environment variable
saveenv - save environment variables to persistent storage
setenv - set environment variables
sleep - delay execution for some time
tftpboot - boot image via network using TFTP protocol
version - print monitor version
MBA2440 #

```

```

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.00.0 | VT102 | Offline
[영어] [완성] [두벌식]

```

[Picture 5.4]

5.2.2.2 Memory command

- 1) **base** : It sets up Offset from Base address. The actual address of SDRAM is 0x30010000 as base address if the base is 0x10000.

```
MBA2440 # help base
base
- print address offset for memory commands
base off
- set address offset for memory commands to 'off'

MBA2440 # base
Base Address: 0x00000000
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.5]

- 2) **crc32** : It calculates checksum from address to count.

```
MBA2440 #
MBA2440 #
MBA2440 # help crc32
crc32 address count [addr]
- compute CRC32 checksum [save at addr]

MBA2440 # crc32 0x31000000 0x100
CRC32 for 31000000 ... 310000ff ==> 0d968558
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.6]

- 3) **mtest** : It tests read / write from start to end address. As a matter to be attended, it doesn't support NOR, NAND because only RAM area is possible.

```
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # help mtest
mtest [start [end [pattern]]]
- simple RAM read/write test

MBA2440 # mtest 0x30100000 0x30200000
Pattern FFFFFFFD Writing... █
[영어] [완성] [두벌식]
```

[Picture 5.7]

- 4) **cp** : It copies as much as count from source address to target address in memory.

Option: Supporting .b(8 bit) .w(16 bit) .l(32 bit) unit

```
MBA2440 #
MBA2440 #
MBA2440 # help cp
cp [.b, .w, .l] source target count
- copy memory

MBA2440 # cp 0x30100000 0x30200000 0x1000
Copy from 0x30100000 to 0x30200000... done
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.8]

- 5) **md** : It prints out memory value as much as the number of objects from memory address established. The number of objects in this case is the size of 4 bites.

Option : Support .b(8 bit) .w(16 bit) .l(32 bit) unit

Ex) md.b 0x30100000 0x1000

=> It means to print out in the byte unit as much as the size of 0x1000 from 0x30100000

```
MBA2440 #
MBA2440 # help md
md [b, .w, .l] address [# of objects]
    - memory display

MBA2440 # md 0x30002000 0x20
30002000: 6c656420 5b207961 35322d31 203a5d35      delay [1-255]:
30002010: 00000000 4e524157 3a474e49 74655320      ....WARNING: Set
30002020: 39385343 414d3030 64644143 73736572      CS8900MACAddress
30002030: 6e49203a 696c6176 414d2064 64612043      : Invalid MAC ad
30002040: 73657264 0a0d2e73 00000000 4f464e49      dress,.....INFO
30002050: 414d203a 64612043 73657264 65732073      : MAC address se
30002060: 6f742074 7825203a 3a78253a 253a7825      t to: %x:%x:%x:%
30002070: 78253a78 0d78253a 0000000a 6e450a0d      x:%x:%x,.....En
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.9]

- 7) **me** : It erases specific memory except the area loaded in U-boot or all memory areas. It is useful for works related to memory area test.

Ex) me all

=> it erases all areas except where u-boot is operated.

```
MBA2440 # help me
me me[b, .w, .l] start_add size - start_addr and size is hex value
me all - erase all memory except the area of u-boot running code

MBA2440 # me 0x30000000 0x1000
erasing memory .. start address: 0x30000000, size: 0x1000(4096 bytes)
.
erase completed..

MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.10]

```
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # me all
erasing memory .. start address: 0x30000000, size: 0x3f00000(66060288 bytes)
.....
erase completed..

MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.11]

5.2.2.3 NAND flash command

- 1) **nandb** : it checks the status of NAND flash memory block currently used

nandb all : it checks all blocks of NAND flash memory

```
MBA2440 # help nandb
nandb - MBA2410 NAND Flash Bad block check program
nandb startblock(0~4095) endblock(0~4095)
nande blocknumber(0~4095) - check one block you select
nande all - check all blocks

MBA2440 # nandb all
check all blocks [0 ~ 4095]
Display block status :
All block is OK..

MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.12]

nandb [block number] : it checks a block conformed NAND flash memory

```
MBA2440 #
MBA2440 # nandb 300
check block number [300]
Display block status :
block[ 300] is OK..
```

```
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.13]

nandb [initial number of block] [last number of block] : it checks from the initial number to the last number of block

```
MBA2440 # nandb 340 399
erase block [340 ~ 399]
Display block status :block[ 340] ~ block[ 399] OK..
```

```
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.14]

2) **nande** : it erases the status of NAND SMC flash memory block currently used

nande all : it erases all areas of NAND flash memory

```
MBA2440 # nande all
erase all blocks [0 ~ 4095]
Erasing block.. \
Complete erasing block
```

```
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.15]

nande [block number] : it erases the block conformed in NAND flash memory

```
MBA2440 #
MBA2440 # nande b 0
erase block [0 ~ 0]
Erasing block.. -
Complete erasing block
```

```
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.16]

nande [initial number of block] [last number of block] : it erases from the initial number to the last number of block in NAND flash memory

```
MBA2440 #
MBA2440 # nande 10 200
erase block [10 ~ 200]
Erasing block.. /
Complete erasing block
```

```
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.17]

3) **andr** : It brings and reads NAND flash memory block currently used into memory

nandr p [mem_addr] [block number] [page number] : it reads the page conformed in the block conformed in NAND flash memory and saves mem_addr

```
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # nandr p 0x30000000 1 10
read page[block:1, page:10].. copy to memory address 0x30000000, size is 0x200
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.18]

nandr b [mem_addr] [block number] : It saves the block conformed in NAND flash memory into mem_addr

```
MBA2440 #
MBA2440 #
MBA2440 # nandr b 0x31000000 2
read block[2].. copy to memory address 0x31000000, size is 0x4000
\
Reading block is Completed
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.19]

nandr t [mem_addr] [base address of nand flash] [size] : It reads as much as specified starting the base address of the area to read in NAND flash memory and saves it in mem_addr. But the base address or the size shall be at least in page unit.

```
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # nandr t 0x30100000 0x400 0x200
Read start point : block[0] page[2] size = 0x200
Reading data ... \
Read complete(0x200): from 0x400(0) NAND flash to 0x30100000 SDRAM
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.20]

4) **nandw** : it uses NAND flash memory block currently used as memory

nandw p [mem_addr] [block number] [page number] : it reads as much as the page size using mem_addr address in memory as base address and uses it on the page conformed on the NAND flash block

```
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # nandw p 0x30000000 1 10
write page[block:1, page:10].. read from memory address 0x30000000, size is 0x200
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.21]

nandw b [mem_addr] [block number] : it reads the block size using m_addr in memory as base address and uses it on NAND flash block conformed


```
MBA2440 #
MBA2440 #
MBA2440 # nandw b 0x30100000 4
write block[4].. read from memory address 0x30100000, size is 0x4000
-
Writing block is Completed
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.22]

nandw t [mem_addr] [base address of nand flash] [size] : It reads as much as specified using mem_addr address in memory as base address and write as much as the size from base address of NAND flash. But the base address or the size shall be at least in page unit.

```
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # nandw t 0x30000000 0x0 0x20000
Write start point : block[0] page[0] size = 0x20000
Writing data .../
Write complete(0x20000): from 0x30000000 SDRAM to 0x0(0) NAND flash
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.23]

5.2.2.4 NOR flash command

1) **flinfo** : It shows information regarding NOR flash memory.

flinfo [bank number] : It prints out information NOR flash if you input bank number marked on NOR flash information when booting u-boot. If you don't input bank number, it prints out all NOR flash information connected.

```
MBA2440 #
MBA2440 # flinfo

Bank # 0: AMD AM29LV800BB
Size: 0x100000 Bytes, 19 Sectors
Sector Start Addresses:
00000000 (RO) 00040000 (RO) 00080000 (RO) 000C0000 (RO)
00100000 (RO) 00140000 00180000 001C0000
00200000 00240000 00280000 002C0000
00300000 00340000 00380000 003C0000
00400000 00440000 00480000 004C0000
00500000 00540000 00580000 005C0000
00600000 00640000 00680000 006C0000
00700000 00740000 00780000 007C0000
00800000 00840000 00880000 008C0000
00900000 00940000 00980000 009C0000
00A00000 00A40000 00A80000 00AC0000
00B00000 00B40000 00B80000 00BC0000
00C00000 00C40000 00C80000 00CC0000
00D00000 00D40000 00D80000 00DC0000
00E00000 00E40000 00E80000 00EC0000
00F00000 00F40000 00F80000 00FC0000

Bank # 1: INTEL 28F128J3A
Size: 0x2000000 Bytes, 128 Sectors
Sector Start Addresses:
08000000 08040000 08080000 080C0000
08100000 08140000 08180000 081C0000
08200000 08240000 08280000 082C0000
08300000 08340000 08380000 083C0000
08400000 08440000 08480000 084C0000
08500000 08540000 08580000 085C0000
08600000 08640000 08680000 086C0000
08700000 08740000 08780000 087C0000
08800000 08840000 08880000 088C0000
08900000 08940000 08980000 089C0000
08A00000 08A40000 08A80000 08AC0000
08B00000 08B40000 08B80000 08BC0000
08C00000 08C40000 08C80000 08CC0000
08D00000 08D40000 08D80000 08DC0000
08E00000 08E40000 08E80000 08EC0000
08F00000 08F40000 08F80000 08FC0000
09000000 09040000 09080000 090C0000
09100000 09140000 09180000 091C0000
09200000 09240000 09280000 092C0000
09300000 09340000 09380000 093C0000
09400000 09440000 09480000 094C0000
09500000 09540000 09580000 095C0000
09600000 09640000 09680000 096C0000
09700000 09740000 09780000 097C0000
09800000 09840000 09880000 098C0000
09900000 09940000 09980000 099C0000
09A00000 09A40000 09A80000 09AC0000
09B00000 09B40000 09B80000 09BC0000
09C00000 09C40000 09C80000 09CC0000
09D00000 09D40000 09D80000 09DC0000
09E00000 09E40000 09E80000 09EC0000
09F00000 09F40000 09F80000 09FC0000

MBA2440 # █
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.00.0 | VT102 | Offline
[영어] [완성] [두벌식]
```

[Picture 5.24]

2) **flread** : It reads data on NOR flash and copies them into memory.

flread [NOR flash address] [memory address] [size] : It reads as much as the size starting the address of NOR flash memory and copies it into memory address. You can find the memory address of NOR flash through flinfo.

```
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # flread 0x80000000 0x30000000 0x20000
Read data from INTEL flash
read from 0x08000000 and copy to 0x30000000 ... \
Complete read data!! size : 0x20000

MBA2440 # ■
[영어][완성][두벌식]
```

[Picture 5.25]

2) **flwrite** : It writes data on NOR flash. It erases the conformed block through flerase first before writing.

flwrite [memory address] [NOR flash address] [Size] : It writes the NOR flash memory address as much as the size starting memory address.

You can see NOR flash memory address through flinfo.

It erases the conformed block first through flerase before writing data.

```
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # flerase 1:0-4
Erase Flash Sectors 0 - 4 in Bank # 1
Erasing INTEL flash now...

Erasing sector 0 ... complete
Erasing sector 1 ... complete
Erasing sector 2 ... complete
Erasing sector 3 ... complete
Erasing sector 4 ... complete
MBA2440 #
MBA2440 # flwrite 0x30000000 0x80000000 0x20000
Write data to INTEL flash
read from 0x30000000 and write to 0x08000000 ... \
Complete write data!! size : 0x20000

MBA2440 # ■
[영어][완성][두벌식]
```

[Picture 5.26]

3) **flerase** : It erases NOR flash memory in block unit.

flerase bank all : It erases the whole NOR flash memory recognized when booting u-boot.

flerase bank [bank number] : It erases the whole NOR flash conformed to bank number.

flerase bank [bank number]:[initial block]-[last block] : It erases blocks between the initial block to the last block in NOR flash conformed to bank number.

Ex) flerase 1:0-9 => It erases block No. 1 ~ block No. 9 on NOR flash
conformed to bank 1.

```
MBA2440 # flerase 1:0-9
Erase Flash Sectors 0 - 9 in Bank # 1
Erasing INTEL flash now...
```

```
Erasing sector 0 ... complete
Erasing sector 1 ... complete
Erasing sector 2 ... complete
Erasing sector 3 ... complete
Erasing sector 4 ... complete
Erasing sector 5 ... complete
Erasing sector 6 ... complete
Erasing sector 7 ... complete
Erasing sector 8 ... complete
Erasing sector 9 ... complete
```

```
MBA2440 # █
```

[영어] [완성] [두벌식]

[Picture 5.27]

- 4) **protect** : It locks or unlocks NOR flash memory block. If the block is locked, you can't write.

protect [on/off] all : 'On' locks blocks, and 'off' unlocks blocks.

'all' locks or unlocks all blocks on NOR flash recognized when booting u-boot.

protect [on/off] bank [bank number] : 'On' locks blocks, and 'off' unlocks blocks.

'bank' locks or unlocks all blocks of NOR flash included in bank number.

protect [on/off] [bank number]:[initial block]-[last block] : 'On' locks blocks, and 'off' unlocks blocks. It locks or unlocks blocks between the initial block and the last block on NOR flash conformed to bank number.

```
MBA2440 #
MBA2440 # flinfo 0

Bank # 0: AMD AM29LV800BB
Size: 0x100000 Bytes, 19 Sectors
Sector Start Addresses:
00000000 (RO) 00004000 (RO) 00006000 (RO) 00008000 (RO)
00010000 (RO) 00020000 00030000 00040000
00050000 00060000 00070000 00080000
00090000 000A0000 000B0000 000C0000
000D0000 000E0000 000F0000 (RO)
```

```
MBA2440 #
MBA2440 # protect off bank 0
Un-Protect Flash Bank # 0
MBA2440 #
MBA2440 # flinfo 0
```

```
Bank # 0: AMD AM29LV800BB
Size: 0x100000 Bytes, 19 Sectors
Sector Start Addresses:
00000000 00004000 00006000 00008000
00010000 00020000 00030000 00040000
00050000 00060000 00070000 00080000
00090000 000A0000 000B0000 000C0000
000D0000 000E0000 000F0000
```

```
MBA2440 # █
```

```
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.00.0 | VT102 | Offline
```

[영어] [완성] [두벌식]

[Picture 5.28]

5.2.2.5 SD memory command

It can read SD memory card only. File system format for SD memory is FAT file system(not FAT32).

- 1) **fatls** : It shows file name lists in SD memory like “ls” command on linux.

fatls sd 0 : “sd” is the memory type. Now, MBA2440 support only sd memory only.
“0” is devide number.

```
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # fatls sd 0
      79      sdtest
      766168  zimage.bin
      2782866 ram.gz
      20971520 nk.nb0.3.5

4 file(s), 0 dir(s)

MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.29]

- 2) **fatload** : It loads file data from SD memory.

fatload sd 0 mem_addr file : It load “file” to “mem_addr” area in SDRAM.

```
MBA2440 # fatls sd 0 0x30c00000 zimage.bin
Usage:
fatls - list files in a directory (default /)

MBA2440 #
MBA2440 # fatload sd 0 0x30c00000 zimage.bin
reading zimage.bin

766168 bytes read
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.30]

5.2.2.6 Execute command

- 1) **go** : You can operate it by setting up the base address of applications downloaded through serial port or network. You can applications if you create basic system function such as console I/O or malloc(), free() and etc. and hardware function on boot in advance.

```

MBA2440 #
MBA2440 # go 30f00000
## Starting application at 0x30F00000 ...
Uncompressing Linux..... done, booting the.
Linux version 2.4.20 elfin-d1.5 (root@localhost.localdomain) (gcc version 2.95.3 20010315 (rT
CPU: ARM/CIRRUS Arm920Tid(wb) revision 0
Machine: MBA2440 Aiji System Co.,LTD1
Warning: bad configuration page, trying to continue
On node 0 totalpages: 8192
zone(0): 8192 pages.
zone(1): 0 pages.
zone(2): 0 pages.
Kernel command line: root=/dev/ram0 rw console=ttyS0
Console: colour dummy device 80x30
Calibrating delay loop... 199.47 BogoMIPS
Use CONFIG_INSTANT_ON_LPJ=997376 for Instant On.
Memory: 32MB = 32MB total
Memory: 26432KB available (1457K code, 305K data, 80K init)
Dentry cache hash table entries: 4096 (order: 3, 32768 bytes)
Inode cache hash table entries: 2048 (order: 2, 16384 bytes)
Mount-cache hash table entries: 512 (order: 0, 4096 bytes)
Buffer-cache hash table entries: 1024 (order: 0, 4096 bytes)
Page-cache hash table entries: 8192 (order: 3, 32768 bytes)
POSIX conformance testing by UNIFIX
Linux NET4.0 for Linux 2.4
Based upon Swansea University Computer Society NET3.039
Initializing RT netlink socket
CPU clock = 399.651840 Mhz, HCLK = 99.912960 Mhz, PCLK = 49.956480 Mhz
PWM-Timers Management Module Loaded.
Disabling the Out Of Memory Killer
[영어][완성][두벌식]

```

[Picture 5.31]

5.2.2.7 Download command

- 1) **tftp** : Files are downloaded from TFTP server as TFTP client and they are saved in Load Address such as 0x30008000. The parameter given next becomes zImage from the example on the picture the name of image which the parameter given next download becomes zImage from the example on the picture. The image which will be downloaded shall exist in root directory of TFTP server.

```

MBA2440 # tftp 30f00000 zImage.bin
TFTP from server 192.168.100.2; our IP address is 192.168.100.5
Filename 'zImage.bin'.
Load address: 0x30f00000
Loading:-
done
Bytes transferred = 765536 (bae60 hex)
MBA2440 #
[영어][완성][두벌식]

```

[Picture 5.32]

Tip) you need to write files which will be downloaded clearly dividing them into big and small letters. If wrong, you might not be able to download files properly.

- 2) **usbdown** : Files are downloaded from Windows PC using DNW and they are saved in Memory. This command is only for DNW.
usbdown memory_address : put the address of SDRAM you want to download.
Select the file using USB Port->Transmit menu on the DNW window.

5.2.2.8 Environment variable command

1) Environment factors list

The list includes environment factors which affect u-boot while u-boot is being operated.

autoload – If it is installed as “no”, rarpboot, tftpboot, bootp will be established by BOOTP/DHCP server, not environment factors.

autostart – If it is installed as “yes”, it starts rarpboot, bootp, dhcp, tftpboot automatically, download images and operate them. (internal use of bootm)

baudrate – set up Console baudrate

bootargs – Save boot factors of Linux kernel

bootcmd – Saves commands operated automatically when booting u-boot

bootdelay – Set up stand-by time for prompt when auto-booting U-boot

bootfile – the name of file downloaded as TFTP

ethaddr – MAC Address

ipaddr – IP Address of MBA2440

loadaddr – the address which will be downloaded as TFTP

loads_echo – echo data downloaded as loads to screen

pram – Protect RAM

serverip – TFTP Server IP

silent – It doesn't print out anything when booting. CONFIG_SILENT_CONSOLE should be declared.

verify – It calculates Checksum of image file header defined on bootm

bootfile – the name of file which will be downloaded in TFTP/BOOTP/DHCP

dnsip – DNS IP

gatewayip – Gateway IP

hostname – Hostname

netmask – Netmask

filesize – the size of file which will be downloaded in TFTP/BOOTP/DHCP

- 2) **printenv** : Environment factors establish script, delay and etc. operated when booting, and there are IP Address, Netmask, TFTP Server IP and etc. related to network as environment factors. Each parameter and the size and available amount of environment possibly to be saved are printed out in printenv.

```
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # printenv
bootcmd=nand read 30200000 200000 1400000; go wince
bootdelay=3
baudrate=115200
ethaddr=08:00:3e:26:0a:5b
ipaddr=192.168.100.5
serverip=192.168.100.2
netmask=255.255.255.0
stdin=serial
stdout=serial
stderr=serial

Environment size: 211/65532 bytes
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.33]

- 3) **setenv** : It sets up IP Address, MAC Address, Netmask and Gateway IP

```
MBA2440 #
MBA2440 #
MBA2440 # setenv ipaddr 10.0.0.1
MBA2440 # printenv
bootcmd=nand read 30200000 200000 1400000; go wince
bootdelay=3
baudrate=115200
ethaddr=08:00:3e:26:0a:5b
serverip=192.168.100.2
netmask=255.255.255.0
stdin=serial
stdout=serial
stderr=serial
ipaddr=10.0.0.1

Environment size: 206/65532 bytes
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.34]

5.2.2.9 The rest command

- 1) **date** : It prints out the date of system.

```
MBA2440 #
MBA2440 # help date
date [MMDDhhmm[[CC]YY] [.ss]]
date reset
- without arguments: print date & time
- with numeric argument: set the system date & time
- with 'reset' argument: reset the RTC

MBA2440 # date
Date: 2003-01-01 (Monday)      Time: 13:42:00
MBA2440 # date 061013452005
Date: 2005-06-10 (Friday)     Time: 13:45:00
MBA2440 # date
Date: 2005-06-10 (Friday)     Time: 13:45:05
MBA2440 # date
Date: 2005-06-10 (Friday)     Time: 13:45:10
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.35]

- 2) **echo** : it prints out message

```
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # help echo
echo [args..]
    - echo args to console; \c suppresses newline

MBA2440 # echo MBA2440 reference board
MBA2440 reference board
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.36]

- 3) **sleep** : it sleeps during seconds fixed

```
MBA2440 #
MBA2440 #
MBA2440 #
MBA2440 # help sleep
sleep N
    - delay execution for N seconds (N is _decimal_ !!!)

MBA2440 # sleep 4
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.37]

- 4) **version** : it prints out u-boot version

```
MBA2440 #
MBA2440 # version

U-Boot 1.1.1 (Jun 29 2005 - 22:27:49)
MBA2440 # █
[영어] [완성] [두벌식]
```

[Picture 5.38]

- 5) **?** - it is the same command as help

5.3 Bootloader Compile

The name of bootloader source provided in CD is **mba2440-uboot1.1.1-v1.3.tar.bz2**.

Let's learn about how to create bootloader images by compiling source.

5.3.1 Extract Source

Put source provided into a directory and extract it.

A directory named "/uboot" of Linux PC will be created and the operation will be performed in the directory

```
[root@localhost]# mkdir /uboot
[root@localhost]# cd /uboot
[root@localhost]# cp /mnt/cdrom/bootloader/mba2440-uboot1.1.1-v1.3.tar.bz2 ./
[root@localhost]# tar xjf mba2440-uboot1.1.1-v1.3.tar.bz2
[root@localhost]# cd mba2440-uboot1.1.1-v1.3
```

5.3.2 U-boot configuration

There are some parts to be adjusted when compiling u-boot according to the size of system memory and what flash memory is used as boot flash.

5.3.2.1 Set up Boot Flash

You need to modify setup header file for u-boot according to which is used as boot flash, NOR flash or NAND flash.

The header file is **mba2440-uboot1.1.1-v1.3/include/configs/mba2440.h** and macro which will be modified is as the following.

CONFIG_INIT_CRITICAL

The macro should be established as "1" for sure when you try to boot with boot flash.

It is too bothering to upload images on bootrom whenever you compile and upload images, because you have to do that frequently to modify or test u-boot. You can load compiled images directly into memory in this case, and you need to set up the value as "0".

CONFIG_NAND_BOOT

The macro is to divide boot flash into NOR and NAND.

In case you try to use NOR flash [AM29LV800 (basic) or 28F128J3C (option)] as boot flash, set up the macro as "0", compile it and upload the image created into the 0 address of NOR flash.

In case you try to use NAND flash [K9F5608 (basic) 또는 SMC (option)] as boot flash, set up the macro as “1”, compile it and upload the image created into 0 address of NAND flash.

5.3.2.2 Set up System Memory Size

MBA2440 basically uses 64MB의 SDRAM as system memory, it can extend it to 128MB. You can use memory properly on bootloader only if you apply those characteristics to u-boot and compile it according to system memory.

In the same way, modify the following macro of **mba2440-uboot1.1.1-v1.3/include/configs/mba2440.h** for this.

CONFIG_MBA2440_MEM_SIZE

In case of 64MB, set “64” and in case of 128MB, set “128”.

One more thing, you need to modify the macro positioning for bootloader in memory.

The file is **mba2440-uboot1.1.1-v1.3/board/mba2440/config.mk**, and modify the following macro.

TEXT_BASE

In case of 64MB, set “0x33F80000” and in case of 128MB, set “0x37F80000”.

Now, basic setup is all completed.

5.3.2.3 Boot Automatically OS Kernel Image

u-boot can boot WinCE or Linux OS kernel after waiting for fixed time at the same time of booting.

In the same way, you need to modify the following macro of **mba2440-uboot1.1.1-v1.3/include/configs/mba2440.h** for this.

CONFIG_BOOTDELAY

The macro shall be set up waiting time in a second unit in case of automatic booting.

CONFIG_BOOTCOMMAND

This macro shows commands which will be performed after fixed time.

You can write down command in “ ”. In case commands which will be performed are many, put ; at the end of every command.

If the macro is treated as a note, automatic booting will not operate.

“nandr t 30200000 200000 1400000; go wince”

The above command is to copy WinCE image in NAND flash into memory and boot WinCE.

```

*****
*
*  Aiji System
*
*  MBA2440
*
*  (http://www.aijisystem.com)
*
*****

U-Boot 1.1.1 (Jun 30 2005 - 11:00:25)

U-Boot code: 0x33F80000 -> 0x33F9D6C0 BSS: -> 0x33FA1C88
RAM Configuration:
nGCS6 : 0x30000000 64 MB

U-Boot is booted from NAND flash

#### NOR flash information ####
Bank # 0 => [missing or unknown FLASH type]
Bank # 1 => [missing or unknown FLASH type]

#### NAND flash information ####
Manufacture ID [0xec], Device ID [0x76]
[Samsung K9S1208V0M mem size(64MB), Block count(4096)]

*** Warning - bad CRC, using default environment

In: serial
Out: serial
Err: serial
Hit any key to stop autoboot: 0
Read start point : block[128] page[0] size = 0x1400000
Reading data ...-
CTRL-A Z for help 115200 8N1 | NOR | Minicom 2.00.0 | VT102 | Offline
[영어] [완성] [두벌식]

```

[Picture 5.39]

[Picture 5.39] shows automatic booting of WinCE after the power turns on.

5.3.3 Compile U-boot

Perform the following commands in order after set up everything needed.

Of course, these commands should be performed in mba2440-uboot1.1.1-v1.3 directory.

```

[root@localhost]# pwd
/uboot/mba2440-uboot1.1.1-v1.3
[root@localhost]# make clobber
[root@localhost]# make mba2440_config
[root@localhost]# make all

```

make clobber

This command removes all object files and etc. compiled before.

make mba2440_config

This command applies things set up earlier.

make all

This command performs actual compiling.

You can perform three commands at once through doing as the following.

```
[root@localhost]# make clobber; make mba2440_config; make all
```

If compiled properly, you can see **u-boot.bin** created as the below.

```
[root@localhost]# ls
CHANGELOG      System.map      drivers         lib_i386        mips_config.mk  u-boot
CHANGELOG.log  arm_config.mk   dtb             lib_m68k        mkconfig        u-boot.bin
COPYING        board           examples        lib_microblaze  net             u-boot.map
CREDITS         common          fs              lib_mips        nios_config.mk  u-
boot.srec
MAINTAINERS    config.mk       i386_config.mk  lib_nios        post
MAKEALL        cpu            include         lib_ppc          ppc_config.mk
Makefile       disk          lib_arm         m68k_config.mk  rtc
README        doc           lib_generic     microblaze_config.mk  tools
```

u-boot.bin

After this file is compiled, you can fuse the file in 1 address of bootrom as the final binary file created.

u-boot

This file is u-boot image in elf format

u-boot.map

This file shows location information in memory for all functions compiled.

6 Linux Kernel

There exist many OS for embedded system. MBA2440 supports three types of OS for embedded system.

1. Embedded Linux
2. WinCE
3. UbiFOS

In this chapter, you will learn about embedded Linux supporting MBA2440 board.

The kernel version of Linux explained in this manual is 2.4.20 version and it is modified to support MBA2440 based on Linux source in this version.

We will explain about assuming that development environment of Linux PC has been constructed properly in this chapter as explained in [3.3 Embedded Linux Development Environment](#) earlier.

Please read [3.3 Embedded Linux Development Environment](#) before this chapter and construct the environment.

You will learn how to compile Linux source for MBA2440 first and how to operate Linux based on many file systems next.

6.1 Kernel compile

There is Linux kernel compressed source in CD provided. The compressed file named `mba2440-linux2.4.20-v1.3-4.tar.bz2` is the kernel source in `linux/kernel` directory.

Turn on the PC which Linux is installed and login with root account when you login Linux. It is to avoid problems which could be occurred because it is not root account in the process of developing Linux.

6.1.1 Extract Linux Kernel

Extract kernel source and create a directory for compiling.

Create a directory named `mba2440` under `/root/` directory.

```
[root@localhost root]# mkdir /root/mba2440
```

Insert CD provided into CD-Rom of Linux PC, mount it and copy Linux kernel source.

```
[root@localhost root]# mount /mnt/cdrom
[root@localhost root]# cp -a /mnt/cdrom/linux/kernel/mba2440-linux2.4.20-v1.3-4.tar.bz2 ./mba2440/
[root@localhost root]# cd /root/mba2440
[root@localhost root]# ls
mba2440-linux2.4.20-v1.3-4.tar.bz2
```

Extract kernel source.

```
[root@localhost root]# tar xjf mba2440-linux2.4.20-v1.3-4.tar.bz2
[root@localhost root]# ls
mba2440-linux2.4.20-v1.3-4 mba2440-linux2.4.20-v1.3-4.tar.bz2
```

After extracting it, you will see a directory named `mba2440-linux2.4.20-v1.3-4` created. Move to the directory.

```
[root@localhost root]# cd mba2440-linux2.4.20-v1.3-4
```

```
[root@localhost mba2440-linux2.4.20-v1.0]# ls
COPYING  Documentation  README      System.map  fs          ipc          mm          vmlinux
CREDITS  MAINTAINERS    REPORTING-BUGS  arch        include     kernel       net
ChangeLog Makefile       Rules.make    drivers     init        lib          scripts
[root@localhost mba2440-linux2.4.20-v1.0]#
```

[영어] [완성] [두벌식]

[Picture 6.1]

6.1.2 Set up Linux Kernel

You don't need to set up Linux source because it has already been compiled.

6.1.2.1 Load Configuration File

Contents regarding Linux setup satisfying each hardware platform are saved in one file. You can find those files in arch/arm/def-configs/ directory of source.

Setup files for MBA2440 is also found in the directory.

- **mba2440ramdisk** : to select ramdisk as root file system
- **mba2440cramfs** : to select cramfs as root file system

There exist two files.

Source provided is basically set up according to **mba2440ramdisk** setup file. (set up to be operated in 3.5"(240x320) TFT-LCD of 64MB memory)

You will learn about this in detail in [6.2 File system](#).

You can set up Linux kernel through "**make menuconfig**" command.

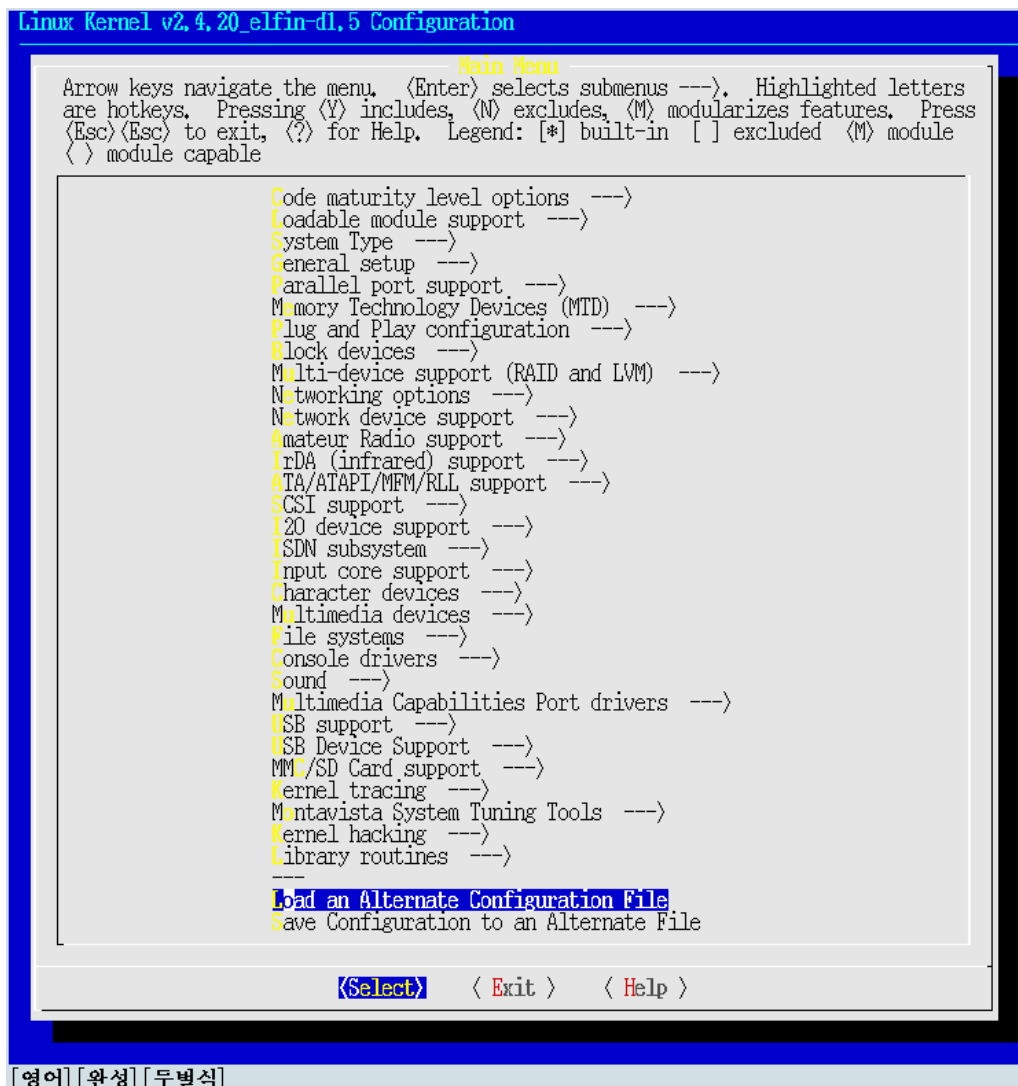
If you operate this command, you will find main menu setup window in text graphic mode, and you can add or omit parts you want into/from kernel on the main menu setup window.

After you set up the menu you want with an arrow on the main menu setup window, you can enter the sub-menu of the menu you selected by pressing enter key or space bar.

After you select <Exit> on the lowest part of setup window using tab key to get out of the menu, you push enter key or space bar in the same way.

If you select <help> with tab key, you will find explanations about the menu selected by direction keys.

```
[root@localhost root]# make menuconfig
```



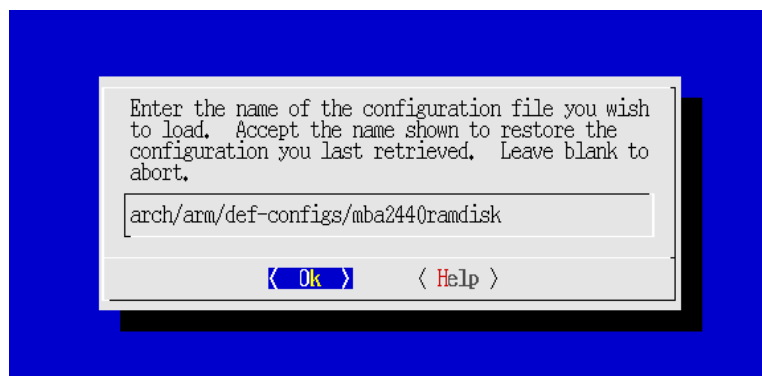
[Picture 6.2] main menu setup window

If you select Load an Alternate Configuration File as shown on [Picture 6.1] and push enter key or space bar, you will see as [Picture 6.2].



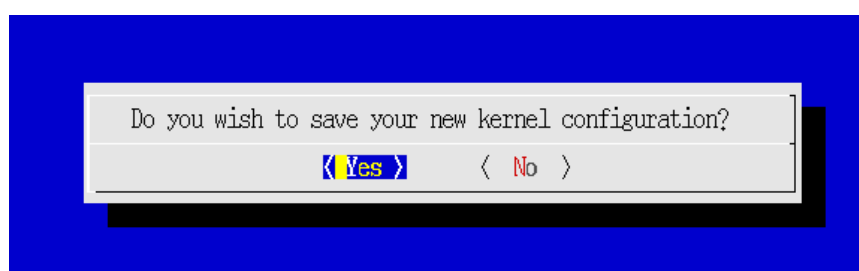
[Picture 6.3]

If you push enter key after erasing “.config” by back space key and inputting “arch/arm/def-configs/mba2440ramdisk”, setup of mba2440ramdisk will be applied to Linux kernel.



[Picture 6.4]

If you set up <Exit> and get out of the main menu setup window, you will find a message asking if you want to save setup as [Picture 6.4], choose <Yes>.



[Picture 6.5]

Compile kernel referring to [6.1.3 Perform kernel compile command](#) after saving it and getting out of there.

6.1.2.2 Set up Memory Size

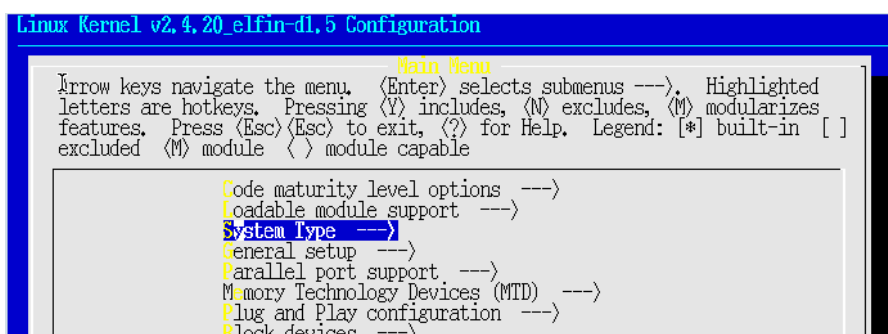
mba2440-linux2.4.20-v1.3-4.tar.bz2 Linux kernel source provided in CD is basically adapted to MBA2440 board attached on 64MB SDRAM.

You need to modify compile setup for MBA2440 board attached on 128MB

Perform make menuconfig command.

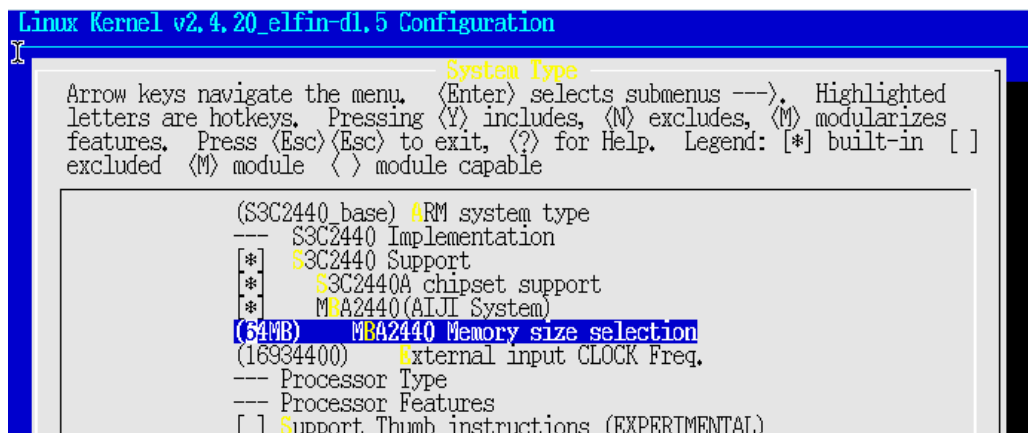
```
[root@localhost root]# make menuconfig
```

Select system type and push enter key.



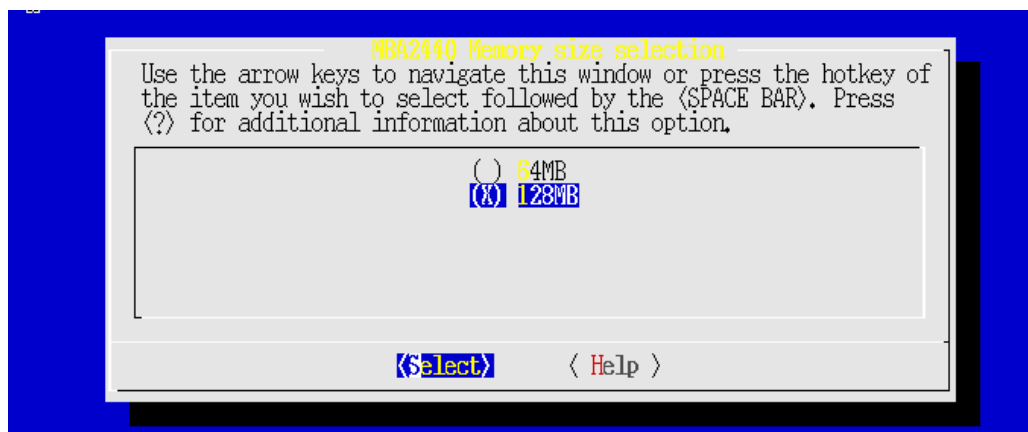
[Picture 6.6]

Select MBA2440 Memory size selection as [Picture 6.7] and push enter key.



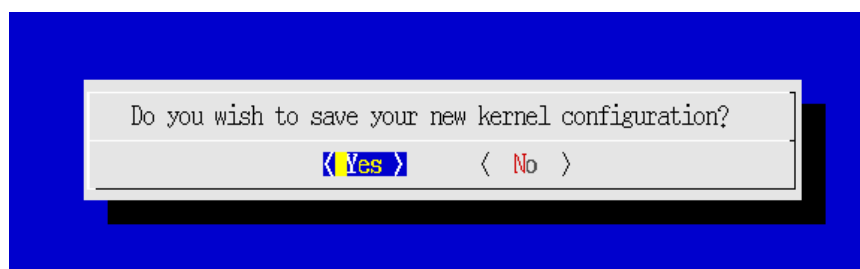
[Picture 6.7]

Select 128MB as [picture 6.8] and push space bar.



[Picture 6.8]

Get out through Exit until you find a menu asking if you want to save as [Picture 6.9].



[Picture 6.9]

After saving and getting out of there, compile kernel referring to [6.1.3 Perform kernel compile command](#).

6.1.2.3 TFT-LCD Setup

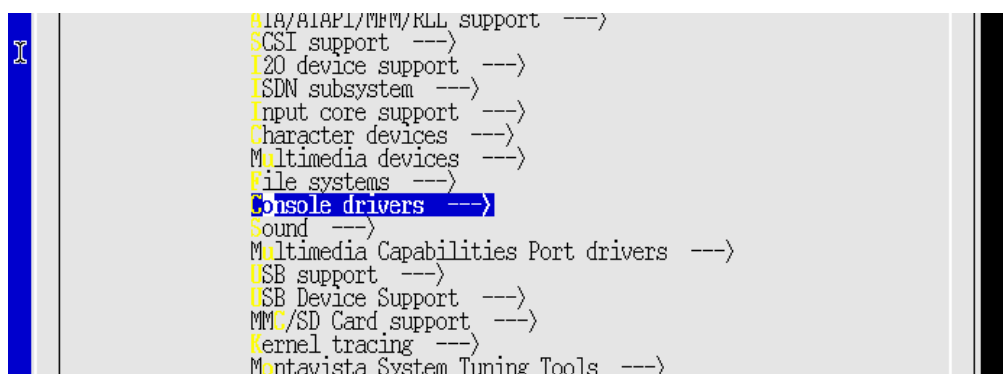
mba2440-linux2.4.20-v1.3-4.tar.bz2 Linux kernel source provided in CD is basically adapted to 240x320 3.5" TFT-LCD.

You need to modify kernel setup to operate it in accordance with 640x480 6.4" TFT-LCD.

Perform make menuconfig command.

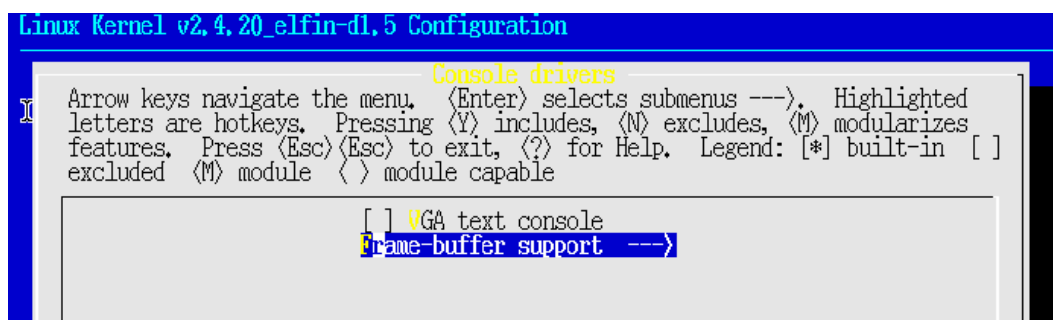
```
[root@localhost root]# make menuconfig
```

Select Console drivers.



[Picture 6.10]

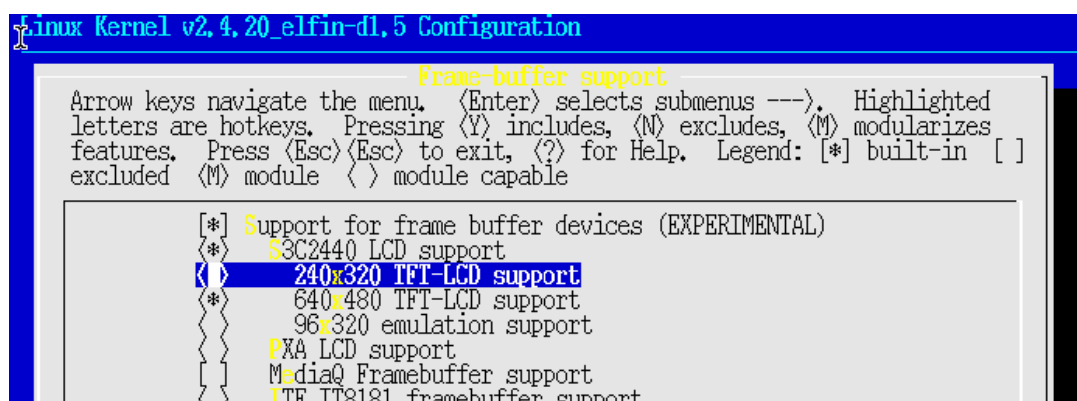
Select Frame-buffer support.



[Picture 6.11]

You will see * mark on 240x320 TFT-LCD support of [Picture 6.12]. Select 240x320 TFT-LCD support and remove * mark by pushing space bar.

Check * mark by pushing space bar twice after selecting 640x480 TFT-LCD support.



[Picture 6.12]

Get out through Exit until you find a message to ask if you want to save as [Picture 6.13].



[Picture 6.13]

Compile kernel referring to [6.1.3 Perform kernel compile command](#) after saving it and getting out of there.

6.1.3 Perform Kernel Compile Command

If kernel environment setup is all completed, you need to go through cleaning process to erase object files or images already created before compiling.

```
[root@localhost root]# make clean
```

“make clean” command removes object files, kernel images, temporary file and etc. created before.

Perform the command checking reliability relations between each source before compiling next.

```
[root@localhost root]# make dep
```

At last, perform the command creating images by compiling kernel.

```
[root@localhost root]# make zImage
```

List files in arch/arm/boot/ directory if kernel compile is completed. `zImage` in the directory is the kernel image.

You can boot Linux if you load the image into MBA2440.

```
[root@localhost root]# ls arch/arm/boot/
Makefile bootp compressed endianswap.c install.sh zImage
[root@localhost root]#
```

Please refer to [2.4 Booting Linux](#) for how to download and boot Linux kernel in MBA2440.

6.2 File System

When you boot Linux, you can determine what file system you select as root file system for booting.

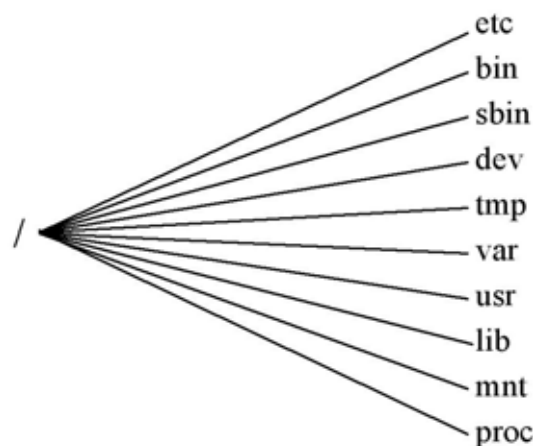
In this chapter, you will learn **RAMDISK file system** and **CRAMFS file system** which are file systems to be used as root file system.

You will also learn how to mount and use flash memory as nonvolatile save device for saving data using JFFS2 file system.

6.2.1 Root File System

Root file system (the highest directory in Linux “/”) is a crucial part in Linux as space where system memory space to operate kernel, library, utility and etc exist.

Root file system is structured as shown on [Picture 6.14], and it doesn't have big structure difference from the root file system operating desktop PC.



[Picture 6.14]

The following is a brief explanation regarding root file system.

[Table 6.1] Explanation regarding root file system

etc	It includes setup files according to each machine /etc/hosts There exist files needed for system management such as /etc/passwd and etc.
bin	There exist user's commands.

sbin	There exist files needed for booting each machine. When boot image boots, it reads them from Ramdisk file system and operate them.
dev	There exist nodes for devices such as tape drive, printer, disc, disc partition, RAMDISK, terminal.
tmp	There exist temporary files used in system.
var	Factors for applications such as log, lock, spool, mail and etc. are used.
usr	It includes files which are possible to operate and ASCII documents which are possible to share and there exist files which don't change. They also can be shared and used by another far-off server.
lib	There exist library files needed for system.
mnt	Space for mounting outer devices
proc	It provides movable information of system temporarily as imaginary file system

6.2.2 RAMDISK File System

Embedded system usually doesn't have hard disk and generally use flash memory for compressing and saving images of system because it is a slow save device which reads fast but writes slow.

Root file system is needed once kernel boots and starts operating.

In this chapter, let's learn about RAMDISK file system as root file system.

Root file system, user applicable program and etc. except kernel among images which will be loaded in MBA2440 board are consisted with disc images, loaded in memory before booted as Linux, and it takes a role of root file system after Linux boots.

RAMDISK image provided by embedded Linux providers is generally provided, and users usually use the way adding applicable programs developed into RAMDISK image.

Please refer to [2.4 Booting Linux](#) for how to boot Linux with RAMDISK as root file system.

6.2.2.1 RAMDISK File System

Let's learn about RAMDISK next.

RAMDISK file system provides a fast file system for kernel to read and write by letting us use a part of RAM as disc file system.

You can obtain RAMDISK image **mba2440-ram.gz** from linux/filesystem/ramdisk directory in CD provided.

You need to set up to use RAMDISK as root file system for Linux kernel compile setup as

explained earlier to use RAMDISK as root file system.

The set up file for this is **mba2440ramdisk** file in `arhc/arm/def-configs/` directory. You can choose and set up this file when you select setup file on [6.1.2.1 Load configuration file](#).

Refer to [6.1 Kernel compile](#) for compile process.

RAMDISK image is a compressed file in zip format, and it should be located on 0x30800000 address of memory. The RAMDISK can be mounted as 16MB size, and you can confirm this through `df` command on console window after booting MBA2440 as Linux.

For Linux kernel, you can find and extract a compressed RAMISK image on 3x30800000 address in the process of booting and mount it as root file system.

You can confirm all directories of root file system mounted on serial console (minicom or DNW and etc.) after Linux mounts and boots RAMDISK as root file system in MBA2440 as shown on [Picture 6.15].

```
syslogd
Starting portmapper: portmap
Starting network Starting network

+-----+
| Welcome to MBA2440 !!! |
+-----+

Linux login: root
[root@Linux /root]$ls /
lost+found  bin          dev          etc          lib          mnt
proc        rd          root         sbin         tmp          usr
var         conf
[root@Linux /root]$
```

[영어] [완성] [두벌식]

[Picture 6.15]

[Picture 6.16] shows the confirmation of RAMDISK mounted through “`df`” command.

```
[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$df
Filesystem      1k-blocks      Used Available Use% Mounted on
rootfs          15863          6226       8818   41% /
/dev/root       15863          6226       8818   41% /

[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$
```

[영어] [완성] [두벌식]

[Picture 6.16]

Rootfs shows root file system.

`/dev/root` means that RAMDISK is recognized as block device driver and mounted.

6.2.2.2 Modify RAMDISK

Files can be added or removed in RAMDISK according to the convenience of developers.
You can insert application operating file developed into RAMDISK image and operate it.

Let's modify RAMDISK.

You will learn how to modify RAMDISK according to the following orders.

1. Create a file named view.txt in /root directory of RAMDISK file system
2. Save a text, "Hello, MBA2440" into view.txt file created
3. Load it into memory using tftp command of u-boot after compressing RAMDISK
4. Load Linux kernel image into MBA2440 through tftp
5. Check if view.txt file is under /root directory after booting as Linux

There should be the existing RAMDISK image (mba2440-ram.gz) because it is modifying RAMDISK. Create a directory named /root/ramdisk and copy RAMDISK image (mba2440-ram.gz) provided in CD.

Create Directory

```
[root@localhost root]# mkdir /root/ramdisk
```

Copy RAMDISK image after inserting CD provided into CD-ROM on Linux PC.

```
[root@localhost root]# mount /mnt/cdrom
[root@localhost root]# cp /mnt/cdrom/filesystem/ramdisk/mba2440-ram.gz /root/ramdisk/
[root@localhost root]# cd /root/ramdisk
[root@localhost root]# ls
mba2440-ram.gz
[root@localhost root]#
```

Create a directory to recognize RAMDISK image compressed as file system after mounted.

The name of directory is ram_tmp.

```
[root@localhost root]# mkdir ram_tmp
[root@localhost root]# ls -al
합계 1852
drwxr-xr-x  3 root  root    4096  6월 27 20:39 .
drwxr-x--- 31 root  root    4096  6월 27 20:39 ..
-rw-r--r--  1 root  root   1878054  6월 27 20:39 mba2440-ram.gz
drwxr-xr-x  2 root  root    4096  6월 27 20:39 ram_tmp
```

Extract RAMDISK image.

```
[root@localhost root]# gzip -d mba2440-ram.gz
[root@localhost root]# ls -al
합계 16416
```



```
drwxr-xr-x    3 root    root          4096  6월 27 20:44 .
drwxr-x---   31 root    root          4096  6월 27 20:44 ..
-rw-r--r--    1 root    root      16777216  6월 27 20:39 mba2440-ram
drwxr-xr-x    2 root    root          4096  6월 27 20:39 ram_tmp
```

You can see the size of mba2440-ram image extracted extended to 16MB.

Connect RAMDISK image extracted with loopback device of Linux

```
[root@localhost root]# losetup /dev/loop0 mba2440-ram
```

Mount the loopback device connected into ram_tmp directory.

```
[root@localhost root]# mount /dev/loop0 ram_tmp
[root@localhost root]# cd ram_tmp
[root@localhost root]# ls
bin  conf  dev  etc  lib  lost+found  mnt  proc  rd  root  sbin  tmp  usr  var
[root@localhost root]#
```

Now create view.txt file in root directory, write "Hello, MBA2440", and save it

```
[root@localhost root]# cd root
[root@localhost root]# vi view.txt
```

Write Hello, MBA2440, save it in vi editor file and get out of there.

Dismount ram_tmp directory.

```
[root@localhost root]# cd ../../
[root@localhost root]# ls
mba2440-ram  ram_tmp
[root@localhost root]# umount ram_tmp
```

Disconnect loopback device.

```
[root@localhost root]# losetup -d /dev/loop0
```

Compress RAMDISK image modified as gzip.

```
[root@localhost root]# gzip mba2440-ram
[root@localhost root]# ls -al
합계 1852
drwxr-xr-x    3 root    root          4096  6월 27 20:53 .
drwxr-x---   31 root    root          4096  6월 27 20:53 ..
-rw-r--r--    1 root    root      1878985  6월 27 20:39 mba2440-ram.gz
drwxr-xr-x    2 root    root          4096  6월 27 20:39 ram_tmp
```

Let's boot Linux after downloading RAMDISK image compressed and kernel image provided in CD using tftp command of u-boot. (refer to [2.4.1 Booting Linux with TFTP](#))

After booting is completed, login with root account.

You can see view.txt created by "Is" command and confirm what the file has in it by "cat" command

```
eth0: using half-duplex 10Base-T (RJ-45)
IP-Config: Incomplete network configuration information.
NET4: Unix domain sockets 1.0/SMP for Linux NET4.0.
NetWinder Floating Point Emulator V0.95 (c) 1998-1999 Rebel.com
RAMDISK: Compressed image found at block 0
Freeing initrd memory: 4096K
EXT2-fs warning: checktime reached, running e2fsck is recommended
VFS: Mounted root (ext2 filesystem).
Freeing init memory: 76K
INIT: versio~ 2.74 booting
eth0: using half-duplex 10Base-T (RJ-45)
INIT: Entering runlevel: 3
Starting system logger: syslogd
Starting portmapper: portmap
Starting network Starting network
```

```
+-----+
| Welcome to MBA2440 !!! |
+-----+
```

```
Linux login: root
[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$ls -al
drwxr-x---  2 root    root      1024 Jun 27  2005 .
drwxr-xr-x 16 root    root      1024 Jun 27  2005 ..
-rw-r----- 1 root    root        208 Mar 15  2002 .profile
-rw-r--r--  1 root    root        15 Jun 27  2005 view.txt
[root@Linux /root]$cat view.txt
Hello, MBA2440
[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$
CTRL-A Z for help |115200 8N1 | NOR | Minicom 2.00.0 | VT102 | Offline
[영어] [완성] [무별식]
```

[Picture 6.17]

6.2.3 CRAMFS

In this chapter, you will learn what CRAMFS is basically and how to boot Linux with CRAMFS as root file system.

6.2.3.1 CRAMFS

CRAMFS is a file system newly introduced in Linux 2.4 version. The file system has been construction as a small sized file system for read-only.

CRAMFS can save all files as compressed and extract them during the operation, which is the biggest strength of CRAMFS. It can be usefully used as a file system of embedded system because it uses small memory space like this.

The space which will be used as system memory in RAMDISK file system becomes smaller because it uses the specific size of space as file system. If you use CRAMFS, you can use more space as system memory space because you don't use the memory space which you use in RAMDISK.

In the other hand, there could be a bothering matter copying applicable programs into RAM to activate applicable programs because files are compressed, and it could need a bigger sized RAM due to that.

It is impossible to write because it is a read-only file system.

Big sized file systems (over 256MB) can't be constructed.

You can use kernel image created by compiled after you perform compile setup to use Linux source for MBA2440 as CRAMFS.

The kernel setup file for this is **mba2440cramfs** file in arch/arm/def-configs/ directory of Linux kernel source for MBA2440. You can set up selecting **mba2440cramfs** file when you chose setup file on [6.1.2.1 Load configuration file](#).

Refer to [6.1.3 Perform kernel compile command](#) for compile process.

You can obtain a kernel image for CRAMFS of MBA2440 board attached with 240x320 3.5" TFT-LCD from linux/image/3.5 directory in CD. **zImage-cramfs3.5.bin**

You can also obtain CRAMFS file system image from zImage-cramfs3.5.bin Linux/image/filesystem/cramfs directory. **root-mba2440.cramfs**

6.2.3.2 Boot Linux with CRAMFS

Let's boot Linux using Linux kernel image for CRAMFS in CD and CRAMFS image.

This is performed by using u-boot on console of MBA2440 board.

CRAMFS images should be loaded in NAND flash memory.

Download CRAMFS image with TFTP and load it NAND flash first.

Don't forget to copy Linux kernel image for CRAMFS and CRAMFS image into root directory of TFTP server.

```
MBA2440 # tftp 30000000 root-mba2440.cramfs
TFTP from server 192.168.100.2; our IP address is 192.168.100.3
Filename 'root-mba2440.cramfs'.
Load address: 0x30000000
Loading:-
done
Bytes transferred = 2199552 (219000 hex)
```

After removing the file system of NAND flash, write on the spot. We will explain about file system area in [6.2.4 JFFS2 File System](#).

```
MBA2440 # nande 128 383
erase block [128 ~ 383]
Erasing block ../
Complete erasing block

MBA2440 # nandw t 30000000 200000 220000
Write start point : block[128] page[0] size = 0x220000
Writing data .../
Write complete(0x140000): from 0x30000000 SDRAM to 0x200000(128) NAND flash
```

You can boot after downloading Linux kernel image for CRAMFS.

```
MBA2440 # tftp 30c00000 zImage-cramfs3.5.bin
TFTP from server 192.168.100.2; our IP address is 192.168.100.3
Filename 'mba2440/zImage-cramfs.bin'.
Load address: 0x30c00000
Loading:/
done
Bytes transferred = 746012 (b621c hex)
MBA2440 # go 30c00000
Uncompressing Linux..... done, booting the kern.
Linux version 2.4.20_elfin-d1.5 (root@localhost.localdomain) (gcc version 2.95.3 20010315 (releaT
CPU: ARM/CIRRUS Arm920Tid(wb) revision 0
.....
.....
.....
+-----+
|      Welcome to MBA2440  !!!      |
+-----+

MBA2440(S3C2440) Linux (experimental)
Kernel 2.4.20_elfin-d1.5 on armv4l

console=/dev/console
init started: BusyBox v0.60.3 (2002.05.13-08:36+0000) multi-call binary
Starting pid 30, console /dev/console: '/etc/init.d/rcS'
```

```
eth0: using half-duplex 10Base-T (RJ-45)
Waiting for enter to start '/bin/sh' (pid 38, terminal /dev/console)

Please press Enter to activate this console.
```

You will find a prompt shown if you push the enter key.

```
USB Function Ethernet Driver Interface
NET4: Linux TCP/IP 1.0 for NET4.0
IP Protocols: ICMP, UDP, TCP, IGMP
IP: routing cache hash table of 512 buckets, 4Kbytes
TCP: Hash tables configured (established 2048 bind 2048)
eth0: using half-duplex 10Base-T (RJ-45)
IP-Config: Incomplete network configuration information.
NET4: Unix domain sockets 1.0/SMP for Linux NET4.0.
NetWinder Floating Point Emulator V0.95 (c) 1998-1999 Rebel.com
VFS: Mounted root (cramfs filesystem) readonly.
Mounted devfs on /dev
Freeing init memory: 76K
mount /etc as ramfs
re-create the /etc/mtab entries

+-----+
| Welcome to MBA2440 !!! |
+-----+

MBA2440(S3C2440A) Linux (experimental)
Kernel 2.4.20_elfin-dl.5 on armv4l

console=/dev/console
init started: BusyBox v0.60.3 (2002.05.13-08:36+0000) multi-call binary
Starting pid 30, console /dev/console: '/etc/init.d/rcS'
eth0: using half-duplex 10Base-T (RJ-45)
Waiting for enter to start '/bin/sh' (pid 38, terminal /dev/console)

Please press Enter to activate this console.
Starting pid 38, console /dev/console: '/bin/sh'

BusyBox v0.60.3 (2002.05.13-08:36+0000) Built-in shell (ash)
Enter 'help' for a list of built-in commands.

#
#
#
CTRL-A Z for help |115200 8N1 | NOR | Minicom 2.00.0 | VT102 | Offline
[영어] [완성] [무별식]
```

[Picture 6.18]

6.2.3.3 Modify CRAMFS

It is impossible for CRAMFS file system images to write even if you mount it into a directory because they are readonly images themselves differently from RAMDISK file system. Only reading is possible on them.

Then, how do you think you can modify CRAMFS?

It's simple. You can unloose CRAMFS images in the directory using utility (**cramfsck**) which extracts, modify them, and use utility (**mkcramfs**) creating the directory as CRAMFS image.

Utilities for CRAMFS are included in linux/filesystem/cramfs directory in CD.

It will be performed in a directory created under the name of /root/cramfs in Linux desktop PC.

Create a directory for CRAMFS images first and copy everything such as utility, CRAMFS images and etc. in it.

Insert CD into CD-ROM of Linux PC.

```
[root@localhost root]# mkdir /root/cramfs
[root@localhost root]# cd cramfs
[root@localhost root]# mount /mnt/cdrom
[root@localhost root]# cp /mnt/cdrom/filesystem/cramfs/* ./
[root@localhost root]# ls
cramfsck mkcramfs root-mba2440.cramfs
```

Create a directory to mount CRAMFS. Name the directory as tmp.

```
[root@localhost root]# mkdir tmp
[root@localhost root]# ls
cramfsck mkcramfs root-mba2440.cramfs tmp
[root@localhost root]# mount -t cramfs -o loop root-mba2440.cramfs tmp
[root@localhost root]# ls tmp
bin dev etc lib linuxrc mnt proc sbin tmp usr var
```

Create another directory.

```
[root@localhost root]# vi tmp/temp.txt
```

Write anything in file contents, save it by wq command and get out of there...

You did ok?

You will see like [Picture 6.19]. Of course it doesn't change properly, if only, those images get damaged.

That's because CRAMFS file system images are read-only as explained earlier.



[Picture 6.19]

Now, let's modify CRAMFS file system images.

Copy another CRAMFS file system image which is in the condition as it is because CRAMFS file system image has been already damaged.

Of course, you need to dismount tmp directory mounted earlier.

```
[root@localhost root]# umount tmp
[root@localhost root]# rm -f root-mba2440.cramfs
[root@localhost root]# cp /mnt/cdrom/filesystem/cramfs/root-mba2440.cramfs ./
```

Extract CRAMFS file system image in tmp directory. The utility used for this is cramfsck. You can get the utility from internet.

```
[root@localhost root]# ./cramfsck -h
usage: ./cramfsck [-hv] [-x dir] file
-h          print this help
-x dir      extract into dir
-v          be more verbose
file        file to test
[root@localhost root]# ./cramfsck -x tmp root-mba2440.cramfs
[root@localhost root]# ls tmp
bin  dev  etc  lib  linuxrc  mnt  proc  sbin  tmp  usr  var
```

Open linuxrc file and modify it as the following

```
[root@localhost root]# vi tmp/temp.txt
```

temp.txt file

```
temp data ..
```

Write a sentence, save it and get out of there.

Now create CRAMFS file system image. The utility used for this is mkcramfs. You can get this utility from internet.

```
[root@localhost root]# ./mkcramfs tmp modified.cramfs
.....
-48.18% (-2168 bytes)  ln_x_init
-53.81% (-31596 bytes) pump
 38.46% (+10 bytes)    pump.sh
150.00% (+12 bytes)    var
Everything: 1240 kilobytes
Super block: 76 bytes
CRC: 112c8b5a
[root@localhost cramfs]# ls
cramfsck  mkcramfs  modified.cramfs  root-mba2440.cramfs  tmp
```

Operate modified.cramfs newly created after downloading it with TFTP. Refer to [6.2.3.2 Boot](#)

Linux with CRAMFS.

```
mount /etc as ramfs
re-create the /etc/mtab entries

+-----+
| Welcome to MBA2440 !!! |
+-----+

MBA2440(S3C2440A) Linux (experimental)
Kernel 2.4.20_elfin-d1.5 on armv4l

console=/dev/console
init started: BusyBox v0.60.3 (2002.05.13-08:36+0000) multi-call binary
Starting pid 30, console /dev/console: '/etc/init.d/rcS'

eth0: using half-duplex 10Base-T (RJ-45)
Waiting for enter to start '/bin/sh' (pid 38, terminal /dev/console)

Please press Enter to activate this console.
Starting pid 38, console /dev/console: '/bin/sh'

BusyBox v0.60.3 (2002.05.13-08:36+0000) Built-in shell (ash)
Enter 'help' for a list of built-in commands.

# ls
bin      etc      linuxrc  proc     temp.txt usr
dev      lib      mnt      sbin     tmp      var
# cat temp.txt
temp data ..
#
CTRL-A Z for help 115200 8N1 | NOR | Minicom 2.00.0 | VT102 | Offline
[영어] [완성] [두벌식]
```

[Picture 6.20]

You will find it printed out reflecting the modification as shown on [Picture 6.20].

6.2.4 JFFS2 File System

Embedded Linux kernel for MBA2440 uses MTD driver. You can NAND flash as a save device using this driver. The file system applied to NAND flash is JFFS2.

JFFS2 (Journaling Flash File System 2) is safe even when the power turns off suddenly and you don't need to check the file system next time you boot it again.

Flash memory has a life as long as it can endure tens of thousands of writing. JFFS2 is designed specially to use flash memory. It provides a function called wear leveling to stop reducing the life of flash memory.

And this function improves the life of flash memory dramatically by letting all areas of flash used evenly when saving data.

JFFS2 is efficient because it compresses and saves files differently from other JFFS versions.

Some areas of JFFS2 are consumed by journaling and garbage collection system.

Linux kernel for MBA2440 can automatically recognize NAND flash (SOP, SMC and all)

partitioning into JFFS2 file system only if NAND flash is connected when kernel booting.

The following two pictures show NAND flash partitioning and recognizing when kernel booting.

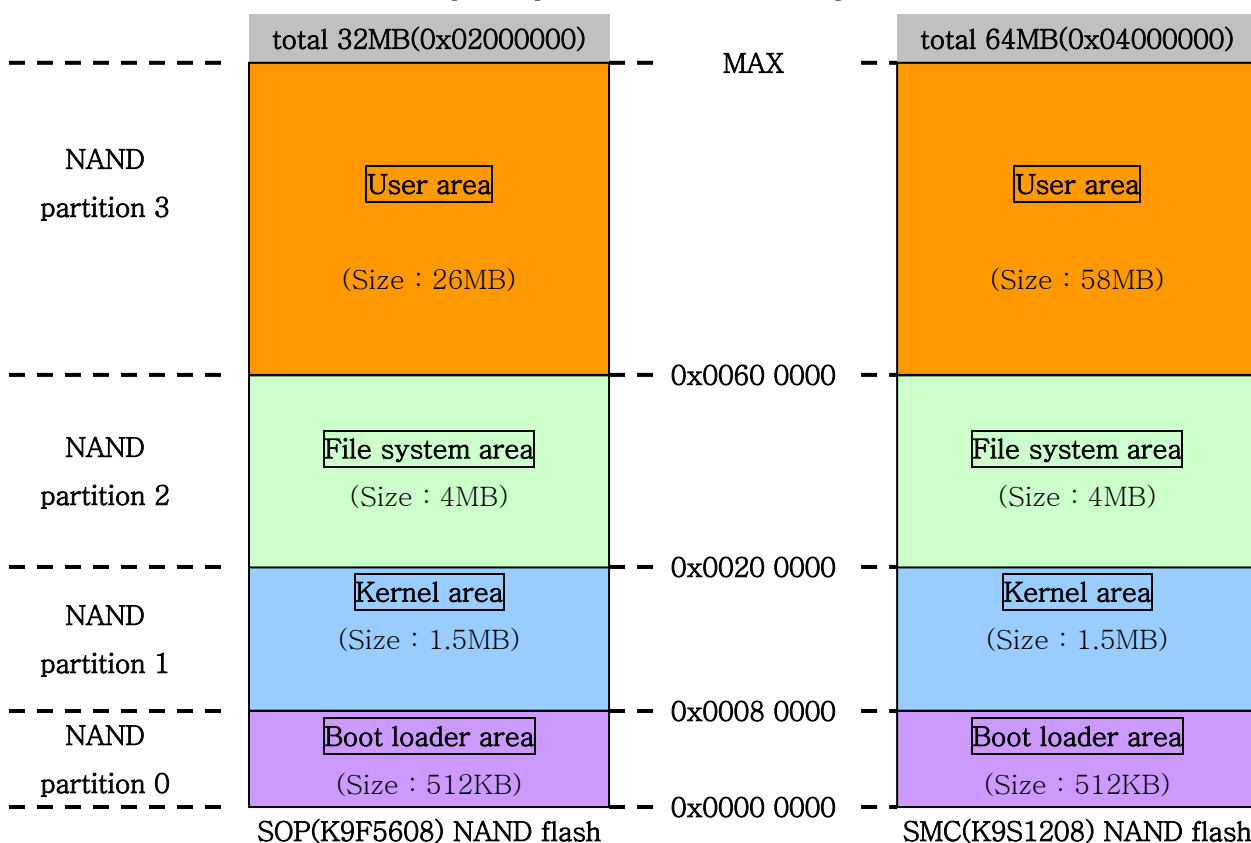
```
elfin init: Initialized IIC on S3CX, 5kHz clock
iic elfin init: initialized iic-bus at 0xf4000000.
ELEFN UDA1341 audio driver initialized
NAND device: Manufacture ID: 0xec, Chip ID: 0x75 (Samsung NAND 32MB 3.3V)
Creating 4 MTD partitions on "NAND 32MB 3.3V":
0x00000000-0x00080000 : "NAND partition 0 : Bootloader"
0x00080000-0x00200000 : "NAND partition 1 : Kernel"
0x00200000-0x00600000 : "NAND partition 2 : File System"
0x00600000-0x02000000 : "NAND partition 3 : User"
usb.c: registered new driver hub
S3C2440 USBD Controller Core Initialized
USB Function Ethernet Driver Interface
NET4: Linux TCP/IP 1.0 for NET4.0
IP Protocols: ICMP, UDP, TCP, IGMP
IP: routing cache hash table of 512 buckets, 4Kbytes
TCP: Hash tables configured (established 2048 bind 2048)
eth0: using half-duplex 10Base-T (RJ-45)
IP-Config: Incomplete network configuration information.
NET4: Unix domain sockets 1.0/SMP for Linux NET4.0.
```

[Picture 6.21] the result of SOP(K9F5608) NAND flash partitioning

```
elfin init: Initialized IIC on S3CX, 5kHz clock
iic elfin init: initialized iic-bus at 0xf4000000.
ELEFN UDA1341 audio driver initialized
NAND device: Manufacture ID: 0xec, Chip ID: 0x76 (Samsung NAND 64MB 3.3V)
Creating 4 MTD partitions on "NAND 64MB 3.3V":
0x00000000-0x00080000 : "NAND partition 0 : Bootloader"
0x00080000-0x00200000 : "NAND partition 1 : Kernel"
0x00200000-0x00600000 : "NAND partition 2 : File System"
0x00600000-0x04000000 : "NAND partition 3 : User"
usb.c: registered new driver hub
S3C2440 USBD Controller Core Initialized
USB Function Ethernet Driver Interface
NET4: Linux TCP/IP 1.0 for NET4.0
IP Protocols: ICMP, UDP, TCP, IGMP
IP: routing cache hash table of 512 buckets, 4Kbytes
TCP: Hash tables configured (established 2048 bind 2048)
eth0: using half-duplex 10Base-T (RJ-45)
IP-Config: Incomplete network configuration information.
NET4: Unix domain sockets 1.0/SMP for Linux NET4.0.
```

[Picture 6.22] the result of SMC(K9S1208) NAND flash partitioning

6.2.4.1 Memory Map of JFFS2 File System



[Picture 6.23] memory map of JFFS2 file system

It is the file system area (NAND partition 2) that writes on NAND flash after downloading CRAMFS image through TFTP on [6.2.3.2 Boot Linux with CRAMFS](#).

You shouldn't touch the files system area when you boot Linux with CRAMFS as root file system.

6.2.4.2 Partition mount

You can mount partition areas according to the intention of developers after kernel boots as shown on [Picture 6.15].

Here are tips!!

- In case of booting with CRAMFS as root file system, you shouldn't touch the file system area. Be aware that the root file system doesn't work if it is damaged while Linux kernel operates.
- You can mount and use all parts except boot loader area in case of booting with RAMDISK as root file system.

In this chapter, you will try to mount and use user area in /mnt/mtd3 directory.

It doesn't matter if you use CRAMFS or RAMDISK as root file system.

The following is how to operate it.

After the booting of embedded Linux kernel for MBA2440 is completed,

1. Erase user area.
2. Mount user area into /mnt/mtd3 directory.
3. Copy a file into /mnt/mtd3.
4. After the power turns off, check if there are files mounted and copied after booting Linux.

Erase user area. Use an utility named erase all to erase. This has been put in the file system in advance.

```
[root@Linux /root]$ eraseall /dev/mtd/3
Erased 26624 Kibyte @ 0 -- 100% complete.
```

Mount user area into /mnt/mtd3 directory.

```
[root@Linux /root]$ mount -t jffs2 /dev/mtdblock/3 /mnt/mtd3/
```

Copy a file. Copy the services file in /etc/ directory.

```
[root@Linux /root]$ cd /mnt/mtd3/
[root@Linux /root]$ ls
[root@Linux /mnt3]$ cp /etc/services ./
[root@Linux /mnt3]$ ls -al
-rw-r--r--  1 root  root    10788 Jul 31 00:01 services
```

Check if the services file has been copied and its size is 10778 bites.

Now turn off the power, wait for a second, and turn on the power again.

After booting Linux, mount it.

```
[root@Linux /root]$ mount -t jffs2 /dev/mtdblock/3 /mnt/mtd3/
[root@Linux /root]$ cd /mnt/mtd3/
[root@Linux /mnt3]$ ls -al
-rw-r--r--  1 root  root    10788 Jul 31 00:01 services
```

You can check if files exist.

6.3 Device Driver

Embedded Linux kernel for MBA2440 provides character GPIO button device driver and RTC (Real Time Clock) device driver for the external interrupt KEY1 ~ KEY4 button.

We only introduce GPIO button device driver for KEY1 ~ KEY4 buttons in the chapter 6.3.

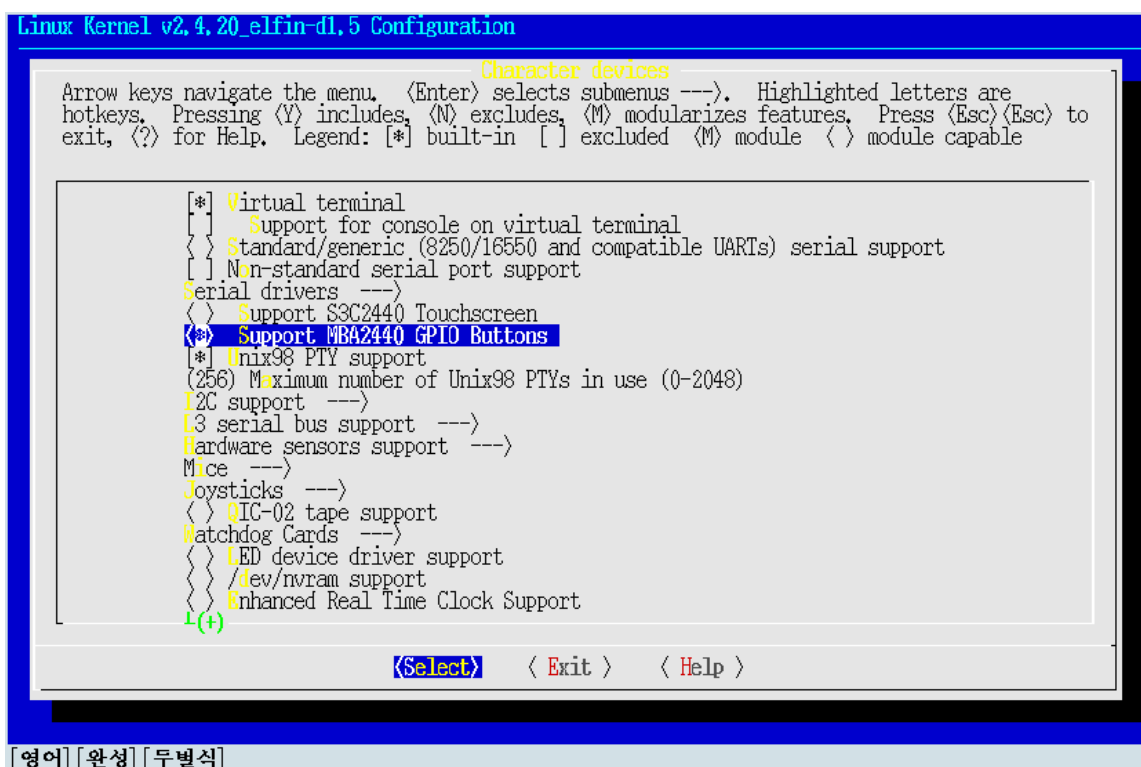
We will explain about RTC device driver and applications in [6.4 Application](#).

RAMDISK file system will be set up as root file system Linux kernel source will be compiled in the 6.3 chapter.

Copy and extract Linux kernel for MBA2440 in /root/mba2440 directory in CD provided. (refer to [6.1 Kernel compile](#))

Set up kernel compile through **make menuconfig**.

Select Character devices ---> on the main menu.



[Picture 6.24]

Push the space bar and keep selecting until you see * mark as shown on [Picture 6.24]. whenever you push the space bar, the mark in < > will change.

Save what you just set up, get out and compile kernel.

```
[root@localhost root]# make zImage
scripts/split-include include/linux/autoconf.h include/config
arm-linux-gcc -D__KERNEL__ -I/root/mba2440/mba2440-linux2.4.20-v1.0/include -Wall -Wstrict-
```

```

prototypes -Wno-trigraphs -Os -fno-strict-aliasing -fno-common -ggdb -Uarm -fno-common -
pipe -mapcs -mno-sched-prolog -g -mapcs-32 -D__LINUX_ARM_ARCH__=4 -march=armv4 -
mtune=arm9tdmi -mshort-load-bytes -msoft-float -DKBUILD_BASENAME=main -c -o init/main.o
init/main.c
.....

```

If compiling is completed, upload zImage kernel image in arch/arm/boot/ directory to MBA2440 through u-boot, and boot kernel. (Refer to [2.4.1 Booting Linux with TFTP](#))

After the booting is completed, push any button among KEY1 ~ KEY4. You will see a sentence shown on the screen.

KEY1 ~ KEY4 is set up as external interrupt, and you will meet interrupt twice when you push the button and you take your finger off from the button because it is set up as both edge detect.

```

eth0: using half-duplex 10Base-T (RJ-45)
INIT: Entering runlevel: 3
Starting system logger: syslogd
Starting portmapper: portmap
Starting network Starting network

+-----+
| Welcome to MBA2440 !!! |
+-----+

Linux login: root
[root@Linux /root]$KEY1(EINT0), st = 1
KEY1(EINT0), st = 0
KEY2(EINT2), st = 1
KEY2(EINT2), st = 0
KEY3(EINT8), st = 1
KEY3(EINT8), st = 0
KEY4(EINT19), st = 1
KEY4(EINT19), st = 0

[root@Linux /root]$
[root@Linux /root]$
CTRL-A Z for help 115200 8N1 | NOR | Minicom 2.00.0 | VT102 | Offline
[영어] [완성] [두벌식]

```

[Picture 6.25]

The source of GPIO button device driver for MBA2440 is the mba2440_gpio_button.c file in drivers/char/ directory of embedded Linux kernel source.

Please analyze the source yourself.

6.4 Application

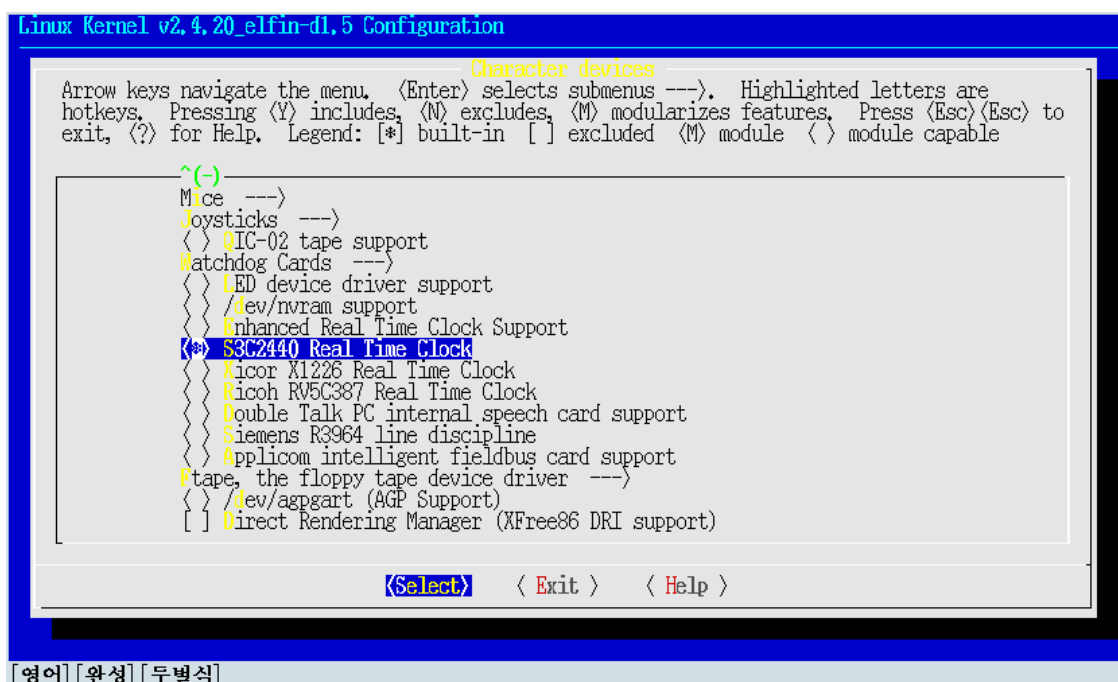
In this chapter, you will operate the application using RTC device driver and RTC device driver.

There are the RTC test application operating file and source in linuxWapplication directory in CD provided.

Compile device driver first.

Set up kernel compile through make menuconfig.

Select Character devices ---> on the main menu.



[Picture 6.26]

Set it up to show * mark as [Picture 6.26], save setup and get out.

Compile kernel source.

```
[root@localhost root]# make zImage
scripts/split-include include/linux/autoconf.h include/config
arm-linux-gcc -D__KERNEL__ -I/root/mba2440/mba2440-linux2.4.20-v1.0/include -Wall -Wstrict-
prototypes -Wno-trigraphs -Os -fno-strict-aliasing -fno-common -ggdb -Uarm -fno-common -
pipe -mapcs -mno-sched-prolog -g -mapcs-32 -D__LINUX_ARM_ARCH__=4 -march=armv4 -
mtune=arm9tdmi -mshort-load-bytes -msoft-float -DKBUILD_BASENAME=main -c -o init/main.o
init/main.c
.....
```

Now, RTC device driver is included in kernel image.

Copy **rtc-app** file which is an application operation file to use RTC in CD into the /nfsroot

directory.

```
[root@localhost root]# cp /mnt/cdrom/app/rtc/rtc-app /nfsroot
[root@localhost root]# ls /nfsroot
ram2440.gz  root-mba2440.cramfs  rtc-app  zImage-cramfs.bin  zImage.bin
```

Upload zImage kernel image in arch/arm/boot/ to MBA2440 through u-boot, and boot kernel.
(Refer to [2.4.1 Booting Linux with TFTP](#))

Mount /mnt/nfs into /nfsroot directory of Linux PC using NFS after booting.

Copy **rtc-app** into /root directory after mounting is completed.

[Picture 6.27] shows the page shown after operating the above process.

```
eth0: using half-duplex 10Base-T (RJ-45)
INIT: Entering runlevel: 3
Starting system logger: syslogd
Starting portmapper: portmap
Starting network Starting network

+-----+
| Welcome to MBA2440 !!! |
+-----+

Linux login: root
[root@Linux /root]$ mount -t nfs 192.168.100.2:/nfsroot /mnt/nfs/
nfs warning: mount version older than kernel
[root@Linux /root]$ ls /mnt/nfs/
zImage.bin          ram2440.gz          zImage-cramfs.bin
root-mba2440.cramfs  rtc-app             s3c2440-rtc.o
mba2440_gpio_button.o
[root@Linux /root]$
[root@Linux /root]$ cp /mnt/nfs/rtc-app .
[root@Linux /root]$ ls
rtc-app
[root@Linux /root]$
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.00.0 | VT102 | Offline
[영어] [완성] [두벌식]
```

[Picture 6.27]

You should create a device file for RTC device driver in /dev/ directory before operating rtc-app application. It takes a role of connecting RTC application and RTC device driver.

Use “mknod” command. Perform the command as [Picture 6.28].

```
[root@Linux /root]$ mknod /dev/rtc c 10 135
[root@Linux /root]$ ls /dev/rtc
/dev/rtc
[root@Linux /root]$ ls -al /dev/rtc
crw-r--r--  1 root   root    10, 135 Dec  1 00:36 /dev/rtc
[root@Linux /root]$
```

[영어] [완성] [두벌식]

[Picture 6.28]

Perform the command as [Picture 6.29].

```
[root@Linux /root]$. /rtc-app
program start

2094년 24월 45일 31시 6분 45초

Set time you want..
enter year : 2005

enter month : 6

enter day : 20

enter hour : 14

enter minute: 24

enter second : 20

2005, 6, 20, 14, 24, 20

2005년 6월 20일 14시 24분 20초

program exit
[root@Linux /root]$
```

CTRL-A Z for help 115200 8N1 NOR Minicom 2.00.0 VT102 Offline

[영어] [완성] [두벌식]

[Picture 6.29]

Perform as [Picture 6.30] after a while.

```
[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$
[root@Linux /root]$. /rtc-app display
2005년 6월 20일 14시 25분 45초
[root@Linux /root]$
```

[영어] [완성] [두벌식]

[그림 6.30]

You can find the second changed based on the time set up.

6.5 MBA2440 Linux Tips

6.5.1 Mount USB memory

S3C2440 support USB HOST controller(USB HOST 1.1 compatible). The Linux kernel of MAB2440 support the function of mount USB mass storage.

Check the following when linux kernel compile...

Following the explanation after performing “**make menuconfig**” command

File systems -->

[*] Quota support

<*> DOS FAT fs support

<*> MSDOS fs support

<*> VFAT (Windows-95) fs support

<*> NTFS file system support (read only)

(- write write support는 체크 안함(2.4.18버전에서는 불안정함))

Partition Types --->

[*] Advanced partition selection

[*] PC BIOS (MSDOS partition tables) support

SCSI support --->

<*> SCSI support?

<*> SCSI disk support (NEW)

(40) Maximum number of SCSI disks that can be loaded as modules (NEW)

<*> SCSI generic support (NEW)

ATA/IDE/MFM/RLL support --->

<*> ATA/IDE/MFM/RLL support

IDE, ATA and ATAPI Block devices --->

<*> Enhanced IDE/MFM/RLL disk/cdrom/tape/floppy support

<*> Include IDE/ATA-2 DISK support (NEW)

<*> SCSI emulation support (NEW)

USB support --->

<*> Support for USB

[*] Preliminary USB device filesystem (NEW)

[*] Long timeout for slow-responding devices (some MGE Ellipse UPSes) (NEW)

--- USB Controllers

<*> S3C2410 OHCI-compatible host interface support

--- USB Device Class drivers

<*> USB Mass Storage support

After completing compile the linux kernel, Booting the MBA2440 using created linux kernel.

Let's connect USB memory stick to USB host port of MBA2440, and then you can see the following message.

```
-----
hub.c: new USB device usb-ohci-1, assigned address 2
scsi1 : SCSI emulation for USB Mass Storage devices
  Vendor: SONY      Model: Storage Media    Rev: 1.00
  Type:   Direct-Access          ANSI SCSI revision: 02
Attached scsi removable disk sda at scsi1, channel 0, id 0, lun 0
SCSI device sda: 507904 512-byte hdwr sectors (260 MB)
sda: Write Protect is off
sda: sda1 sda2 sda3 sda4
-----
```

Let's mount the USB memory stick.

```
[root@localhost root]# mount -t vfat /dev/sda /mnt/usbmem
[root@localhost root]# ls /mnt/usbmem
matrix.mpg          nangman_teaser.wmv  starwars.mpeg
[root@localhost root]# df
```

Filesystem	1k-blocks	Used	Available	Use	Mounted on
rootfs	15863	7195	7849	48/	
/dev/root	15863	7195	7849	48/	
/dev/sda	252944	84202	168742	33/	/mnt/usbmem

7 WinCE

In this chapter, you will learn how to compile and create images through WinCE 5.0 BSP (Board Support Package) and WinCE 5.0 Platform Builder, WinCE development process and eboot for booting WinCE.

Platform Builder of WinCE5.0 should be installed before you perform this chapter.

7.1 WinCE 5.0 BSP for MBA2440

You can obtain BSP for WinCE5.0 named **MBA2440_ARM4_v2.0.zip** from WinCE5.0 directory in CD.

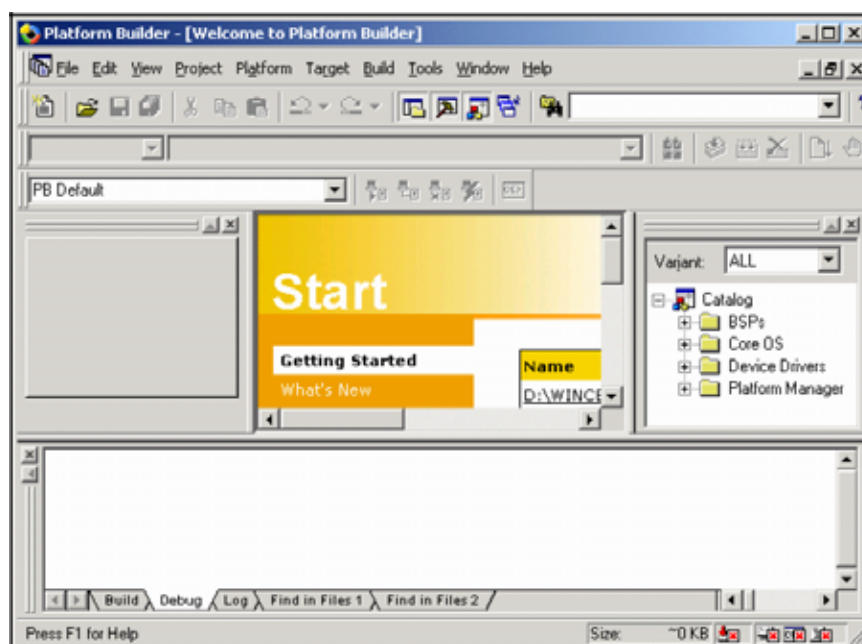
MBA2440 is designed to be able to load 3.5”(240x320)TFT-LCD and 6.4”(640x480) TFT-LCD. You can use WinCE5.0 BSP for MBA2440 modifying source according to each TFT-LCD.

BSP provided in CD is basically designed to operate in 3.5” TFT-LCD and support English.

7.1.1 BSP Build

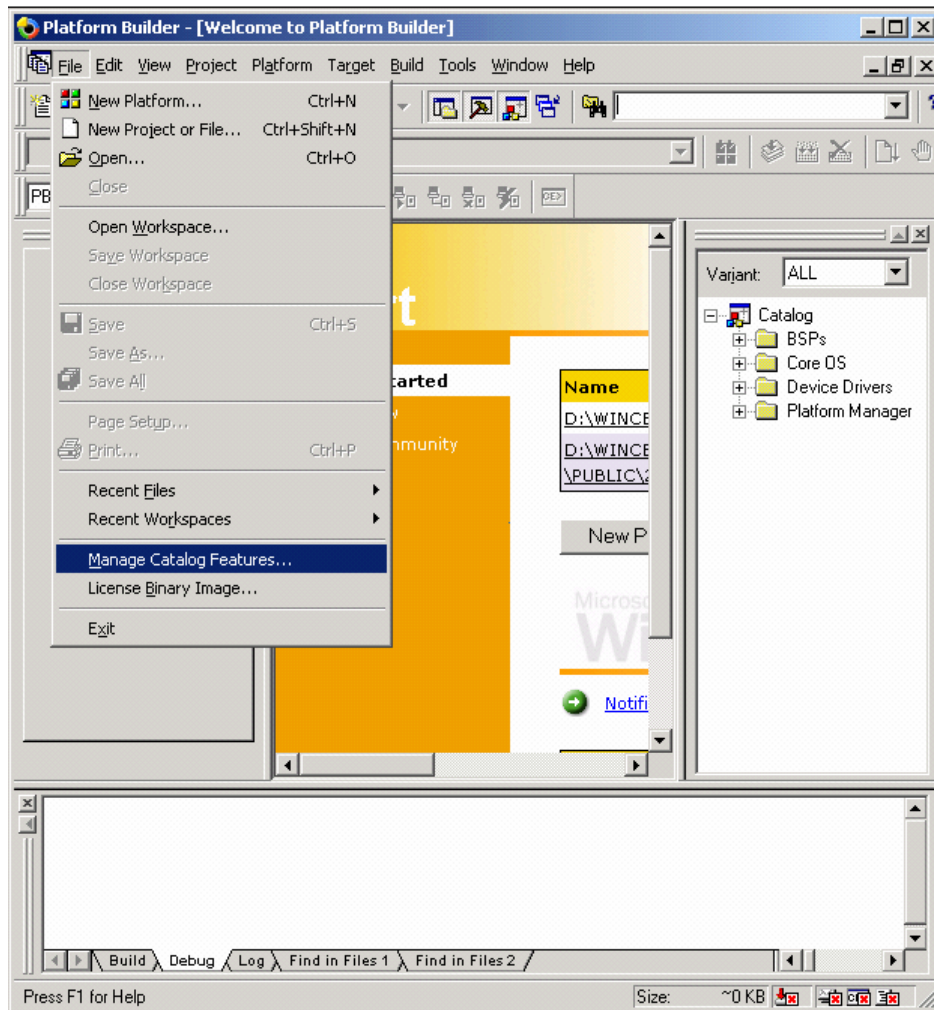
Let's compile BSP now.

1. If you copy **MBA2440_ARM4_v2.0.zip** in WinCE5.0 directory in CD mentioned above and extract it, a directory named **MBA2440_ARM4_v2.0** will be created. Modify its name to **MBA2440_ARM4** and copy it into C:/WINCE5.0/PLATFORM/ as it is.
2. Operate WinCE.

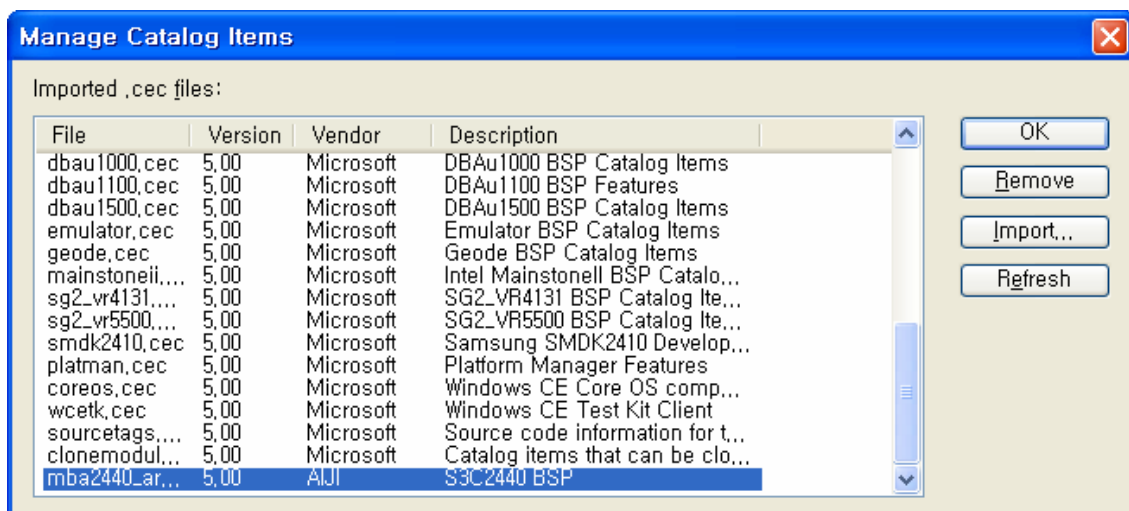


[Picture 7.1] Showing WinCE5.0 Operated

3. Operate Files→Mange Catalog Features..., select MBA2440_ARM4.cec file in BSP directory copied from Manage Catalog Features window, and click import button.

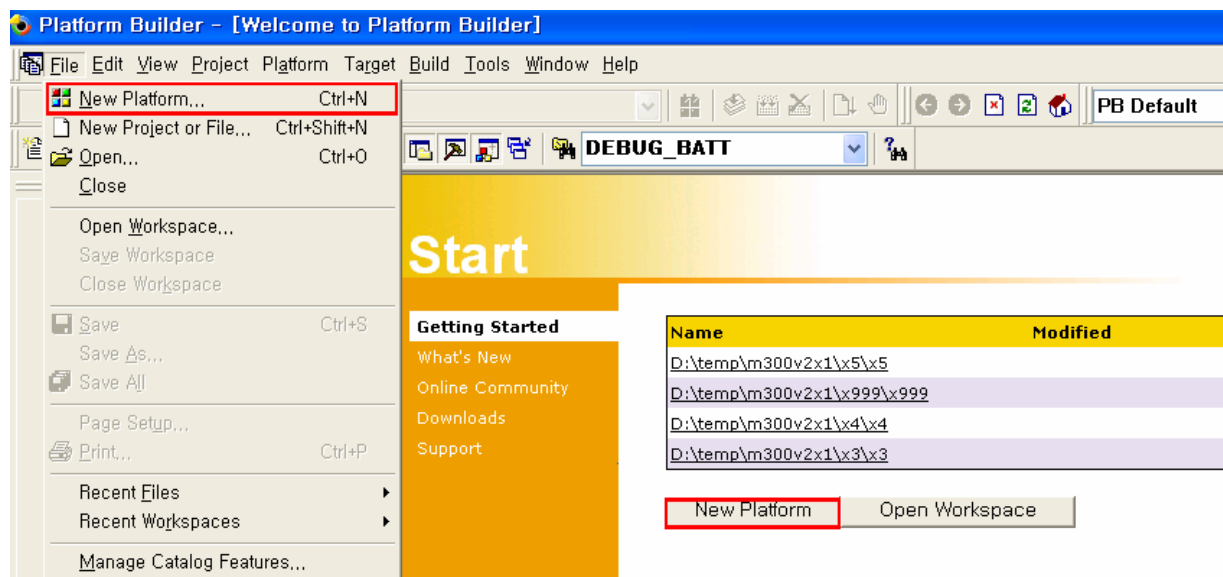


[Picture 7.2]



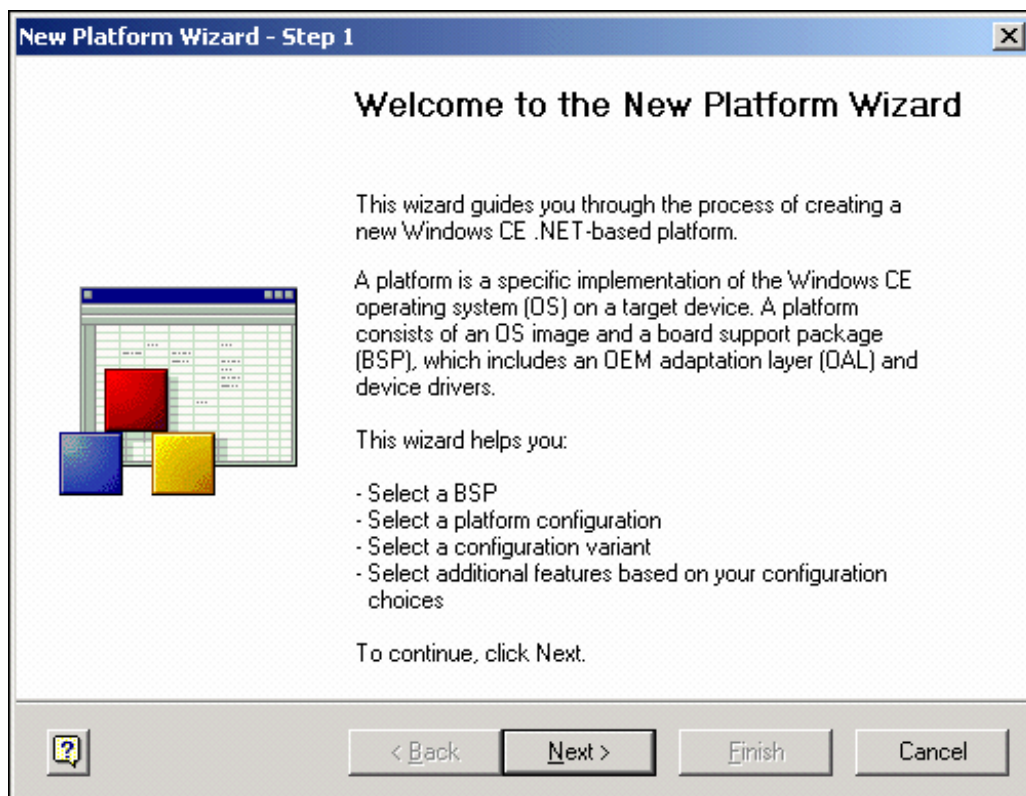
[그림 7.3] MBA2440_ARM4.cec file Import

4. Select Files→New Platform..., or Push New Platform button in the center of Platform builder.



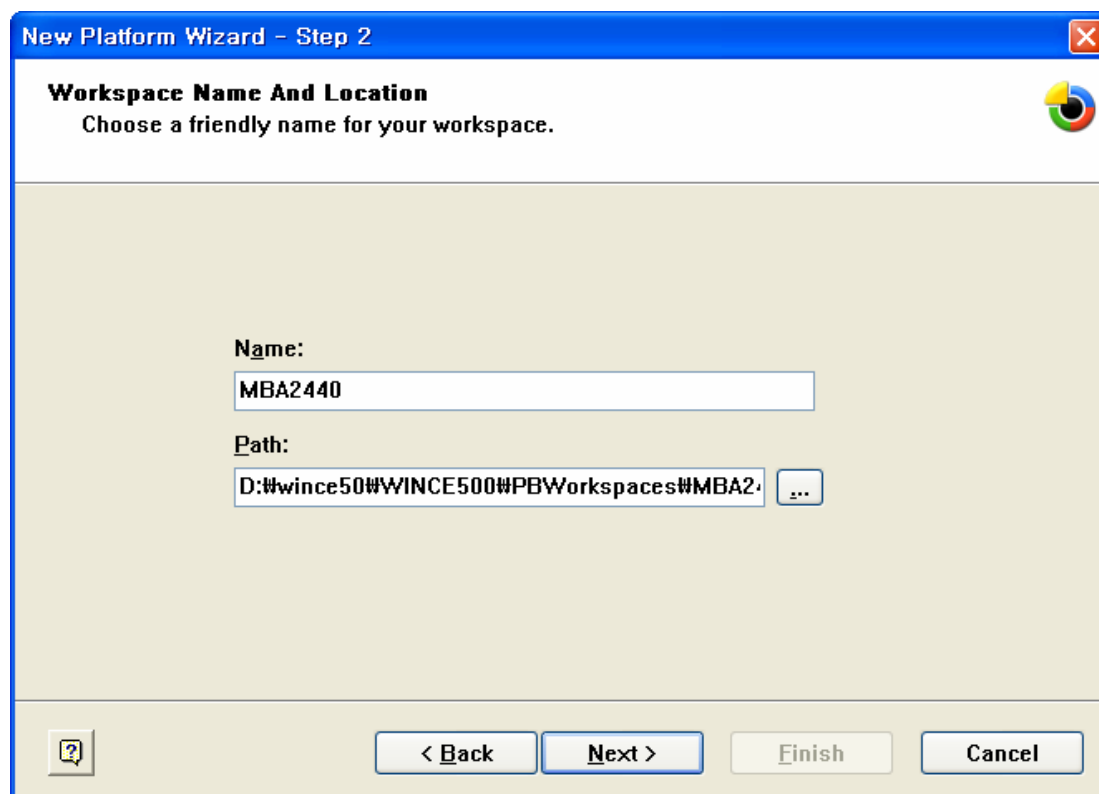
[Picture 7.4]

5. Push Next button on Step 1 window.



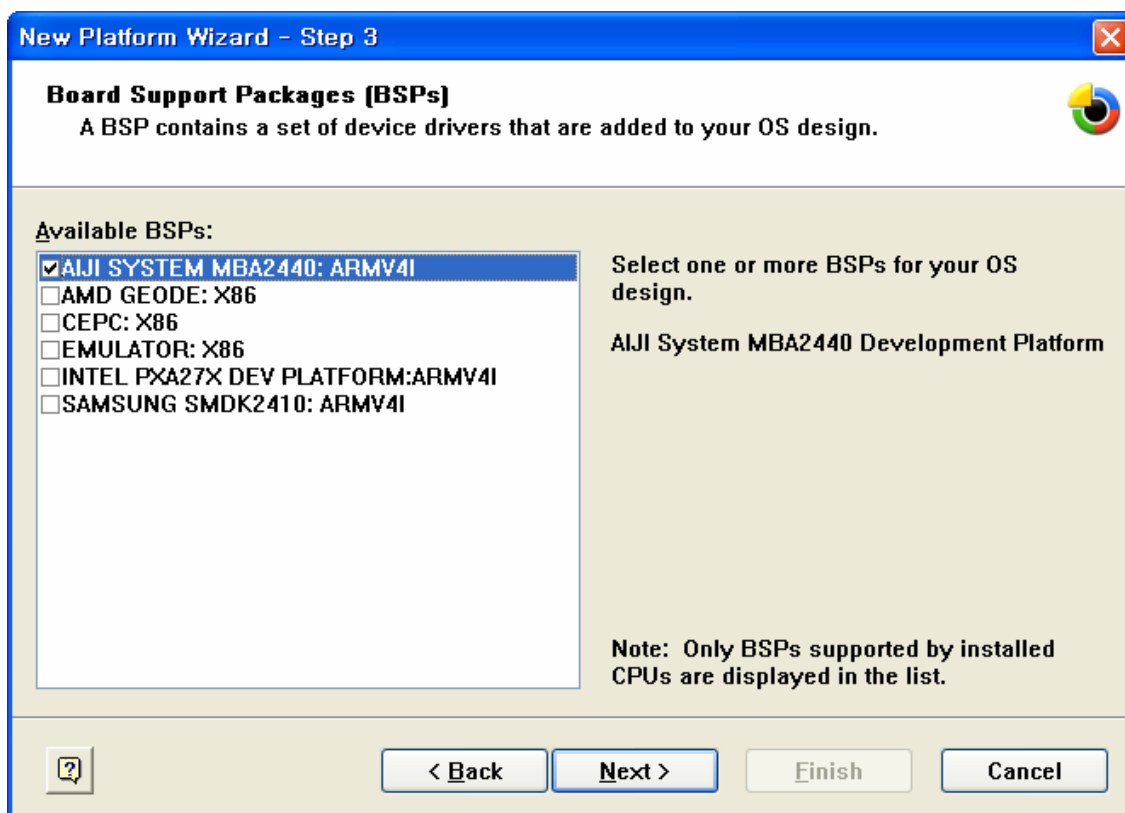
[Picture 7.5]

6. Set up a workspace name you want in Step 2.



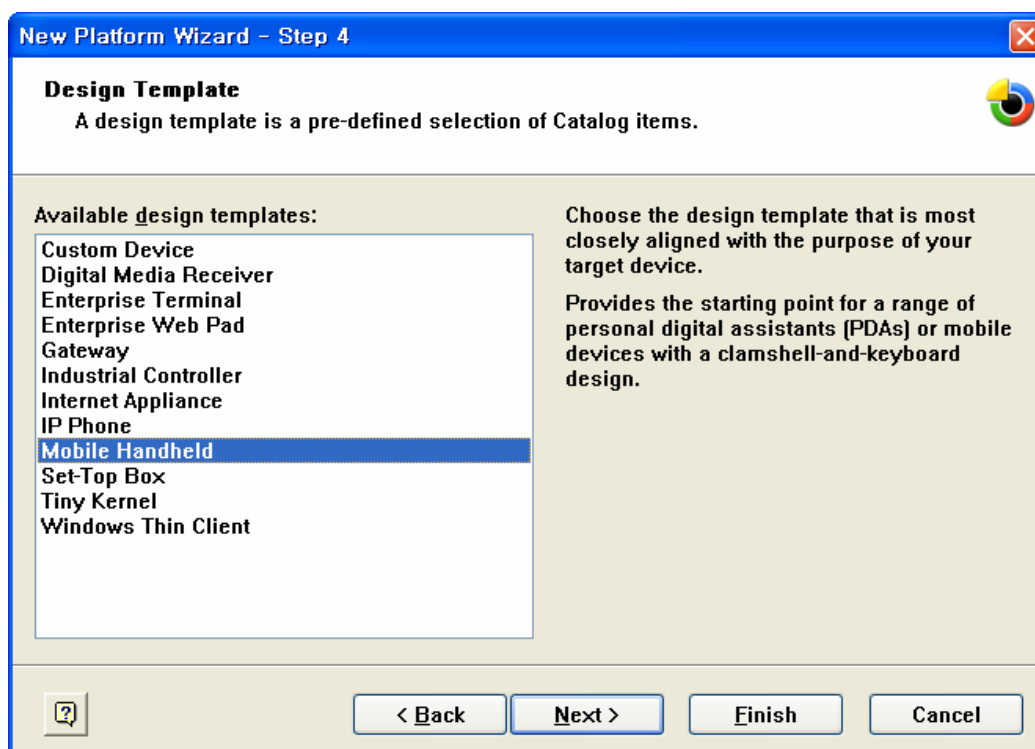
[Picture 7.6]

7. Select AIJI SYSTEM MBA2440: ARMV4I in Step 3 window, and push Next button.



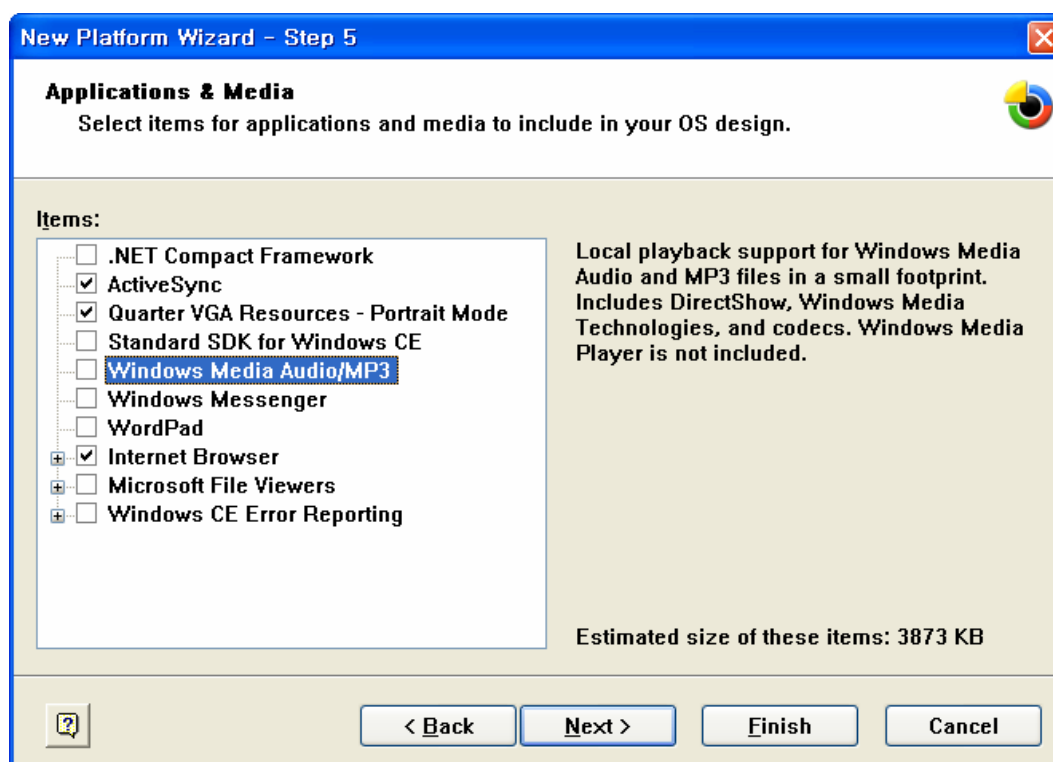
[Picture 7.7]

8. Select Mobile Handheld in Step 4 and input a platform name you want on Platform name. You will input mba2440_test in this document.



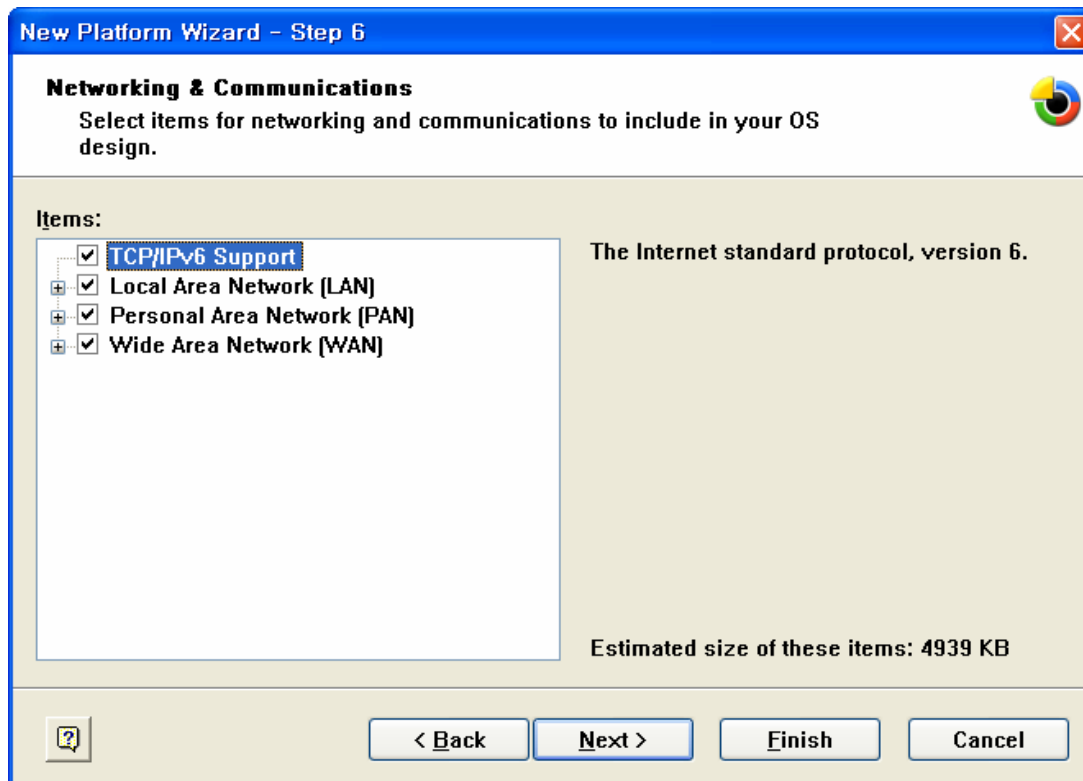
[Picture 7.8]

9. Check as [Picture 7.9] in step 5.



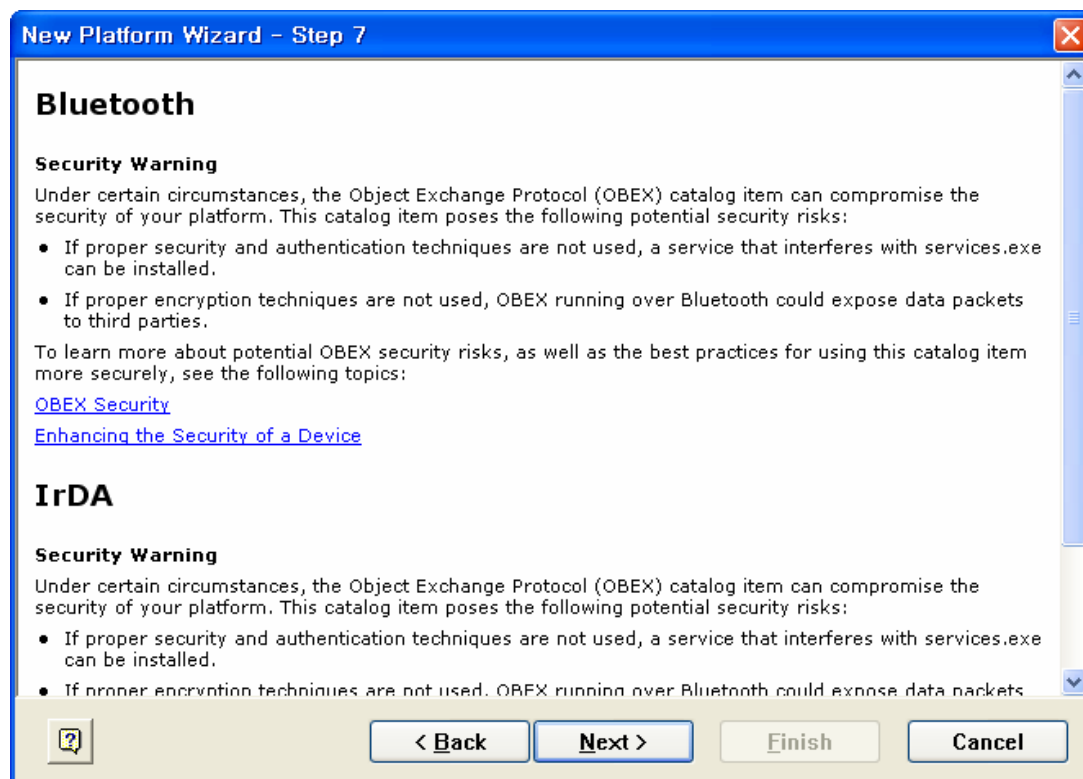
[Picture 7.9]

10. Just push Next button without any modification in Step 6.



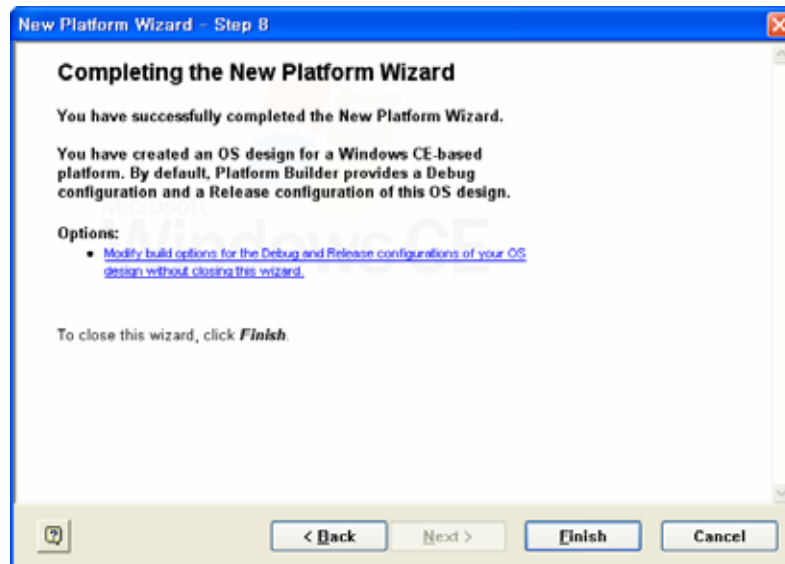
[Picture 7.10]

11. Just push Next button in Step 7.



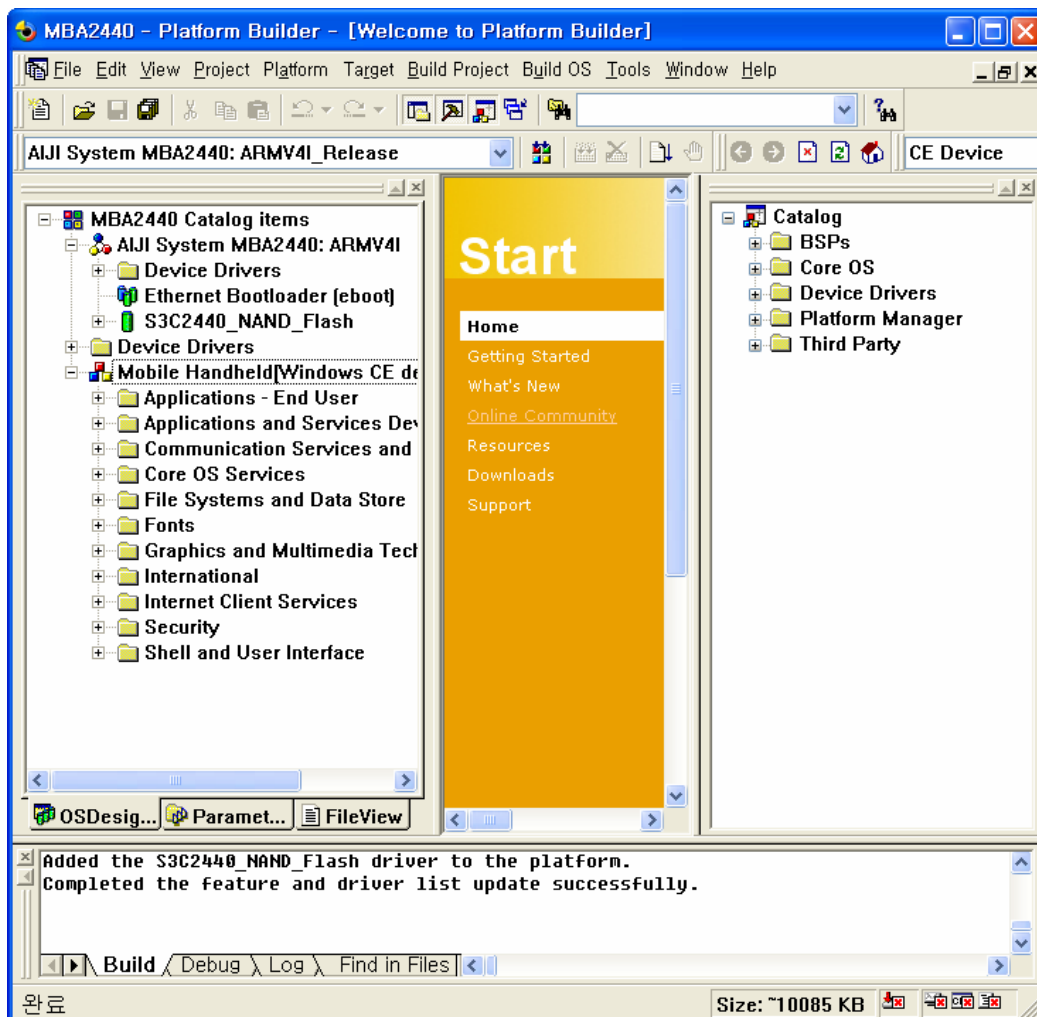
[Picture 7.11]

12. Complete New Platform Wizard by pushing Finish button in Step 8.



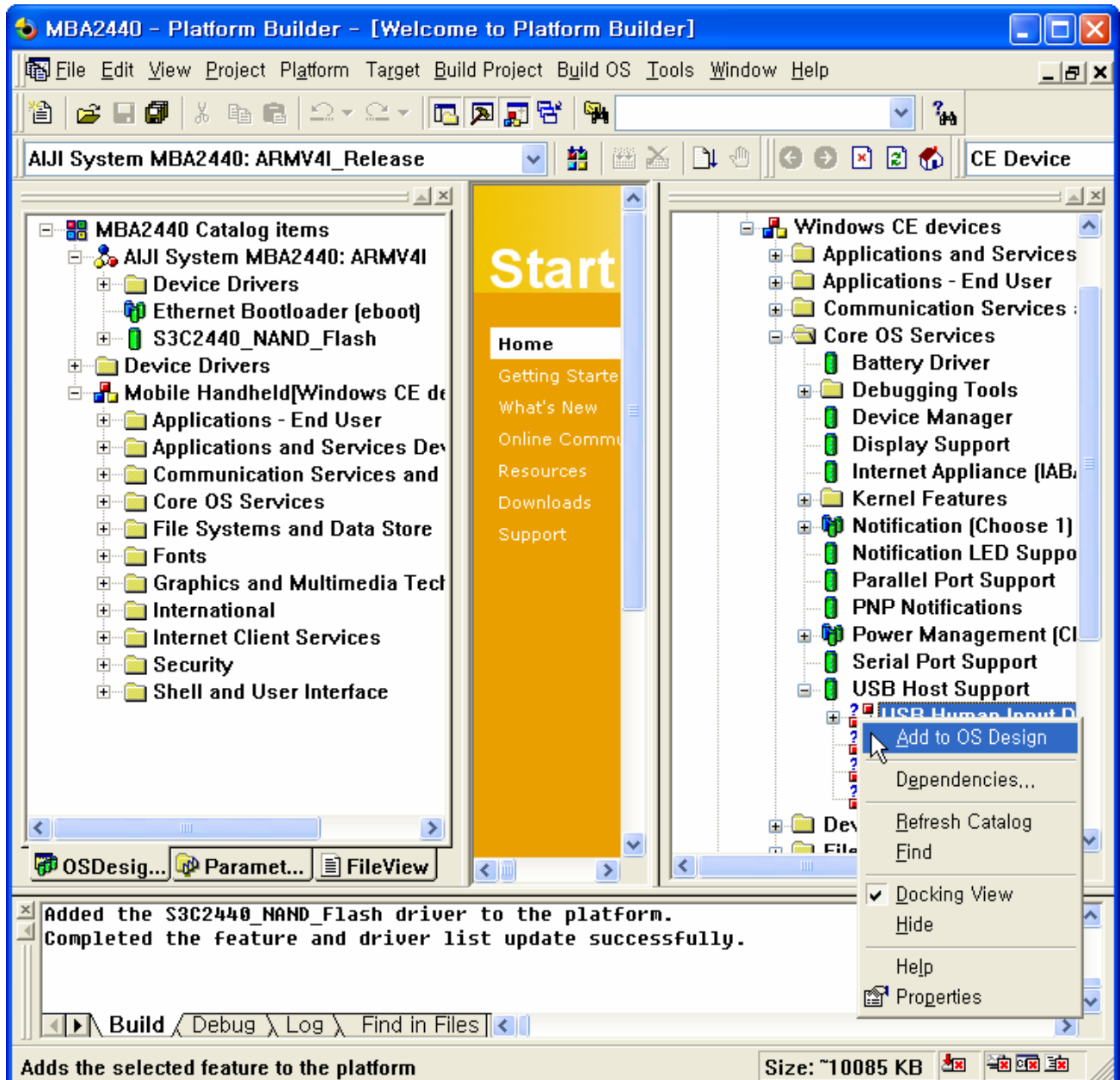
[Picture 7.12]

13. The page like on [Picture 7.13] will be shown after completing New Platform Wizard.



[Picture 7.13]

14. Add the device driver on a catalogue window on the right side and a part needed on Core OS into the platform newly created.



[Picture 7.14]

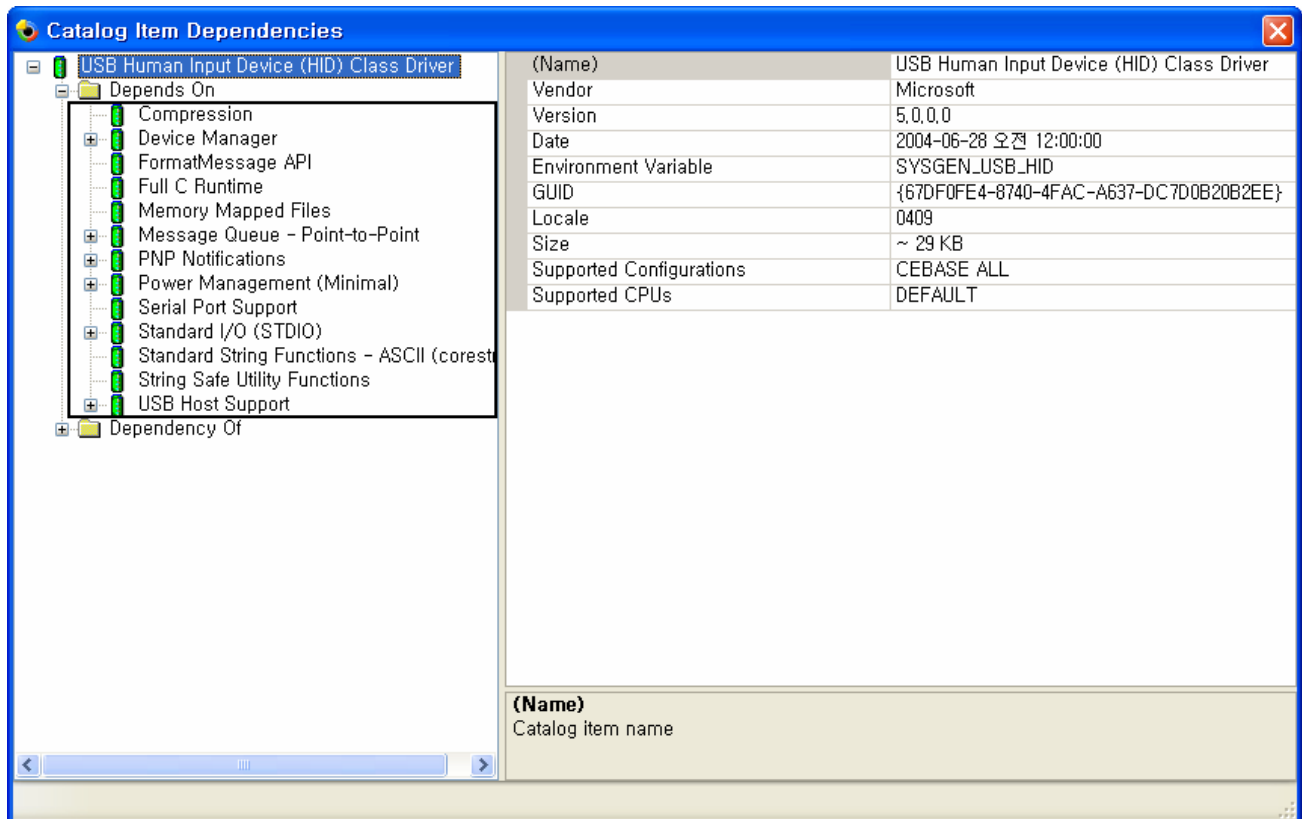
Developers themselves can add functions they need newly added in OS.

According to what they add in this part, WinCE will provide different functions. All functions needed are added in Images provided in CD. Developers should search and add each image themselves in the process of newly creating WinCE.

If you put the mouse-pointer on the function you want to add and click the right button of mouse, a popup window will be shown like [Picture 7.14], and if you push “add to OS design”, it will be included in OS.

If you push Dependencies on a popup window for the function you want to add, Item Dependencies window will be shown. And you will confirm it on this window.

You can search and add what exist in Depends On area on the left on [Picture 7.15].

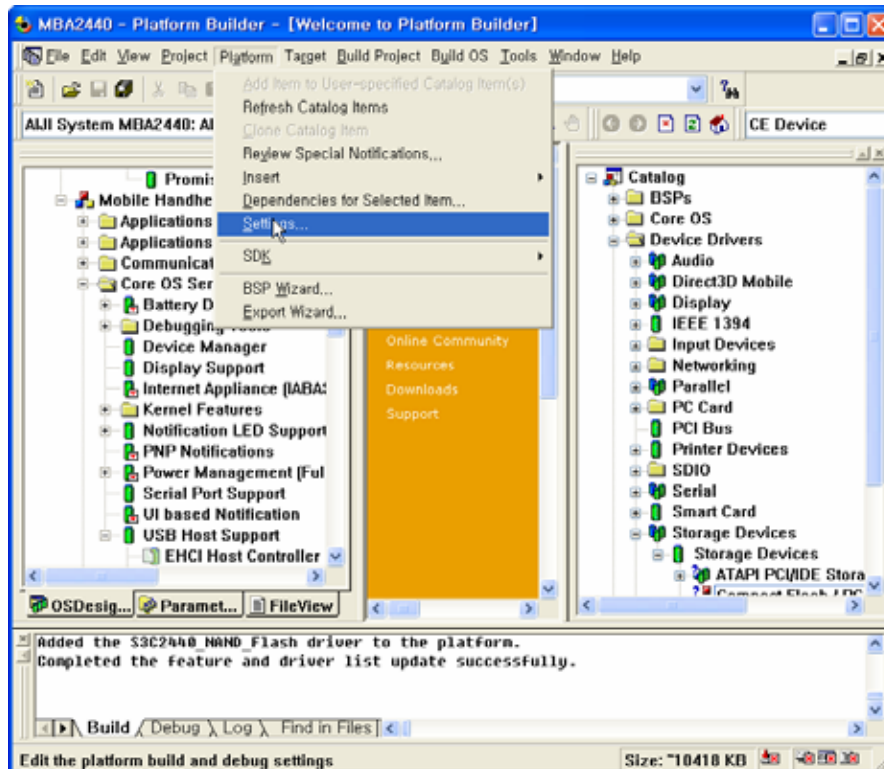


[Picture 7.15] Item Dependencies window

Add the following on the right window of WinCE 5.0 platform builder.

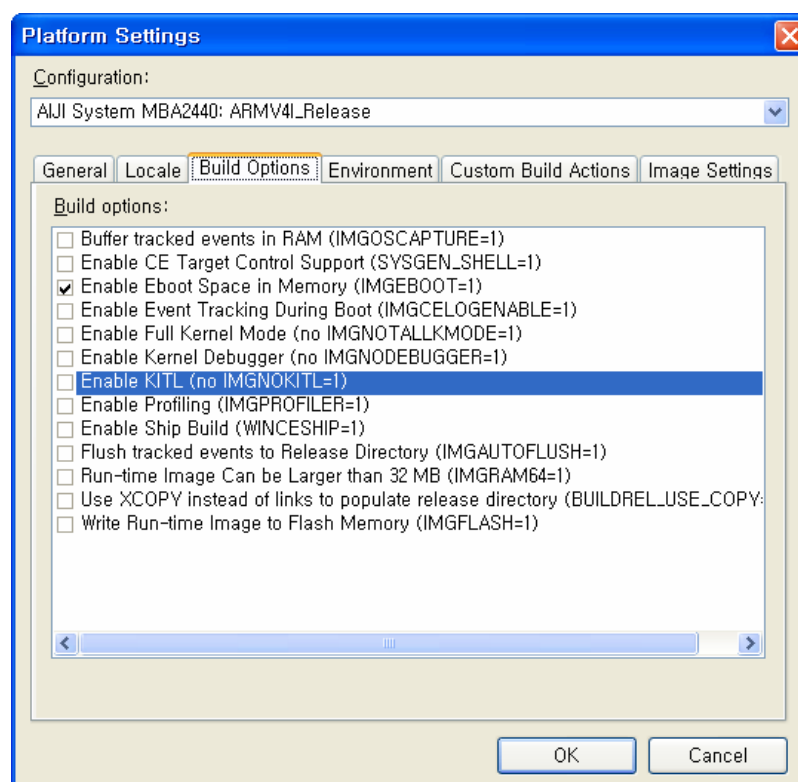
- ▶ Catalog → Core OS → Windows CE devices → Core OS Services → USB Host Support, Adding **USB Human Input Device (HID) Class Driver**
- ▶ Catalog → Core OS → Windows CE devices → Core OS Services → USB Host Support, Adding **USB Storage Class Driver**
- ▶ Catalog → Core OS → Windows CE devices → Core OS Services → USB Host Support → USB Human Input Device(HID) Class Driver, Adding **USB HID Keyboard and Mouse**
- ▶ Catalog → Core OS → Windows CE devices → File Systems and Data Store → Storage Manager , Adding **FAT File System**
- ▶ Catalog → Device Drivers → Storage Devices → Storage Devices → ATAPI PCI/IDE Storage Block Driver, Adding **ATAPI PCI/IDE Storage Block Driver**
- ▶ Catalog → Device Drivers → Storage Devices → Storage Devices, Adding **Compact Flash/PC Card Storage[ATADISK]**

15. Select Platform->Settings....



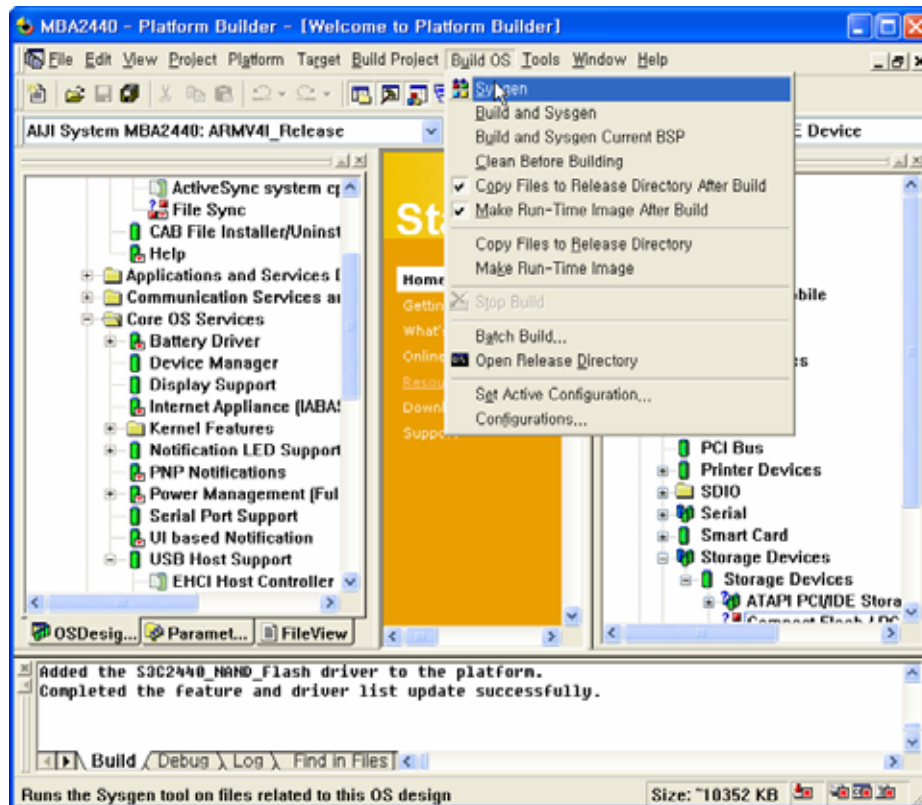
[Picture 7.16]

16. Select only Enable Eboot Space in Memory on Build Option tab on Platform settings window.

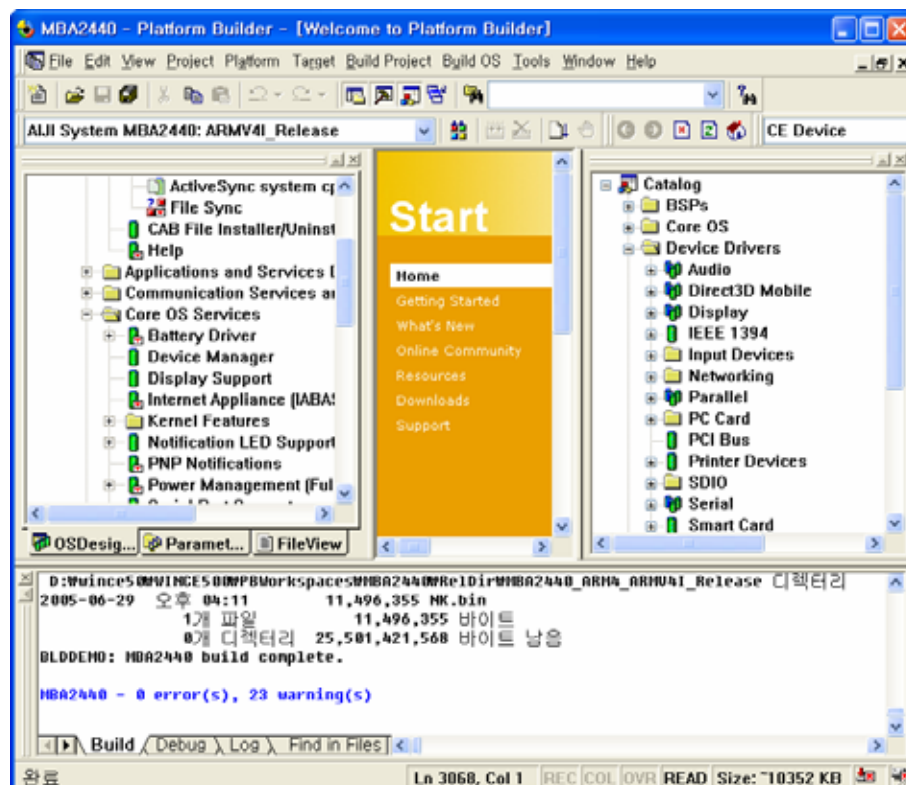


[Picture 7.17]

17. Now you can compile it. If you select Build OS -> Sysgen, compiling will start. It will take several or several tens minutes to compile it according to PC's capability and options added.



[Picture 7.18]



[Picture 7.19] When build is completed

[Picture 7.19] shows when build is completed.

If compiling is completed, you can find NK.nb0 file and NK.bin file created in WINCE500W

PBWorkspaces\WMBA2440\WRelDir\WMBA2440_ARM4_ARMV4I_Release directory.

You can boot WinCE by loading WinCE image into MBA2440 using u-boot or NAND boot loader.
(Refer to [2.3 Booting WinCE](#))

7.1.2 Support 6.4" TFT-LCD

WinCE BSP provided in CD basically supports 3.5" TFT-LCD, however, if you modify BSP, it can also support 6.4" TFT-LCD.

In this chapter, you will learn about parts which will be modified in BSP.

1. Modify LCD_TYPE macro in s2440.h

There is a header file named s2440.h in inc directory of BSP.

Modify LCD_TYPE macro in this header file.

Before modification...

```
#define LCD_TYPE      TFT240_320
// #define LCD_TYPE      TFT640_480
```

After modification...

```
// #define LCD_TYPE      TFT240_320
#define LCD_TYPE      TFT640_480
```

2. Modify CalibrationData in platform.reg

Modify CalibrationData part in platform.reg file of Files directory of BSP.

Before modification...

```
; for 640x480 display
;      "CalibrationData"="529,672 185,1293 188,51 862,57 864,1299"  ;(6.4" LCD ver1.3)
; for 240x320 display
      "CalibrationData"="500.664 166,140 120,1172 888,1160 876,152"
```

After modification...

```
; for 640x480 display
      "CalibrationData"="529,672 185,1293 188,51 862,57 864,1299"  ;(6.4" LCD ver1.3)
; for 240x320 display
;      "CalibrationData"="500.664 166,140 120,1172 888,1160 876,152"
```

After modification, recompile it.

7.1.3 Set up SDRAM Memory

WinCE 5.0 BSP provided in CD is adapted to SDRAM 64MB MBA2440 board. BSP should be modified to adapt to 128MB SDRAM to WINCE on MBA2440 board.

1. Modifying map.a file

Modify map.a file in KERNEL\WHALEWARM directory of BSP.

Before modification...

```
DCD 0x8C000000, 0x30000000, 64 ; 64 MB DRAM BANK 0, 1
```

After modification...

```
DCD 0x8C000000, 0x30000000, 128 ; 128 MB DRAM BANK 0, 1
```

After completing modification, recompile it.

7.1.4 Support Korean Language

C Basically, WinCE 5.0 BSP provided in CD only supports English language. You need to modify quite few things such as BSP modification and build option on platform builder.

You need to adjust the size of image created after compiled because it becomes much bigger when Korean font and etc. is included in WinCE Image.

In case of supporting only English, the size of WinCE image is 0x1400000(20MB size), however, you need to readjust the size of image to 0x2000000(32MB) size in case of supporting Korean as well.

Modify as the following for this.

1. Modifying config.bib file

Modify config.bib file in Files directory of BSP as the following.

Before modification...

```
NK      8C200000  01400000 RAMIMAGE
RAM      8D600000  01800000 RAM
.....
.....
ROMSIZE=01400000
```

After modification...

```
NK      8C200000  02000000 RAMIMAGE
RAM      8E200000  01800000 RAM
.....
```



```
.....
ROMSIZE=02000000
```

2. Modifying loader.h file

Modify loader.h file in inc directory of BSP.

Before modification...

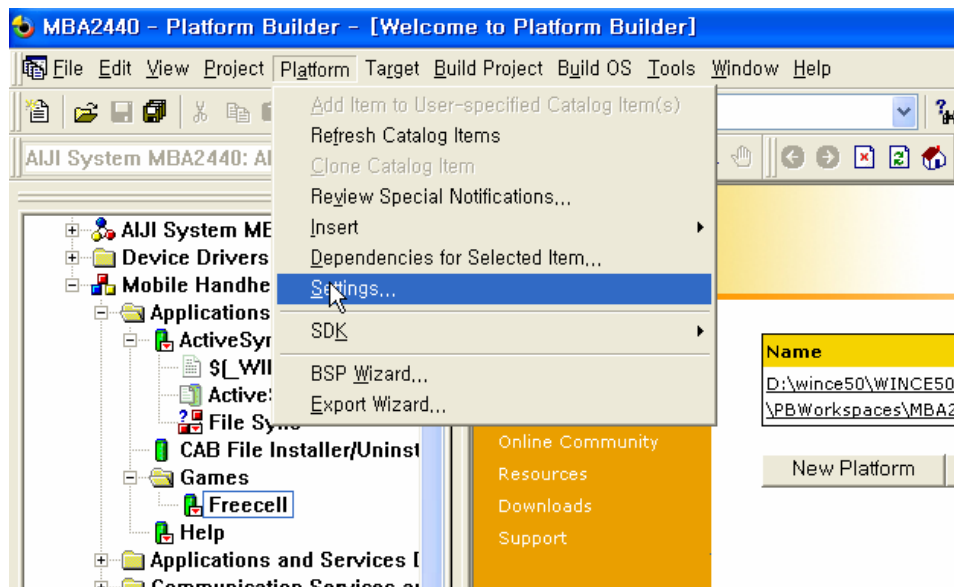
```
#define OS_RAM_IMAGE_SIZE      0x01400000
.....
.....
#define FLASH_CACHE_SIZE      0x01400000UL
```

After modification...

```
#define OS_RAM_IMAGE_SIZE      0x02000000
.....
.....
#define FLASH_CACHE_SIZE      0x02000000UL
```

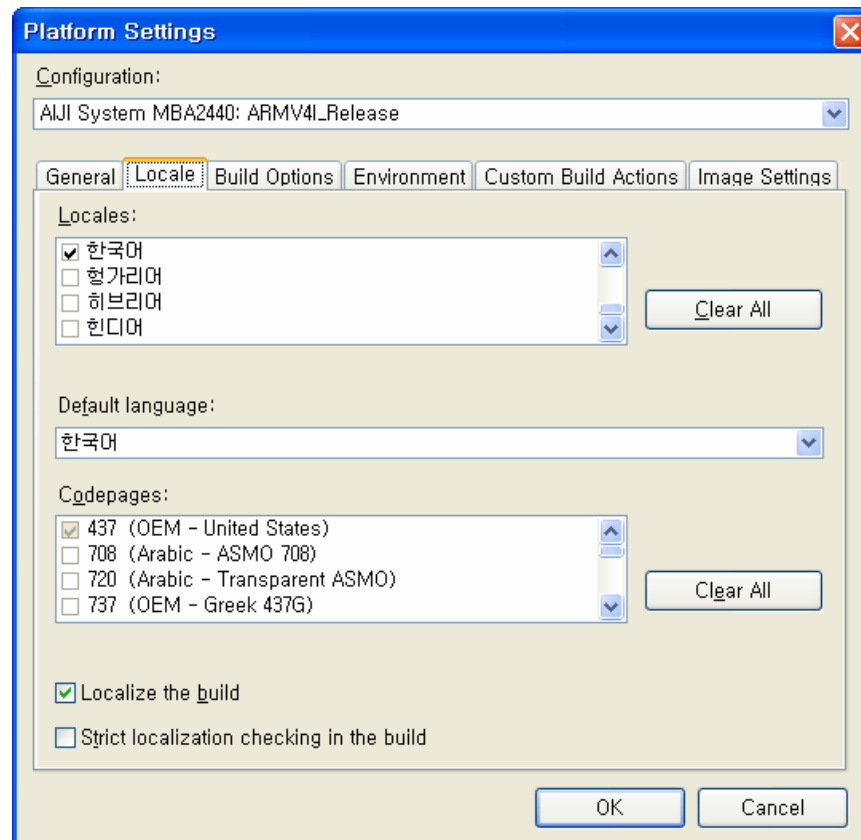
3. Modifying locale in platform → settings

Select and click platform → settings on WinCE platform builder.



[Picture 7.20]

Check locales as Korea on platform settings and select Default Language.



[Picture 7.21]

After pressing Ok button, recompile it.

After compiling is completed, upload WinCE image on MBA2440 and operate WinCE.

7.2 Eboot boot loader

Eboot is compiled and created when compiling BSP. It is performed in eboot directory within BSP source.

In this chapter, you will learn how to boot and upload WinCE image on the board using eboot.

1. PC which WinCE 5.0 is installed in and MBA2440 should be connected through LAN cable first and network setup should be done.

Set up the IP address of desktop PC as 192.168.100.1, and the IP address of MBA2440 as 192.168.100.3. (refer to [3.1 Examples of establishing development environment](#))

2. Compile MBA2440 BSP. (refer to [7.1.1 BSP build](#))

If compiling is done,

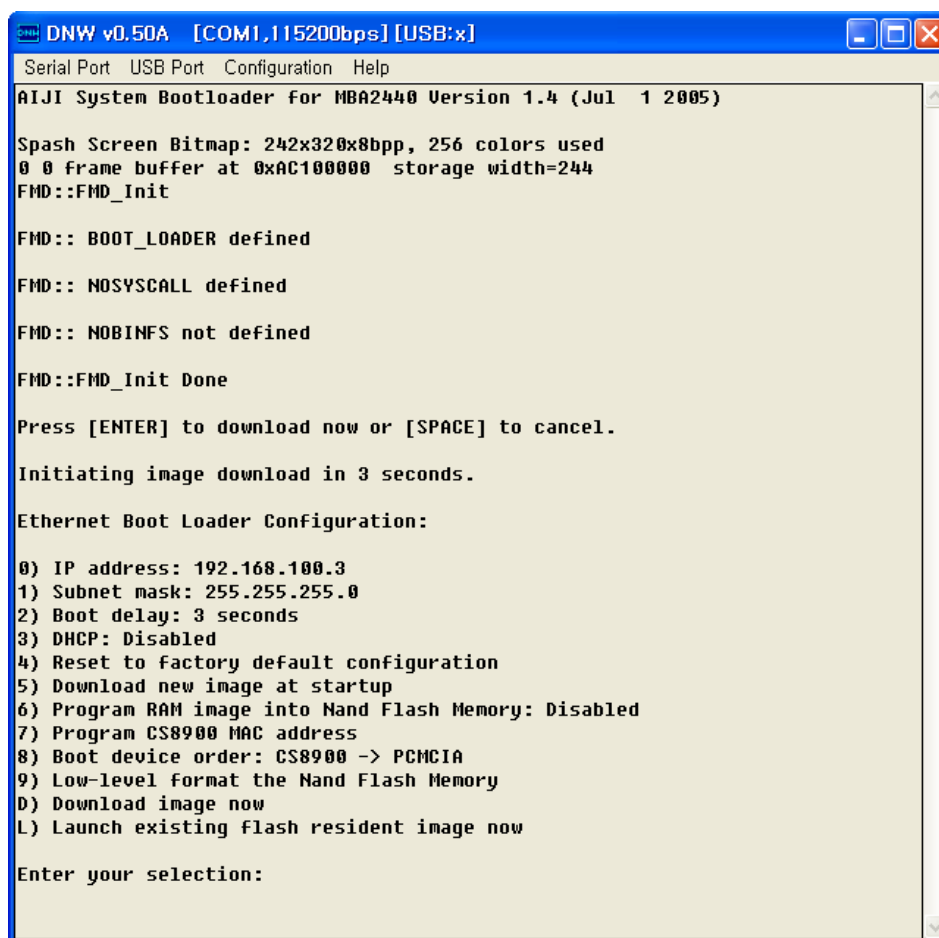
You will find EBOOT.nb0 file and EBOOT.bin file created in WINCE50/PBworkspaces/MBA2440/RelDir/MBA2440_ARM4_ARMV4I_Release

You can use EBOOT.nb0 image in WinCE5.0 directory in CD provided.

3. Write EBOOT.nb0 on AMD flash. (Refer to [3.5.2 Write image on boot flash](#))

4. If you boot it setting up AMD flash as boot flash after writing on AMD flash, eboot will boot.

If you push the space bar as soon as it boots, eboot menu will be shown



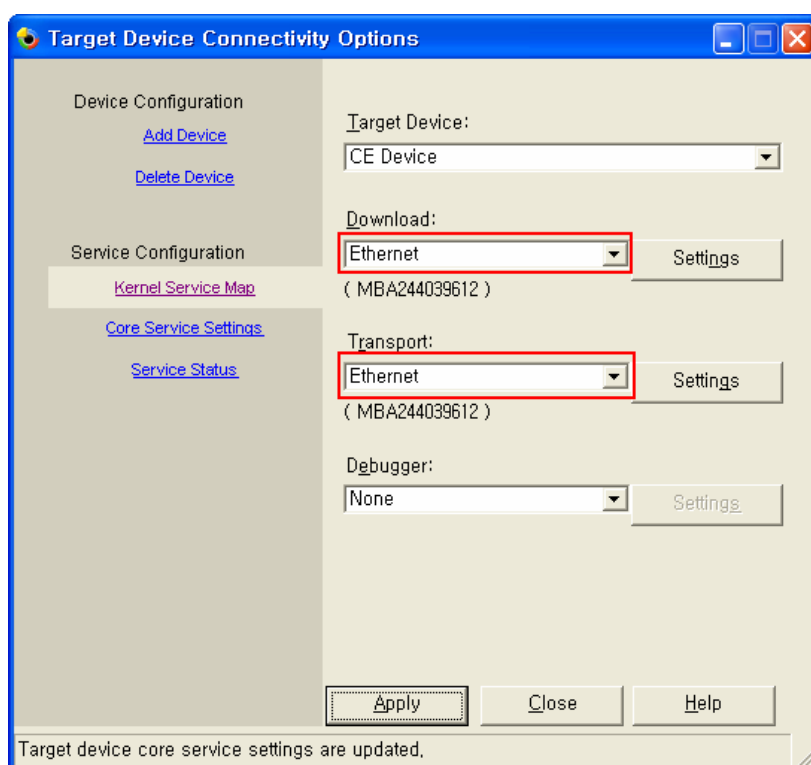
[Picture 7.22]

Set up IP address of MBA2440 on the menu No. 0 : 192.168.100.3

Set up Subnet mask on the menu No. 1: 255.255.255.0

Set up a MAC address on the menu No. 7 if needed : 12:34:56:78:9a:bc

5. Check if LAN cable is connected properly between MBA2440 and PC.
6. Select Target->Connectivity Options of WinCE 5.0 platform builder, set up as [Picture 7.23] on Target Device Connectivity Options window, push Apply button and finish it by pushing Close button.



[Picture 7.23]

7. Push D) Download image now on Eboot menu. Search for device (MBA2440) connected like [Picture 7.24].

```
Enter your selection: d
CS8900: MAC Address: 12:34:56:78:9A:BC

INFO: Probe: CS8900 is detected.

INFO: Init: CS8900_Init OK.

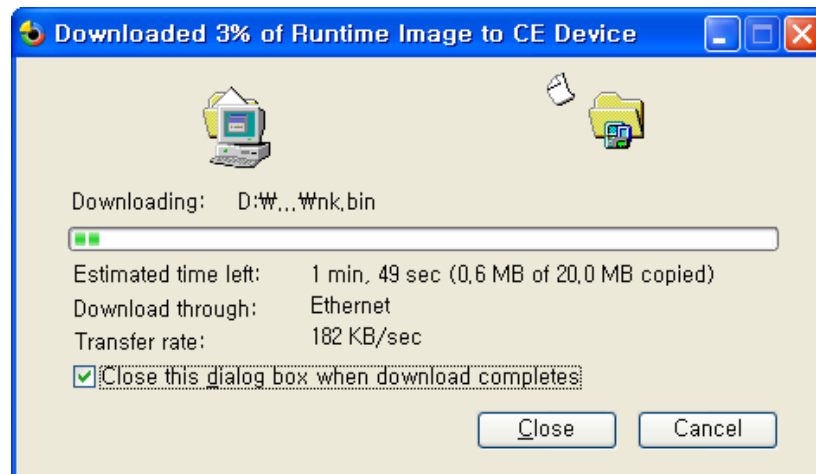
System ready!
Preparing for download...
INFO: Using static IP address 192.168.100.3.

INFO: Using subnet mask 255.255.255.0.

INFO: Using device name: 'MBA244039612'
+EbootSendBootmeAndWaitForTftp
Sent BOOTME to 255.255.255.255
Sent BOOTME to 255.255.255.255
Sent BOOTME to 255.255.255.255
```

[Picture 7.24]

8. Push Target->Attach Device of WinCE 5.0 platform builder.



[Picture 7.25]

Download nk.bin file on MBA2440 like [Picture 7.25].

If downloading is completed, WinCE will boot.

7.3 Nboot boot loader

Nboot loader explained in this chapter is not in charge of resetting hardware such as eboot or other hardware like eboot. It is a firmware used to operate WinCE image and copy WinCE image in memory after setting up NAND flash a boot flash on MBA2440 and booting the board loaded in block 0 of NAND flash. It is smaller than eboot or u-boot

The way Nboot operates is very simple.

Nboot image is smaller than 4KB. If you boot NAND flash on MBA2440, S3C2440 will bring and read the first 4KB data of NAND flash to the buffer first and operate it.

It is 4KB size of SRAM buffer existing in S3C2440, and it's called stepping stone buffer. That's why the size of nboot is smaller than 4KB and it is saved in the block No. 0 of NAND flash.

WinCE image is saved in the block No.1 of NAND flash, and nboot will read the same size of WinCE image starting the block No. 1 from NAND flash and copy it to 0x30200000 address of SDRAM system memory if nboot starts operating after it is loaded in stepping stone buffer.

If copying is completed, it will give the right of control to WinCE image, and after that, WinCE image will operate.

Nboot source is a compressed file named MBA2440_nand_loader.zip in firmwares\Nboot directory in CD.

The main function has been developed as ADS1.2 in 2440loader.c file.

There is a macro needed to refer in 2440loader.c.

```
#define BOOT_IMAGE_SIZE                (0x1400000)
    . . .
#define BOOT_START_ADDR_OFFSET        (0x200000)
    . . .
#define DOWNLOAD_ADDRESS  (_RAM_STARTADDRESS+ BOOT_START_ADDR_OFFSET)
```

The above macro is the heart of nboot.

BOOT_IMAGE_SIZE

: This macro shows WinCE image. It is set up as 0x14000000 because the size of WinCE is 20MB. When you newly build WinCE image modifying the size of image, you can load it in memory properly only if you modify it in nboot.

DOWNLOAD_ADDRESS

: This macro determines where to load WinCE image in system memory (SDRAM) after copying it NAND flash.

The value of DOWNLOAD_ADDRESS becomes 0x30200000 address because the physical address of SDRAM is 0x30000000 (_RAM_STARTADDRESS) and the start offset of WinCE image is 0x200000 (BOOT_START_ADDR_OFFSET).

This macro can be modified and used according to the convenience of developers in the future.

WinCE can boot only if it exists in 0x30200000 address of memory.

Because it is set up in WinCE bsp.

Please analyze source yourself for details.

7.3.1 Booting WinCE with Nboot

Load Nboot image and WinCE image with tftp into memory on u-boot, load it into NAND flash memory again, set up NAND flash as boot flash, and operate WinCE at the same time you turn off the computer.

Nboot operate for the first time in case of booting NAND flash

And Nboot is a bootloader which take the only role of loading WinCE image into memory and giving the right of control to the image.

The way of operating it is as the following.

1. Download WinCE image and Nboot image by tftp command into memory.
2. Write images downloaded on NAND flash.
3. Set up NAND flash written as boot flash and boot it.

You need to determine on what NAND flash to write through J3 whichever u-boot is operated in NOR flash or NAND flash.

It will write images on SOP (K9F5608) if J3 is set up as 1-2 and on SMC(K9S1208) if J3 is set up as 2-3.

Download WinCE image and NAND bootloader image on u-boot.

Let' say nboot image is nboot.bin and WinCE image is NK.nb0.

Download Nboot image through tftp command.

```
MBA2440 # tftp 30000000 nboot.bin
```

```
TFTP from server 192.168.100.2; our IP address is 192.168.100.3
```

```
Filename 'nboot.bin'.
Load address: 0x30000000
Loading:-
done
Bytes transferred = 4024 (fb8 hex)
```

Download WinCE image through tftp.

```
MBA2440 # tftp 30004000 NK.nb0
TFTP from server 192.168.100.2; our IP address is 192.168.100.3
Filename 'NK.nb0'.
Load address: 0x30004000
Loading:/
done
Bytes transferred = 20971520 (1400000 hex)
```

And then, write the image on NAND flash.

```
MBA2440 # nande all
erase all blocks [0 ~ 2047]
Erasing block ..-
Complete erasing block
MBA2440 # nandw t 30000000 0 1500000
Write start point : block[0] page[0] size = 0x1500000
Writing data .../
Write complete(0x1500000): from 0x30000000 SDRAM to 0x0(0) NAND flash
MBA2440 #
```

Turn off the board, set up NAND flash you write on as boot flash, and turn it on.

```
Image loading Start
Image loading End
Boot time=nTCNT*82uS. nTCNT=0x6784.
I/O Strength Min

CE Kernel for ARM (Thumb Enabled) Built on Jun  1 2004 at 16:38:37
ProcessorType=0920 Revision=0
sp_abt=ffff5000 sp_irq=ffff2800 sp_undef=ffffc800 OEMAddressTable = 8c201480

Windows CE Firmware Init
INFO: Initializing system interrupts...
```

```

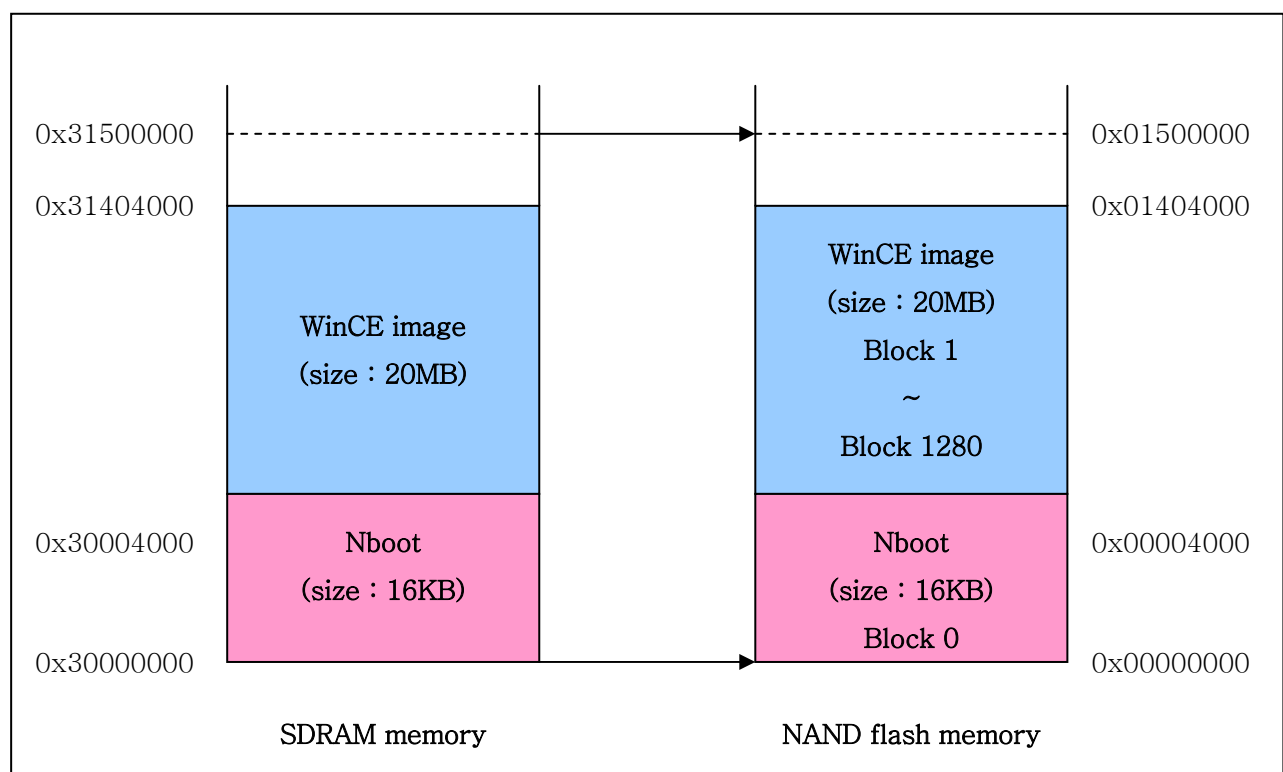
INFO: Initializing system clock(s)...
INFO: Initializing driver globals area...
INFO: Initializing driver globals area...-> ok!
INFO: Initializing Display grid -> ok!
### SDMMC config current rGPGCON: ff95ffba
OEMInit Done...
Sp=ffffc7cc
OEMSetRealTime: Year: 2003, Month: 1, Day: 1, Hour: 12, Minute: 0, second: 0 rcnr=1h
OEMSetRealTime(register): Year: 3, Month: 1, Day: 1, Hour: 12, Minute: 0, second: 0 rcnr=1h
FMD::FMD_Init
.....

```

You can see that it copied NK.nb0 WinCE image through image loading start and image loading end message.

You can see WinCE image boots through CE Kernel for ARM (Thumb Enabled) Built on Jun 1 2004 at 16:38:37 message.

The following is a memory map between NAND flash and memory which helps easy understanding about the above process.



[Picture 7.26] memory map between NAND flash and memory

8 UbiFOS

8.1 What is UbiFOS...

UbiFOS™ is a domestic operating system originally developed by AIJI system and Professor Cheol-Hoon Lee (System software) in computer engineering department of Chungnam National University under a catchphrase, “internet-speeding RTOS” in 2003, and it operates stable Kernel by the development process for years from 1996.

UbiFOS™ made each function of UbiFOS™ Kernel into modules considering characteristics of embedded system requiring a small size of memory, and it can make the most suitable image size because it can be used optionally accordance with the requirements of application program.

i The suitability of UbiFOS™ has been authorized by loaded in SoC for multimedia produced in Korea. It is very efficient to reduce the price and size because its size is small, about 50% compared to other well-known foreign RTOS. It provides stability and transplantation supporting various platforms (ARM type, CalmRISC type, MPC type and Intel type).

These are general characteristics of UbiFOS™ kernel.

-
- **Real time operation system in multi-tasking type based on the order of priority for embedded system**
 - **Components**
 - Scheduling
 - Scheduling in a preoccupying type based on the order of priority
 - RR (Round-Robin) for the order of same priority / Supporting FIFO(First-In First-Out)
 - Patented with “Task Scheduling Method”
 - Dynamic Memory Management
 - Heap storage manager (Variable-Size)
 - Memory Pool(Fixed-Size)
 - ITC (Inter Task Communication)
 - Message queue, message mailbox, message port, task port
 - Inter Task Synchronization
 - Semaphore, event flags, signal
 - Efficient handling for interrupt
 - LISR (Low-Level ISR)
 - HISR (High-Level ISR)
 - Software Timer
 - **Making into Module**
 - UbiFOS™ can be in the most suitable condition by making each function into module.
 - **Source Code**

- BSP (Board Support Package), Scheduler, Handling Exception
: Assembly language
- Etc.
: ANSI C
- **Support Processor**
 - ARM type (S3C2400, S3C2800, S3C2440, S5H5002)
 - CalmRISC type (CalmRISC 16, CalmRISC 32)
 - Motorola type (MPC 750)
 - Intel type (80x86)

UbiFOS™ is a real time operating system in an occupying type based on the order of priority in multi-tasking environment for embedded system.

Task can approach joint work area freely with a moving structure based on thread. For source code of UbiFOS™ kernel, the scheduler part for hardware reset which is dependent to platform and scheduler part for improving system capability are created with assembly language, and the rest of source code are created based on ANSI C

UbiFOS™ provides functions such as message queue, message mailbox, message port and task port for the communication between tasks, and provides semaphore, event flags and signals for Inter Task Synchronization.

[Picture 8.1] shows a diagram of UbiFOS™ organization.



[Picture 8.1] Overall organization of UbiFOS™

8.1.1 Kernel

Basic components of UbiFOS™ kernel are as the following.

- ❑ Task Management
- ❑ Interrupt Handling
- ❑ Storage Allocation
- ❑ Inter Task Communication
- ❑ Inter Task Synchronization
- ❑ Timer

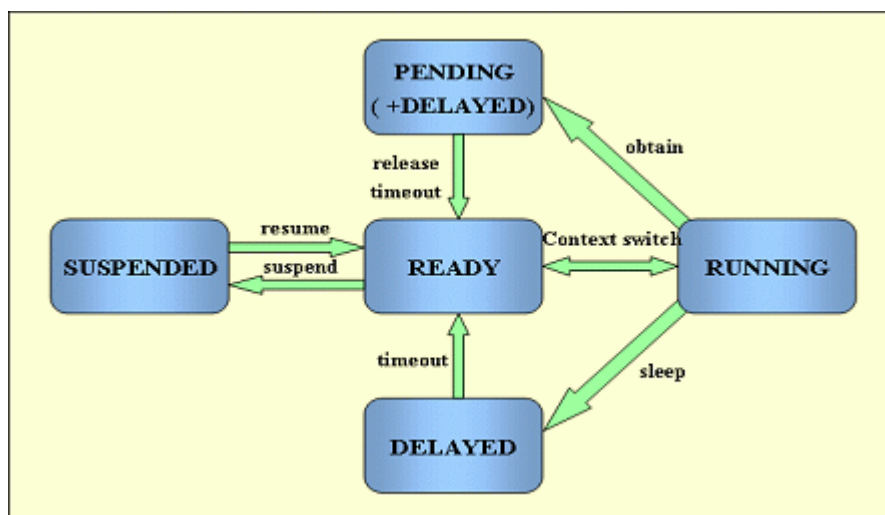
8.1.1.1 Task Management

Applicable programs of embedded systems have a very close functional relation because they perform many functions at the same time and are divided logically. Multi-tasking has been introduced to perform many tasks at the same time because it is hard to program in order like this. UbiFOS™ supports the basic multi-tasking environment.

○ Task status on UbiFOS™

- SUSPEND : Initial status task is created in, or waiting status until a certain event is originated.
- READY : Waiting status until operated.
- RUNNING : Tasks occupied CPU / operating status.
- DELAYED : Status being excluded from scheduling objects during specific time.
- PENDING : Stand by status to achieve specific resource during performing tasks.
- PENDING+ DELAYED : Status setting up timeout during stand by to achieve specific resource.

[Picture 8-2] shows the change of task status.



[Picture 8-2] Change of Task status on UbiFOS™

○ Scheduler of UbiFOS™

UbiFOS™ schedules in a preoccupying type based on the order of priority, basically schedules Round Robin for the same order of priority, and supports FIFO (First-in First-Out) scheduling

when needed. The order of priority of each task from 0 to 255 on UbiFOS™ has priority, and 0 has the highest order of priority.

8.1.1.2 Interrupt Handling

Interrupt means events originated inside and outside of system, and it ceases performing a task and performs ISR (Interrupt Service Routine) if an interrupt is originated. Interrupt handling requires short response time due to characteristics of real time operation system. UbiFOS™ is divided and designed into LISR(Low-Level ISR) and HISR(High-Level ISR) to guarantee flexibility for short response time about interrupt.

[Table 1] explains characteristics of LISR and HISR.

LISR(Low-level ISR)	HISR(High-level ISR)
- Interrupt requiring short response time - Same as handling existing interrupt - Using task stack - No use of system call modifying the status of task	- Interrupt having flexible time compared to LISR - Using its own stack (HISR stack) - Giving the order of priority in the step 3 (0~2) - Scheduling HISR before scheduling tasks - Same as task scheduling skills

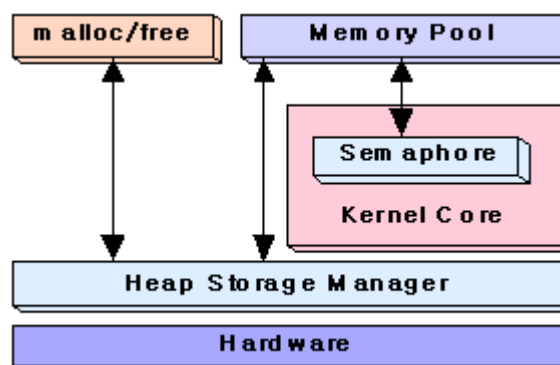
[Table 8-1] Characteristics of LISR and HISR

8.1.1.3 Storage Allocation

UbiFOS™ provides a memory management skill in two steps for storage allocation.

The lower step is Heap-Storage Manager which can allocate and remove memory in the changeable size, and the upper step is Memory Pool which can allocate and remove memory in the fixed size.

[Picture 8-3] shows a dynamic memory management system.



[Picture 8-3] Dynamic memory management system of UbiFOS™

○ Heap Storage Manager

Heap storage manager has high flexibility because it can allocate memory in an optional size. But it takes $O(n)$ of time to find memory block in a reasonable size, and it decreases time determination. Also outside fragmentation phenomenon appears.

○ Memory Pool

Memory pool allocates memory in a fixed size. It is good for an applicable program which allocates memory quite often or has the same size of memory. When allocating memory, it decreases flexibility. But it can reduce outside fragmentation phenomenon and time determination because it can use the area allocated and removed as it is.

The applicable program can use both management skills together and it can also a reasonable management skill considering strength and weakness of each.

8.1.1.4 Inter Task Communication

UbiFOS™ provides multi-tasking environment which can operate many independent tasks at the same time. Inter task communication is required to give and receive information between tasks in multi-tasking environment.

○ Message Mailbox

Message mailbox on UbiFOS™ can be used comfortably when giving and receiving simple message as a structure which can send a message at once.

○ Message Queue

Message queue is used when delivering many messages differently from message mailbox. UbiFOS™ provides message queue in a changeable length and in a fixed length according to changeableness of the message delivered. It delivers message by copying

○ Message Port

UbiFOS™ provides message ports as a message port delivering a pointer itself. Delivering by message copy is better for data security compared to delivering a pointer, but overhead for copying is followed. It is much easier to deliver a pointer than copying a message for transmitting messages between tasks on TCP/IP and etc.

○ Task Port

UbiFOS™ provides a task port for 1:1 communication between tasks differently from message port of message queue and message mailbox.

○ Semaphore

Semaphore makes the mutually exclusive approach possible when approaching to shared Tasks only achieved semaphore can use shared resource without interference of other tasks on UbiFOS™.

8.1.1.5 Inter Task Synchronization

Inter task synchronization between tasks mean the coordination of order among tasks, and

ways to synchronize tasks are semaphore, event flags and signal.

○ Semaphore

Semaphore can be used as a way to synchronize tasks besides used for the mutually exclusive approach.

○ Event Flags

Event flags are used when synchronization for many events is needed as an extended concept of semaphore. Event flags of UbiFOS™ are flag variables of 32bit, and each bit shows a separate event. OR and AND operation are possible for handling events.

○ Signal

Signal of UbiFOS™ is a skill to do software interrupt for a specific task, consisting of signal variables of 32bit like event flags. But signal exists in each task differently from event flags, and motivate signal handler registered with stack newly constructed for handling signal.

8.1.1.6 Timer

UbiFOS™ provides software timer, it can be used in case there are tasks which will be performed periodically. The unit of time used for timer is a tick. A tick means the period hardware timer interrupt originates in.

8.1.1.7 Dynamic Loader

Loader doesn't load programs at once but a part needed for operation in order.

8.2 Booting UbiFOS

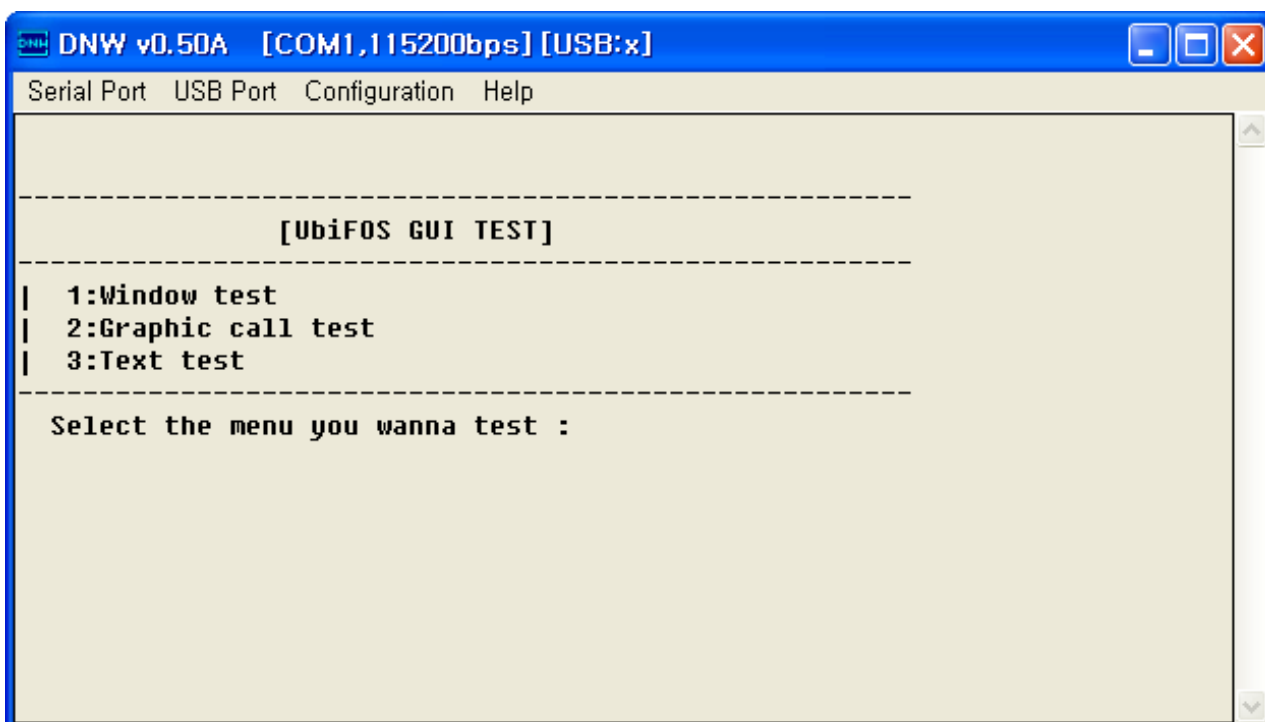
UbiFOS only provides images.

There are **UbiFOS_kor.bin** file and **UbiFOS_eng.bin** in UbiFOS directory in CD. Write **UbiFOS_eng.bin** file on 0 address of AMD flash. Refer to [3.5.2 Write image on Boot Flash](#) or [5.2.2.4 NOR flash command](#) for how to write.

Connect serial cable between MBA2440 and PC, and use hyper terminal such as DNW or minicom as console. (Refer to [2.1.3 Hyper-terminal setup](#) and [3.3.4 Minicom setup](#))

(Tip: Use other console emulators if letters are broken when used on DNW)

If you set up AMD flash as boot flash and turn up the power, you can see a menu page like [Picture 8.4].



[Picture 8.4]

These are menus to display GUI on LCD using UbiFOS.
Please select a menu and test it yourself.